ROBUST ROBOT TEAMS; MAKING PLANNING FOR MULTI-AGENT SYSTEMS REFLECTIVE

by

Jonathan A. Diller

© **Copyright by Jonathan A. Diller, 2025**

All Rights Reserved

A thesis submitted to the Faculty and the Board of Trustees of the Colorado School of Mines in partial fulfillment of the requirements for the degree of Doctor of Philosophy (Robotics).

Golden, Colorado

Date _____

Signed: _____

Jonathan A. Diller

Signed: _____

Dr. Qi Han
Thesis Advisor

Golden, Colorado

Date _____

Signed: _____

Dr. Andrew J. Petruska
Robotics Program Director
Department of Computer Science

ABSTRACT

Robot teams can extend the reach of what is possible by humans and automate dangerous or monotonous tasks. However, robots are complex systems and poor planning leads to autonomous agents that lack robustness. This thesis demonstrates how to apply a *Reflective Planning* design philosophy when crafting and validating algorithms for multi-robot planning and tasking applications. The *Reflective Planning* approach seeks to make the decision-making process reflective of how a robot's actions impact the status of its own systems, how the environment impacts the actions of the robot, and how the actions of an individual robot impact its ability to interact with others within a team. More specifically, we investigate how to best model the way that robots interact with their environment and each other. After acquiring a deeper understanding of these models, we integrate them into planning algorithms, seeking to not only abide by the limitations of these high-fidelity models but seek out methods for exploiting their attributes to further the operational capabilities of our robots.

Within this larger scope, we are specifically interested in energy and communication. We review how drones consume energy and then demonstrate how to exploit the relationship between drone travel distance and speed in a drone-ground vehicle path finding problem. We then investigate how unmanned ground vehicles can be used to transfer energy to drones in support of patrolling applications over indefinite time horizons. Furthermore, we show how to assign a probability to a drones operational capabilities based on energy stores and integrate this into a decentralized tasking algorithm. On the communication side, we investigate how to best create maps that capture the potential for robot-to-robot communication and show how to apply these maps to a variety of multi-robot applications. We then investigate the intersection of energy and communication by looking at how to appropriately allot energy between preplanned robot actions and adaptive behavior during deployment in response to the stochastic nature of communication. Building on this concept, we also investigate how to bound allowable model error for both energy and communication, with an emphasis on adapting team-level decisions in response to external disturbances during deployment.

TABLE OF CONTENTS

ABSTRACT iii

LIST OF FIGURES xii

LIST OF TABLES xv

ACKNOWLEDGMENTS 1

CHAPTER 1 INTRODUCTION 2

1.1 Context and Motivation 2

1.2 Our Research Question 4

1.3 Reflective Planning 4

1.4 Scope 5

1.5 Research Methodology 5

1.6 Contributions of this Thesis 6

CHAPTER 2 LITERATURE REVIEW 9

2.1 Multi-Robot Routing Problems 9

2.2 Multi-Robot Task Allocation Problems 11

2.3 Multi-Agent Planning 12

2.4 System Models 12

2.4.1 Energy Models 12

2.4.2 Wireless Communication Models 13

CHAPTER 3 ENERGY-AWARE DRONE PATH FINDING WITH A FIXED-TRAJECTORY GROUND VEHICLE 15

3.1 Introduction 15

3.2 Related Work 17

3.2.1 Multi-Drone Path Finding 17

3.2.2 Mixed Drone & Ground Vehicle Path Finding 18

3.2.3 Drone Energy Models 19

3.3 Problem Formulation . . . 20
3.3.1 Adaption of Drone Speed . . . 24
3.4 Our Solution Framework . . . 26
3.5 Fixed-Start MT-OTM Solutions . . . 27
3.5.1 MINLP Formulation for the fixed-MdMtHPP . . . 30
3.5.2 Heuristic Solution for the fixed-MdMtHPP . . . 33
3.6 Generalized MT-OTM Solutions . . . 34
3.6.1 MINLP Formulation for the MT-OTM Problem . . . 34
3.6.2 Heuristic Solutions for the fixed-MT-OTM Problem . . . 35
3.7 Simulation Evaluation . . . 38
3.7.1 Simulation Setup . . . 38
3.7.1.1 Realistic Energy Model from Field Tests . . . 38
3.7.1.2 Metrics and Baseline Approaches . . . 39
3.7.2 Comparison of Fixed-Start Approaches . . . 40
3.7.3 Comparison of Dynamic Approaches . . . 42
3.7.3.1 Mission Completion Time Compared to Optimal Solution . . . 42
3.7.3.2 Larger Inputs with Linear Trajectory Gound Vehicle . . . 43
3.7.3.3 Larger Inputs with Sinusoidal Trajectory Gound Vehicle . . . 44
3.7.4 Impact of Speed-Scheduling . . . 46
3.7.5 Computational Efficiency . . . 47
3.7.6 Summary of Findings . . . 48
3.8 Problem Scenario Prototypes . . . 49
3.8.1 Urban Environment Simulation . . . 49
3.8.2 Field Test . . . 50
3.9 Conclusions and Discussion . . . 51
CHAPTER 4 COLLABORATIVE PATH PLANNING OF ENERGY-SHARING DRONE-UGV TEAMS FOR PERSISTENT PATROLLING . . . 54
4.1 Introduction . . . 54

4.2 Literature Review . . . 56
4.2.1 Related Drone-UGV Problems . . . 56
4.2.2 Path-Finding Algorithms . . . 58
4.3 Problem Formulation . . . 59
4.3.1 Patrolling Kernels . . . 59
4.3.2 Penalty Accumulation Rate . . . 60
4.3.3 Robot Energy Models . . . 62
4.3.4 Formal Problem Definition . . . 64
4.4 Proposed Algorithms . . . 65
4.4.1 Launch Optimizer with Rescheduling and Swapping Algorithm . . . 65
4.4.2 Finding an Initial Solution . . . 67
4.4.3 Optimizing Launch and Land Actions . . . 68
4.4.4 Obstacle Avoidance . . . 70
4.4.4.1 Square Constraints . . . 71
4.4.4.2 Action-to-Action Constraints . . . 72
4.4.5 Time Complexity Analysis . . . 73
4.5 Numerical Simulation Results . . . 73
4.5.1 Simulation Setup & Baseline Approach . . . 74
4.5.2 Simulation Results . . . 75
4.5.2.1 *IncN* Data Set . . . 75
4.5.2.2 *IncO* Data Set . . . 76
4.5.2.3 $IncM_a$ Data Set . . . 76
4.5.2.4 $IncM_g$ Data Set . . . 77
4.5.2.5 Computation Time . . . 78
4.5.3 Results Discussion & Limitations . . . 80
4.6 Case Study . . . 81
4.7 Prototype on Real Drone-UGV Team . . . 82

4.8 Conclusions . . . 83
CHAPTER 5 FAILURE-AWARE TASKING FOR TEAMS OF DRONES . . . 84
5.1 Introduction . . . 84
5.2 Related Works . . . 86
5.2.1 Task Assignment Problems for Robotics . . . 86
5.2.2 Distributed Task Assignment Algorithms . . . 87
5.3 Maximum Probability of Task Success Problem Formulation . . . 88
5.3.1 Drone Energy-based Probability of Contribution . . . 89
5.3.2 Maximum Probability of Task Success Problem . . . 90
5.4 Solution Existence and Upper Bound . . . 93
5.5 Algorithms to Solve the MPTS Problem . . . 95
5.5.1 Match + Iterated-Local-Search Algorithm . . . 95
5.5.2 Optimal Solution: Branch-and-Match (BnM) . . . 98
5.6 Performance Evaluation Via Simulation . . . 98
5.6.1 Realistic Inputs . . . 99
5.6.2 Baseline Methods . . . 100
5.6.3 Simulation Results . . . 101
5.6.3.1 Performance Versus Optimal Solution . . . 101
5.6.3.2 Performance at Scale . . . 101
5.6.3.3 Impact Of Floating Agents . . . 102
5.6.3.4 Finding Valid Solutions . . . 103
5.6.3.5 Computation Time (Centralized) . . . 103
5.6.3.6 Packet Count and Computation Time (Distributed) . . . 104
5.6.3.7 Summary of Major Findings . . . 105
5.6.4 Case Study Results . . . 105
5.7 Performance Evaluation in the Field . . . 105
5.8 Conclusions . . . 107

CHAPTER 6 COMMUNICATION MAPPING FOR ROBOT TEAMS 108
6.1 Introduction 108
6.2 Related Work 110
6.2.1 Communication-Aware Robot Team Deployment 110
6.2.2 Predicting & Mapping Communication for Robotics 112
6.3 Background on Cumulative Link Quality Metrics 114
6.4 Link Quality Communication Map 115
6.4.1 Link Quality Communication Map 116
6.4.2 Predicting ETX Between Robots 118
6.5 Applying Link Quality Communication Maps 119
6.5.1 Properties of Link Quality Communication Maps 119
6.5.2 Finding Minimum Cost Paths in an LQCM 121
6.5.3 Applications of Link Quality Communication Maps 124
6.6 Evaluation in the Field 124
6.6.1 ETX Prediction Accuracy 126
6.6.1.1 Model Accuracy 127
6.6.1.2 Training on Sparse Datasets 129
6.6.2 Maintaining Connectivity with Another Device 131
6.6.3 Generating and Searching Maps 132
6.6.4 Environmental Monitoring Application 134
6.7 Conclusions, Limitations, and Future Work 137
CHAPTER 7 HOLISTIC FRAMEWORK FOR MULTI-DRONE DATA COLLECTION IN IOT SYSTEMS 140
7.1 Introduction 140
7.2 Related Work 142
7.3 Problem Formulation 144
7.3.1 Communication Model 144
7.3.2 Energy Model 145

7.3.3 Travel-To-TX Tradeoff . . . 146
7.3.4 Problem Definition . . . 146
7.4 Holistic Drone Planning Framework . . . 147
7.5 Offline Global Planner . . . 149
7.5.1 Mixed Integer Non-Linear Solution . . . 150
7.5.2 Polynomial Time Solution . . . 151
7.6 Online Adaptive Planner . . . 154
7.7 Drone & Network Simulator . . . 154
7.7.1 Drone Autopilot . . . 155
7.7.2 Network Simulator . . . 155
7.8 Numerical Evaluations . . . 156
7.8.1 Data Transmission Experiments . . . 156
7.8.2 Framework Evaluation . . . 158
7.8.3 Offline Global Planner Evaluation . . . 159
7.8.3.1 Small Varying Number of Sensors . . . 160
7.8.3.2 Large Varying Number of Sensors . . . 162
7.8.3.3 Varying Data Volume . . . 162
7.8.3.4 Computation Time . . . 163
7.8.3.5 Offline Global Planner Evaluation Summary . . . 163
7.8.4 Online Adaptive Planner Evaluation . . . 163
7.9 Field Test . . . 164
7.10 Conclusions . . . 165
CHAPTER 8 SEMPER GUMBY: ADAPTIVE MULTI-ROBOT TASKING IN DYNAMIC ENVIRONMENTS . . . 166
8.1 Introduction . . . 166
8.2 Related Work . . . 168
8.2.1 Stochastic System Models in Robot Tasking . . . 168
8.2.1.1 Energy Models in Tasking . . . 168

8.2.1.2 Communication Models in Tasking . . . 169
8.2.2 Other Related Topics . . . 170
8.3 Problem Formulation . . . 170
8.3.1 Energy Model . . . 170
8.3.2 Communication Model . . . 171
8.3.3 Formal Problem Definition . . . 172
8.4 Bounding System-Model Error . . . 172
8.4.1 Energy Model Error . . . 173
8.4.2 Communication Model Error . . . 174
8.5 Error-Aware Tasking Framework . . . 175
8.6 Multi-Robot Tasking Scenarios . . . 177
8.6.1 Multi-Drone Site Inspection . . . 177
8.6.1.1 Solver Speed-Up . . . 179
8.6.2 Drone-UGV Teaming for Disaster Response . . . 181
8.6.2.1 Solver Speed-Up . . . 184
8.7 Evaluation . . . 184
8.7.1 Experimental Setup . . . 184
8.7.2 Multi-Drone Site Inspection . . . 186
8.7.3 Collaborative Drone-UGV Teams . . . 188
8.7.4 Discussion & Limitations . . . 190
8.8 Conclusions & Future Work . . . 190
CHAPTER 9 CONCLUSIONS & FUTURE WORK . . . 192
9.1 Conclusions . . . 192
9.2 Future Work . . . 193
9.2.1 Modeling More Robot Capabilities . . . 193
9.2.2 Expanding To New Problem Domains . . . 194
9.2.3 Guiding and Validating Embodied Intelligence . . . 194

REFERENCES 196

APPENDIX A LIST OF PUBLICATIONS 218

APPENDIX B COPYRIGHT 221

B.1 Permission to Reuse Material Already Published 221

B.1.1 Association for Computing Machinery 221

B.1.2 International Foundation for Autonomous Agents and Multiagent Systems 221

B.1.3 Institute of Electrical and Electronics Engineers 221

B.2 Permission from Co-Authors 221

LIST OF FIGURES

Figure 1.1 Crashed Drones . . . 3

Figure 1.2 Contributions of Thesis . . . 6

Figure 3.1 Energy-Sharing Problem Setup . . . 21

Figure 3.2 Speed-to-Energy Graph . . . 24

Figure 3.3 Speed-to-Distance Graph . . . 24

Figure 3.4 Fixed-Start MT-OTM Algorithm Diagram . . . 28

Figure 3.5 General MT-OTM Algorithm Diagram . . . 36

Figure 3.6 MILP Sample Output for MT-OTM Problem . . . 40

Figure 3.7 k-IP Sample Output for MT-OTM Problem . . . 40

Figure 3.8 Fixed-Start MT-OTM Results – Heuristics . . . 41

Figure 3.9 Fixed-Start MT-OTM Results – Optimal . . . 42

Figure 3.10 Dynamic MT-OTM Results – Heuristics . . . 43

Figure 3.11 Fixed-Start MT-OTM Outputs on Sinusoids . . . 44

Figure 3.12 Dynamic MT-OTM Outputs on Sinusoids . . . 45

Figure 3.13 Dynamic MT-OTM Results – Sinusoidal . . . 45

Figure 3.14 MT-OTM Speed Scheduling Results . . . 46

Figure 3.15 MT-OTM Case Study in Urban Environment . . . 50

Figure 3.16 MT-OTM Field Prototype . . . 51

Figure 4.1 Examples of PoI Latency . . . 61

Figure 4.2 Examples of Penalty Accumulation . . . 61

Figure 4.3 LORS Algorithm Example . . . 65

Figure 4.4 Square and Action-to-Action Constraint Example . . . 71

Figure 4.5 Impact of Increasing PoI on PAR . . . 75

Figure 4.6 Impact of Increasing Obstacles on PAR . . . 76

Figure 4.7 Impact of Increasing Drones on PAR . . . 77

Figure 4.8 Impact of Increasing Teams on PAR . . . 77
Figure 4.9 Failure Rate on the *IncN* and *IncO* Data Sets . . . 78
Figure 4.10 Failure Rate on the $IncM_a$ and $IncM_g$ Data Sets . . . 78
Figure 4.11 Computation Time on the *IncN* and *IncO* Data Sets . . . 79
Figure 4.12 Computation Time on the $IncM_a$ and $IncM_g$ Data Sets . . . 79
Figure 4.13 Output Path Example for Case Study . . . 82
Figure 4.14 Drone-UGV Prototype Setup . . . 83
Figure 4.15 Drone and UGV Paths from Prototype . . . 83
Figure 5.1 Forest Fire Case Study . . . 85
Figure 5.2 Balanced Matching Example . . . 93
Figure 5.3 Matching Example fo M+ILS Algorithm . . . 96
Figure 5.4 Impact of Increasing Drones . . . 101
Figure 5.5 Impact of Increasing Drones at Scale . . . 102
Figure 5.6 Impact of Floating Agents . . . 103
Figure 5.7 Packet Count and Runtime . . . 104
Figure 5.8 Raspberry Pi Testbed . . . 106
Figure 6.1 Communication Map Example . . . 116
Figure 6.2 Hardness Proof Diagram . . . 122
Figure 6.3 Jackal Robots . . . 124
Figure 6.4 Experiment Locations . . . 125
Figure 6.5 RMSE of Prediction Models I . . . 129
Figure 6.6 RMSE of Prediction Models II . . . 130
Figure 6.7 Connectivity Results . . . 131
Figure 6.8 LQCMs From the Field I . . . 132
Figure 6.9 LQCMs From the Field II . . . 133
Figure 6.10 Environmental Monitoring Application . . . 135
Figure 6.11 Data Streaming . . . 137

Figure 7.1 Data-Muling Scenario . . . 141
Figure 7.2 Overview of Framework . . . 147
Figure 7.3 Hardware for Field Experiments . . . 156
Figure 7.4 Determining Data Transfer Rates . . . 157
Figure 7.5 Optimal Distances to Collect Data . . . 158
Figure 7.6 Average Latency Compared to Fest-Found Solution . . . 159
Figure 7.7 Average Latency Compared to Optimal . . . 161
Figure 7.8 Average Latency with Increasing Sensors . . . 161
Figure 7.9 Average Latency with Increasing Data Quantity . . . 162
Figure 7.10 Field Experiment Results . . . 164
Figure 8.1 Proposed Error-Aware Tasking Framework . . . 176
Figure 8.2 Field Experiment Results For Energy Consumption Rate . . . 185
Figure 8.3 Field Experiment Results For Data Transmission Rate . . . 186
Figure 8.4 Multi-Drone Site Inspection Case Study . . . 186
Figure 8.5 Multi-Drone Site Inspection Results I . . . 187
Figure 8.6 Multi-Drone Site Inspection Results II . . . 187
Figure 8.7 Drone-UGV collaboration case studies. Triangles depict the waiting area for the UGV for each sector. . . . 188
Figure 8.8 Drone-UGV collaboration case study results. . . . 189
Figure 8.9 Drone-UGV collaboration case study results. . . . 189
Figure B.1 Copyright Permission From Yee Shen Teoh . . . 222
Figure B.2 Copyright Permission From Robert Byers . . . 223

LIST OF TABLES

Table 3.1 Symbols Used in MILP 20
Table 3.2 MINLP Formulation Symbols 31
Table 3.3 Average Computation Time 47
Table 4.1 Decision Variables for Optimizing Launch and Land Actions 69
Table 5.1 Symbols 90
Table 5.2 Drone Specifications 99
Table 5.3 Computation Time (Seconds) – Exact Methods 103
Table 5.4 Computation Time (Seconds) – Heuristics-Based (Centralized) 104
Table 5.5 Computation Time (seconds) on Testbed 106
Table 5.6 Found solution over best found solution on Testbed 106
Table 6.1 Motivating Results From Previous Study 111
Table 6.2 Prediction Accuracy at *Mock-Town* and *Courtyard* 127
Table 6.3 Prediction Accuracy at *Camp* and *Office* 128
Table 6.4 Computation Times 134
Table 6.5 Transmission Times for Environmental Monitoring Application 135
Table 7.1 Sets for Mixed Integer Non-Linear Program 149
Table 7.2 Decision Variables for Mixed Integer Non-Linear Program 149
Table 7.3 Average Computation Times 163
Table 8.1 Sets for Multi-Drone Site Inspection 177
Table 8.2 Decision Variables for Multi-Drone Site Inspection 177
Table 8.3 Sets for Drone-UGV Collaboration 180
Table 8.4 Decision Variables for Drone-UGV Collaboration 180

ACKNOWLEDGMENTS

I would like to thank and acknowledge my advisor, Dr. Qi Han, for all of her support and guidance during this journey. She gave me the freedom to pursue research topics that I found interesting and provided unwavering support in my academics. She also allowed me to invest time into supporting the CS and Robotics graduate communities and to teach CSCI 128, and I am very grateful for that. I feel honored and lucky to have had the opportunity to be her advisee.

I would like to thank my wife, Joaquina Lidiane Candido Diller, for all of her support throughout this journey. She not only encouraged me but held me accountable for my studies. I feel very lucky to have had her by my side. *Te amo.*

I would like to thank my parents for all their support, both during my time in graduate school and throughout life. I would also like to thank my entire family, starting with my brothers: Joshua, Jason, Joseph, Joel, and Jesse. All of you have helped me arrive at this point in various ways, and I feel very fortunate to call you all my brothers.

I would like to thank everyone who has contributed to my research in some form at the Pervasive Computing Systems Group. A large part of this work was possible thanks to your help. I would also like to acknowledge and thank all of my external collaborators. You have had an impact on shaping me as a researcher, and I am grateful for the opportunity to work with you. In no particular order, this includes: Shneka Muthu Kumara Swamy, Robert Byers, Matthew Desaulniers, Yee Shen Teoh, Chenyang Wang, Jack Sigler, Cody Fellinge, Matthew Hatch, Peter Hall, Ava Moon, Corey Schanker, Kristen Ung, Philip Belous, John G. Rogers, Neil T. Dantam, James Dotterweich, and James Humann.

I would like to thank everyone I served with in the U.S. Marine Corps. All of you have helped me develop into the person I am today. I do not think I would have ever accomplished this feat had I not joined the Marines, and my brothers and sisters that served alongside me are what made the experience so meaningful. *Semper Fi!*

I would like to thank Nora Frick for teaching me how to read.

Finally, I would like to thank the committee. You all have given me a lot of really helpful feedback and helped shape this body of work.

CHAPTER 1
INTRODUCTION

Heterogeneous robot teams have the potential to automate tasks, removing humans from dangerous environments or simply taking over mundane actions that humans dislike completing. For example, drones can easily fly over rough terrain and can quickly be deployed in emergency scenarios. Drones can use onboard sensors, such as cameras, to capture information about the environment or they can connect with wirelessly-enabled sensor nodes to retrieve stored data. Common applications include disaster response, security, and monitoring for agriculture or industry, among many others [1]. Similarly, Unmanned Ground Vehicles (UGVs) can operate for extended time periods and can carry a large variety of sensors and equipment, making them ideal for automation and augmenting humans in arduous tasks.

1.1 Context and Motivation

This thesis looks at planning algorithms for enabling autonomous behavior in robots. We are generally interested in planning problems that revolve around path finding and tasking for robot teams. Examples include determining how a drone-UGV team should inspect industrial equipment or sending out a swarm of drones to capture images of a forest fire.

Common approaches to solving these types of planning problems are to represent the problem as a well-known graph theory problem and then using existing algorithms for solving these related problems. For example, suppose that first responders would like to leverage the capabilities of drones during a search and rescue mission by taking high quality images of n different locations that are of interest to the search effort. If given only a single drone, this problem can be modeled as finding a minimum-cost cycle in a fully connected graph. This is known as the Traveling Salesman Problem, or TSP, and is a widely understood topic. Yet drones are inexpensive and widely available. Deploying multiple drones would allow first responders to cover a larger area more quickly. If given m drones, the problem can be modeled as finding m minimum-cost cycles in a graph and is known as the Vehicle Routing Problem (VRP), another well studied problem.

However, robots are complex systems and there are many challenges that are unique to robots that must be considered when determining how to effectively deploy robot teams for intricate tasks. Drones, like most mobile robots, have limited onboard energy storage and can only operate for a limited period of time before needing to land and recharge or replace their batteries. Failing to accurately model how a drone consumes energy in our path finding algorithm puts our drones at risk of running out of energy, causing them to plummet to the ground and destroy hardware. This limitation means that we cannot simply put

our path finding problem into a classical VRP algorithm and expect to get a solution that respects the limitations of our vehicle's capabilities.

Wireless communication is another key factor that helps facilitate both operation and collaboration but can complicate planning. Robot-to-robot communication is required when robot teams must adapt their behavior during deployment and is particularly crucial when exploring new environments or when information sharing is consequential to the given application. Similarly, robot-to-infrastructure communication is often needed when robots extend the reach of human operators or rely on information from external, static devices. Failing to correctly represent robot-to-robot communication in our planning process could mean robots fail to receive mission-critical information, resulting in failed missions or robot crashes.

There is often a gap between offline, mission-level planning and the actual behavior seen during robot deployment. We argue that this gap is usually caused by over-simplified models in planning algorithms for how robots operate. Using simplified characterization for system status, such as energy or communication, can certainly aide in the planning process by eliminating complex mathematical models and allowing us to more readily reduce our planning problem into some well-studied graph theory problem. However, what we have found in our own field experiments is that this approach to planning creates two major problems: (1) the solution cannot be accurately replicated on physical robots and (2) the quality of the solution is poor.

(a) (b) (c)

Figure 1.1 Drone swarm from [2] (a) and several drones that crashed after running out of energy during deployment (b and c).

For example, in [2] we used a swarm of drones to imitate satellite orbits and used a time-based energy constraint in our orbit planner. As shown in Fig.1.1, we constructed five drones for this research but routinely had drones sporadically running out of energy and crashing, leaving us with only three functional drones for our experiments. We have seen this same issue when modeling communication. Our solution to avoiding malicious jamming in [3] assumed that wireless robot-to-robot communication could be accurately modeled by estimating signal strength based on distance and line of site, then utilizing this estimation in a planning algorithm. However, when we actually deployed our proposed solution on physical hardware we

found that there was very little correlation between network performance and the metrics used in our communication model.

1.2 Our Research Question

Mobile robots are complex conglomerates of multiple sub-systems. These various sub-systems will interact differently, and at times non-deterministically, when deployed in the real world. Misrepresenting these complex behaviors can lead to failing to meet application objectives in the best case and destructive robot crashes in the worst case. Not only do planning algorithms need accurate models for representing how robots operate in their environment and interact with one another, but they need to integrate the characteristics of these models into the planning process.

This leads us to ask a similar set of questions for any system and application when considering how to effectively, and safely, deploying teams of mobile robots. In a more general sense, this thesis seeks to address the question:

This leads us to the general research question that this thesis seeks to address:

> *How do we accurately model robot behavior and how do we incorporate these models into the decision-making process for robot teams?*

1.3 Reflective Planning

Our stated research question is quite broad and the answer will depend greatly on the application and the robots used. Rather than proposing a single solution, we recommend the philosophy of *Reflective Planning*. This is a design philosophy for how to approach the various stages of modeling robots, integrating these models into algorithms, and validating solutions that can be reapplied to different multi-robot applications. It seeks to make robots self-aware to how the robot's actions impact the status of its own systems, how the environment impacts the actions of the robot, and how the actions of individual robots impact their ability to interact with each other in teams.

Poor representations of how robots operate may suffice in simple cases, but demanding applications require a deeper understanding of platform limits. For example, consider a forest fire monitoring application with drone teams: a simplified model of vehicle endurance may be sufficient for short missions near the operator, but rapidly shifting fire conditions demand repeated deployments over growing areas. As the need to gathering information quickly grows, first responders will push their drones to the edge of their capabilities. In such cases, relying on crude models not only limits usefulness but risks catastrophic vehicle loss.

In the *Reflective Planning* approach, we utilize results from preliminary experiments to refine how we model robot systems. We then investigate the impact that these models have on specific multi-robot applications, with a particular interest in identifying tradeoffs in our decision-making process and how these tradeoffs impact application-level performance. Using these insights, we design algorithms that exploit these tradeoffs to extend the operational capabilities of our robot teams. We then deploy our proposed solution back on our robots, using physical hardware when possible, to validate our theoretical results and identify gaps in our solution. These validation experiments often lead us to new, open research questions that can be solved by reapplying our *Reflective Planning* philosophy.

The rest of this thesis demonstrates how *Reflective Planning* not only improves solution quality in a multitude of multi-robot applications but also allows robots to safely extend the reach of their capabilities.

1.4 Scope

Within the larger scope of modeling robots in planning algorithms, we are particularly interested in energy and communication. We consider various sub-questions within the scope of modeling energy, such as how modulating a drone's speed impacts performance, how to plan for energy transfer from other vehicles, or how should the stochastic nature of energy storage and consumption inform planning decisions. For communication, there is an ongoing debate on how to best model communication for various multi-robot applications involving both robot-to-robot and robot-to-infrastructure links. Other open questions include finding a balance between improve communication quality versus the effort required to make this improvement or how to account for the stochastic nature of wireless communication in tasking.

Additionally, there exists an interesting relationship between communication and energy, particularly with drones. Hovering to transmit data is energy expensive and we should expect a poor communication link to have a larger impact on how we coordinate the actions of these energy-limited platforms.

We acknowledge that there are many other topics that fall within the larger scope of modeling robot behavior in tasking algorithms that are not considered in this thesis. Examples include mobility, perception, computation, and data storage. These various facets of mobile robots are expected to have an impact in how mobile robots operate in the environment and should also be incorporated into the decision-making process, but are not directly considered here and left for future work.

1.5 Research Methodology

Our research methodology is designed to bridge the gap between theory and practice in robotics. We begin by seeking formal mathematical formulations of the problems we study, enabling the use of advanced solvers to compute optimal solutions for novel inputs. Because these solvers are often intractable in

real-world applications, we also develop heuristic-based algorithms that yield near-optimal solutions in polynomial time. We rigorously validate our algorithms through both numerical simulations and theoretical analysis – such as characterizing time complexity and formal proofs of performance guarantees. A key focus of our work is to understand the tradeoffs that arise when incorporating high-fidelity models of robot systems into our problem formulation.

Given our emphasis on applying theoretical insights to physical systems, we prototype our algorithms in field experiments and compare empirical results against those from simulation. Whenever we are not able to deploy our solutions on physical hardware, we use a data-driven approach to create simulated environments that closely imitate the behavior we would expect to see on real robots. This end-to-end approach supports our design philosophy of *Reflective Planning*.

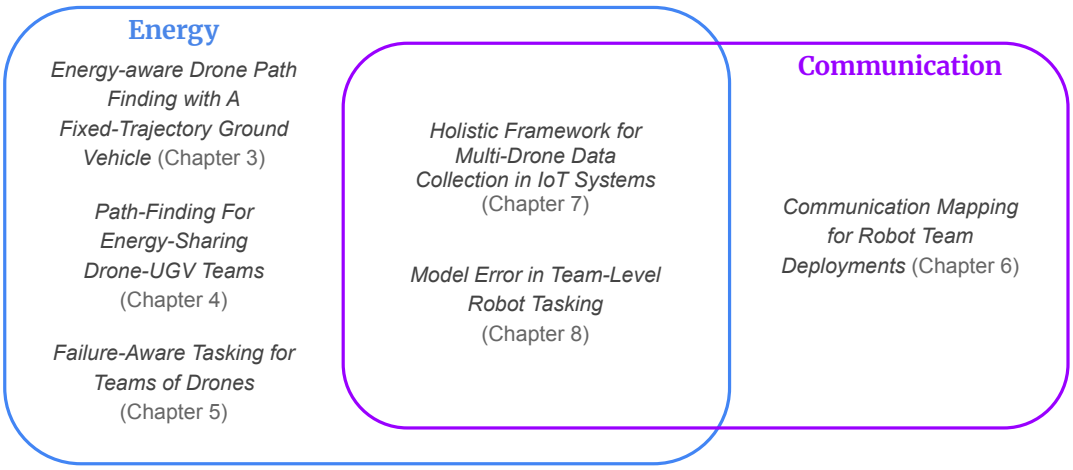


Figure 1.2 Contributions of this thesis.

1.6 Contributions of this Thesis

Below are the main contributions of this dissertation research. Figure 1.2 shows how the various chapters of this thesis relate back to modeling energy and communication.

Energy-Aware Planning: We take a deeper look at how drones consume energy. Recent works have found that drone energy consumption depends on the speed of the drone [4, 5]. From this theoretical model of energy consumption, we identify a major trade-off between maximizing the distance that a drone can travel versus maximizing the speed that the drone travels at. To investigate how this trade-off impacts application performance, we apply it to an information gathering application where a drone is launched from a moving ground vehicle that moves along a fixed trajectory, which we show to be NP-Hard. To exploit the distance-to-speed trade-off in our considered application, we integrate speed scheduling into

both optimal and heuristics-based path finding algorithms. Our results show that incorporating speed scheduling into planning can improve solution quality by up to 47% and finds solutions for over 200% more problem inputs when compared to using fixed speeds. This line of work is presented in detail in Chapter 3.

Planning for Energy Sharing: We investigate how UGVs can transfer energy to drones where, we review a model for energy transfer and integrate this model into planning actions for drone-UGV teams. We consider an infrastructure monitoring problem over an indefinite time horizon, where UGVs transfer energy to and ferry around hitchhiking drones. We present a new algorithm that incorporates a Second-Order Cone program to find locally optimal patrolling solutions in polynomial time. We demonstrate how to incorporate ground obstacles into our optimization formulation while maintaining the benefits of convexity. We show that our proposed algorithm achieves more than a 37% improvement in solution quality when compared to existing solutions. We also present a novel metric for evaluating performance in indefinite patrolling problems and propose a new taxonomy for classifying drone-UGV team monitoring problems. The details of this work are found Chapter 4.

Planning for Energy-Related Failures: We look at how to assign a probability to a drone failing to complete a task due to miss calculated energy capacity. We apply this probability model to a forest fire monitoring scenario, where drones must work together to complete various monitoring tasks, and investigate a new class of task assignment problems where the objective function is a product of Poisson-Binomial trials. We show how to determine if a solution exists to this problem and explain how to find an upper bound on solution quality. We then use these two pieces together to find optimal solutions in a customized branch-and-bound algorithm. We also present a decentralized algorithm that quickly finds near-optimal solutions without relying on a centralized decision maker. This line of research is present in Chapter 5.

Communication-Aware Planning: Decades of research within the networking community has led to various approaches for representing communication at the application level [6]. However, the robotics community has not followed this same trend and instead continues to use agnostic methods for representing communication in robot planning. We present a new method for mapping the potential for robot-to-robot and robot-to-infrastructure communication and propose a method for predicting communication link quality. We prove that our communication map can be used to determine the optimal robot configurations to maximize network throughput between any two arbitrary positions in the environment. We show how this map can be integrated in to planning actions for various robotics applications, such as environmental monitoring or exploration, with a strong emphasis on demonstrating that our proposed system works in the field. The details of this work are found in Chapter 6.

Holistic Approach for Combining Communication and Energy in Planning: To address the gap between offline planning and online behavior, we combine our work on both energy and communication into a holistic planning framework for multi-drone data collection applications. Our experimental results show that our holistic framework is able to find a balance between idealistic, offline planning and allowing enough energy for drones to adapt their behavior during deployment without crashing. Using a data-driven approach to modeling energy and communication, our framework was able to reduce the error between offline planning and online behavior from 45% (found in [7]) to only 7.5%. Additionally, we show that there is a trade-off between increasing the data transfer rate versus the time spent moving closer to a wireless sensor. Integrating this tradeoff into planning improves solution quality when compared to current planning approaches. This full body of work is found in Chapter 7.

Bounding Allowable Model Error and Adapting to External Disturbances: We define model error, as a result of both external disturbances and the inherent nature of stochastic systems, for both energy and communication. Using a user specified risk tolerance, we show how to find an upper bound on allowable model error without breaking a given robot-to-task assignment. We integrate these bounds on error into a team-level tasking framework that uses a pull-based design to enable robots to adapt tasking decisions online in response to changing environments. Our simulation results demonstrate the impact that model error has on solution quality in various multi-robot tasking scenarios. Furthermore, we show that our proposed tasking framework can improve application-level performance by up to 38% when compared to a baseline approach. More details on this work can be found in Chapter 8.

CHAPTER 2
LITERATURE REVIEW

In this chapter we review recent literature on planning actions for robot teams. We first look at various path finding algorithms often employed for various applications, we then look at algorithms for handling tasking. We then discuss how these problems fall into the larger field of multi-agent planning. These applications and algorithms usually rely on system models for constraining behavior, which we also investigate.

There is a large variety of considered problem setups and applications for robot teams. We focus on just a handful of problem scenarios relevant to this thesis but acknowledge that this is not an exhaustive list. All of these problems can involve drone-only teams, UGV-only teams, and mixed drone-UGV teams. We will not differentiate between these different scenarios here, but later chapters will focus on particular problem setups that are tailored to specific applications.

2.1 Multi-Robot Routing Problems

One of the most generalized problem setups is sending out robots to visit Points of Interest (PoI). In the general PoI problem, we are given n PoI and m robots and are asked to find efficient paths for each robot such that each PoI is visited while minimizing some objective [8]. Common applications for this problem setup include agricultural monitoring [9], surveillance and security [10], environmental monitoring [11], forest fire monitoring [12], and search-and-rescue [13]. The specific application often dictates whether the robot is required to stop at the PoI and perform some task or simply pass by the PoI, treating it as a waypoint. These problems are a generalization of coverage problems, where robots must move in sweeping patterns so that their footprint covers the entirety of some given area [14]. Common metrics for visiting PoI include minimizing the time it takes for the last PoI to be visited (minimizing the maximum) [15], minimizing robot travel distance [16], minimize robot energy spent (which is often treated as minimizing distance), or minimizing some application-specific penalty [17], among others.

A closely related problem is using robots to collect data from static sensors. This is commonly referred to as data-muling [18]. In the general data muling problem, we are given n sensors and m robots and must find efficient routes such that the data from every sensor is collected by one of the robots while minimizing some objective. One major difference between the data-muling problem and the PoI problem is that the latter requires the robots to visit a discrete point while the former usually (although not always) relaxes this requirement so that the robot only needs to travel close enough to communicate with a sensor (i.e. visiting any location within a continuous space versus visiting a specific, discrete point). Common

applications normally fall within monitoring scenarios, such as infrastructure monitoring [19] and environment monitoring [20]. Minimizing the maximum data latency is one of the most common metrics applied to the class of problem [21, 22]. Other metrics include maximizing the volume of data collected [23] and minimizing energy spent to collect data [24].

All of these problems are variations of well-studied graph theory problems and in general are all considered hard to solve to optimality. For example, the PoI problem is a variation of the Vehicle Routing Problem (VRP), a well-known NP-Hard problem [25] that can be defined using the same definition as before. The data-muling problem can be modeled as a variation of the Orienteering Problem – another known NP-Hard problem where we are given sets of waypoints and must visit at least one waypoint in each set [26] – or by treating the problem as yet another variation of the VRP.

The most common method for finding optimal solutions to these problem setups is to formulate some variation of an Integer Program (IP). Integer Programs are very generalizable and can be used to solve these various routing problems while also considering various constraints, such as limited robot energy or communication. For example, see [27] for the PoI variation and [28] for data muling. We usually employ powerful commercial solvers such as Gurobi [29] to compute solutions for IP. However, solving IPs to optimality is considered hard in the general case [30]. These formulations can be used to benchmark other approaches at small problem sizes but do not scale well with the problem input (in our case, the size of n and m), making them impractical for quickly finding solutions to routing problems for robot teams.

There have been several proposed heuristics-based algorithms for multi-robot routing problems. Most of these algorithms break down the problem into smaller pieces and then solve these individual parts. The most common example of this is finding subsets (or clusters) of PoI or sensors, and then treating these clusters as an instance of the TSP. Although TSP is itself an NP-Hard problem [30], there is extensive work on heuristics-based algorithms for solving the TSP, such as the Lin-Kernighan-Helsgaun heuristic [31], genetic algorithms [32], or ant colony optimization [33]. Therefore, practitioners usually search for methods for breaking down their considered routing problem into the TSP and then leverage these existing algorithms to solve individual instances of the TSP. There are also more localized approaches for solving multi-robot routing problems, such as iteratively building up a solution [34], evaluating nearest-neighbors [35], or focusing on a shortened time horizon [36].

It is worth noting that there exist approximation algorithms to solve many of the underlying routing problems, such as the TSP and VRP. Christofides algorithm [37] guarantees a solution that is within a $\frac{3}{2}$ of the optimal solution and runs in polynomial time. Similarly, there is ongoing research in finding approximation algorithms for various forms of the VRP [38]. Although these algorithms could be used to solve derived variations of multi-robot routing problems, heuristics-based solutions tend to outperform

these approaches [31, 39]. Furthermore, these algorithms tend to use simplifying assumptions about systems that allow them to make such guarantees. Incorporating accurate system models (such as energy consumption or energy transfer) into the planning process can break these assumptions and complicate maintaining guarantees. For example, we can make solution guarantees for the Capacitated VRP – where vehicles have a limited capacity – if we assume that we can assign a constant cost for a vehicle for visiting a PoI [40]. However, how much energy a robot consumes depends on the robot's speed and the cost for visiting a PoI will no longer be constant if part of our considered problem is to select what speed the robot should travel at.

2.2 Multi-Robot Task Allocation Problems

A related topic to multi-robot routing problems is the Multi-Robot Task Allocation (MRTA) problem [41]. In the most general form of the MRTA problem, we are given n tasks and m robots and must find robot-to-task assignments such that we optimize some objective. The MRTA problem can be thought of as a variation of the PoI routing problem where each point requires the robot to spend a non-negligible period of time performing some task and is applicable to many of the same applications and metrics discussed before. Popular applications of the MRTA problem include responding to natural disasters [42] and national defense [43]. The considered metrics are often determined by the motivating application and therefore can be wide-ranging. Some examples include robot energy [44], task rewards [45], or the probability of completing tasks [46].

The chosen algorithm for the MRTA problem depends greatly on the problem setup and objective. When the number of tasks is equal to the number of robots ($n = m$) and the objective can be expressed as a summation of robot-to-task rewards or costs, then this is a variation of the linear assignment problem and can be solved to optimality using the Hungarian method (Kuhn-Munkres algorithm) in polynomial time [47]. If robots are only required to complete a single task and we can accept that some tasks will not be completed, then the unbalanced version of this problem ($n \geq m$) can be reduced to the balanced version and solved using the same approach [48]. However, the general case of the MRTA problem, where $n \geq m$ and robots can complete more than a single task, has been shown to be NP-hard [49] and is often itself a generalization of the machine shop scheduling problem – an extension of the TSP and well-known NP-hard problem.

Similar to routing problems, there are both optimal and heuristics-based solutions to these MRTA problems. Formulating an IP can again be used to integrate systems constraints into the considered problem and provide an optimal solution [44] but normally does not scale with the problem size. Most of the computationally-efficient heuristics-based approaches will start with a greedy solution and then run

some type of local search to improve solution quality [50]. Market-based strategies such as auctions [51, 52] are also common for these types of problems.

2.3 Multi-Agent Planning

Both multi-robot routing and task allocation fall into the more general scope of multi-agent planning. Multi-agent planning looks at how to enable agents to independently make decisions while interacting with other agents. In this context, an "agent" is broadly defined as any entity that can perceive its environment and interact with the environment through actions – which includes but is not limited to robots. Beyond robot routing and tasking, there are several other problem types that fall within the larger scope of multi-agent planning.

One popular problem in this space is the Multi-Agent Path Finding (MAPF) Problem. There are many variations to the MAPF problem, but it can be generalized as follows: given m agents, each with a start location and a goal location, find collision free paths for each agent such that we minimize some metric, such as distance or time [53]. This problem setup is often seen in grid worlds, with applications in warehouse logistics [54]. Other applications include directing autonomous cars through traffic [55], moving and rearranging objects [56], and air traffic control [57]. Other problems that fall within this domain include formation control [58], swarm robotics [59], and pursuit-evasion games [60].

2.4 System Models

Robots rely on various system status that enable them to operate and collaborate. This work focuses on two major system conditions: energy and communication. How we represent these models and integrate them in the planning process will not only impact performance, but will also determine how representative our planning process is of the behavior that we see when actually deploying robot teams. We will first discuss methods for modeling energy and then look at communication.

2.4.1 Energy Models

There are a variety of methods for modeling robot energy. We will focus on energy consumption here, but later discuss energy transfer in Chapter 4 and risk of failure as it relates to energy in Chapter 5. Most of our discussion on energy models will be focused on drones, but these models tend to generalize to those of their ground-based counterparts.

In general, there are four common approaches for modeling energy on drones: time-based, distance-based, speed-based, and action-based. The time-based model discretizes time and assumes that a drone will consume some fixed quantity of energy for each unit of time that it operates [36, 61]. Similarly, the distance based method dictates that a drone can only travel a predetermined distance before running

out of energy [62]. Taking a slightly different approach, the speed-based model assumes that energy consumption rate is determined by the speed that the drone travels at, to include hovering in-place [4]. This method is based on physics [5] and has been shown to accurately represent energy consumption on physical hardware [63]. Finally, the action-based method assigns an energy cost to different actions [8]. The action-based model can usually encompass one of the other models, for example assigning different costs for hovering for a predetermined time versus traveling a known distance [9].

As previously stated, these drone energy models are generalizations of how ground robots consume energy. Field tests have shown that UGV energy also depends on speed [64], and can be modeled using the same mathematical equations.

2.4.2 Wireless Communication Models

There are generally two main models for communication in robot planning: the disk-based model and using Signal-to-Noise Ratio (SNR). The disk-based model greatly simplifies communication by assuming that two robots will have full, uninterrupted communication when within some predetermined distance apart and will not be able to communicate at all outside of this distance [65, 66]. Although this approach simplifies planning for robot-to-robot communication, it is agnostic to obstacles and interference. Alternatively, the SNR approach attempts to better account for obstruction and noise by calculating robot-to-robot signal strength. This method is usually based on the International Telecommunication Union's path loss model for peer-to-peer radio communication [67]. To apply the SNR method, planning algorithms will again assume that robots will have full bandwidth when the SNR between two robots is above some predetermined threshold [68]. In general, these models are either represented in discrete space or continuous space. The former allows us to more readily use algorithms based in graph theory for planning [22] while the latter complicates the graph representation but is more intuitive for mathematical optimization [69].

Within the context of planning, there are two general approaches to handling communication in robot system: (1) intermittent connectivity and (2) continuous connectivity. Intermittent connectivity accepts that robots will occasionally disconnect from one another during execution [70]. Alternatively, continuous connectivity requires the robots to maintain connection to one another, or to a centralized machine, through one or more communication hops [71].

However, years of networking research have found that neither of these methods accurately represent application-layer performance [72, 73]. We will address this gap in existing multi-robot planning research in Chapter 6, where we investigate how to use more advanced methods for representing peer-to-peer communication in multi-robot planning. We also propose a data-driven communication model for

representing robot-to-infrastructure communication in Chapter 7.

CHAPTER 3

ENERGY-AWARE DRONE PATH FINDING WITH A FIXED-TRAJECTORY GROUND VEHICLE

Modified from [74], as published in The ACM Journal on Autonomous Transportation Systems[1]. This manuscript extends the work in [75], presented at the 22^{nd} International Conference on Autonomous Agents and Multiagent Systems.

Jonathan Diller[2,3], Qi Han[2]

Rotary-wing Unmanned Aerial Vehicles (commonly referred to as drones) are versatile autonomous transportation platforms that can be used for a variety of data collection applications including emergency response, environmental monitoring, surveillance, and many others. In this work, we investigate how to plan efficient paths that minimize mission completion time for drone data collection where the drone must rendezvous with a moving ground vehicle (GV) that cannot stop and wait for the drone. Moreover, we address the limited onboard energy storage issue by adapting drone speed. We propose a mixed-integer nonlinear program (MINLP) solution to solve this problem to optimality and provide two heuristics-based alternative solutions (the k-TSP and D-TSP approaches) that are more computationally tractable. We evaluate these approaches in extensive simulations using real drone characteristics to highlight their trade-offs. Our results show that the k-TSP algorithm performs well when data collection points are closer to the GV, averaging within 4.5% of the optimal solution, while the D-TSP approach is more versatile, finding solutions in situations where the k-TSP algorithm tends to fail. Furthermore, we show that adapting drone speed can improve solution quality by up to 47.1% compared to fixed-speed approaches. In summary, this article serves as an exploratory study in energy-aware planning and scheduling for drones and other autonomous transportation systems.

3.1 Introduction

Rotary-wing Unmanned Aerial Vehicles (i.e., drones) can easily move over rough terrain and water, are low-cost, commercially available, and can be deployed quickly. This makes them well-suited for various data collection applications such as emergency response, environmental monitoring, agriculture, and surveillance. Studying how to deploy drones will better enable them as autonomous systems and facilitate their adoption into society.

[1]Reprinted, with permission, from Jonathan Diller, Energy-Aware Drone Path Finding with a Fixed-Trajectory Ground Vehicle, Journal on Autonomous Transportation Systems, 02/2025
[2]Department of Computer Science, Colorado School of Mines, USA
[3]Primary researcher and author

A common requirement in these applications is minimizing mission completion time. Intuitively, this implies that we want to find an optimal path for a drone to follow while maximizing the drone's speed. However, drones have limited onboard energy storage and there is a trade-off between speed and energy consumption [5, 63], where maximizing speed does not maximize travel distance due to a higher energy consumption rate (which is further discussed in Section 3.3.1). We see a lack of existing research on how to integrate this tradeoff into planning algorithms. Any robust path planning approach should consider the speed-energy consumption trade-off and plan for recharging or battery swaps.

In many applications, drones often work with GVs which provide battery recharging, battery swap, data offloading, etc. Collaborative drone-GV problems have already been well studied [8, 9, 76]. However, these existing works assume that the GV can stop and wait for the drone to return. While a drone completes its given tasks, the GV deploying the drone may need to continue moving rather than wait at the starting location for the drone to return. One example of this is a large ship on a maritime search and rescue mission, such as those recently seen in the Mediterranean sea where migrants frequently become stranded on failing vessels [77]. The drone can be deployed to inspect minor areas of interest while the main search and rescue ship continues along a preplanned trajectory. Other examples include drones used in urban environments for package delivery or to assist first responders while relying on volunteer vehicles for ride hitching [78]. In such applications, drone path planning algorithms must account for the movement of a GV that is following a preplanned trajectory.

In this work, we seek to answer the research question: *How do we plan energy-aware drone paths and rendezvouses with a non-stopping GV for general data collection applications?* To this end, we consider the trade-offs between drone speed and total travel distance and propose various speed-adapting algorithms. This article expands on our previously published conference paper [75]. For completeness and context, major parts of our previous publication are included in this manuscript. We expand on the previous publication in the following ways.

1. We remove the assumptions made in our previous work that the drone will be launched at mission start time ($t = 0$) and that subsequent launches happen immediately after swapping batteries on the drone (found in Section 3.6). Selecting a launch time is now part of the considered problem.

2. We provide a complete solution to the problem as a whole (in Section 3.6.1) and prove that the problem is NP-Hard (in Section 3.3). Our previous publication only provided a complete solution for an underlying Hamiltonian Paths Problem found when fixing the start and stop time of each drone sub-tour and commented on the hardness of the underlying problem.

3. We expand our literature review to include more recent related work on this topic (in Section 3.2).

4. We expand on the simulation evaluations. In addition to more randomly generated inputs (presented in Fig. 3.9, 3.10, and 3.14(b)), we also evaluate our solution on non-linear GV trajectories (presented in Fig. 3.13).

In the next section, we discuss related works and how they compare to our body of research. In Section 3.3 we formally define the considered problem, termed Minimum-Time while On-the-Move (MT-OTM), then discuss our drone energy model in Section 3.3.1. In Section 3.4 we present our solution framework for the MT-OTM Problem and then further discuss details of our various proposed algorithms in Sections 3.5 and 3.6. In Section 3.7 we provide simulation results and discuss the details of our simulation setup while Section 3.8 details our field test prototype of the MT-OTM Problem using a physical drone. Finally, we discuss the conclusions of our study and provide closing remarks in Section 3.9.

3.2 Related Work

In this section we review recent works in literature with a focus on drone path planning algorithms for mixed drone and GV problems. We also review drone energy models and how they have been applied in drone path finding. For clarity, we refer to unmanned variations of GVs as Unmanned Ground Vehicles (UGVs). The term "GV" may include both manned and unmanned vehicles.

3.2.1 Multi-Drone Path Finding

Outside of coordinated drone-GV teams, there has been extensive research on path finding for drone swarms with common proposed applications in data collection, package delivery, and supporting wireless infrastructure [79–82]. These path finding problems are usually presented as variations of the Vehicle Routing Problem (VRP) or the Multi-Traveling Salesman Problem (MTSP) [27, 83, 84]. In the MTSP, we are given a set of salesmen that are deployed from the same depot and a set of stops that must be visited by at least one salesman and are ask to find efficient paths for the salesmen such that all stops are visited [85]. The VRP extends the MTSP by allowing vehicles to start from multiple depots, with variations including capacity constraints, time windows, or pick-ups and deliveries, just to name a few [86].

Common solution approaches include both exact methods and heuristics-based algorithms. Math optimization approaches (such as mixed integer programming) are largely popular for exact methods [27, 83, 84, 87]. Heuristics-based solutions commonly simplify the problem into smaller graph search problems [22, 36] or simplify the problem into the classical Traveling Salesman Problem (TSP) and utilize well-studied heuristics-based solutions, such as the Lin-Kernighan-Helsgaun heuristic [88, 89], genetic algorithms [32], or ant colony optimization [33].

3.2.2 Mixed Drone & Ground Vehicle Path Finding

Many previous works look at cooperative problems involving drones and GVs where a series of waypoints must be visited. In [9] they consider an application where either a drone or a UGV must be within some distance of a set of waypoints, which is modeled as an orienteering problem, a problem with known algorithms. In [90] they consider drone path planning where a GV can be used to swap-out batteries on the drone but is constrained to a network of streets. They propose treating all drone waypoints as a single TSP, then breaking up the route into sub-tours. This work was further expanded in [91, 92] where they propose a mixed-integer linear programming (MILP) solution. A related scenario is considered in [8, 93], where a UGV can ferry around the drone. The authors model the problem as a Generalized TSP and apply a known solution approach [94]. A similar problem is found in [95], where they first plan GV routes along a road network then plan drone routes using Conflict-Based Search. A similar problem was looked at in [16] and extended in [96], where multiple drones are deployed with a single UGV. The drones must visit mission waypoints while the UGV provides recharging for the drone but is not restricted to a road network. A further extension is found in [97], where the authors use convex optimization to select rendezvous points. They solve this problem by first using a clustering algorithm and a TSP solver to find a path for the UGV then plan drone paths with a set of fixed recharging locations. An iterative solution to this problem setup is found in [98] while [99] looks at how to represent the problem as an agent-based model with behavior trees and finite state machines. A common theme in all of these works is that the GV can stop and wait for the drone to finish flying a sub-tour of waypoints, which often allows the problem to be modeled as a traditional graph theory problem and solved using known techniques. In contrast, our work considers how to plan drone paths when the GV cannot stop and wait for the drone. This means that we must consider the trajectory of the GV as part of the problem.

A related set of drone and GV problems is the drone parcel delivery problem, where a delivery truck, bound to a network of streets, acts as a launching point for a drone to deliver a package in a last-mile delivery system [100]. These scenarios can largely be broken into two categories: (1) where the drones can only make a single delivery stop per sortie, as seen in [101–104], and (2) where the drones are allowed to deliver multiple packages per sortie [84, 105–107]. These problems are usually solved by breaking down the larger problem into smaller ones that can be handled using math programming techniques [76]. Most of the literature on this set of problems allows for one of the vehicles to stop and wait for the other. However, in applications over water or rugged terrain, the drone cannot land to wait for the GV and hovering is energy intensive.

Another set of related problems looks at selecting rendezvous locations for a drone to be recharged by a GV. In [108], the authors plan where a GV should meet up with a drone on a fixed route by converting the problem into the Generalized TSP and solve it using both integer programming and the Lin-Kernighan-Helsgaun (LKH) heuristic [109]. In [110] both the drone and UGV have established paths and must select rendezvous points that are ideal for both vehicles, which is done using a Markov Decision Process. The work in [110] was further extended in [111] to include multiple UAVs and multiple UGVs. Rather than looking at the trade-off between varying drone speeds, these works looked at the risk of running out of battery and used this metric to determine how often the drone should recharge with the UGV. Their problem setup assumes that the paths for both the UGV and UAV are already determined and is similar to the baseline approach in our work. Although our work also selects rendezvous locations, our scenario differs from the current literature because we consider planning rendezvouses with a non-stopping GV and fit this problem into a larger, combined rendezvous and path planning problem.

For this work, we assume that the drone is capable of landing on the GV after rendezvousing using techniques such as the ones discussed in [112, 113]. Landing on a moving vehicle falls out of the scope of this research.

3.2.3 Drone Energy Models

Drones have limited onboard energy, motivating the need to model and integrate drone energy consumption into planning algorithms. There have been many proposed methods for modeling energy consumption in drones including: the distance-based method [62, 90], the time-based method [36, 61], discretized approaches [7–9], and the velocity-based method [4]. The distance-based model assigns a max travel distance and ignores or excludes time spend hovering. The time-based method assigns a total operating time to the drone and usually does not differentiate between time spent hovering versus time spent traveling. Discretized approaches allot an energy budget to the drone and reduce the budget based on actions, such as flying a certain distance or hovering for some period of time. However, the authors in [101] found that these energy modeling approaches are not as accurate as the velocity-based approach. The velocity-based model maps the drone's speed to power consumption based on characteristics specific to the drone. This model was formulated separately in [5] and [4] and validated in [4, 63] through field testing on physical testbeds. Due to its accuracy, we use the velocity-based model in this work.

The velocity-based energy model has been used in various drone path planning problems. In [101], it was applied to the drone parcel delivery problem and in [110] it was used to better model the risk of the drone running out of energy in a rendezvousing problem. In [102], they used this energy model to adapt drone speed as a post-processing step to further improve mission completion time. However, mission

completion time is the product of both the drone's speed and the distance that the drone must travel. To directly optimize completion time the energy model should be embedded in the solution formulation. In our work, we add speed adaptation using the velocity-based energy model directly in our mathematical formulation and evaluate speed adaptation against fixed-speed approaches for a multi-waypoint drone problem.

Table 3.1 Symbols used in this chapter

Symbol	*Description*
$\mathbf{\Delta}$	Set of m sub-tours
δ_k	Sub-tour k, an ordered set of waypoints
d_k	Distance of sub-tour k
Γ	Set of m start-end time tuples
P	Set of waypoints to visit
p_k^d	Depot waypoint for sub-tour k
p_k^t	Terminal waypoint for sub-tour k
Φ	Complete drone tour with m sub-tours
S^m	Set of speeds for sub-tour m
t_b	Time required to swap batteries between sub-tours
τ_k^d	Start time for sub-tour k
τ_k^t	End time for sub-tour k
$\mathcal{X}$	Application space (assumed to be in $\mathcal{R}^2$)

3.3 Problem Formulation

In this section, we describe the system setup and formally define the MT-OTM problem. In general, in the MT-OTM Problem, we are given a drone that must visit a series of waypoints while being launched from and received by a moving GV. The drone has limited onboard energy and may need to stop several times on the GV to swap out its batteries or recharge. Our objective is to minimize the time it takes for the drone to visit every waypoint and return to the GV. We focus on the offline, mission planning problem of identifying efficient drone paths for visiting every waypoint in minimal time. Online control for the drone as well as communication and coordination with the GV, although ongoing areas of research, are assumed to function fully as intended and fall out of scope of this work.

Table 3.1 includes a list of symbols used in our problem formulation. Our problem formulation considers drone energy and battery storage, which is further discussed in detail in Section 3.3.1. We acknowledge but leave out various factors in our problem formulation that could affect flight performance such as wind and the energy consumed by making turns. Although we do not consider these and other minor factors that can impact energy consumption, our proposed solution is versatile and can be reapplied with a more comprehensive energy model.

Let $\mathcal{X} \in \mathcal{R}^2$ be a large area with several navigational waypoints that must be visited. A GV that acts as a moving base station moves through $\mathcal{X}$ on a predetermined, fixed route described by $p_b(t)$, a function that returns the GV's position at time t. That is, $p_b(t) \in \mathcal{X}$ for any $t \geq 0$. Without loss of generality, we assume that the origin of the two-dimensional coordinate system for $\mathcal{X}$ is at the GV's initial position at the beginning of the considered time window. Figure 3.1 shows the general problem setup.

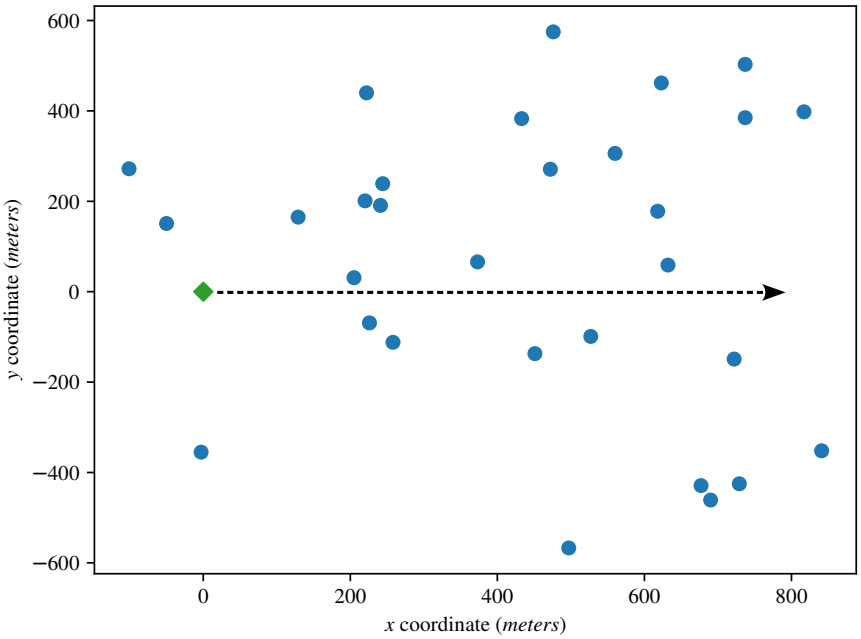


Figure 3.1 General problem setup: the blue circles are navigational waypoints that must be visited by the drone, the green diamond is the GV's starting location, and the black, dashed arrow shows the GV's fixed trajectory.

Let P be the set of all waypoints that must be visited by a drone and p_i be the i^{th} waypoint in P. Due to energy constraints, the drone may not be able to visit all of the waypoints in P in a single tour. Let m be the number of sub-tours required to visit every waypoint in P, which is initially unknown. We define the k^{th} sub-tour, denoted as δ_k with path length d_k, as an ordered set of waypoints containing a starting position in $\mathcal{X}$ (a *depot*), an ending position in $\mathcal{X}$ (a *terminal*), and at least one waypoint in P. We denote the depot of sub-tour δ_k as p_k^d and the terminal as p_k^t. Let $\mathbf{\Delta}$ be a set of m sub-tours such that every waypoint in P is visited exactly once and let the set of speeds for each sub-tour in $\mathbf{\Delta}$ be S^m. The time it takes a drone to travel δ_k while moving at speed $s_k \in S^m$ will be

$$t_\delta(\delta_k, s_k) = \frac{d_k}{s_k}. \tag{3.1}$$

We denote the start time of sub-tour k as τ_k^d and the end time of sub-tour k as τ_k^t. By definition, for any sub-tour k, $\tau_k^t = \tau_k^d + t_\delta(\delta_k, s_k)$. Suppose that it takes t_b seconds to land the drone on the GV and change out the battery between each sub-tour. The earliest that sub-tour $k+1$ can start is $\tau_k^t + t_b$. That is, $\tau_{k+1}^d \geq \tau_k^t + t_b$. We assume that the drone cannot be sent out earlier than the considered time horizon (i.e. $\tau_1^d \geq 0$). Let Γ be a set of m $\{\tau_k^d,\ \tau_k^t\}$ tuples, for $k \in \{1 \cdots m\}$, where each $\{\tau_k^d,\ \tau_k^t\}$ tuple is the start and end time for sub-tour k.

We define a complete drone tour as $\Phi = \{\mathbf{\Delta}, S^m, \Gamma\}$. We say that Φ is *consistent* if the following constraints hold:

$$\tau_1^d \geq 0 \tag{3.2}$$

$$\tau_{k+1}^d \geq \tau_k^t + t_b,\ \forall k \in \{1 \cdots m-1\} \tag{3.3}$$

$$\tau_k^t = \tau_k^d + t_\delta(\delta_k, s_k),\ \forall k \in \{1 \cdots m\} \tag{3.4}$$

$$p_k^d = p_b(\tau_k^d),\ \forall k \in \{1 \cdots m\} \tag{3.5}$$

$$p_k^t = p_b(\tau_k^t),\ \forall k \in \{1 \cdots m\} \tag{3.6}$$

Constraint 3.2 states that the first sub-tour cannot start earlier than the considered time horizon while Constraint 3.3 requires any subsequent sub-tour to not start earlier than the end time of the previous sub-tour plus the time required to swap out batteries on the drone. Constraint 3.4 requires that the end time of any sub-tour is equal to the start time of that sub-tour plus the time required to travel the distance of that sub-tour at the designated speed. Constraint 3.5 requires that the position of the depot waypoint for sub-tour k be based on the start time of that sub-tour and the GV trajectory function $p_b(t)$ while Constraint 3.6 holds the same requirements for the terminal waypoint of the sub-tour.

We formally define the MT-OTM problem as:

Definition 1 (MT-OTM Problem): *Given application-space $\mathcal{X}$ with waypoint set P and vehicle position function $p_b(t)$, determine the number of sub-tours m and a corresponding consistent Φ such that τ_m^t is minimized and all waypoints in P are visited at least once.*

Solving the MT-OTM Problem is challenging. To show the problem is at least as hard as any problem in the NP-Hard problem space, we first introduce a slightly simplified version of the problem, the Minimum-Time while On-the-Move with a fixed number of sub-tours (fixed-Mt-OTM) problem, defined as:

Definition 2 (fixed-MT-OTM Problem): *Given application-space $\mathcal{X}$ with waypoint set P, vehicle position function $p_b(t)$ and a number of sub-tours m, determine a corresponding consistent Φ such that τ_m^t is minimized and all waypoints in P are visited at least once.*

The fixed-MT-OTM Problem can easily be shown to be NP-Hard by a reduction from the TSP, a well known NP-Hard problem [30]. In the classic TSP, we are given n vertices in a fully connected weighted graph and are asked to find a minimum weight cycle in the graph that visits every vertex. Our strategy for reducing TSP to the fixed-MT-OTM Problem is to make the GV stationary.

Theorem 1: *The fixed-MT-OTM problem is NP-Hard.*

Proof. Select an arbitrary vertex in the TSP and set this as the starting location for the GV in the MT-OTM Problem and setting $p_b(t)$ to be this location at all input times. Map the remaining vertices in the TSP to waypoint set P by treating the edge weights as distances between waypoints and setting the drone's energy profile such that it can visit every vertex from the TSP without requiring a battery swap. An optimal solution to this fixed-MT-OTM Problem will also be an optimal solution to the original TSP. This reduction can be performed in linear time in terms of the number of vertices in the TSP.

As we have found a polynomial time reduction from the TSP to the fixed-MT-OTM Problem and it has been well established that TSP is NP-Hard, then the fixed-MT-OTM Problem is at least as hard as any NP-Hard problem. □

The last part of the reduction can be done by assuming the drone has an arbitrarily large and weightless battery. Drone energy models are further discussed in the following section.

We next show that the original MT-OTM Problem is NP-Hard by repeatedly reducing the fixed-MT-OTM Problem to the MT-OTM Problem.

Theorem 2: *The MT-OTM problem is NP-Hard.*

Proof. With $m = 1$, solve the MT-OTM Problem input as a fixed-MT-OTM Problem. Increment the value of m, solve the fixed-MT-OTM Problem with this new value of m, then repeat the process until $m = n$. The value of m and corresponding consistent Φ with a minimal value for τ_m^t that was found when solving the fixed-MT-OTM Problem will be the optimal solution to the MT-OTM Problem. This reduction can be performed in polynomial time in terms of the number of waypoints in P.

As we have found a polynomial time reduction from the fixed-MT-OTM Problem to the MT-OTM Problem and we showed that the fixed-MT-OTM Problem is NP-Hard, then the MT-OTM Problem must also be at least as hard as any NP-Hard problem. □

As shown in our hardness proof, solving the MT-OTM Problem could be simplified to finding a single drone sub-tour (i.e. finding an optimal Φ with one sub-tour) if the drone has enough on-board energy to complete Φ without needing to stop to swap out batteries. However, this will not be the case in more

interesting application scenarios and the drone will have to perform multiple sub-tours. This requires us to consider how drones consume energy in our solution.

3.3.1 Adaption of Drone Speed

To find a realistic solution to the MT-OTM Problem for non-arbitrary inputs, we need an accurate model for how drones consume energy. In this section, we summarize previous works that looked at how drones consume energy and we discuss the trade-off between maximizing a drone's speed versus maximizing a drone's travel distance.

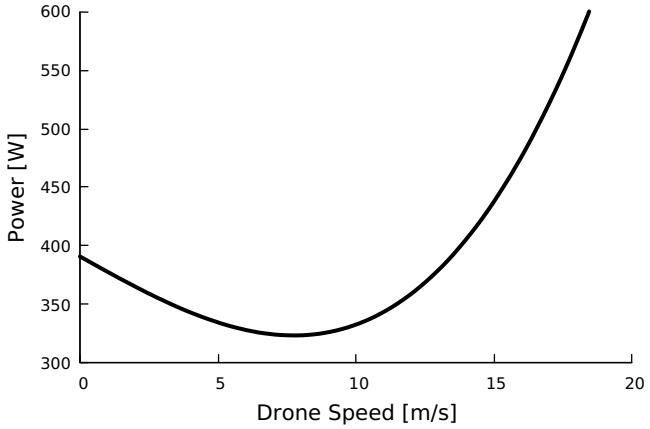


Figure 3.2 The relationship between a drone's speed and power consumption (Eq. 3.7) using experimental data in Ref. [63].

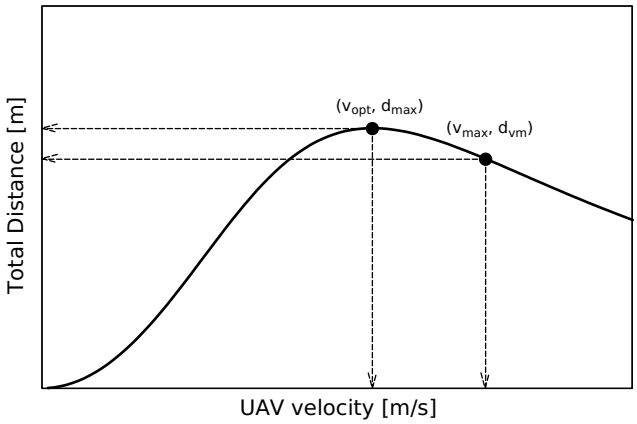


Figure 3.3 The relationship between a drone's speed and its maximum travel distance (Eq. 3.9).

Previous work has determined the amount of power consumed by a drone at varying speeds [4, 5]. From [5], we see that propulsion power consumption of a rotary-wing drone as a function of speed v can be approximated as:

$$\mathcal{P}(v) \approx C_0 \left(1 + \frac{3v^2}{U_{tip}^2}\right) + \frac{C_i v_0}{v} + \frac{1}{2} d_o \rho s_r A v^3, \tag{3.7}$$

where C_0 and C_i are constants representing blade profile power and induced power, respectively, U_{tip} represents the tip speed of the drone's propellers, v_0 is what is known as the mean rotor induced velocity while hovering, d_0 is an aircraft-specific drag ratio, s_r is rotor solidity, ρ is the air density and A is the rotor disk area.

Equation 3.7 is highly dependent on specific aircraft parameters and will change from aircraft to aircraft, but in general, this function has the shape of an upwards-facing parabola as shown in Fig. 3.2. The exact values for each parameter in Eq. (3.7) can be experimentally determined for individual drones, as demonstrated in [63]. Drone payloads will impact the values of these parameters and we assume that these values are known *a priori* for the drone and its payload. For a more detailed summary on drone energy models over varying drone types, we refer the reader to more comprehensive studies on this topic in [5, 114].

Many drone applications depend more on the total distance that a drone can travel as opposed to just the amount of energy consumed. The relationship between speed and the total distance depends on the voltage that a battery supplies to the drone and the rate that the battery discharges. We can represent this relationship as:

$$\mathcal{D}(v) = \frac{B_{rate} V_{bat} v}{\mathcal{P}(v)}, \tag{3.8}$$

where B_{rate} is the rate of battery discharge in amp-seconds and V_{bat} is the voltage of the battery. In practice, V_{bat} should be set lower than what the battery is rated for to ensure a safety buffer for uncertainty. We also treat the energy required for the drone to takeoff and land again on the GV as a sunken cost and assume that it has already been discounted from the battery capacity.

Equation (3.8) has the following general form (Fig. 3.3):

$$\mathcal{D}(v) = \frac{v}{v^3 + v^2 + \frac{1}{v}}. \tag{3.9}$$

Intuitively, to minimize total mission time, we will want to maximize the drone's speed. It was found [5] and then verified experimentally [63] that in order to achieve maximum traveling distance, the drone must travel at a lower speed than its maximum possible speed. Let d_{max} be this maximum achievable distance and v_{opt} be the speed that achieves d_{max}. Let v_{max} be the maximum speed that the drone is capable of traveling and d_{vm} be the distance that the drone can travel when moving at v_{max}. As shown in Figure 3.3, if $v_{opt} < v_{max}$ then $d_{max} > d_{vm}$.

Inspired by this finding, we formulate a function $v(d)$ that takes a distance and gives us the maximum speed that a drone can travel to achieve this distance.

$$v(d) = \begin{cases} v_{max} & \text{if } d \leq d_{vm} \\ \mathcal{D}^{-1}(d) & \text{if } d_{vm} < d \leq d_{max} \\ infeasible & \text{if } d > d_{max} \end{cases} \tag{3.10}$$

where $\mathcal{D}^{-1}(d)$ is the inverse of Eq. (3.8). Note that in the third case (i.e., if $d > d_{max}$) the drone is not capable of actually traveling distance d. This is the adaptive speed we use in our approaches.

If given realistic parameters for a drone, we could determine $\mathcal{D}(v)$. However, $\mathcal{D}^{-1}(d)$ is not easy to work with. Therefore, we instead propose approximating Eq. (3.8) between d_{vm} and d_{max} as a second order polynomial then finding the inverse of this polynomial to approximate Eq. (3.10). The inverse of such a polynomial will have the form

$$v(d) = \frac{\sqrt{c_1 + c_2 d}}{c_3} + c_4 \tag{3.11}$$

where c_1, c_2, c_3 and c_4 will be constants.

In the MT-OTM Problem, we want the drone to visit every waypoint and return to the GV in minimal time. To minimize time, we would like to minimize the distance that the drone must travel while maximizing the speed that the drone travels at. However, if moving faster means that the drone runs out of battery before visiting every waypoint then we may waste time swapping batteries. This creates a tradeoff between maximizing the speed of the drone and maximizing the distance that the drone can travel - potentially avoiding the need to swap out batteries as often. Drone path finding algorithms must be cognizant of this tradeoff and seek to exploit it when battery swaps or charging is required to complete the considered task. In the following section we show how to minimize the distance that the drone must travel and discuss our strategy for creating an energy-aware path planner that exploits the tradeoff between maximizing travel distance and maximizing speed.

3.4 Our Solution Framework

In this section, we summarize our framework for finding a solution to the MT-OTM problem. Our framework follows the same strategy that we used in Theorem 2 where we fixed the number of sub-tours (m) then solve the fixed-MT-OTM Problem.

Algorithm 1 depicts the process described in Theorem 2 to determine m^o, the optimal number of sub-tours, and a corresponding Φ^o, an optimal drone path, for the MT-OTM problem. We first set $m = 1$ (lines 1 and 3), solve for Φ using the function *path-planning()* (line 4), then increment m until $m = |P|$

(lines 3 and 8, respectively). On each iteration we check to see if we found a better solution (line 5) and update m^o and Φ^o (line 6), as needed. Note that the *path-planning()* function may fail to find a valid solution, particularly when there are a large number of waypoints to visit and m is small. In these scenarios, the condition on line 5 is false and the algorithm continues.

Algorithm 1 *MT-OTM Solver*

Input: P : set of waypoints to visit, $p_b(t)$: GV position function
Output: m^o : optimal number of sub-tours, Φ^o : optimal complete drone tour
1: $m^o \leftarrow 0,\ \Phi^o \leftarrow \emptyset,\ m \leftarrow 0$
2: **Do:**
3: $m \leftarrow m + 1$
4: $\Phi \leftarrow$ *path-planning*$(P, p_b(t), m)$
5: **if** $\Phi.\tau_m^t < \Phi^o.\tau_m^t$ **then**
6: $\Phi^o \leftarrow \Phi$, $m^o \leftarrow m$
7: **end if**
8: **While** $m < |P|$
9: **return** m^o, Φ^o

The runtime of Algorithm 1 greatly depends on the runtime of the *path-planning()* function. Assume that the *path-planning()* function runs in $\mathcal{O}(\psi)$. The loop that starts at line 2 will run $|P| = n$ times, making the runtime of Algorithm 1 $\mathcal{O}(n\psi)$. In practice, we can reduce the number of times we call the *path-planning()* function by terminating the algorithm when we stop seeing an improvement after incrementing m. However, this modification will not improve the worst-case run-time and provides no guarantee of optimality.

If we can write a function *path-planning()* that finds an optimal solution to the fixed-MT-OTM Problem, then Algorithm 1 will find an optimal solution and is an exact algorithm for the MT-OTM problem. In Section 3.5, we simplify the fixed-MT-OTM Problem by assuming that the first sub-tour starts at the beginning of the considered time horizon and that every consecutive sub-tour begins as early as possible. This assumption allows us to design both a MINLP and a heuristics-based solution for the *path-planning()* function. By building on the algorithms presented in Section 3.5, we remove our simplifying assumption on sub-tour start times in Section 3.6 and formulate an exact solution to the fixed-MT-OTM Problem and propose a flexible heuristics-based solution.

3.5 Fixed-Start MT-OTM Solutions

To solve the MT-OTM problem, Algorithm 1 breaks the problem into repeated instances of the fixed-MT-OTM problem which are solved using a suitable implementation of the function *path-planning()*. In this section we propose a method for implementing *path-planning()* that assumes the first drone sub-tour begins at the beginning of the considered time horizon and that all consecutive sub-tours will

start as early as possible. That is, we assume $\tau_1^d = 0$ and $\tau_{k+1}^d = \tau_k^t + t_b$ for $m \geq k \geq 1$, where τ_k^d and τ_k^t are the start and end times of the k^{th} sub-tour, respectively. We term this the *fixed-start assumption*. The motivation for using the fixed-start assumption is that it simplifies the *path-planning()* function and can outperform generalized approaches that remove the assumption (as shown in Section 3.7).

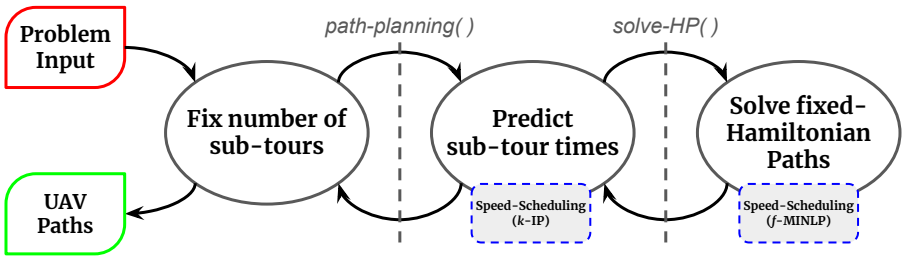


Figure 3.4 A flow graph for solving the MT-OTM Problem using the fixed-start assumption. We initially fix the number of sub-tours, then iteratively predict sub-tour times and find fixed-Hamiltonian paths until the solution is consistent in the *path-planning()* function. We then increase the number of sub-tours and rerun the *path-planning()* function.

Algorithm 2 *path-planning*

Input: P : set of waypoints to visit, $p_b(t)$: GV position function, m : number of sub-tours
Output: Φ : complete drone tour with m sub-tours

$t \leftarrow$ *guess-time*$(P, p_b(t), m)$
$A \leftarrow \{\frac{t}{m}, \cdots, \frac{t}{m}\}$
Do:
  $t' \leftarrow t$, $A' \leftarrow A$
  $G'_m \leftarrow$ *form-graph*$(P, p_b(t), m, A')$
  $\mathbf{\Delta}, S \leftarrow$ *solve-HP*(G'_m)
  $t \leftarrow t(\mathbf{\Delta}, S)$
  $A \leftarrow$ *sub-tour-times*$(\mathbf{\Delta}, S)$
While $\left(\left|t - t'\right| \geq \epsilon_t$ **or** $\left|max_i(A - A')\right| \geq \epsilon_A\right)$ **and** iterations $<$ iteration-limit
return Φ

Figure 3.4 shows the flow of our proposed *path-planning* function based on the fixed-start assumption while Algorithm 2 provides more details on our implementation. We start by guessing at the total time, t, that it will take to visit the waypoints in P based on $p_b(t)$ and m. Vector A represents the time required for each of the m sub-tours. In the while loop, we set our guess for the total time and sub-tour times to t' and A', respectively. We then form the graph G'_m using P, $p_b(t)$, m, and A in function *form-graph*(), which determines the locations of all p_k^d and p_k^t (the depots and terminals of each sub-tour) using A. This graph can be described mathematically as $G'_m = (V_m, E_m)$, where $V_m = P \cup \{p_1^d, \cdots, p_m^d\} \cup \{p_1^t, \cdots, p_m^t\}$ and

$$E_m = \{(p_i, p_j) \mid 1 \leq i, j \leq n \text{ and } i \neq j\} \\ \cup \{(p_k^d, p_i) \mid 1 \leq k \leq m \text{ and } 1 \leq i \leq n\}$$

$$\cup \ \{(p_i, p_k^t) \mid 1 \leq i \leq n \ \textit{and} \ 1 \leq k \leq m\}.$$

Once we find G'_m, the problem becomes solving an underlying Hamiltonian Path problem (function *solve-HP()*). We set $\mathbf{\Delta}$ and S as the found set of sub-tours and set of assigned speeds, respectively, then update the value of t based on the actual total mission time to run all sub-tours in $\mathbf{\Delta}$ given S. We update A using the function *sub-tour-times*(), which updates each sub-tour k's entry in A based on the time required to travel sub-tour δ_k while moving at speed s_k.

The loop continues until one of the following conditions is met:

1. The predicted mission time, t, is within some epsilon of the actual time, t', and every entry of our predicted vector A' is within some epsilon of the actual vector A, or

2. Some iteration limit has been met.

We then return the last found graph G' and the corresponding set of sub-tours $\mathbf{\Delta}$. If an iteration time-out condition occurs, we propose updating G' based on t and A without changing the found set of tours $\mathbf{\Delta}$.

To make an initial guess for t, we find the minimum spanning forest of m trees on $P \cup \{p_1^d\}$, determine the minimum time that a UAV needs to fly the total distance of the forest while making $m-1$ stops to swap batteries. We can use this time to predict where the base station will be located for each of the $m-1$ stops. Using this intermediate guess, we can create a G'_m, find a new minimum forest of m trees in G'_m then use the distance of this new forest to get a guess on the time required to complete the search.

Our fixed-start solution framework reduces the MT-OTM problem into an underlying fixed multi-depot, multi-terminal Hamiltonian paths problem (fixed-MdMtHPP). This is a special case of the more general MdMtHPP, which is formally defined as: "Given m salesmen that start from distinct depots, m terminals and n destinations, the problem is to choose paths for each of the salesmen so that (1) each salesman starts at his respective depot, visits at least one destination and reaches any one of the terminals not visited by other salesmen, (2) each destination is visited exactly once, and (3) the cost of the paths is minimum among all possible paths for the salesmen" [115]. fixed-MdMtHPP differs from the traditional MdMtHPP in that the depots and terminals are fixed. Each tour that starts at some depot (p_k^d) must end at a specific terminal (the corresponding p_k^t).

MdMtHPP is NP-Hard ([115]). Fixing the matching between depots and terminals does not make the problem easier to solve.

Theorem 3: *The fixed multi-depot, multi-terminal Hamiltonian path problem (fixed-MdMtHPP) is NP-Hard.*

Before beginning the proof for Theorem 3, we would like to remind the reader of the fixed destination multi-salesmen, multi-depot TSP (MmTSP). In MmTSP, there is a given set of depots with one or more salesmen and a set of destination vertices that must be visited by exactly one salesman. In fixed-destination MmTSP, each salesman must end their tour at the vertex they started at, which is known to be NP-Hard.

Proof. We form a reduction from fixed-destination MmTSP to fixed-MdMtHPP as follows. Let G be the graph for the fixed-destination MmTSP problem. Form a new graph G' by taking the set of destination vertices and depots from G. If any depot has more than one salesman that starts at it, then form a new depot at this same location and assign the additional salesman to this depot. For every salesman, create a terminal vertex that lies on top of the salesman's starting vertex. Solve the fixed-MdMtHPP that we have just formed. The routes from our solution to fixed-MdMtHPP will also be a solution to the fixed-destination MmTSP.

This reduction can be performed in linear time in terms of the number of vertices in G. As we have found a polynomial time reduction from fixed-destination MmTSP to fixed-MdMtHPP, and we know that fixed-destination MmTSP is NP-Hard, then fixed-MdMtHPP must also be NP-Hard. □

We present a MINLP that provides an exact solution to the fixed-MdMtHPP in the following subsection. However, as we have just shown, the fixed-MdMtHPP is NP-Hard and MINLP are not tractable for larger inputs. To address this limitation of our MINLP solution, we present a heuristics-based solution in Section 3.5.2.

3.5.1 MINLP Formulation for the fixed-MdMtHPP

In this subsection we formulate a MINLP to solve the underlying fixed-MdMtHPP for our drone path finding problem. We term this approach the *fixed*-MINLP, or f-MINLP. We also show how a simplified version of the formulation can be used to solve the general case of the fixed-MdMtHPP.

Our f-MINLP formulation minimizes mission completion time by jointly minimizing the distance of each sub-tour and maximizing the speed that the drone travels on each sub-tour. Due to the relationship between travel distance and speed discussed in Section 3.3.1, we cannot simply use a fixed drone speed but must adapt the speed based on distance, which is why we optimize these two jointly. We use a variation of the Miller-Tucker-Zemlin formulation [116, 117] for the capacitated VRP because it provides us with the flexibility to force fixed depots and terminals. Although our fixed-start assumption prevents us from making guarantees on the optimality of a solution to the MT-OTM problem, our f-MINLP can find an optimal solution to the underlying fixed-MdMtHPP.

In general, we use capital letters for sets, capital letters with subscripts as decision variables, and lowercase letters as constants. We refer the reader to Table 3.2 for a list of symbols used in our formulation. For the set of waypoints $i, j \in P$ and sub-tour k in the set of tours K, let E_{ijk} be a binary decision variable that determines if edge $(i,\ j)$ is included in tour k. We denote the Euclidean distance between i and j as d_{ij}. We use binary variables D^d_{ki} and D^t_{jk} to connect each depot k to some waypoint i, and some waypoint j to some terminal k, respectively. Let L_k and S_k be continuous variables for the total distance that the UAV must fly on sub-tour k and the constant speed of the UAV on k, respectively. To prevent cycles within each sub-tour, we use the integer variable U_i to give ordering assignments to each waypoint.

Table 3.2 MINLP Formulation Symbols

Symbol	*Type*	*Description*
D^d_{ki}	Binary Variable	1 if waypoint i is the first stop on sub-tour k, 0 otherwise
D^t_{jk}	Binary Variable	1 if waypoint j is the last stop on sub-tour k, 0 otherwise
E_{ijk}	Binary Variable	1 if edge (i, j) is in sub-tour k, 0 otherwise
K	Set	Set of sub-tours, where $k \in K$ is a sub-tour
L_k	Continuous Variable	Total distance of sub-tour k
L^t_{jk}	Continuous Variable	Distance from waypoint j to terminal of sub-tour k
L^d_{ki}	Continuous Variable	Distance from depot of sub-tour k to waypoint i
P	Set	Set of waypoints to visit, where $i, j \in P$ are waypoints
S_k	Continuous Variable	Assigned drone speed for sub-tour k
T_k	Continuous Variable	Start (T^d_k) and terminal (T^t_k) times for sub-tour k
X_k	Continuous Variable	x coordinate for depot (X^d_k) and terminal (X^t_k) of sub-tour k
Y_k	Continuous Variable	y coordinate for depot (Y^d_k) and terminal (Y^t_k) of sub-tour k

Our f-MINLP formulation of the problem is as follows.

$$\min \sum_{k \in K} \frac{L_k}{S_k} \tag{3.12}$$

subject to:

$$L_k = \sum_{i \in P} \sum_{j \in P} d_{ij} E_{ijk} + \sum_{i \in P} d_{ki} D^d_{ki} + \sum_{j \in P} d_{jk} D^t_{jk}, \qquad \forall k \in K \tag{3.13}$$

$$S_k \leq \frac{\sqrt{c_1 - c_2 L_k}}{c_3} + c_4, \qquad \forall k \in K \tag{3.14}$$

$$1 - \mathcal{M}(1 - E_{ijk}) \leq U_j - U_i \leq 1 + \mathcal{M}(1 - E_{ijk}), \qquad \forall i, j \in P, \forall k \in K \tag{3.15}$$

$$1 \leq U_i - U^d_k D^d_{ki} \leq 1 + \mathcal{M}(1 - D^d_{ki}), \qquad \forall i \in P, \forall k \in K \tag{3.16}$$

$$\sum_{j \in P} E_{j,i,k} + D^d_{ki} = \sum_{j \in P} E_{i,j,k} + D^t_{ki}, \qquad \forall i \in P, \forall k \in K \tag{3.17}$$

$$\sum_{j \in P} \sum_{k \in K} E_{j,i,k} + \sum_{k \in K} D^d_{ki} + \sum_{j \in P} \sum_{k \in K} E_{i,j,k} + \sum_{k \in K} D^t_{ki} = 2, \qquad \forall i \in P \tag{3.18}$$

$$\sum_{i \in P} D^d_{ki} = 1, \qquad \forall k \in K \tag{3.19}$$

$$\sum_{i \in P} D^t_{ki} = 1, \qquad \forall k \in K \tag{3.20}$$

$$S_k \leq v_{max}, \qquad \forall k \in K \tag{3.21}$$

$$L_k \leq d_{max}, \qquad \forall k \in K \tag{3.22}$$

Our objective function (3.12) is minimizing the total time required for the UAV to travel all sub-tours. Note that we can remove possible domain violations in the objective using an additional auxiliary variable, A_k, to separate L_k and S_k as $L_k = A_k S_k$ then minimizing A_k. We forego adding additional variables here for brevity. Constraint (3.13) forces variable L_k to equal the distance of sub-tour k. Constraint (3.14) adapts a sub-tour speed based on the distance of sub-tour k and is derived from Eq. (3.11).

Constraint (3.15) enforces a tight numbering scheme for consecutive waypoints in each sub-tour where $\mathcal{M}$ is some sufficiently large number which will be further discussed shortly. Constraint (3.16) forces the waypoint after depot k to be assigned a sequence number of $U^d_k + 1$, where U^d_k is an implied, fixed sequence number assigned to depot k that remains constant. For each waypoint i on sub-tour k, we want to assign a sequence number to U_i that is within a designated range to force waypoints that are on the same sub-tour to be numbered together. We define the bounds of this numbering range for sub-tour k using implied depot and terminal sequence numbers U^d_k and U^t_k, respectively. If l_m is the maximum number of waypoints that can be assigned to a sub-tour then we want to have l_m sequence numbers available between U^d_k and U^t_k. When given n waypoints to visit on m sub-tours with at least one waypoint on each sub-tour, by the pigeon hole principle, $l_m = n - m + 1$. For the first sub-tour, if we set $U^d_1 = 0$ then U^t_1 must be $l_m + 1$. This makes $U^d_2 = l_m + 2$. Following this trend, we find that for any sub-tour k, $U^d_k = (l_m + 2)(k - 1)$ and $U^t_k = k(l_m + 1) + (k - 1)$. In constraints (3.15) and (3.16) we want a value for $\mathcal{M}$ that is large enough to allow for all feasible sequence number assignments for each U_i. To keep a tight bound on our constraints, we set $\mathcal{M} = U^t_m = ml_m + 2m - 1$.

Constraint (3.17) ensures that the in-degree is equal to the out-degree of each waypoint while constraint (3.18) forces each waypoint to have a degree of two. Constraints (3.19) and (3.20) force every depot and terminal to be used, respectively. Finally, constraints (3.21) and (3.22) bound the maximum allowable speed of the UAV and maximum allowable distance of any sub-tour, respectively.

The numbering scheme described above is what allows us to fix the depots with their corresponding terminals. We are also ensuring that each sub-tour contains at least one waypoint by not defining an edge from depot k to terminal k in our formulation.

We can modify our f-MINLP formulation to solve the general case of fixed-MdMtHPP by making the objective function to be

$$\min \sum_{k \in K} L_k \tag{3.23}$$

and removing constraints (3.14), (3.21) and (3.22). The general fixed-MdMtHPP formulation avoids the additional complexity of variable multiplication seen in our f-MINLP formulation but does not adapt UAV speed and is not directly optimizing mission completion time.

Our f-MINLP formulation provides an exact solution to the fixed-MdMtHPP created by fixing the drone's launch and receive locations. However, the fixed-MdMtHPP is an NP-Hard problem and MINLPs in general take a long time to solve to completion. Therefore, we do not expect our f-MINLP formulation to be able to solve large problem inputs in a reasonable amount of time. In the following subsection, we address this limitation by introducing a heuristics-based algorithm that can handle large problem inputs.

3.5.2 Heuristic Solution for the fixed-MdMtHPP

Mixed-Integer Non-Linear Programs are often very hard to solve, even for commercial solvers on high-performance computers, so in this subsection we propose a more tractable approach that combines a heuristics-based k-means clustering algorithm and a TSP solver. The general concept is to partition the waypoints into m groups then form sub-tours by solving the TSP. After finding waypoint grouping and ordering each sub-tour, we schedule speeds for the sub-tour by finding the distance of the sub-tour and using Eq. (3.10).

To partition the waypoints, we use Lloyd's algorithm to form m k-means clusters [118]. We use the centroid of each p_k^d and p_k^t pair as the initial cluster centroids. We then limit the number of iterations that the algorithm runs which prevents the cluster centroids from migrating too far away from their corresponding p_k^d and p_k^t pairs. Once we have put each waypoint into a cluster, we combine the clusters with the corresponding p_k^d and p_k^t. We then solve a TSP on the resulting graph and force the edge connecting p_k^d and p_k^t to be part of the solution.

There are several well-studied approaches to solving the TSP. In this work we use both an Integer Program (IP) and the Lin-Kernighan-Helsgaun (LKH) heuristic [109]. We use the IP formulation found in [119] that uses sub-cycle cuts to enforce closed tours. To avoid an exponential number of sub-cycle cuts, we treat these as lazy constraints where the constraint is only added to the solver when a found solution would break the constraint, a feature available in many commercially available solvers such as Gurobi. To get the LKH solver to force edge connecting p_k^d and p_k^t to be part of the solution we set this edge equal to 0 and multiply all other edges going out of each of these vertices by a constant. We term the IP version of this solution the k-IP approach and the LKH solution the k-TSP approach. Although neither the k-IP nor the

k-TSP algorithms provide theoretical guarantees on solution quality, we should expect both algorithms to run faster than solving the f-MINLP formulation.

The f-MINLP formulation and the k-IP and k-TSP algorithms all make the fixed-start assumption, where the drone is launched at the beginning of the considered time horizon and all subsequent sub-tours start as early as possible. The results of our numerical simulations in Section 3.7 show that this assumption is valid when all waypoints are located close to the GV at the start of the considered time horizon but can cause these algorithms to fail to find a solution when the waypoints are further away. To address this problem, we next consider more flexible solutions that remove the fixed-start assumption.

3.6 Generalized MT-OTM Solutions

Algorithm 1 can find a solution to the MT-OTM problem if given function *path-planning()* - which solves instances of the fixed-MT-OTM problem. In the previous section we discussed how to implement *path-planning()* by assuming that the first sub-tour should start at the beginning of the considered time horizon and that every consecutive sub-tour should start as early as possible (the "fixed-start assumption"). In this section, we remove the fixed-start assumption and propose more generalized solutions.

We first look at a MINLP that provides an exact solution to the MT-OTM problem then propose a heuristic-based solution that is more tractable. These approaches solve the fixed-MT-OTM problem directly rather than solving for a fixed-MdMtHPP as presented in the previous section.

3.6.1 MINLP Formulation for the MT-OTM Problem

In this subsection we formulate a new MINLP to solve the generalized MT-OTM Problem. Our formulation minimizes mission completion time (the end time of the final sub-tour, τ_m^t) by jointly minimizing the distance of each sub-tour and maximizing the speed that the drone travels on each sub-tour. Additionally, our formulation for the generalized problem dynamically selects p_k^d and p_k^t, the starting and end location of sub-tour k. We term this solution *dynamic*-MINLP, or D-MINLP.

Our D-MINLP formulation builds on the f-MINLP formulation presented in Section 3.5.1 with a few additional variables. Let L_{ki}^d be a continuous variable that tracks the distance from the depot of sub-tour k to waypoint i while continuous variable L_{jk}^t tracks the distance from waypoint j to the sub-tour's terminal. Let T_k^d and T_k^t be continuous variables for the start and end time of sub-tour k, respectively. Continuous variables X_k^d and Y_k^d are the x and y coordinates, respectively, of the depot for sub-tour k and variables X_k^t and Y_k^t are the coordinates for the terminal of the sub-tour. Similarly to the f-MINLP formulation, the D-MINLP formulation uses the S_k variable and Eq. (3.10) to schedule speeds as part of the optimization problem.

Our D-MINLP formulation of the problem is as follows:

$$\min T_m^t \tag{3.24}$$

subject to Eq. (3.13), (3.14) (3.15), (3.16), (3.17), (3.18), (3.19), (3.20), (3.21), (3.22), and:

$$L_{ki}^d = \sqrt{(X_k^d - x_i)^2 + (Y_k^d - y_i)^2}, \qquad \forall k \in K, \forall i \in P \tag{3.25}$$

$$L_{jk}^t = \sqrt{(X_k^t - x_j)^2 + (Y_k^t - y_j)^2}, \qquad \forall j \in P, \forall k \in K \tag{3.26}$$

$$(X_k^d, Y_k^d) = p_d(T_k^d), \qquad \forall k \in K \tag{3.27}$$

$$(X_k^t, Y_k^t) = p_d(T_k^t), \qquad \forall k \in K \tag{3.28}$$

$$T_1^d \geq 0 \tag{3.29}$$

$$T_k^d \geq T_{k-1}^t + t_b, \qquad \forall k \in K, k \geq 2 \tag{3.30}$$

$$T_k^t = T_k^d + \frac{L_k}{S_k}, \qquad \forall k \in K \tag{3.31}$$

Our objective function (3.24) minimizes mission completion time by minimizing T_m^t, the end time of the final sub-tour. Constraint (3.25) sets variable L_{ki}^d based on the distance from depot k to waypoint i while constraint (3.26) sets variable L_{ki}^d to be the distance from waypoint j to terminal k. Constraints (3.27) and (3.28) ensure that the location of each depot and terminal, respectively, is based on $p_d(t)$, the function for the GV's trajectory discussed in Section 3.3. Constraints (3.29) and (3.30) bound the start time of sub-tour k based on the considered time window (for the former) and the previous sub-tour's end time (for the latter). Constraint (3.31) sets the sub-tour's end time.

Our D-MINLP formulation finds an optimal solution to the fixed-MT-OTM Problem and makes Algorithm 1 an exact algorithm to the larger MT-OTM Problem when used as function *path-planning()*. However, even on the strongest commercial solvers, MINLPs are not easy to solve to optimality and we have already shown that the MT-OTM Problem is NP-Hard in Section 3.3. In the following section we propose a heuristics-based method for solving the fixed-MT-OTM Problem that is more tractable than our D-MINLP formulation.

3.6.2 Heuristic Solutions for the fixed-MT-OTM Problem

Our heuristics-based solution for solving the fixed-MT-OTM Problem builds on the concepts presented in Section 3.5.2. We use a clustering algorithm to form m clusters that serve as our m sub-tours. We pick start and end locations for each sub-tour then treat the clusters with start and end locations as a TSP and iteratively solve this TSP and re-selecting start and end locations until the solution becomes consistent and stops changing from iteration to iteration. We term this approach the *Dynamic*-TSP, or D-TSP. This solution requires $p_b^{-1}(x_c, y_c)$, the inverse of the GV's trajectory function that gives you the time that the

GV will pass through (x_c, y_c), or a method for approximating the inverse of $p_b(t)$.

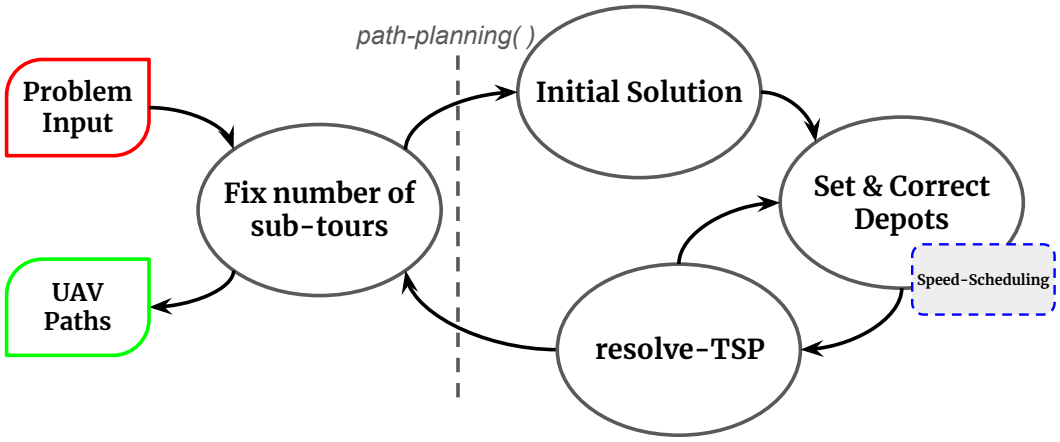


Figure 3.5 A flow graph for solving the MT-OTM Problem directly. We initially fix the number of sub-tours, run the *path-planning()* function, and then increase the number of sub-tours. The *path-planning()* function finds an initial solution by clustering together waypoints then iteratively solves a TSP on each cluster and adjusts depot and terminal locations until the solution is consistent.

Figure 3.5 shows the general flow for our D-TSP algorithm for directly solving the MT-OTM Problem while Algorithm 3 lists the details of our implementation. Starting on line 1, we use a k-means cluster algorithms to form m sets of waypoints (P_m). Using the function *create-depots()* on line 2, we create m pairs of depots and terminals (stored in set Γ). For each cluster, the function finds the closest point along the GV's trajectory to the cluster's centroid. Suppose this position is (x_c, y_c). The function then queries $p_b^{-1}(x_c, y_c)$ to determine the time that the GV will be at this location. Suppose this time is t_c. The *create-depots()* function then estimates the time required for a drone to visit all waypoints in the cluster using a minimum spanning tree and assuming the drone moves at v_{max}. Suppose this time is t'_c. The function then places the depot for the cluster at the location of the GV's position at $t_c - \frac{t'_c}{2}$ and the terminal at $t_c + \frac{t'_c}{2}$. The algorithm then treats the pairs of depots and terminals in Γ and waypoint clusters in P_m as a set of TSP, which is solved using the LKH heuristic on line 3.

On line 8, the function *set-depots()* sets sub-tour speeds based on the distance of the sub-tour and Eq. (3.10) then corrects the position of the depots and terminals so the solution meets problem constraints (3.4), (3.5), and (3.6). This logic is based on centering the depot and terminal around the mid-point time t_c that was discussed earlier. This function also attempts to move the start time of a sub-tour to an earlier time if the distance of the sub-tour is less than d_{vm}, the distance that the drone can travel if moving at v_{max}. The idea behind moving up the start time of a sub-tour is that if the tour can start earlier then it can potentially end earlier and reduce the time required to visit all waypoints.

Algorithm 3 *D*-TSP

Input: P : set of waypoints to visit, $p_b(t)$: GV position function, m : number of sub-tours
Output: Φ : complete drone tour with m sub-tours

1: $P_m \leftarrow k\text{-}means(P, m)$
2: $\Gamma \leftarrow create\text{-}depots(P_m)$
3: $\mathbf{\Delta} \leftarrow solve\text{-}TSP(P_m, \Gamma),\ \mathbf{\Delta}' \leftarrow \emptyset$
4: **while** $\mathbf{\Delta} \neq \mathbf{\Delta}'$ **do**
5: $\quad \mathbf{\Delta}' \leftarrow \mathbf{\Delta},\ \Gamma' \leftarrow \emptyset$
6: $\quad$ **while** $\Gamma \neq \Gamma'$ **do**
7: $\quad\quad \Gamma' \leftarrow \Gamma$
8: $\quad\quad S^m, \Gamma \leftarrow set\text{-}depots(\mathbf{\Delta})$
9: $\quad\quad \Gamma \leftarrow correct\text{-}depots(\Gamma)$
10: $\quad$ **end while**
11: $\quad \mathbf{\Delta}_m \leftarrow solve\text{-}TSP(P_m, \Gamma)$
12: **end while**
13: **return** $\Phi \leftarrow \{\mathbf{\Delta}, S^m, \Gamma\}$

Both the *create-depots()* and *set-depots()* functions ignore that a consecutive sub-tour cannot start earlier than the end time of the previous sub-tour plus the time required to swap out batteries (constraint 3.3). Function *correct-depots()* on line 9 fixes constraint 3.3 violations by solving the following convex optimization problem:

$$\min \sum_{k=1}^{m} (T_k^d - t_k^d)^2 \tag{3.32}$$

subject to:

$$T_{k-1}^d + \hat{t}_k + t_b \leq T_k^d, \qquad 2 \leq k \leq m \tag{3.33}$$

where T_k^d is a continuous variable for the start time and t_k^d is a constant that represents the desired start time for sub-tour k. Constant $\hat{t}_k$ is the time duration of sub-tour k and t_b is the time required to swap out batteries on the drone at the GV.

After finding a consistent solution that meets constraints (3.2), (3.3), (3.4), (3.5), and (3.6), we run the TSP solver again on line 11 to verify that the sub-tour ordering cannot be improved based on how we moved the depots and terminals. The while-loop on line 4 runs until the sub-tours stop changing between iterations and the while-loop on line 6 runs until the location of the depots and terminals stops changing. In practice, these while-loops should have a time-out criteria to avoid an infinite loop should the algorithm never converge onto a consistent solution.

Algorithm 3 can be performed in polynomial time. To ensure that the k-means clustering algorithm converges in a timely manner we can add a time-out condition based on $n = |P|$. The *create-depots()* function finds a minimum spanning tree, which can be done in $\mathcal{O}(n^2 log(n))$. The LKH solver is

approximated to run in $\mathcal{O}(n^{2.2})$ [31]. The function *set-depots()* can be done in $\mathcal{O}(n)$. Convex optimization problems can be solved to optimality in cubic time based on the number of decision variables [120], which means function *correct-depots()* can run in $\mathcal{O}(m^3)$. Suppose we limit both while-loops to time-out after a constant number of iterations, then the inner loop at line 6 will have a runtime of $\mathcal{O}(n)$ while the outer loop at line 4 will be dominated by the runtime of the LKH solver at $\mathcal{O}(n^{2.2})$. This makes the run-time of Algorithm 3 dependent on the LKH solver at $\mathcal{O}(n^{2.2})$. If we use Algorithm 3 as function *path-planning()* from Algorithm 1, then the runtime complexity of the D-TSP approach will be $\mathcal{O}(n^{3.2})$.

Similar to the k-IP and k-TSP algorithms, the D-TSP algorithm provides no theoretical guarantees on solution quality. However, we should expect the D-TSP algorithm to compute solutions to the general MT-OTM problem much faster than solving the D-MINLP formulation.

3.7 Simulation Evaluation

In this section we discuss our evaluation of our framework for solving the MT-OTM Problem in simulation using parameters from previous field testing and commercially available hardware. We start this section with a discussion on our evaluation setup. We then look at our proposed methods for the MT-OTM Problem where the start points of each sub-tour are fixed and then remove this assumption and evaluate our generalized solutions. We conclude this section with a review of run times for all considered approaches.

3.7.1 Simulation Setup

Our simulations were conducted on a machine with an Intel 3.4 GHz 16-Core CPU and 64 GiB of RAM. We use Gurobi Optimizer version 10.0.3 for our optimization solver. Our solution framework is implemented in C++ and provided as open-source[4].

3.7.1.1 Realistic Energy Model from Field Tests

In Section 3.3.1 we presented $\mathcal{P}(v)$, a theoretical equation that represents the power consumed by a drone based on speed v. $\mathcal{P}(v)$ was verified through field testing in [63]. For their specific drone, $\mathcal{P}(v)$ can be approximated as

$$\mathcal{P}(v) = \ 0.07v^3 + 0.0391v^2 - 13.196v + 390.95 \tag{3.34}$$

If we equip the drone with a commercially available LiPo battery rated at 2,200 mAh, 12.6 volt, then we can plug Eq. (3.34) into Eq. (3.8) and get

$$d(v) = \ \frac{99,792v}{0.07v^3 + 0.0391v^2 - 13.196v + 390.95} \tag{3.35}$$

[4] github.com/pervasive-computing-systems-group/MT-OTM-Solver

We can use Eq. (3.35) to determine d_{max}, the maximum distance that the drone can travel. By approximating the curve of Eq. (3.35) between d_{vm} and d_{max} as a polynomial, we can find an approximation for $\mathcal{D}^{-1}$ between v_{opt} and v_{max} in the form of Eq. (3.8).

3.7.1.2 Metrics and Baseline Approaches

In all simulations, our primary metric was the total time required to visit all waypoints and then return to the GV. This includes the time required to swap out batteries between consecutive sub-tours, incentivizing each approach to reduce the number of sub-tours required to visit all waypoints. We also looked at the percentage of inputs where each approach was able to find a solution and the computation time for each approach.

For approaches that assume a fixed start time we compared against a baseline adapted from the heuristics-based approach found in [91]. We refer to this approach as *tour-splitting*, which is further discussed below. When evaluating our generalized MT-OTM Problem solutions we compared against the approaches that assume a fixed start time and as well as a dynamic implementation of the *tour-splitting* algorithm.

The *tour-splitting* (TS) approach finds a TSP tour on the entire set of waypoints using the LKH heuristic [109] then has the drone follow this tour until it runs out of energy. We chose this approach because it avoids repeatedly solving for Hamiltonian paths, allows us to adapt drone speeds to sub-tour distances, and has been proposed for similar problems in recent literature [91, 92, 111]. Additionally, this approach gives us an example of simplifying the problem into a variation of the traditional TSP and then using a TSP-based heuristic to solve the problem.

This approach as proposed in [91] finds a cycle while in the MT-OTM Problem we wish for the drone to generally follow the path of the GV. This requires us to slightly adapt the original approach instead of using it directly. To adapt this approach, we find a minimum distance Hamiltonian path using the LKH heuristic that starts from the waypoint closest to the vehicle's starting point and ends where we predict the GV to be at the end of the entire data-collection mission, as described in Section 3.4. We then divide this total path into m roughly equal segments. For each segment, we form a sub-tour by determining where the GV will be at for the beginning of the sub-tour based on the previous sub-tour. We then iteratively approximate the maximum possible drone speed allowed for the sub-tour using Eq. (3.10) and determine a corresponding sub-tour terminating location until we settle on a consistent solution. The approach is plugged into Algorithm 1 as the *path-planning()* function.

The basic TS approach holds the fixed-start assumption that was discussed in Section 3.5. To compare against our dynamic solutions (D-MINLP and D-TSP), we further modify the TS approach to allow it to

dynamically set sub-tour start and end locations. The adaptive TS (a-TS) selects depot positions by finding the closest point along the GV's trajectory to the beginning of each tour that does not break problem constraints (3.2), (3.3), (3.4), (3.5), and (3.6).

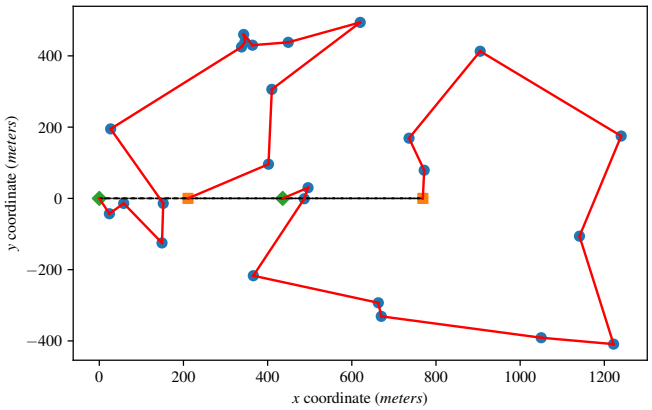


Figure 3.6 Sample drone paths generated using the MILP approach. The blue circles are the navigational waypoints, the dashed, black line is the GV's trajectory, the red lines are the drone's path where the green diamonds are drone launch points and the orange squares are drone receiving points.

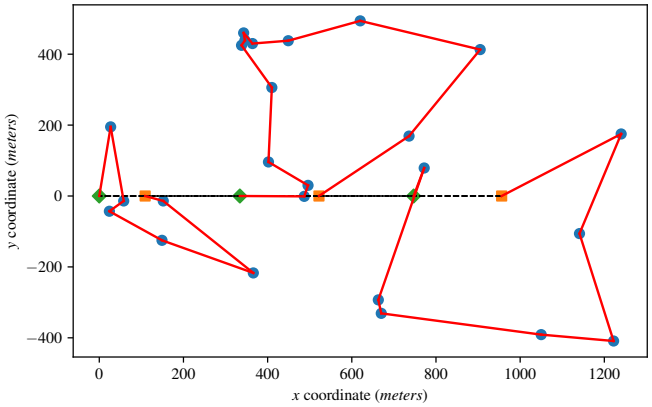


Figure 3.7 Sample drone paths generated using the k-IP approach.

3.7.2 Comparison of Fixed-Start Approaches

We ran all three fixed-start approaches on randomized graphs with n ranging from 5 up to 80 at increments of 5. We generated 50 graphs at each increment. To keep the results comparable across each input graph, we had the GV move at a fixed speed of 2.5 m/s along the x-axis.

Figure 3.6 shows an example output of the f-MINLP approach and Figure 3.7 shows the output of the k-IP approach on a randomly generated graph with 25 waypoints and a linear GV trajectory. The f-MINLP approach found a superior solution with a mission completion time of 307.5 s while the k-IP

approach found a solution with a completion time of 382.4 s. However, the k-IP only took 0.038 s to compute this solution while using the f-MINLP formulation took 130.9 s to compute a solution. This tradeoff is further demonstrated in Section 3.7.2.

Figure 3.8 (*top*) shows how the number of waypoints affects the average mission completion time. The error bars show the standard deviation for each approach. We stopped the f-MINLP at $n = 30$ due to long computation times, which are further discussed below. The TS and k-IP approaches were not always able to find a feasible solution and their failure rates are also documented below. On average, the f-MINLP and the k-IP approaches provide a 23.8% and 14.5% improvement over the TS approach, respectively. The k-IP's performance is not as good as the f-MINLP solution but only averages a 3.8% increase in mission completion time over the f-MINLP approach and provides a nice alternative to the f-MINLP in larger sized problems. We note that the results in Figure 3.8 were first presented in our conference publication [75].

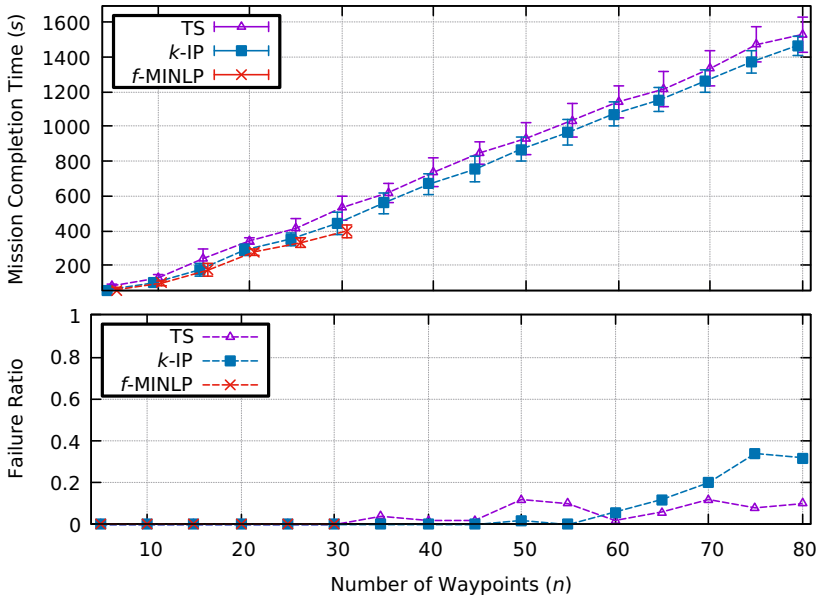


Figure 3.8 Simulation results for fixed-start solutions. The top plot shows the impact of the number of waypoints on mission completion time for the f-MINLP, k-IP, and TS approaches. The bottom plot shows the ratio of graphs where each approach failed to find a solution.

Figure 3.8 (*bottom*) shows the ratio of graphs where each approach failed to find a solution. The k-IP approach failed to find a valid solution for 6.6% of the graphs while the TS approach failed at 4.3% of the graphs. This suggests that although k-IP outperforms TS in mission completion time, TS may be able to find more solutions than k-IP. The f-MINLP approach was able to find a solution of all graphs up to the cut-off point ($n = 30$).

3.7.3 Comparison of Dynamic Approaches

In this section we evaluate our proposed generalized solutions for the MT-OTM Problem that remove the fixed-start time assumption. We ran each approach on multiple randomized sets of graphs with varying values for n. In addition to linear GV trajectories, we also compared our various solutions on instances where the GV moves along a sinusoidal path. The details for each set of inputs is further described below.

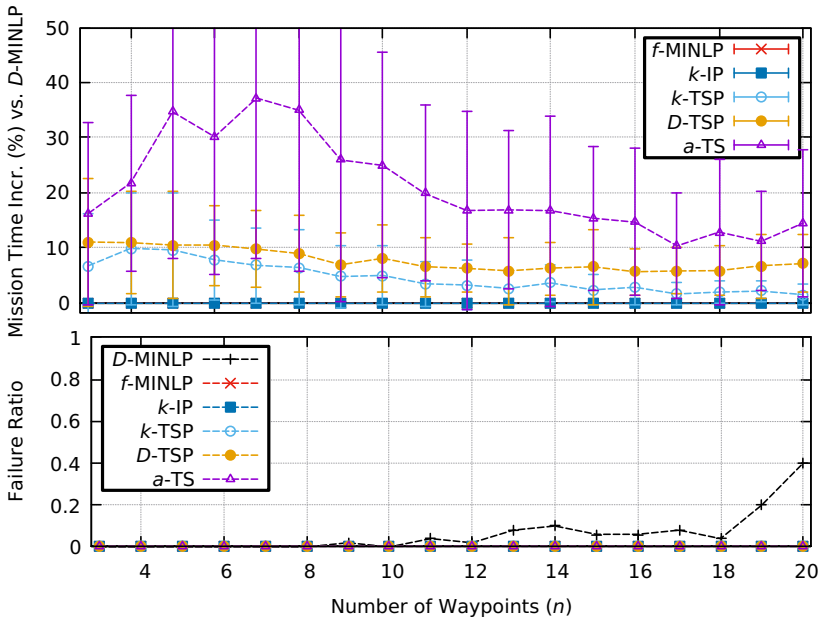


Figure 3.9 Simulation results for dynamic solutions. The top plot shows the impact of the number of waypoints on mission completion time for the f-MINLP, k-IP, k-TSP, D-TSP, and adaptive TS approaches when compared against an optimal solution found using the D-MINLP approach. The bottom plot shows the ratio of graphs where each approach failed to find a solution.

3.7.3.1 Mission Completion Time Compared to Optimal Solution

For our generalized approaches we first ran the D-MINLP, f-MINLP, k-IP, k-TSP, D-TSP and adaptive TS on a test set where the GV moved along a linear trajectory at 2.5 m/s. To avoid letting the solver get stuck on hard inputs, we set the D-MINLP solution to time-out at 1,200 s. The input set had n ranging from 3 up to 20 at increments of 1. We generated 50 graphs at each increment. We stopped at 20 because this is when the D-MINLP began to struggle to find a solution in a reasonable amount of time.

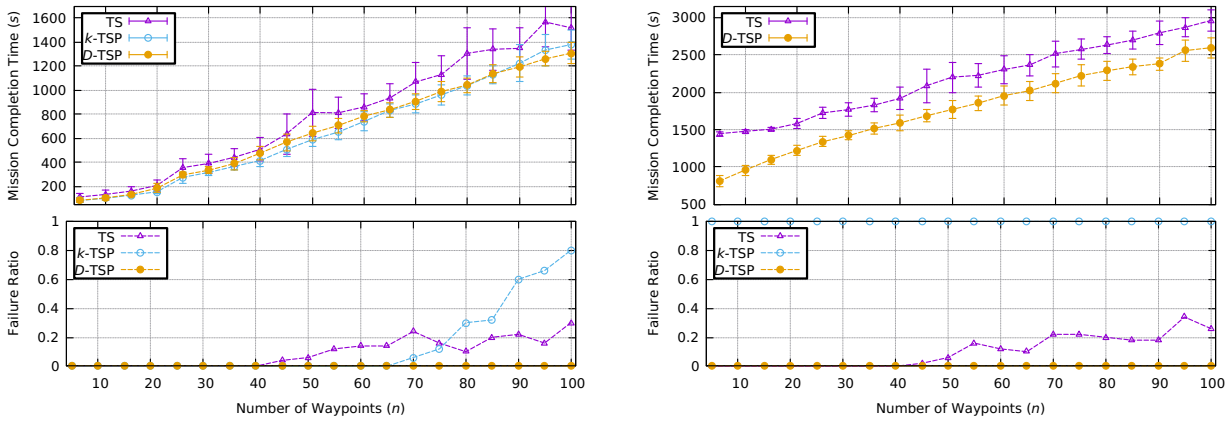
Figure 3.9 (*top*) shows the percent increase in mission completion time for each method when compared against the exact D-MINLP approach. The error bars show the standard deviation. The f-MINLP and k-IP approaches found a comparable solution to the D-MINLP for all inputs. On average, the k-TSP and D-TSP approaches found a solution that was within 4.5% and 7.7% of the optimal solution, respectively, while the adaptive TS approach averaged 20.8% increase in completion time compared to the optimal.

Figure 3.9 (*bottom*) shows the ratio of graphs where each approach failed to find a solution. All approaches except for the D-MINLP were able to find a solution on this data set. We set a timeout of 1,200 s for the D-MINLP and all failures were from the solver timing out.

3.7.3.2 Larger Inputs with Linear Trajectory Gound Vehicle

We then ran the k-TSP, D-TSP and adaptive TS on larger inputs with a linear GV trajectory. We chose to use the k-TSP approach here instead of the k-IP because we wanted to compare the D-TSP approach against another polynomial time solution. The GV moved at 1.5 m/s and n ranged from 5 up to 100 in increments of 5 with 50 randomly generated plots at each increment. We generated two data sets with the setup. The first has the waypoints beginning close to the GV's position at the beginning of the considered time window, which we refer to as the "*near-start*" linear data set. The second has the GV start 2 kilometers before the waypoints, which we refer to as the "*far-start*" linear data set.

Figure 3.10 shows the results of these two test scenarios with Figure 3.10(a) showing the near-start results and Figure 3.10(b) showing the far-start results. On the near-start linear data set, the k-TSP and D-TSP approaches performed nearly the same with the former slightly outperforming the latter. When compared against adaptive TS, the k-TSP and D-TSP approaches improved completion time by an average of 18.3% and 14.8%, respectively. As shown in the bottom of Figure 3.10(a), the k-TSP approach struggled to find a solution as n grew above 70. We note that Figure 3.10(a) is only showing results for inputs where all three algorithms were able to find a solution.



(a) Near-start linear data set

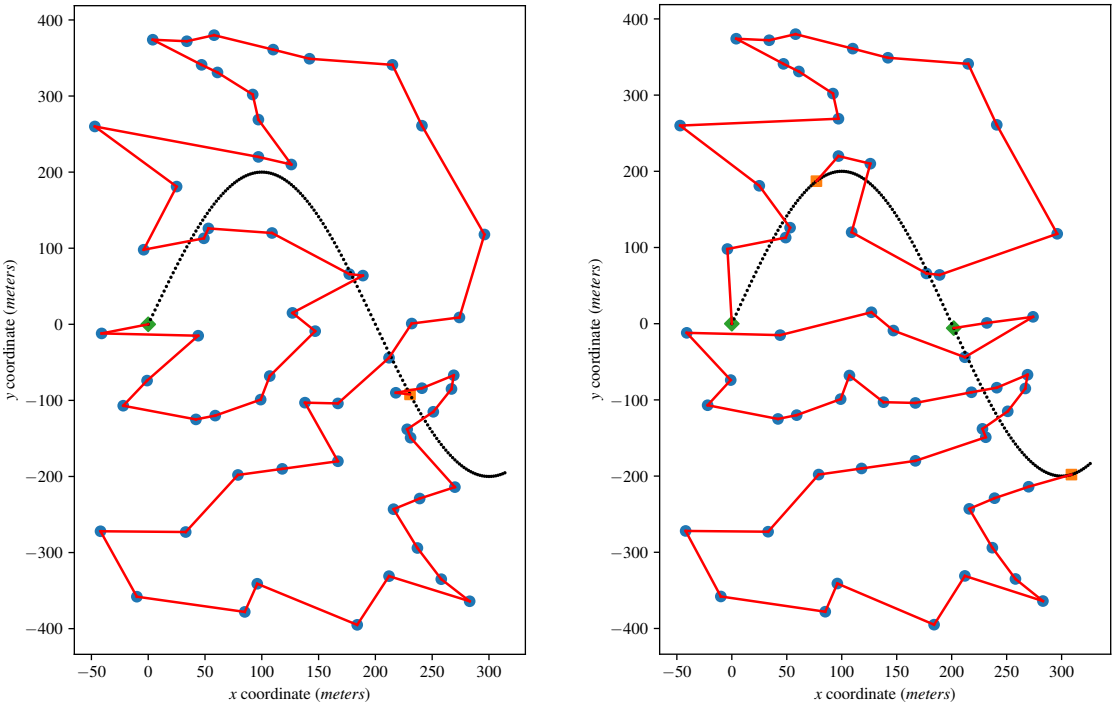
(b) Far-start linear data set

Figure 3.10 Simulation results for dynamic solutions on inputs with linear GV trajectories. The top plots show the impact of the number of waypoints on mission completion time. The bottom plots show the ratio of graphs where each approach failed to find a solution.

Figure 3.10(b) shows the results for the far-start linear data set. The k-TSP approach failed to find any solution on these inputs because the GV started at a distance from the first waypoints and the fixed-start assumption caused the drone to launch too early. However, both the D-TSP and adaptive TS approaches were able to solve inputs from this data set with the former finding a solution to all given inputs. The D-TSP improved completion time by an average of 19.2% when compared to the adaptive TS.

3.7.3.3 Larger Inputs with Sinusoidal Trajectory Gound Vehicle

We next ran the k-TSP, D-TSP and adaptive TS test data sets where the GV moves along a sinusoidal trajectory. The GV moved in the positive x direction at 1 m/s while the vehicle's y-coordinate was defined as $y(t) = 200sin(\frac{2\pi t}{400})$. We again generated two data sets with this setup; the near-start sinusoidal data set where the waypoints begin close to the GV's starting position and the far-start sinusoidal data set where the GV starts 2 kilometers before the waypoints.



(a) Drone path generated using k-TSP

(b) Drone path generated using D-TSP

Figure 3.11 Sample drone paths for a GV moving along a sinusoidal trajectory. The blue circles are the navigational waypoints, the dashed, black line is the GV's trajectory (which moves from left to right), the red lines are the drone's path where the green diamonds are drone launch points and the orange squares are drone receiving points.

Figure 3.11 shows example outputs for the k-TSP approach (3.11(a)) and the D-TSP approach (3.11(b)) on a graph with 60 waypoints from the near-start sinusoidal data set. The k-TSP approach found

a superior solution with a single sub-tour and mission completion time of 230.5 s while the D-TSP approach found a solution with two sub-tours and a 308.9 s completion time. Figure 3.12 shows the solution found using the D-TSP approach on graph with 15 waypoints from the far-start sinusoidal data set. In this example, at the beginning of the time window ($t = 0$) the GV began at position (0, 0). The D-TSP approach found a solution that begins at $t = 884.2$ and ends at $t = 1042.1$ while the k-TSP approach was unable to find any solution.

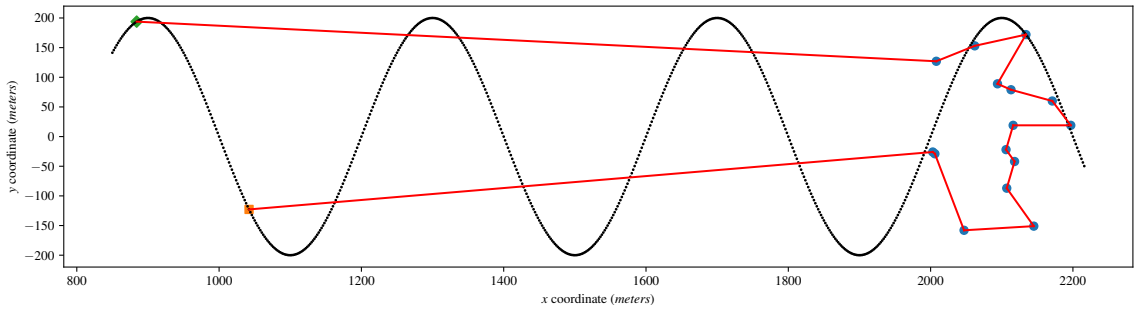
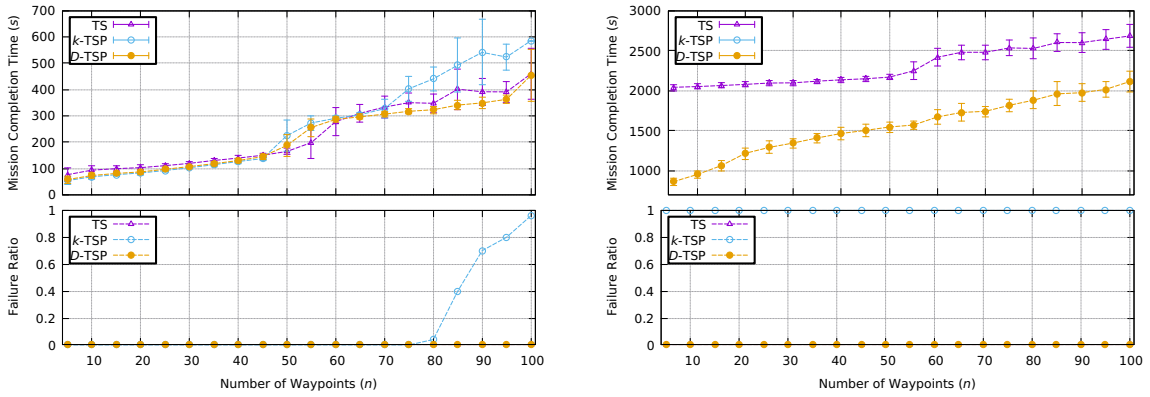


Figure 3.12 Drone path generated using D-TSP for a GV moving along a sinusoidal trajectory. The blue circles are the navigational waypoints, the dashed black line is the GV's trajectory (which moves from left to right and begins at position (0, 0)), the red lines are the drone's path where the green diamonds are drone launch points and the orange squares are drone receiving points.



(a) Near-start sinusoidal data set

(b) Far-start sinusoidal data set

Figure 3.13 Simulation results for dynamic solutions on inputs with sinusoidal GV trajectories. The top plots show the impact of the number of waypoints on mission completion time. The bottom plots show the ratio of graphs where each approach failed to find a solution.

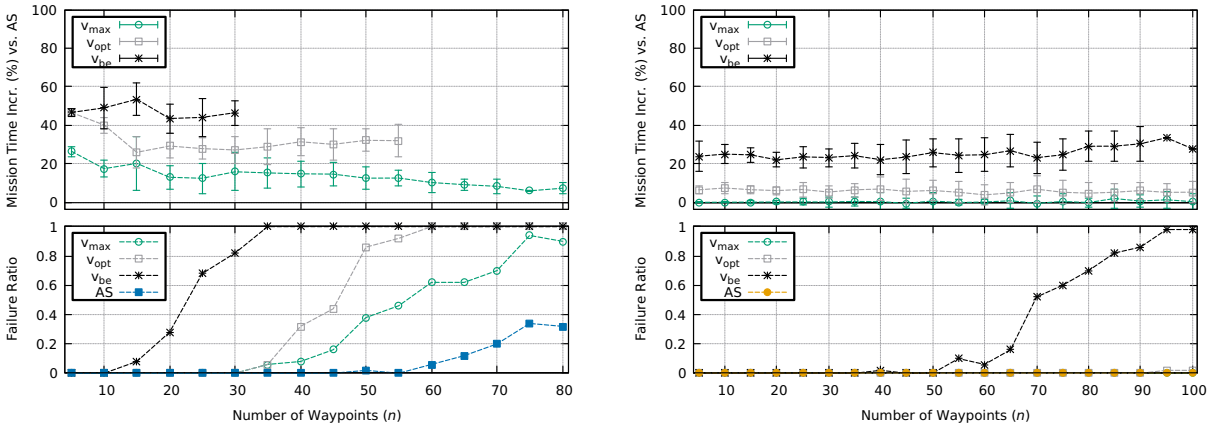
Figure 3.13 shows the results of these two test scenarios with Figure 3.13(a) showing the near-start results and Figure 3.13(b) showing the far-start results. On the near-start sinusoidal data set, the k-TSP struggled to compete against both the D-TSP and adaptive TS approaches, with the D-TSP improving mission completion time by an average of 7.8% compared to k-TSP. The D-TSP also improved mission

completion time by an average of 7.0% when compared against the TS approach. Similar to the larger linear data set, the k-TSP also struggled to find solutions as n grew above 80.

On the far-start sinusoidal data set, the k-TSP approach again failed to find a solution for all inputs because of the assumption that the first sub-tour should start at the beginning of the considered time horizon. Both the D-TSP and adaptive TS approaches were able to find solutions to all inputs with the D-TSP improving mission completion time by an average of 33.3% when compared against the adaptive TS approach.

3.7.4 Impact of Speed-Scheduling

We also evaluated how our approach to adaptive speed affected mission completion time. We compared both the k-IP and D-TSP approaches using adaptive speed (AS) against using these same algorithms when the drone's speed is fixed at v_{max}, v_{opt}, and the speed that minimizes energy consumption (termed best endurance, or v_{be}). We chose these different speed settings because they have all been proposed for drone path planning problems in recent literature [101, 105, 110]. Figure 3.14 shows the impact of using adaptive speed in path finding.



(a) Impact of speed scheduling for k-IP

(b) Impact of speed scheduling for D-TSP

Figure 3.14 Simulation results showing the impact of using adaptive speed in path finding approaches. The top plot of (a) shows the percent improvement that speed scheduling yields with the k-IP approach when compared to fixing the speed while (b) shows this impact using the D-TSP approach. The bottom plots for both (a) and (b) show the ratio of graphs where each approach failed to find a solution.

Figure 3.14(a) shows the results from using the k-IP algorithm on the same input set that was used to generate the data shown in Figure 3.8. In this data set, the GV moves along a linear trajectory and starts close to the waypoints. On average, the adaptive speed approach improved the mission completion time by 11.9%, 31.9%, and 47.1% compared against fixing the velocity at v_{max}, v_{opt}, and v_{be}, respectively. With

speed fixed at v_{max}, v_{opt}, and v_{be} the solver only found solutions for 69.3%, 52.5%, and 25.9% of the graphs, respectively, while using an adaptive speed approach found a solution for 93.4% of the inputs. In fact, when fixing speed at v_{be} no graphs were solved with $n \geq 35$.

Figure 3.14(b) shows the results of running the D-TSP approach on the far-start linear data set. Adaptive speed had negligible impact when compared to fixing speed at v_{max} while when compared to fixing the speed at v_{opt} adaptive speed only improved mission completion time by an average of 5.7% across all inputs. When compared against v_{be}, using adaptive speed improved the results by 25.5%. Using v_{be} again made it hard for our solution framework to find solutions as the number of waypoints grew but the D-TSP approach was able to find solutions for all inputs when using speed scheduling and when fixing speed at v_{max} and v_{opt}.

Table 3.3 Average computation time (in *seconds*) and 95% confidence interval (95% CI) for the D-MINLP (D-MP), f-MINLP (f-MP), k-IP, k-TSP, D-TSP and adaptive tour-splitting (a-TS) approaches with varying number of waypoints (n).

$n =$		5	10	15	20	30	40	50	60	70	80
d-MP	Avg.	0.24	77.1	79.3	**162.9**	-	-	-	-	-	-
	95% CI	0.039	59.7	53.0	127.8	-	-	-	-	-	-
f-MP	Avg.	0.08	0.73	14.4	135	**5139**	-	-	-	-	-
	95% CI	0.01	0.15	7.64	48.34	3215	-	-	-	-	-
k-IP	Avg.	0.02	0.02	0.03	0.05	0.08	0.13	0.19	0.23	0.30	**0.37**
	95% CI	0.002	0.002	0.004	0.006	0.010	0.011	0.017	0.027	0.032	0.033
k-TSP	Avg.	0.02	0.03	0.03	0.04	0.13	0.19	0.40	0.51	0.64	0.73
	95% CI	0.003	0.007	0.005	0.009	0.022	0.024	0.049	0.068	0.072	0.097
D-TSP	Avg.	0.06	0.07	0.08	0.09	0.16	0.18	0.29	0.40	0.45	**0.54**
	95% CI	0.005	0.005	0.007	0.008	0.015	0.018	0.026	0.038	0.047	0.045
a-TS	Avg.	0.003	0.005	0.006	0.008	0.018	0.04	0.05	0.06	0.08	0.12
	95% CI	0.0	0.001	0.002	0.002	0.003	0.005	0.004	0.010	0.015	0.022

3.7.5 Computational Efficiency

Table 3.3 shows the average and 95% confidence interval on computation time for randomized graph inputs on the various approaches. The f-MINLP results are from the data set used for Figure 3.8 and D-MINLP results come from the data set used for Figure 3.9, while the rest are from the near-start linear data set discussed in Section 3.7.3.2. This data only includes the results from graphs that each approach was able to find a feasible solution on.

The table shows that the computational time for both the D-MINLP and the f-MINLP increases exponentially with the number of waypoints, with the f-MINLP able to solve slightly larger inputs. As expected, the a-TS approach averaged the lowest computation time. Surprisingly, the k-IP approach was the second fastest despite having no guarantee to run in polynomial time, which we credit to the speed of

the Gurobi solver. Out of the solutions that are iteratively running the LKH solver, the D-TSP approach was slightly faster than the k-TSP. We believe this is because the D-TSP algorithm converges on a solution faster than the k-TSP does.

3.7.6 Summary of Findings

Our numerical simulations demonstrate various tradeoffs between our different considered approaches. We will first summarize the performance of the fixed-start algorithms introduced in Section 3.5 then look at when the fixed-start assumption should and should not be used. We will then summarize our findings on the generalized algorithms introduced in Section 3.6 and remark on the impact of speed scheduling for drone path finding.

Among the fixed-start solutions to the MT-OTM problem, the f-MINLP outperforms the other approaches in solution quality but at the cost of computation time, taking too long to reasonably solve inputs with 30 or more waypoints. The k-IP algorithm out performs the k-TSP and the fixed-start TSP-based baseline approach (TS) in both solution quality and in computation time. A downside to both the k-IP and k-TSP approaches is that they occasionally fail to find a solution in scenarios where the fixed-start assumption should otherwise work (a short-fall not as common in the TS approach). In our evaluation, we observed that the k-IP and k-TSP approaches struggle on inputs with sub-regions with a high concentration of waypoints while the rest of the graph is sparse. On these graphs, the clustering algorithm forms a single cluster with too many waypoints while the rest of the clusters are fairly small creating many short sub-tours with one very long tour that cannot be completed in a single flight. Computation time should also be considered when deploying the k-IP algorithm. Although our implementation of the k-IP algorithm has a reasonable computation time (averaging 0.37 seconds for inputs with 80 waypoints), this approach is still solving an instance of the TSP to optimality. On more limited hardware and with less powerful optimization tools, we expect the k-IP solution to struggle and recommend the k-TSP algorithm as an alternative.

Our results show that the fixed-start assumption is valid when the waypoints are located near the GV at the beginning of the considered time horizon. In fact, on smaller inputs where the waypoints were all located close to the GV, the f-MINLP and k-IP algorithms found comparable solutions to the D-MINLP algorithm (an exact method for the general MT-OTM problem) and the k-TSP algorithm averaged solutions that were within 4.5% of the optimal. These results show us that, in scenarios where the waypoints are located close to the GV's starting position, the fixed-start assumption is valid and the k-IP and k-TSP algorithms should be preferred. However, the fixed-start assumption begins to fail for inputs where the waypoints were located much further away from the GV's starting position. In such scenarios,

the fixed-start algorithms (k-IP and k-TSP) fail to find valid solutions.

Among the algorithms that remove the fixed-start assumption, we found that the D-TSP provides a nice balance between performance, flexibility, and computation time. The algorithm performed slightly worse than the fixed-start solutions (k-IP and D-TSP) on smaller inputs where the waypoints were all located close to the GV. However, when it is not clear when to launch the drone or when the waypoints are spread across a large area, the D-TSP algorithm's versatility helps it find valid solutions when the fixed-start assumption fails. The D-MINLP performs well as an exact algorithm when the number of waypoints is less than 20 but becomes too slow to solve after this point.

Although case dependent, our numerical simulation results also show that speed scheduling can have a major impact on performance. We found that adapting the drones speed did not perform any different from fixing the speed at the drone's max speed (v_{max}) when the waypoints are located further away from the GV. We believe that this is because the D-TSP algorithm is able to stretch each sub-tour out when the waypoints are far enough away from the GV's starting location, which removes the benefit of speed scheduling. However, on inputs where the waypoints are close to the GV's starting location, speed scheduling within the k-IP algorithm not only improved solution quality but also made it easier for the algorithm to find valid solutions. Among the fixed-speed approaches, fixing the drone's speed at v_{max} was the next best option in this scenario but still struggled to consistently find valid solutions at the same rate as our variable-speed approach. We also found that moving at the speed that minimizes energy consumption (v_{be}) performs particularly poor in both near-start and far-start scenarios. We hypothesize that on more structured path finding problems, such as the VRP where the base station is static and vehicles must swap out batteries, speed scheduling will improve performance in the same way that it improved the performance of the k-IP approach and recommend incorporating it into path finding algorithms when planning autonomous behavior for drones.

3.8 Problem Scenario Prototypes

In this section we demonstrate various prototype scenarios to demonstrate how the MT-OTM Problem could be applied in different environments and on physical hardware. We first give an example of the MT-OTM Problem in an urban environment. We then discuss our field test on a physical drone to show the practicality of our considered problem scenario.

3.8.1 Urban Environment Simulation

To demonstrate how our MT-OTM solution framework can be used in real-world scenarios, we apply it to an example in an urban environment where the drone must visit a set of waypoints in a city while the

GV follows a set route on city streets, i.e., not a straight line as in the previous simulation settings. We selected 20 waypoints for the drone to visit in the urban environment. We selected these points using Google Earth, then converted the GPS coordinates into relative distances and generated a graph for our MT-OTM solution framework to solve. The GV follows a series of city streets, moving at a constant 3 $\frac{m}{s}$.

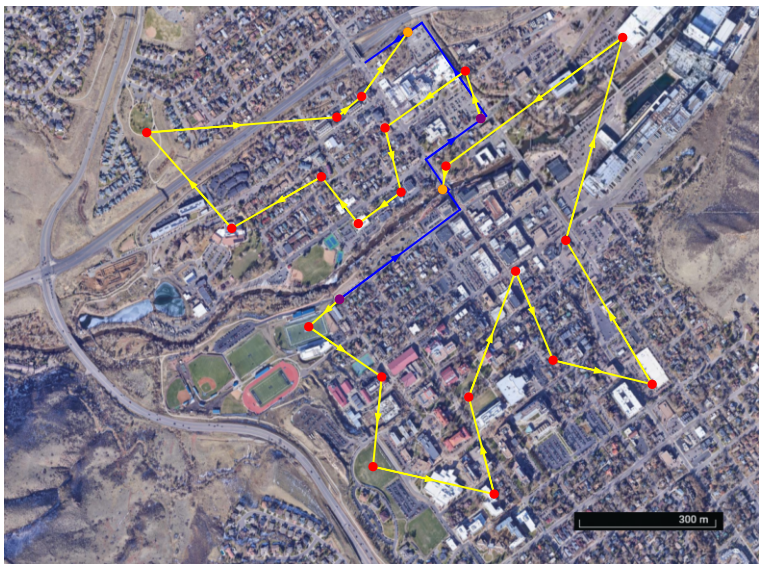
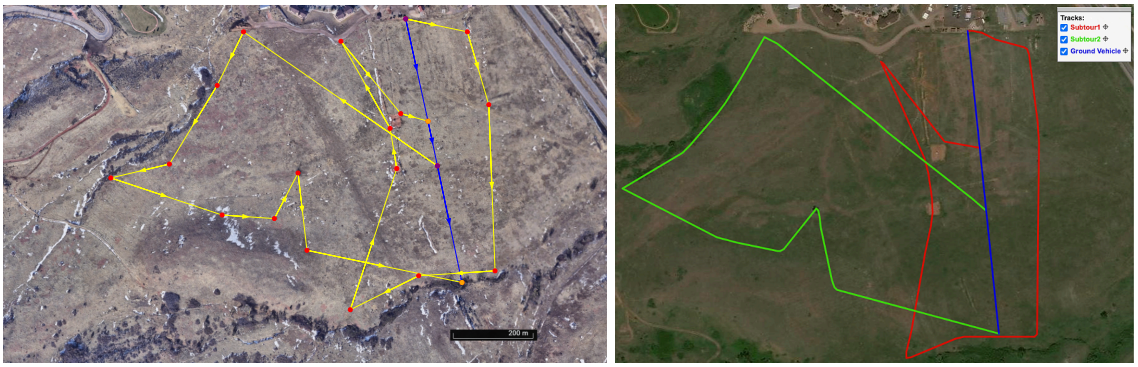


Figure 3.15 Drone path generated using f-MINLP approach for a case study in an urban environment. The red circles are drone waypoints, the purple circles are drone launch locations, and the orange circles are drone landing points. The yellow line shows the path of the drone and the blue line shows the path of the GV.

Figure 3.15 shows the drone path generated using the f-MINLP approach. It took the f-MINLP approach 43.1 seconds to find this solution. We argue that this demonstrates that for most common scenarios the number of waypoints will be small enough that the f-MINLP approach can be used to find superior solutions. The solution found using the f-MINLP has a mission completion time of 355.8 seconds while the k-IP finds a solution with 373.1 seconds.

3.8.2 Field Test

To further validate our solution framework we created a field prototype of the MT-OTM Problem using our own physical drone testbed [2]. We selected 18 waypoints in an empty field using Google Earth with the GV moving in a straight line across the field at 2.5 $\frac{m}{s}$. For simplicity, we landed the drone manually at the end of each sub-tour and substituted the GV by walking the path of the vehicle on foot. Because the drones in [2] can travel up to 8 km, we shortened the max flying distance to 1.7 km and set a max velocity of 11 $\frac{m}{s}$ to scale down the physical prototype. Figure 3.16 shows the details of our prototype and the outcome.



(a) Drone paths generated using k-IP (b) GPS trace of drone path from deployment

Figure 3.16 Field prototype results. Image (a) shows the drone path generated using k-IP approach and image (b) drone's GPS trace from our field experiment.

Figure 3.16(a) shows the drone path found using the k-IP approach. Figure 3.16(b) shows the GPS trace of the drone while following the found paths. The result on the physical prototype demonstrates that our approach works well when the MT-OTM problem is applied in real world scenarios.

3.9 Conclusions and Discussion

In this work, we formulated the Minimum-Time while On-The-Move (MT-OTM) Problem and presented several algorithms that solve this problem. This work builds on our previous conference proceedings [75].

We first looked at how to solve the MT-OTM Problem when assuming that the first drone sub-tour must start at the beginning of the considered time window and that all consecutive sub-tours must start as early as possible. This assumption allowed us to simplify the problem into an underlying fixed multi-depot, multi-terminal Hamiltonian paths problem (fixed-MdMtHPP). We developed two approaches for solving fixed-MdMtHPP, a MINLP (termed f-MINLP) that optimizes drone speeds and a k-means clustering algorithm paired with a TSP solver (the k-IP and k-TSP approaches).

We then looked at how to solve the MT-OTM Problem with the fixed-start assumption removed. We modified our f-MINLP formulation so that it selects the start and end location of each sub-tour (the D-MINLP solution), which provides an admissible algorithm to the MT-OTM Problem. We also provided a dynamic version of our k-TSP approach (the D-TSP approach) that dynamically sets sub-tour start and end locations and runs in polynomial time.

The main findings from this larger body of work are:

1. We presented a framework for drone path finding that allows a drone to be launched from a GV moving along a fixed trajectory.

2. Through numerical simulations, we found that the D-MINLP solution can solve smaller problem inputs to optimality but begins to struggle as the number of waypoints grows.

3. We showed that our k-IP approach is ideal when the user knows when to start the time horizon but is not as flexible for more general inputs.

4. We found that the D-TSP approach at times does not perform as well as the k-IP approach, but is much more flexible than the k-IP and was able to find a solution to a more diverse range of inputs.

5. We discovered a tradeoff between maximizing drone speed and maximizing the distance that a drone can travel.

6. We showed that drone speed adaptation can reduce mission completion time in well-structured problems and should be considered when planning paths for drones.

We believe that the tradeoff between speed and travel distance, leading to our speed scheduling approach within our algorithms, should be of particular interest to the research community and argue that our results support this position. Speed scheduling not only led to better quality solutions, but also allowed our algorithm to find solutions on inputs where following a fixed-speed approach failed to find a solution. Beyond speed scheduling, we also believe that our proposed solution framework provides a template for stakeholders to build off of when implementing planning algorithms for autonomous drones.

One possible application of this research is maritime search and rescue, where drones can be quickly launched from large ships to search for survivors after a disaster [77]. Another possible application is to further integrate our problem setup into a smart city environment [121]. The drone and GV could utilize static infrastructure to aide with communication [122], navigation [123], and battery charging [13]. In a smart city scenario, the drone could be delivering packages, monitoring traffic conditions, or aiding first responders [124] while the GV performs some other task within the city but allows the drone to ride hitch [78].

For future work, we plan to expand the MT-OTM problem to include multiple drones and multiple GVs. Variations of this multi-vehicle extension could include limiting the number of drones that can be at a GV, using the GV to ferry around drones, and considering repeated patrolling scenarios over long time horizons. We made the assumption that the drone does not need to stop and hover at each waypoint to collect data in this work (e.g. taking images while passing through a waypoint). Future work could expand on this application setup by requiring the drone to hover at each waypoint for some predetermined time period. Other expansions to the current work include addressing communication delays and online coordination between the drone and the GV, further investigating how to apply our general framework for

package delivery applications, and application-specific integration of the drone and GV into smart cities. Possible solutions to these open research questions include reinforcement learning and adaptive algorithms for online decision making.

More work is also needed to further validate this energy profile on a physical testbed and to study the probability distribution associated with our theoretical model. We believe that a probabilistic model for drone energy could be used to further improve the performance and accuracy of drone path finding algorithms by combining our speed scheduling approach with works such as [110] and [111] where they consider the risk of the drone running out of energy before completing its assigned task. Future work should also investigate if the tradeoff between maximizing speed and maximizing travel distance exists for other autonomous transportation systems, such as GVs, and determine how speed scheduling for these vehicle types impacts path finding.

CHAPTER 4

COLLABORATIVE PATH PLANNING OF ENERGY-SHARING DRONE-UGV TEAMS FOR PERSISTENT PATROLLING

Modified from a paper submitted to the *ACM Journal on Autonomous Transportation Systems*. This manuscript extends the work in [97], presented at the 24^{nd} International Conference on Autonomous Agents and Multiagent Systems.

Jonathan Diller[1,2], Jack Sigler[1], Qi Han[1]

Teams of unmanned ground vehicles (UGVs) and drones are often proposed for various applications in autonomous transportation, where drones quickly move from one point of interest to the next while UGVs act as moving base stations that can both recharge and ferry the drones around. In this paper, we look at how to plan collaborative actions among multiple teams of energy-sharing drones and UGVs for patrolling applications over an indefinite time horizon. We demonstrate how to form a second-order cone (SOC) program that finds optimal solutions, in polynomial time, when the order of drone and UGV actions are fixed. We propose an algorithm that uses various heuristics and our SOC program to find locally optimal solutions while considering the limited energy of both vehicle types. Additionally, we present a new nomenclature for classifying drone-UGV planning problems and propose a novel metric for evaluating multi-agent patrolling plans. We ran various numerical simulations using field data to evaluate our proposed approach. Our algorithm improves solution quality by up to 37.7% compared to a baseline method from the literature. Furthermore, we demonstrate the authenticity of our problem setup through a proof-of-concept experiment on a physical UGV and drone testbed.

4.1 Introduction

Mixed teams of multirotor unmanned aerial vehicles (drones) and unmanned ground vehicles (UGVs) form a symbiosis that extends the capabilities of each platform. Drones are fast and agile, allowing them to quickly traverse rugged terrain while visiting points of interest (PoI), but have limited payload and energy capacity. Unmanned ground vehicles can move heavy payloads, including larger energy stores, but tend to be slower than drones, must maneuver around ground obstacles, and may struggle to traverse rough terrain. When combined, the strengths of one platform help overcome the challenges of the other. The

[1]Department of Computer Science, Colorado School of Mines, USA
[2]Primary researcher and author
This work was supported in part by the Army Research Laboratory's Tactical Behaviors for Autonomous Maneuver Collaborative Research Alliance (W911NF-22-2-0235) and by the Journeyman Fellowship (W911NF-24-2-0140).

UGVs can act as mobile base stations for drones, recharging drone batteries and ferrying the drones to new deployment locations. The drones can quickly perform a variety of tasks such as package delivery, infrastructure inspection, or perimeter monitoring, among many other applications in autonomous transportation, while utilizing the UGVs as mobile base stations.

Pathfinding and rendezvous planning for drone-UGV teams is a well-studied area of interest within the community. Commonly considered problems include planning how to use UGVs to ferry around drones [10], selecting recharging rendezvouses [111], and launching drones off of non-stopping ground vehicles [74]. However, most previous works have failed to consider how to plan for long-term or indefinite deployments while accounting for the energy requirements of both platforms. Moreover, many existing works on this subject tend to simplify models for energy consumption and energy transfer from the UGV to the drone, or ignore energy constraints altogether.

Additionally, a large body of existing research focuses on minimizing the maximum latency that a PoI is visited, often referred to as minimizing the "worst latency" or "makespan." For example, if there are 10 PoIs and two robots, minimizing the worst latency would mean minimizing the time it takes for the last PoI to be visited. This metric puts emphasis on optimizing the actions of the robot that visits the last PoI but ignores what the other robot does, as long as it is not visiting the last PoI. However, we argue that prioritizing the actions of the robot with the worst latency does not measure the efficiency of all robots.

For a more concrete example that motivates our argument for a new metric, consider a scenario where robots must routinely inspect equipment at a petroleum refinery for hazardous air pollutant leaks [125, 126]. Ignoring a leak for too long will not only lead to a larger pollutant spill but may cause the leak to become worse over time. Therefore, the penalty for neglecting any individual inspection site should grow exponentially with time. Another example is routine package pickup and delivery, where robots must visit customers to make deliveries and check for outgoing packages. Just because one customer is much further away from the rest does not mean that we should only prioritize how fast we service the farthest away customer. In applications where a penalty can be assigned for neglecting any individual PoI, such as package handling, maintenance inspections, or security, we argue that minimizing the rate at which the penalty accrues over an indefinite time horizon should be preferred over minimizing the maximum latency.

We seek to fill these gaps in the existing literature. *In this work, we address the problem of planning patrolling routes over indefinite time horizons for energy-constrained drone and UGV teams, where UGVs can transfer energy to and ferry around hitchhiking drones.* This article extends our previous work in [97]. The contributions of our original work include:

1. a Second-Order Cone program for finding a local optimum in polynomial time for drone-UGV team monitoring problems,
2. an algorithm for solving persistent drone-UGV team monitoring problems that considers energy sharing and UGV energy constraints,
3. a taxonomy of drone-UGV team monitoring problems, and
4. a novel metric for evaluating solution quality in multi-agent patrolling problems.

We extend our previous work by:

1. considering ground obstacles for the UGVs and proposing a method for integrating obstacle avoidance into our algorithm (in Section 4.4.4),
2. adding a new action-swapping heuristic to our proposed algorithm to further refine solution quality (in Section 4.4.1), and
3. expanding our numerical evaluation through ablation experiments on more input data sets (in Section 4.5) and a new case study (in Section 4.6).

To promote good research integrity, we have made the source code for our algorithm and our data openly available to the public[3].

We provide a literature review on our considered topic in the following section. In Section 4.3, we formally define our problem and lay out our proposed solution to this problem in Section 4.4. In Section 4.5 we describe our numerical simulation setup and summarize our evaluation results. We present a case study based on a wind farm inspection scenario in Section 4.6 and in Section 4.7 we discuss our field prototype of the considered scenario and lessons learned from testing on physical hardware. Finally, we summarize our findings and provide closing remarks in Section 4.8.

4.2 Literature Review

In this section we define a new taxonomy for Drone-UGV routing problems and summarize common approaches for solving these problems.

4.2.1 Related Drone-UGV Problems

In this article, we look at a Drone-UGV teams monitoring problem where mixed drone-UGV teams must visit a set of PoIs. The specific application may vary widely, such as parcel handling, agriculture monitoring, border patrolling, sanitation, environmental monitoring, maintenance inspections, or perimeter

[3]https://github.com/JonD07/TeamPatrolling

patrolling, and may involve different types of tasks at each PoI, such as delivering and retrieving objects, taking images, taking a sensor measurement, or collecting data from wireless nodes. However, the fundamental planning problem remains the same: find paths that visit each PoI while optimizing some metric.

Variations of the PoI visiting problem can be classified based on a variety of factors. These factors include:

1. the number of times a PoI is visited (fixed visit requirement – **F**, or repeated visits indefinitely – **I**),
2. the considered metric (minimizing completion time – **T**, minimizing maximum latency – **L**, minimizing distance – **D**, or some other metric – **O**),
3. which vehicle can visit a PoI (drone only – **C**, or drone and UGV – **B**), and
4. what support the ground vehicle provides to the drone (energy transfer only – **E**, position transfer only – **P**, or both energy and position transfer – **H** – a.k.a hitchhiking).

Note that the difference between the T and L metrics is that minimizing completion time (T) considers the time required to visit *all* PoIs while minimizing maximum latency (L) considers the time to visit the single most neglected PoI. Metric T tends to occur when there is only a single robot (e.g., a single drone) that is visiting PoIs, while metric L tends to come up when there is more than one robot visiting PoIs in parallel (e.g., multiple drones), although these trends are not strictly followed. Examples of other, less common metrics (O) include time penalties [17] and average latency [127].

The majority of existing work falls within F:T variations of the problem. Works looking at minimizing completion time for fixed visits where only drones can visit PoIs (F:T:C) include [8, 10, 14, 74, 75, 128]. The natural extension to this scenario setup is to allow UGVs to also visit PoIs (F:T:B), as seen in [98, 110, 129–131]. Examples of distance as a metric instead of time (the F:D:C variation) are be found in [16, 132].

Work on indefinite PoI monitoring with drone-UGV teams, often termed patrolling, is more sparse. Examples where drones are responsible for visiting PoIs while minimizing completion time (I:T:C) are found in [11, 133] while minimizing maximum latency examples (I:L:C) are seen in [15, 134]. In [17], they consider a patrolling problem where both the drone and UGV can visit PoIs while using a penalty that is measured at the end of a fixed time period (I:O:B). There are also many examples of minimizing maximum latency in patrolling problems outside of drone-UGV teams, such as [7, 22, 36], which fall into the generalized multi-robot patrolling problem [135]. Yet another metric is found in [136], where they seek to maximize the number drones that each UGV can recharge in an I:O:C:E scenario.

How UGVs are used in path finding problems vary across both fixed visit and repeated visits scenarios. Works that only use the UGV for energy transfer (E) include [8, 11, 14, 16, 17, 129–131]. The more complex hitchhiking setup, where the UGV can both recharge and ferry about drones, is less common but seen in [10, 15, 75, 134].

In this work, we consider a variation of the Drone-UGV teams PoI monitoring problem where PoIs are visited indefinitely by drones that hitchhike off of the UGV while minimizing the rate that a penalty is accumulated for neglecting PoIs (I:O:C:H). To the best of our knowledge, we are the first to consider this variation of the Drone-UGV teams PoI monitoring problem. Perhaps the most similar work to ours is found in [15, 134], where they have a similar problem setup but are minimizing the maximum latency (I:L:C:H). Although minimizing maximum latency may be preferable for certain applications, we argue that this metric is less desirable in applications where a penalty can be assigned for neglecting any individual PoI. We prove that minimizing the maximum latency does not decrease the penalty accumulation rate and use the algorithm proposed in [134] as our baseline in this paper.

Furthermore, none of the summarized existing literature considers limited onboard energy for the UGV. This is particularly problematic for indefinite patrolling problems, where the UGV will eventually run out of energy. Additionally, all existing works assume that energy transfer from the UGV to the drone is completed at a fixed time interval. Although this assumption may be reasonable for scenarios where "energy transfer" refers to swapping batteries, the time required to recharge drone batteries will depend on how much energy was consumed by the drone prior to recharging. In this work, we consider the limited energy of both drones and UGVs and incorporate a model for wirelessly recharging drones on a UGV.

4.2.2 Path-Finding Algorithms

There are many approaches to solving the different variations of the Drone-UGV teams PoI monitoring problem. The most common approach is to first find routes for UGVs then build drone routes off of static UGV routes, as seen in [15, 16, 75, 130, 134], although some have considered UGV and drone routing as separate problems and focus on finding rendezvous points [14, 111, 131]. An example of planning routes together is found in [8].

Regardless of how these problems are broken up, these algorithms usually run iteratively [75, 129, 130], where different sub-problems are addressed individually on each iteration. The iterative approach commonly uses Integer Programming [8, 75, 129, 133], though Integer Programs do not run in polynomial time. A common method for separating the work done by different robots is to use a cluster algorithm, as seen in [10, 16] or other partitioning algorithms [15, 134]. Finding routes themselves is often treated as a Traveling Salesman Problem (TSP), for which there are well studied polynomial time solutions and

complete solutions [31].

Our proposed algorithm plans both drone and UGV routes together. We build off of the algorithm proposed in [15, 134], using this as a method to find an initial solution. We propose improving this initial solution by employing a SOC program, which optimizes both drone and UGV actions jointly. We then further refine the solution by replanning sub-parts of the problem using a TSP solver and utilizing an action-swapping heuristic, before solving the SOC program again. To the best of our knowledge, we are the first to propose combining an action-swapping heuristic with an SOC program for a multi-robot coordination problem.

4.3 Problem Formulation

We consider a problem where collaborative teams of UGVs and drones must repeatedly visit PoIs with the objective of reducing the time that any PoI is left unattended. For this problem, we are given m_g UGVs, m_a drones, and n PoIs that must be routinely visited by one (or more) of the drones. Let I be the set of all PoIs, R^a be the set of drones, and R^g be the set of UGVs, where $|I| = n$, $|R^a| = m_a$, and $|R^g| = m_g$. Each UGV has an assigned team of drones that can land and launch off of the UGV and recharge from the UGV. Let D_j be the set of drones assigned to UGV j. We assume that all D_j are fixed and leave mixing UGV-drone teams to future work.

In the following subsections we mathematically define a patrolling kernel and the penalty accrued for leaving a PoI unmonitored, discuss energy models for both drones and UGVs, including wireless energy transfer from a UGV to a drone, and formally define the Team-PITH Problem.

4.3.1 Patrolling Kernels

To define a patrolling route for each robot, we use sequences of actions that dictate what each vehicle does. We define action i as the tuple $a_i = \{(x_i, y_i), t_i, \varsigma\}$, where:

- (x_i, y_i) are the x, y-coordinates where the action takes place,
- t_i is the time that the action is completed, and
- ς represents the actions type, defined over a set of possible action types for each robot type.

For drones, action types include *take-off*, *visit-PoI*, *return-to-UGV*, and *land*. For UGVs, action types include *launch-drone*, *receive-drone*, *intermediate-waypoint*, *return-to-base*, and *swap-battery*. Let A^g_j be a sequence of actions for UGV j and A^a_k be a sequence of actions for drone k. Note that consecutive pairs of actions in an action sequence may require the robot to move from one location to the next and perform some procedure at that location. For example, a *launch-drone* action a_i at (x_i, y_i) position followed by a

receive-drone action a_{i+1} at a different (x_{i+1}, y_{i+1}) position will require the UGV to physically move from (x_i, y_i) to (x_{i+1}, y_{i+1}) and wait for the drone to land before action a_{i+1} can be considered complete. We say that an action sequence, A_j^g or A_k^a, is *valid* if every consecutive pair of actions in the action sequence adhere to the dynamic limitations of the robot and allow time for the actions type to be completed.

Let Θ be a set of ground obstacles. Without loss of generality, we represent obstacles as circles for convenience but any geometric shape could be used. Each obstacle $o \in \Theta$ has a corresponding position, (x_o, y_o), and radius r_o. Action $a_i \in A_j^g$ is said to be "*obstacle-free*" if there does not exist an obstacle $o \in \Theta$ where $\sqrt{(x_i - x_o)^2 (y_i - y_o)^2} \leq r_o$. A pair of consecutive UGV actions $a_i, a_{i+1} \in A_j^g$ are said to be "*pairwise-free*" if both a_i and a_{i+1} are *obstacle-free* and the straight line connecting a_i and a_{i+1} does not pass through any obstacle in Θ. Action sequence A_j^g is said to be "*collision-free*" if every consecutive action pair $a_i, a_{i+1} \in A_j^g$ is *pairwise-free*.

Each time a drone performs a *take-off* action followed by some number of *visit-PoI* actions, a *return-to-UGV* action, and a *land* action, it is termed a sortie (that is, a complete drone flight). We formally define a drone sortie as the tuple $Q_k = \{{}^s a_k,\ {}^e a_k, I_k\}$, where

- ${}^s a_k$ is the *take-off* action that starts the sortie,
- ${}^e a_k$ is the *land* action that ends the sortie, and
- I_k is a sequence of PoIs that the drone will visit along the sortie.

Each A_k^a can contain multiple Q_k.

We define a patrolling kernel as the set $S = \{A^g, A^a\}$, where

- A^g is a set of UGV action sequences, and
- A^a is a set of drone action sequences.

Observe that certain drone and UGV actions will be correlated. For example, if drone k is launching from UGV j, then there should be a *take-off* action $a_k \in A_k^a$ that corresponds with some *launch-drone* action $a_j \in A_j^g$ where $t_k = t_j$ for $t_k \in a_k,\ t_j \in a_j$. We say that S is *consistent* if *take-off* and *land* actions in A^a have a corresponding *launch-drone* and *receive-drone* action in A^g.

4.3.2 Penalty Accumulation Rate

In this work, we propose imposing a penalty for neglecting a PoI and increasing how much penalty is applied as time passes. The idea is to prevent the robots from ignoring a PoI for too long while also factoring in to the objective how often *all* PoIs are monitored.

Let the instantaneous penalty be linearly proportional to the units of time of the PoI latency. That is, for each hour that passes that we do not remove trash from a trash bin, the rate that we gain a penalty grows by one unit. The instantaneous penalty resets to zero each time a PoI is visited. An example of this is shown in Figure 4.1, where three PoIs are visited over varying intervals and the instantaneous penalty grows linearly with time. Suppose that a robot removed the trash every t_i hours. The total penalty that we gain over t_i hours will be $\int_0^{t_i} t\,dt = \frac{1}{2}t_i^2$. The total penalty gained at any given time is the area under the latency lines, which is the black line in Figure 4.2.

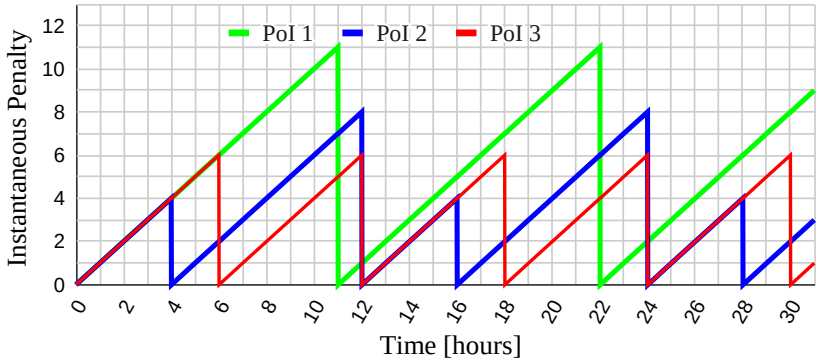


Figure 4.1 Examples of Instantaneous Penalty for PoIs.

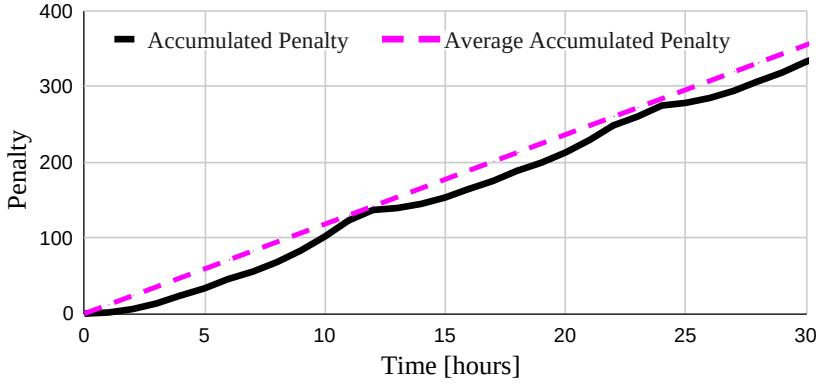


Figure 4.2 Point of Interest penalty accumulation example.

We chose hours as our unit of time because it lends itself well to something easily interpretable by humans (removing trash every 5 hours is more tangible than saying every 18,000 seconds), but one could switch to any other unit of time (seconds, days, galactic-years) as is appropriate for the application and the following math would still apply.

Given patrolling kernel S, we define a PoI visit sequence as $S_i^v = \{t_1^i, t_2^i, \cdots, t_L^i\}$ for each PoI $i \in I$, where each t_l^i is the time lapsed in hours from the beginning of S until i is visited by a drone in an action

sequence in S for the l^{th} time, for $l \in \{1, 2, \cdots, L\}$. We assume that the patrolling kernel S, and all S_i^v, will be repeated indefinitely. From visit sequence S_i^v, we can derive the latency sequence of kernel S as $S_i^L = \{t_1^i, (t_2^i - t_1^i), \cdots, (t_L^i - t_{L-1}^i)\}$, which gives us the latency intervals that i is visited in. From Figure 4.1, $S_1^L = \{11\}$, $S_2^L = \{4, 8\}$ and $S_3^L = \{6\}$.

The penalty accumulated for a single PoI by the end of a latency sequence will be

$$F_p(S_i^L) = \sum_{t_l \in S_i^L} \frac{(t_l)^2}{2}. \tag{4.1}$$

The average rate that penalty across all PoIs is accrued as patrolling continues indefinitely will then be

$$PAR(S) = \sum_{i \in I} \frac{1}{t_L^i} F_p(S_i^L). \tag{4.2}$$

We term (4.2) the Penalty Accumulation Rate (PAR). From Figure 4.1, $PAR(S) = (\frac{11^2}{2})(\frac{1}{11}) + (\frac{4^2+8^2}{2})(\frac{1}{12}) + (\frac{6^2}{2})(\frac{1}{6}) = 11.833$, which is the slope of the pink dashed line in Figure 4.2.

Using PAR as a metric gives us a way to check how well a given S addresses all PoIs, as opposed to using minimizing the maximum latency, which does not tell us how well a patrolling kernel is monitoring the non-maximum latency PoIs. In fact, we can make a strong distinction between these two metrics:

Theorem 4: *Minimizing the maximum latency across all PoIs does not always reduce PAR.*

Proof. By example, suppose that we found a new patrolling kernel S' with latency sequences $S_1^L = \{10\}$, $S_2^L = \{10\}$ and $S_3^L = \{8\}$. Kernel S' has a maximum latency of 10 while S has a maximum latency of 11. However, $PAR(S') = 14 \not< PAR(S) = 11.833$. □

4.3.3 Robot Energy Models

To formulate our patrolling problem, we require accurate models for UGV energy consumption, drone energy consumption, and wireless energy transfer from a UGV to a drone.

Energy models for UGVs: Previous works have shown that the rate that a UGV consumes energy (in watts) depends on the speed of the vehicle and can broadly be modeled as

$$\mathcal{P}_g(v) = c_1 v + c_2, \tag{4.3}$$

where v is the speed that the vehicle is moving at (in meters per second), and c_1 and c_2 are constants. An example of this model is found in [64], where they experimentally found $c_1 = 464.8$ and $c_2 = 156.3$ for a standard Clearpath™ Warthog UGV.

Energy models for drones: Similarly, works such as [4, 5] have shown that the power consumption on drones also depends on the speed of the drone and can generally be approximated as

$$\mathcal{P}_a(v) = c_3v^3 + c_4v^2 + c_5v + c_6, \tag{4.4}$$

where c_3, c_4, c_5 and c_6 are again constants. An example of this model is found in [63] where they experimentally determined these values for a custom build hexacopter. We experimentally found $c_3 = 0, c_4 = 0, c_5 = -1.695$ and $c_6 = 396.74$ on the custom built drone found in Section 4.7. In addition to Equation (4.3), we assume that launching and receiving a drone will consume a constant amount of energy. Let ${}^{l}j_k$ and ${}^{r}j_k$ be the energy required to launch and receive drone j, respectively.

Wireless energy transfer from UGVs to drones: Unmanned ground vehicles can hold significantly larger volumes of usable energy when compared to drones, making them ideal for sharing stored energy with their energy-limited drone teammates. To charge a drone's battery, we charge at constant current up to a "crossover point", E^* (in joules), where energy transfer slows down. After reaching E^* jules, we apply a constant voltage until the battery is full. Therefore, the relationship between energy transferred per unit of time can be modeled as

$$E_t(t) = \begin{cases} c_7t^2 + c_8t, & 0 \leq t \leq t^* \\ E^* + \frac{P^*}{\alpha}(1 - e^{-\alpha(t-t^*)}), & t^* \leq t \leq t_{max}, \end{cases} \tag{4.5}$$

where t is the charge time in seconds, t^* is the time required to reach the crossover point E^*, P^* is the power level at E^*, and α, c_7 and c_8 are all constants determined by battery and hardware characteristics.

From Equation (4.5), we find that the time required to recharge a drone after consuming E joules will be:

$$t_c(E) = \begin{cases} \frac{-c_8+\sqrt{c_8^2+4c_7E}}{2c_7}, & E_{min} \leq E < E^* \\ t^* - \frac{ln\left(1+\frac{\alpha}{P^*}(E^*-E)\right)}{\alpha}, & E^* \leq E \leq E_{max}, \end{cases} \tag{4.6}$$

where E_{min} is the minimum allowable energy level that the battery can safely be discharged to and E_{max} is the capacity of the battery. We expect to lose γ percent of energy in the wireless transfer process.

Applying Energy Models: As an example on how to apply these equations, suppose that drone k performs sortie Q_k and moves at speed v_k. The drone will consume

$$\mathbf{J}_a(Q_k) = {}^{l}j_k + \frac{d_k}{v_k}\mathcal{P}_a(v_k) + {}^{r}j_k \tag{4.7}$$

joules of energy, where d_k is the total distance required to travel from the *take-off* action to each PoI in I_k then return to the *land* action in Q_k. It will then take $t_c\big(\mathbf{J}_a(Q_k)\big)$ seconds to recharge the drone on the UGV and the UGV will lose $(1+\gamma)\mathbf{J}_a(Q_k)$ joules from its own battery. We say that drone action sequence A_k^a is *safe* if for all consecutive sorties $Q_k, Q_{k+1} \in A_k^a$, sortie Q_{k+1} does not begin until $t_c\big(\mathbf{J}_a(Q_k)\big)$ seconds

after sortie Q_k ends. That is, A_k^a is *safe* if we fully recharge the drone before launching it again.

Let $\bar{\jmath}_k$ be the maximum usable energy stored on drone k. Each second that the drone is operating it will consume $\mathcal{P}_a(v_k)$ joules per second and can operate for

$$\bar{t}_k = \frac{\bar{\jmath}_k - ({}^{l}j_k + {}^{r}j_k)}{\mathcal{P}_a(v_k)} \tag{4.8}$$

seconds and travel $\bar{d}_k$ meters. If the drone consumes all usable energy on-board, it will take ${}^{mc}t_k = t_c(\bar{\jmath}_k)$ seconds to recharge the battery completely.

In a similar manner, the energy consumed by UGV j after completing action sequence A_j^g (excluding energy transferred to drones), will be

$$\mathbf{J}_g(A_j^g) = \frac{d_j}{v_j}\mathcal{P}_g(v_j), \tag{4.9}$$

where d_j is the total travel distance to complete the actions in A_j^g and v_j is the speed of the UGV. Let $\bar{\jmath}_j$ be the maximum usable energy stored on UGV j.

We acknowledge that these energy models abstract other factors in energy consumption, such as environmental conditions or payload [114]. However, we argue that these aspects could be included into the models without changing the fundamental approach.

4.3.4 Formal Problem Definition

Given m_g UGVs, m_a drones, and n PoIs, find a patrolling kernel S where

$$\min \ PAR(S) \tag{4.10}$$

subject to:

$$S \text{ is } \textit{consistent} \tag{4.11a}$$
$$S \text{ is } \textit{valid} \tag{4.11b}$$
$$\forall A_j^g \in S,\ A_j^g \text{ is } \textit{collision-free} \tag{4.11c}$$
$$\mathbf{J}_g(A_j^g) + \sum_{k \in D_j} (1+\gamma)\mathbf{J}_a(A_k^a) \leq \bar{\jmath}_j, \qquad \forall A_j^g \in S \tag{4.11d}$$
$$\mathbf{J}_a(A_k^a) \leq \bar{\jmath}_k, \qquad \forall A_k^a \in S \tag{4.11e}$$
$$A_k^a \text{ is } \textit{safe}, \qquad \forall A_k^a \in S. \tag{4.11f}$$

We term this the Team Patrolling over Indefinite Time Horizons (Team-PITH) Problem.

We can show that Team-PITH is $\mathcal{NP}$-Hard through a reduction from the TSP, a well-known $\mathcal{NP}$-Hard problem [30].

Theorem 5: *The Team-PITH problem is $\mathcal{NP}$-Hard.*

Proof. Given graph $G = \{V, E\}$ as an input to the TSP, where V is a set of vertices and $E = V \times V$ is a set of edges, we can reduce the TSP to the Team-PITH problem via the following procedure: Set $I = V$, place the base station at the center of V, and $m_g = m_a = 1$ (a single UGV and a single drone). Set $\bar{j}_k$, the usable energy onboard the drone, to an arbitrarily high value and set the UGV speed to 0. After solving the Team-PITH problem, $A^a \in S$ will contain a closed cycle over the points in I, which provides a solution to the original TSP. Because TSP is $\mathcal{NP}$-Hard, and we provided a polynomial time reduction from TSP to the Team-PITH problem, the Team-PITH problem must also be $\mathcal{NP}$-Hard. □

4.4 Proposed Algorithms

In this section we introduce the Launch Optimizer with Rescheduling and Swapping (LORS) algorithm for the Team-PITH problem. This algorithm utilizes related work to find an initial solution and then builds on our algorithms in [97], along with newly proposed heuristics, to improve solution quality while avoiding obstacle collisions. We will discuss each of these individual parts in detail in the subsequent subsections. We also provide a time complexity analysis on these algorithms.

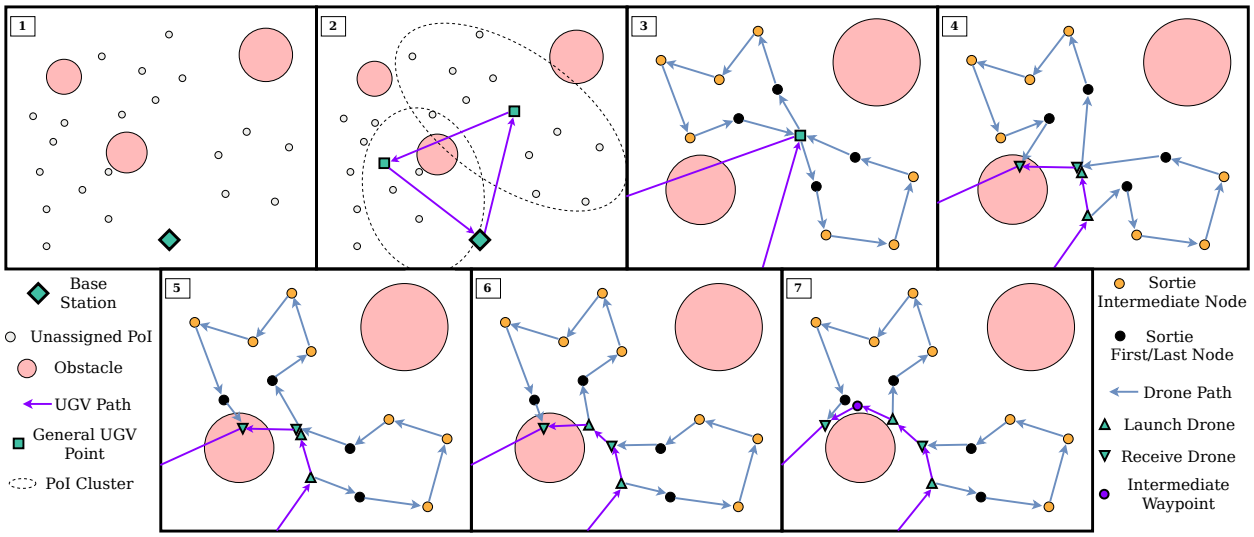


Figure 4.3 Example of the LORS algorithm running on a simple input.

4.4.1 Launch Optimizer with Rescheduling and Swapping Algorithm

In general, the LORS algorithm takes an initial solution (Figure 4.3, panels 1-3), optimizes drone launch and land locations across each UGV tour using a SOC program (Figure 4.3, panel 4), and then attempts to further improve the solution by replanning drone sorties (Figure 4.3, panel 5), swapping overlapping actions (Figure 4.3, panel 6), and finding routes around obstacles using a local path planner (Figure 4.3, panel 7).

Listing 4 lays out the details of the LORS algorithm. The algorithm finds an initial solution, S, using the *Initial-Solution()* function on line 1. We propose using the method presented in [15, 134], while ignoring obstacles, to find an initial solution, though any method for finding a *consistent*, *valid*, and *collision-free* S may be used. We then make a first attempt to improve the initial solution using a SOC program in the *Launch-Optimizer()* function while ignoring obstacles. The details of the *Initial-Solution()* and *Launch-Optimizer()* functions are further discussed in Sections 4.4.2 and 4.4.3, respectively.

Algorithm 4 Iterative Launch Optimizer Algorithm

Input: PoI set I, drone set R^a, UGV set R^g, drone-to-UGV set D_j
Output: Patrolling kernel S

1: $S \leftarrow$ *Initial-Solution*(I, R^a, R^g, D_j)
2: $S \leftarrow$ *Launch-Optimizer*(S)
3: **for each** $j \in R^g$ **do**
4: **for each** $k \in D_j$ **do**
5: $S \leftarrow$ *Update-Subtour*(S, k)
6: **end for**
7: $S \leftarrow$ *Action-Swap*(S, j)
8: **while** A_j^g is not *collision-free* **do**
9: $S, Z_j \leftarrow$ *Local-Planner*(S, j)
10: $S \leftarrow$ *LO-Obstacles*(S, Z_j)
11: **end while**
12: **end for**

For each UGV $j \in R^g$, we attempt to improve the ordering of PoI visit actions of each drone k assigned to UGV j in the *Update-Subtour()* function on line 5. Observe that we can form a closed cycle out of each drone sortie Q_k by connecting the *take-off* and *land* locations. We can treat Q_k as a TSP instance by setting the set $V = \{{}^s a_k,\ {}^e a_k\} \cup I_k$ as vertices, $E = V \times V$ as edges and fixing the edge $({}^e a_k,\ {}^s a_k)$ to be in the solution. The *Update-Subtour()* function improves each set of drone actions by transforming each sortie into this special instance of a TSP and solving it. We use the polynomial time TSP solver presented in [109], but any user-preferred algorithm for the TSP can be used [137].

After running the *Update-Subtour()* function for each drone k assigned to UGV j, we attempt to further improve solution quality by swapping actions in the *Action-Swap()* function on line 7. This function cycles through consecutive action pairs $a_i, a_{i+1} \in A_j^g$. If $(x_i, y_i) = (x_{i+1}, y_{i+1})$, then the function swaps their order in A_j^g and runs the *Launch-Optimizer()* function to determine if the swap lowered $PAR(S)$, only keeping swaps that improve solution quality. The motivation for this heuristic is that the *Launch-Optimizer()* function will often place consecutive actions on top of one another, creating a "snagged" look (see the overlapping actions in Figure 4.3, panels 4 and 5). This suggests that the optimizer would prefer to swap the ordering of these actions. Note that ground obstacles are ignored on lines 1 through 7.

After running the *Action-Swap()* function, we check if A_j^g is *collision-free* on line 8. While A_j^g is not *collision-free*, we run the *Local-Planner()* and *LO-Obstacles()* functions to handle obstacle avoidance. The *Local-Planner()* function pushes actions out of obstacles (making them *obstacle-free*) and finds paths through free-space for each $a_i, a_{i+1} \in A_j^g$ pair that is not *pairwise-free* while the *LO-Obstacles()* function is used to refine the timing and position of actions. Further details on the *Local-Planner()* function and how to integrate obstacle avoidance in the *LO-Obstacles()* function is discussed in Section 4.4.4.

4.4.2 Finding an Initial Solution

We use a modified version of the algorithm in [15, 134] to find an initial solution. The original algorithm runs as follows: (1) break up the environment into sub-regions using predetermined region sizes, (2) order sub-regions using a TSP solver, (3) order and divide discrete nodes by their quadrant angles from the centroid of the sub-region, and (4) order nodes for each drone using a TSP solver. We modify this approach by using a VRP solver for steps 1 and 2 then use a VRP solver in each sub-region to accomplish steps 3 and 4. We also incorporate UGV energy limitations into the algorithm.

Algorithm 5 Initial-Solution Algorithm

Input: PoI set I, drone set R^a, UGV set R^g, drone-to-UGV set D_j
Output: Patrolling kernel S
1: $m' \leftarrow |R^g|,\ c \leftarrow m'$
2: $c \leftarrow \max(c, m')$
3: $C \leftarrow k\text{-}means(c)$
4: $Q^g \leftarrow VRP(C, m')$
5: **for each** $j \in R^g$ **do**
6: **for each** cluster C' assigned to j in Q^g **do**
7: $Q^a \leftarrow VRP(C', D_j)$
8: **if** $\exists Q_k \in Q^a$ such that $\text{dist}(Q_k) > \bar{d}_k$ **then**
9: $c \leftarrow c + 1$, repeat steps 2–8
10: **end if**
11: **end for**
12: **end for**
13: **if** $\exists Q_j \in Q^g$ such that $\text{energy}(Q_j) > \bar{d}_j$ **then**
14: $m' \leftarrow m' + |R^g|$, repeat steps 2–13
15: **end if**
16: $A^g, A^a \leftarrow Form\text{-}Actions(Q^g, Q^a)$
17: **return** $S = \{A^g, A^a\}$

Listing 5 provides details of the *Initial-Solution()* function. The algorithm forms $c = m_g$ clusters using Lloyd's k-means clustering algorithm on line 3. After finding PoI clusters C, the algorithm treats the centroids of the PoI clusters in C as stops for a Vehicle Routing Problem with $m' = m_g$ vehicles [138], which is solved using the *VRP()* function on line 4. To implement a VRP solver in the *VRP()* function, we further divided the PoI centroids into m' clusters and then solve a TSP on each cluster using the Lin-Kernighan-Helsgaun TSP solver [109], though this method could be swapped out with other VRP

solvers such as [139] and the rest of the *Initial-Solution()* function algorithm would remain the same. Figure 4.3 shows an example of the PoI clustering and VRP for the UGV steps in panel 2.

For each cluster C' assigned to UGV j, the algorithm solves another VRP on line 7 that sends each drone assigned to j to visit the PoI in C' using the centroid of the cluster as the depot for the VRP (where the UGV will wait while each drone visits PoI in C'). After finding a set of drone sorties Q^a for cluster C', the algorithm verifies that none of the drone sorties $Q_k \in Q^a$ violate drone energy constraints. If there is a drone sortie that exceeds the drone's energy limit, the algorithm increments c and restarts. Figure 4.3 demonstrates finding drone sorties on a PoI cluster while the UGV waits at the cluster centroid in panel 3. If there exists a UGV tour that runs a UGV out of energy, then the algorithm increments m' by m_g and restarts. Otherwise, the algorithm forms action sequences that form a patrolling kernel to the Team-PITH problem.

4.4.3 Optimizing Launch and Land Actions

The *Launch-Optimizer()* function is given sets of actions for a drone-UGV team and optimizes the location of each of these actions via a SOC program while holding a set of assumptions. The function returns optimal action locations, within the limitations of the assumptions. These assumptions are:

1. the order of drone launch and land actions is known a priori,
2. the drones must recharge completely between consecutive sorties,
3. vehicle position accuracy when visiting any given PoI or the base station can be relaxed to some constant ϵ, and
4. the travel distance limits of any vehicle can be relaxed by ϵ.

The *Initial-Solution()* function assigns an ordering to each drone sortie, giving us assumption 1. Our *Initial-Solution()* function pushes each drone as close as it can to the limit of their operating distance, meaning that each drone is expected to need a near-full recharge, validating assumption 2. Furthermore, standard commercially available GPS have limited accuracy, validating assumption 3, and the distance that each vehicle will be traveling is far enough that assumption 4 should not impact the validity of a solution.

Table 4.1 lists decision variables. Given a patrolling kernel S, we formulate a SOC program to optimize the action list of each UGV j and the corresponding drones assigned to j, given in D_j, using the following variables: For each sortie $Q_k \in S$, where $k \in D_j$, we create continuous variables $(\mathbf{x}_k, \mathbf{y}_k)$ and $(\mathbf{x}_{k'}, \mathbf{y}_{k'})$ for the location of the launch and land actions, ${}^s a_k$ and ${}^e a_k \in Q_k$, respectively. Let set $\tilde{A}_j$ be the collection of all corresponding launch and land actions, ${}^s a_k$ and ${}^e a_k$, in S for all $k \in D_j$. Let $({}^s\mathbf{x}_k,\ {}^s\mathbf{y}_k)$ and $({}^e\mathbf{x}_k,\ {}^e\mathbf{y}_k)$

be the location of the first and last PoI in $I_k \in Q_k$, respectively, which are continuous variables that we constrain to stay within an $\epsilon \times \epsilon$ box centered on their respective PoI. Let ${}^{t}d_k$ be the distance required to traverse sequence $I_k \in Q_k$, which we assume to be constant. Let continuous variables ${}^{s}\mathbf{d}_k$ and ${}^{e}\mathbf{d}_k$ represent the distance from the launch action, ${}^{s}a_k$, and the land action, ${}^{e}a_k$, to the first and last PoIs in I_k, respectively. Let ${}^{a}t_i$ be the time required to perform action $a_i \in A_j^g$ and t_r be the time required for a drone to land, both of which are constant. We use variable $\mathbf{t}_i$ to track the end time for each action $a_i \in A_j^g$. Let variable $\mathbf{t}_b$ specifically be the time the "end" action is completed (the time that the UGV returns to the base station). Let d_j be the distance from a_i to a_j, for consecutive actions a_i and $a_j \in A_j^g$.

Table 4.1 Decision Variables for Optimizing Launch and Land Actions

Variable	Range	Description
$(\mathbf{x}_k, \mathbf{y}_k)$	$\mathbb{R}$	x, y-coordinate of the launch action for sortie k
$(\mathbf{x}_{k'}, \mathbf{y}_{k'})$	$\mathbb{R}$	x, y-coordinate of the land action for sortie k
$({}^{s}\mathbf{x}_k,\ {}^{s}\mathbf{y}_k)$	$\mathbb{R}$	x, y-coordinate of first PoI on sortie k
$({}^{e}\mathbf{x}_k,\ {}^{e}\mathbf{y}_k)$	$\mathbb{R}$	x, y-coordinate of first PoI on sortie k
${}^{s}\mathbf{d}_k$	$\mathbb{R}_{\geq 0}$	Distance of first leg of on sortie k
${}^{e}\mathbf{d}_k$	$\mathbb{R}_{\geq 0}$	Distance of last leg of on sortie k
$\mathbf{t}_i$	$\mathbb{R}_{\geq 0}$	End time of action i
$\mathbf{t}_b$	$\mathbb{R}_{\geq 0}$	Time that the UGV returns to the base station

We formulate a SOC program to optimize the location of the launch and land actions as follows:

$$\min \mathbf{t}_b \tag{4.12}$$

Subject to

$$\mathbf{t}_i \geq \mathbf{t}_j + {}^{mc}t_k, \quad \forall \text{ launch } {}^{s}a_i \text{ after land } {}^{e}a_j \in \tilde{A}_j \tag{4.13a}$$

$$\mathbf{t}_{i+1} \geq \mathbf{t}_i + \frac{1}{v_g} d_{i+1} + {}^{a}t_{i+1}, \quad i \in \{1, 2, \cdots k-1\} \tag{4.13b}$$

$$\mathbf{t}_j = \mathbf{t}_i + \frac{1}{v_k}({}^{s}\mathbf{d}_k + {}^{t}d_k + {}^{e}\mathbf{d}_k) + t_r, \quad \forall\{ {}^{s}a_i,\ {}^{e}a_j\} \in Q_k \tag{4.13c}$$

$${}^{s}\mathbf{d}_k^2 \geq (\mathbf{x}_k - {}^{s}\mathbf{x}_k)^2 + (\mathbf{y}_k - {}^{s}\mathbf{y}_k)^2, \quad \forall Q_k \in S, k \in D_j \tag{4.13d}$$

$${}^{e}\mathbf{d}_k^2 \geq (\mathbf{x}_{k'} - {}^{e}\mathbf{x}_k)^2 + (\mathbf{y}_{k'} - {}^{e}\mathbf{y}_k)^2, \quad \forall Q_k \in S, k \in D_j \tag{4.13e}$$

$$d_k^2 \geq (\mathbf{x}_k - \mathbf{x}_{k-1})^2 + (\mathbf{y}_k - \mathbf{y}_{k-1})^2, \quad \forall Q_k \in S, k \in D_j, k > 1 \tag{4.13f}$$

$$\bar{d}_k \geq {}^{s}\mathbf{d}_k + {}^{t}d_k + {}^{e}\mathbf{d}_k, \quad \forall Q_k \in S, k \in D_j \tag{4.13g}$$

For each drone sortie, constraint (4.13a) does not allow a drone to be launched on a consecutive sortie until after ${}^{mc}t_k$ seconds – the time required to fully recharged the drone's battery (enforcing assumption 2). Constraint (4.13b) prevents action a_{i+1} from completing until after action a_i completes plus the time required for the UGV to move from a_i to a_{i+1} and perform action a_{i+1}. Constraint (4.13c)

forces the end time of a drone sortie to be equal to the end time of the sortie's launch action plus the time required for the drone to traverse the distance of the sortie and land again. Constraints (4.13d) and (4.13e) control the distance of the start and end leg of each sortie, respectively, while constraint (4.13f) controls the total distance from one action to the next in A_j^g. Finally, constraint (4.13g) enforces each drone's energy constraint.

Observe that constraints (4.13d), (4.13e), and (4.13f) are convex quadratic constraints while all other constraints and the objective are linear. Therefore, Objective (4.12) with constants (4.13) form a SOC program, which can be solved in polynomial time to optimality using interior point [140]. This means that the LORS algorithm can solve for (4.12) in polynomial time to find a locally optimal solution to the Team-PITH problem.

4.4.4 Obstacle Avoidance

The general strategy behind our obstacle avoidance method is to push actions out of obstacles, find free-space paths between actions by generating *intermediate-waypoint* actions, and then bound these actions into convex hulls. The first two steps of this process are handled by the *Local-Planner()* function and the final step is done in the *LO-Obstacles()* function.

The *Local-Planner()* function first cycles through each launch and land action $a_j \in A_j^g$, for UGV j. In the case where a_j is a *launch-drone* action, let a_k be the first *visit-PoI* action for the drone being launched. If a_j is a *receive-drone* action, then let a_k be the last *visit-PoI* action for the associated drone. If a_j is not *obstacle-free* (that is, a_j lies within some obstacle $o \in \Theta$), then we push a_j towards a_k until the action is no longer overlapping an obstacle. This design is meant to reduce the distance that the drone must travel to rendezvous with the UGV. Our motivation for this decision (as oppose to reducing UGV travel distance) is because the drones tend to move much faster than the UGVs, causing the SOC program to push each drone sortie to the max allowable distance. Forcing the drone to travel any further could lead to an infeasible solution that runs the drone out of energy.

After every action $a_j \in A_j^g$ is *obstacle-free*, we check that every pair of consecutive actions, $a_j, a_{j+1} \in A_j^g$, is *pairwise-free* (that is, the straight line between a_j and a_{j+1} does not bisect an obstacle). If a_j, a_{j+1} are not *pairwise-free*, then we find a path through clear space from a_j to a_{j+1} and insert *intermediate-waypoint* actions ${}^i a_j^1, \ldots, {}^i a_j^k$ between a_j and a_{j+1} in A_j^g. There are many well studied solution from finding a path through free space [141], such as RRT* [142], which is asymptotically optimal.

While performing the above, we create set Z_j that holds action-obstacle pairs (a_j, o). These action-obstacle pairs form an association between each UGV action a_j that was moved out of an obstacle and the corresponding obstacle o. These actions can be any *launch-drone*, *receive-drone*, or

intermediate-waypoint action that is modified by the *Local-Planner()* function.

The *LO-Obstacles()* function solves the same SOC program described in Section 4.4.3 for function *Launch-Optimizer()* but with added constraints to prevent actions in Z_j from reentering an obstacle. These constraints form a convex hull that allow us to maintain convexity in the SOC program while also preventing troublesome actions from entering obstacles. We form these convex hulls by adding two types of new constraints to our optimization problem: squares and action-to-action constraints. The square constraints are added to all action types while the action-to-action constraints are only used for *intermediate-waypoint* actions. Figure 4.4 gives an example of how we generate each of these constraints.

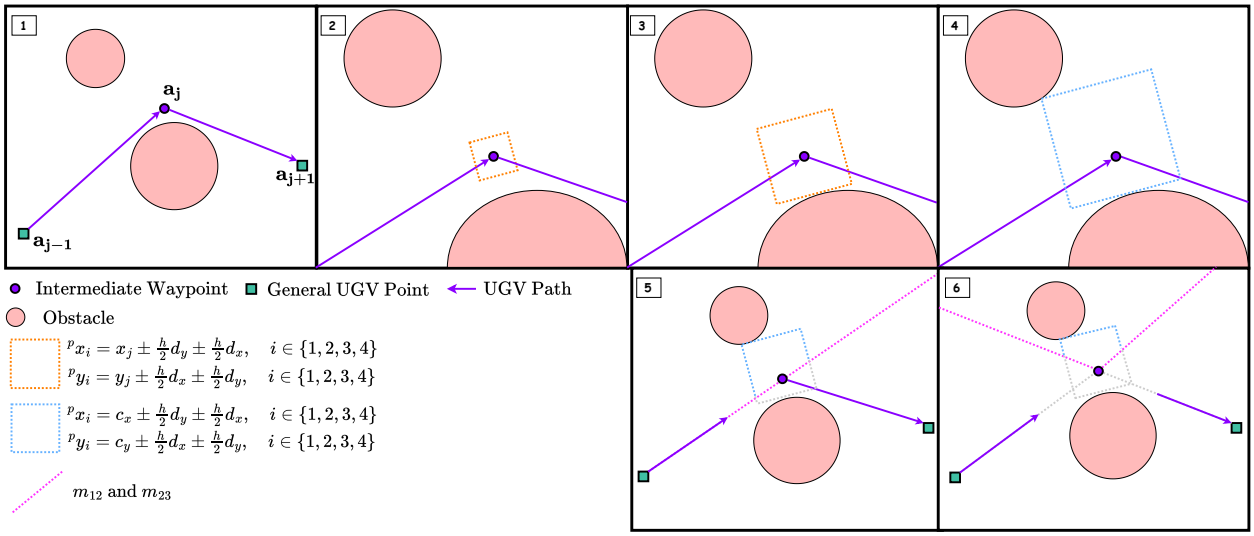


Figure 4.4 Example of how to build the Square and Action-to-Action Constraints.

4.4.4.1 Square Constraints

For all pairs $(a_j, o) \in Z_j$, where a_j is located at (x_j, y_j) and o at (x_o, y_o) with radius r_o, we create a bounding box around a_j as follows: Let $({}^p x_1,\ {}^p y_1)$, $({}^p x_2,\ {}^p y_2)$, $({}^p x_3,\ {}^p y_3)$, and $({}^p x_4,\ {}^p y_4)$ define the four corners of a square centered over a_j. These points are defined as

$$ {}^p x_i = x_j \pm \frac{h}{2} d_y \pm \frac{h}{2} d_x, \quad i \in \{1,2,3,4\} \tag{4.14a} $$

$$ {}^p y_i = y_j \pm \frac{h}{2} d_x \pm \frac{h}{2} d_y, \quad i \in \{1,2,3,4\}, \tag{4.14b} $$

where h is the side length of the square and d_x and d_y are the x and y components the normalized vector from the centroid of o to a_j. We iteratively increase h until either $h = 2\big(r_o - \sqrt{(x_j - x_o)^2 (y_j - y_o)^2}\big)$ (i.e. the square either collides with o) or the square intersects with some other obstacle in Θ. Panels 2 and 3 of Figure 4.4 demonstrate this process, with the square stopping after colliding with o.

If the square hit some obstacle in Θ other than o, then we use (4.14) to set the values of ${}^{p}x_i$ and ${}^{p}y_i$. If the square collides with o, then we bound the square to the side of the obstacle and continue to increase h until we collide with another obstacle. Mathematically, we can bound the square to the side of the obstacle by defining the four points of the square as

$$ {}^{p}x_i = c_x \pm \frac{h}{2} d_y \pm \frac{h}{2} d_x, \quad i \in \{1,2,3,4\} \tag{4.15a} $$

$$ {}^{p}y_i = c_y \pm \frac{h}{2} d_x \pm \frac{h}{2} d_y, \quad i \in \{1,2,3,4\}, \tag{4.15b} $$

where $c_x = x_o + (r_o + \frac{h}{2}) d_x$ and $c_y = y_o + (r_o + \frac{h}{2}) d_y$. We continue to increase h until we either strike another obstacle in Θ or hit some application-appropriate size limit for h. Panel 4 of Figure 4.4 depicts this scenario.

After fixing the value of h and computing the location of the four corners of the square, as defined in either (4.14) or (4.15), we restrict action a_j to lie within the square by adding the following constraints to (4.13):

$$ y_j \begin{cases} \leq \frac{{}^{p}y_3 - {}^{p}y_4}{{}^{p}x_3 - {}^{p}x_4}(x_j - {}^{p}x_4) + {}^{p}y_4, & \text{if } x_a < x_o, \\ \geq \frac{{}^{p}y_3 - {}^{p}y_4}{{}^{p}x_3 - {}^{p}x_4}(x_j - {}^{p}x_4) + {}^{p}y_4, & \text{otherwise,} \end{cases} \tag{4.16a} $$

$$ y_j \begin{cases} \leq \frac{{}^{p}y_1 - {}^{p}y_2}{{}^{p}x_1 - {}^{p}x_2}(x_j - {}^{p}x_2) + {}^{p}y_2, & \text{if } x_a > x_o, \\ \geq \frac{{}^{p}y_1 - {}^{p}y_2}{{}^{p}x_1 - {}^{p}x_2}(x_j - {}^{p}x_2) + {}^{p}y_2, & \text{otherwise,} \end{cases} \tag{4.16b} $$

$$ y_j \begin{cases} \leq \frac{{}^{p}y_2 - {}^{p}y_4}{{}^{p}x_2 - {}^{p}x_4}(x_j - {}^{p}x_4) + {}^{p}y_4, & \text{if } y_a < y_o, \\ \geq \frac{{}^{p}y_2 - {}^{p}y_4}{{}^{p}x_2 - {}^{p}x_4}(x_j - {}^{p}x_4) + {}^{p}y_4, & \text{otherwise,} \end{cases} \tag{4.16c} $$

$$ y_j \begin{cases} \leq \frac{{}^{p}y_1 - {}^{p}y_3}{{}^{p}x_1 - {}^{p}x_3}(x_j - {}^{p}x_3) + {}^{p}y_3, & \text{if } y_a > y_o, \\ \geq \frac{{}^{p}y_1 - {}^{p}y_3}{{}^{p}x_1 - {}^{p}x_3}(x_j - {}^{p}x_3) + {}^{p}y_3, & \text{otherwise.} \end{cases} \tag{4.16d} $$

We acknowledge an ambiguous use of notation regarding action locations. The initial position of action j is denoted as (x_a, y_a) while the decision variable to adjust where the action occurs is denoted as (x_j, y_j). Note there there are unlisted edge cases for the above constraints when ${}^{p}x_i = {}^{p}x_j$, making the slope formula undefined. These edge cases can be handled by setting the corresponding constraints to be a vertical line.

4.4.4.2 Action-to-Action Constraints

For all pairs $(a_j, o) \in Z_j$, where action a_j is of type *intermediate-waypoint*, we add two new constraints to (4.13) that prevent the action from moving behind the associated obstacle o. Let a_{j-1} and a_{j+1} be the actions immediately before and after action a_j, respectively. Add the following constraints to the SOC program:

$$y_j \begin{cases} \leq m_{12}x_j + (y_{j-1} - m_{12}x_{j-1}), & \text{if } {}^1o_y < y_o, \\ \geq m_{12}x_j + (y_{j-1} - m_{12}x_{j-1}), & \text{otherwise,} \end{cases} \tag{4.17a}$$

$$y_j \begin{cases} \leq m_{23}x_j + (y_j - m_{23}x_j), & \text{if } {}^3o_y < y_o, \\ \geq m_{23}x_j + (y_j - m_{23}x_j), & \text{otherwise,} \end{cases} \tag{4.17b}$$

where $m_{12} = \frac{y_j - y_{j-1}}{x_j - x_{j-1}}$, $m_{23} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j}$, ${}^1o_y = m_{12}x_o + (y_{j-1} - m_{12}x_{j-1})$, and ${}^3o_y = m_{23}x_o + (y_j - m_{23}x_j)$. Panels 5 and 6 of Figure 4.4 show an example of how we form (4.17). The same undefined slope formula edge case exists for these constraints as well, which can be handled in the same manner as discussed above.

4.4.5 Time Complexity Analysis

The *Initial-Solution()* function will run the *VRP()* function n^2 times in the worst case. The *VRP()* function depends on the LKH solver, which has a time complexity of $\mathcal{O}(n^{2.2})$ [31]. If we bound the number of times that Lloyd's algorithm iterates, then the *Initial-Solution()* function has a runtime of $\mathcal{O}(n^{4.2})$. The *Update-Subtour()* function also depends on the LKH solver, giving it runtime of $\mathcal{O}(n^{4.2})$.

The efficiency of both the *Launch-Optimizer()* and *LO-Obstacles()* functions depend on the performance of the solver used, though we know for certainty that the SOC program can be solved in polynomial time [143]. Let ζ be the time required to solve the SOC program. The *Action-Swap()* function relies on the *Launch-Optimizer()*, which it will run n times in the worst case, giving the *Action-Swap()* function a runtime of $\mathcal{O}(n\zeta)$.

The runtime of the *Local-Planner()* function depends on the selected path planner. For example, the RRT* algorithm runs in $\mathcal{O}(\eta log(\eta))$, where η is the number of iterations that the algorithm is allowed to run for [142].

The LORS algorithm will run the *Initial-Solution()* function once and the *Launch-Optimizer()* function m_g times. The algorithm then runs the *Update-Subtour()* function m_a times and the *Action-Swap()* function m_g times. Each time the obstacle avoidance logic is run, we further box in any action that may have intersected with an obstacle in a previous iteration of the while-loop on line 8. In the worst case, this logic is repeated n times to bound each PoI individually. This gives the LORS algorithm complexity of $\mathcal{O}(m_a n^{4.2} + m_g n\zeta + m_g n(\eta log(\eta) + \zeta))$. If the selected mathematical optimizer is efficient, then we should expect the TSP solver and the local path planner to dominate the run time.

4.5 Numerical Simulation Results

To test the effectiveness of our proposed LORS algorithm, we ran a series of ablation experiments in a numerical simulation based on field test data from real robots. We also compared our algorithm against the proposed algorithm for a similar problem setup found in recent literature. We use the PAR metric

defined in Section 4.3.2 as our primary metric and include worst-latency as an additional reference metric.

In this section, we first look at our simulation setup and baseline. We then discuss the results of each experiment. This section concludes with a discussion on our main findings and the limitations of our proposed algorithm.

4.5.1 Simulation Setup & Baseline Approach

We generated four sets of data inputs for our numerical simulations. Unless otherwise stated, each data set has 50 PoI selected uniform randomly over a 10 km x 10 km environment with 100 randomly placed obstacles. The obstacles have varying radii, from two m up to 300 m.

We deployed two robots teams, each consisting of one UGV and two drones, for a total two UGVs and four drones. The UGV is modeled after the Clearpath™ Warthog and used the energy profile described in [64]. We used the data from our own, in-house drone and experimentally determined the coefficients for (4.4) to be $c_3 \approx 0.0, c_4 \approx 0.0, c_5 \approx 396.743$ and $c_6 \approx -1.695$. Our Warthog is equipped with wireless energy transfer equipment that allows us to recharge the drone while the drone is perched on top of the UGV. We experimentally found the following values for the energy transfer equation (4.5): $c_7 = 0.0652902, c_8 = 403.856, t^* = 555.6, E^* = 244537, P^* = 476.407, \alpha = 0.003, t_{max} = 991.4, E_{max} = 360381$.

To create our four data sets, we generated 50 samples at varying settings. Our first data set has an increasing number of PoI, from five to 100, in increments of 5, for a total of 1,000 inputs. We will refer to this as the *IncN* data set. The second data set has an increasing number of obstacles, from zero to 200, in increments of 10, for a total of 1,000 inputs. We term this the *IncO* data set. Our third data set has a single UGV with an increasing number of drones, from one to 10, in increments of one, for a total of 500 inputs. We will refer to this as the $IncM_a$ data set. Our fourth and final data set has an increasing number of teams, from one to 10, in increments of one, where each team consists of one UGV and two drones. With a total of 500 inputs, we refer to this as the $IncM_g$ data set.

As a baseline method, we use the *Initial-Solution()* function by itself. This function is a modification of the proposed algorithm in [15, 134], which was used for a closely related patrolling problem, and is similar to the proposed solutions in [10, 16]. In addition to the steps described in 5, we run the *Local-Planner()* function to make the initial solutions *collision-free*. Additionally, we compared our LORS algorithm against its own sub-components. The first of these is the Launch Optimizer (LO) by itself. This algorithm runs Algorithm 4 but skips the replanning (lines 4 through 6) and the swapping (line 7) steps. The next version is the Launch Optimizer with Swapping (LOS) algorithm, which skips the replanning step. The third variation is the Launch Optimizer with Replanning (LOR) algorithm, which skips the swapping step. All algorithms were implemented in C++. We used Gurobi version 12 to solve the SOC program and LKH

version 3.0.8 [109]. We used The Open Motion Planning Library (OMPL) [144] implementation of the Informed RRT* algorithm [145] as our local path planner in the *Local-Planner()* function.

4.5.2 Simulation Results

We calculated the PAR for each output of each of the considered algorithms. To keep our figures easier to read, we calculated the percent increase in PAR of each algorithm's output when compared against the best found solution across all algorithms. We also measured the worst latency of each solution as a reference to the more commonly used metric in patrolling problems. In addition to solution quality, we measured the computation time of each algorithm on each input and the percentage if inputs that each algorithm failed to find a valid solution to. We note that failure rate is a percentage of binomial trials (calculated as the number of failed inputs over total number of inputs). We do not include standard deviation in our failure rate graphs (see Figures 4.9 and 4.10) as this value is not particularly meaningful when running binomial trials.

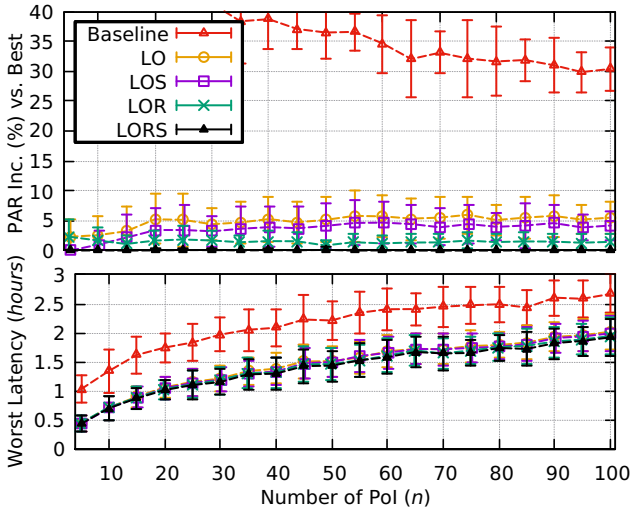


Figure 4.5 Results from the *IncN* data set showing percent increase in PAR compared to the best found solution (*top*) and worst latency (*bottom*), with standard deviation.

4.5.2.1 *IncN* Data Set

Figure 4.5 shows our solution quality results from the *IncN* data set. The LORS algorithm consistently finds the best quality solution and reduces PAR by an average of 1.58%, 3.49%, 4.83%, and 37.7% compared to the LOR, LOS, LO, and Baseline algorithms, respectively. We see a similar performance trend on the worst latency results. Figure 4.9(a) shows the failure rate of each algorithm for the *IncN* data set. On average, the LO, LOS, LOR, LORS algorithms failed on 6.4%, 4.4%, 6.8%, and 5.0% of the inputs,

respectively, while the Baseline averages a 10.3% failure rate. Although the LOS and LORS algorithms tend to perform slightly better than the others, no approach strictly outperforms the rest.

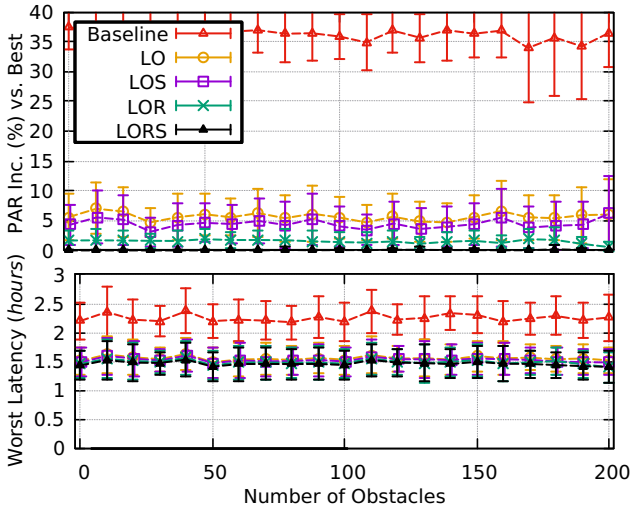


Figure 4.6 Results from the *IncO* data set showing percent increase in PAR compared to the best found solution (*top*) and worst latency (*bottom*), with standard deviation.

4.5.2.2 *IncO* Data Set

Figure 4.6 shows our solution quality results from the *IncO* data set. The LORS algorithm again performs the best, with an average reduction in PAR of 1.60%, 4.60%, 6.07%, and 36.4% when compared against the LOR, LOS, LO, and Baseline algorithms, respectively. Figure 4.9(b) shows the failure rate of each algorithm on this data set. The LOR and LORS algorithms tend to outperform the other approaches, failing an average of 7.3% and 6.4%, respectively, while the LO algorithm failed on 9.9%, LOS 9.5%, and the Baseline 14.8% of inputs.

4.5.2.3 *IncM*$_a$ Data Set

Figure 4.7 shows our solution quality results from the *IncM*$_a$ data set. The LORS algorithm averages a reduction in PAR of 7.42%, 1.72%, 8.67%, and 36.3% when compared to the LOR, LOS, LO, and Baseline algorithms, respectively. Figure 4.10(a) shows the failure rate of each algorithm for the *IncM*$_a$ data set. The baseline algorithm initially struggles, failing on 20% of inputs with a single drone, but then gradually improves as the number of drones increases, failing an average of 11.4% of all inputs in the data set. The other algorithms follow a converse trend, where they begin to struggle as the number of drones increases, with the LO, LOS, LOR, and LORS algorithms failing 22.2%, 13.6%, 20.2%, and 14.4% of the inputs, respectively.

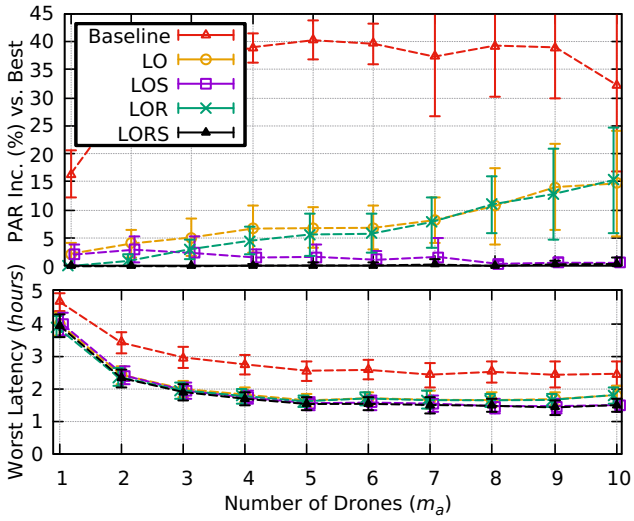


Figure 4.7 Results from the $IncM_a$ data set showing percent increase in PAR compared to the best found solution (*top*) and worst latency (*bottom*), with standard deviation.

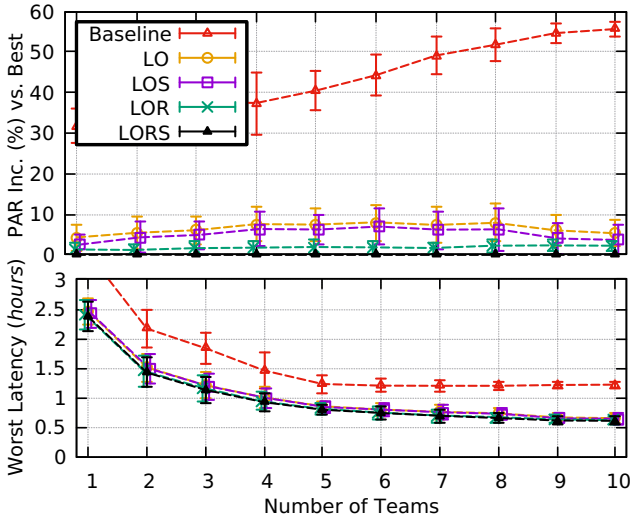
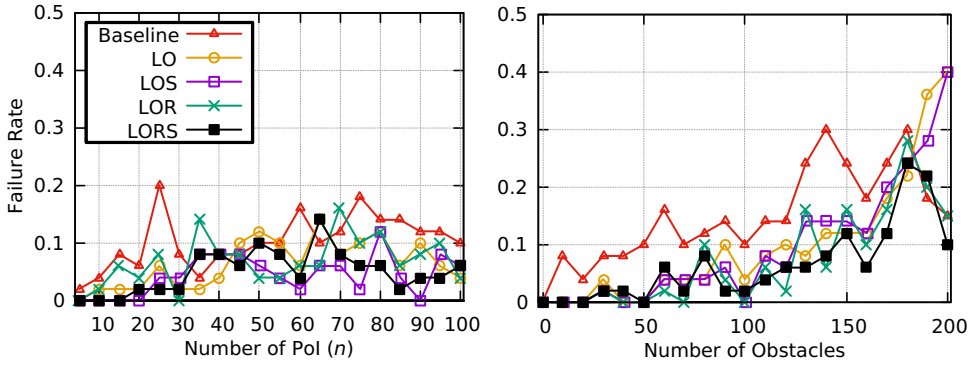


Figure 4.8 Results from the $IncM_g$ data set showing percent increase in PAR compared to the best found solution (*top*) and worst latency (*bottom*), with standard deviation.

4.5.2.4 $IncM_g$ Data Set

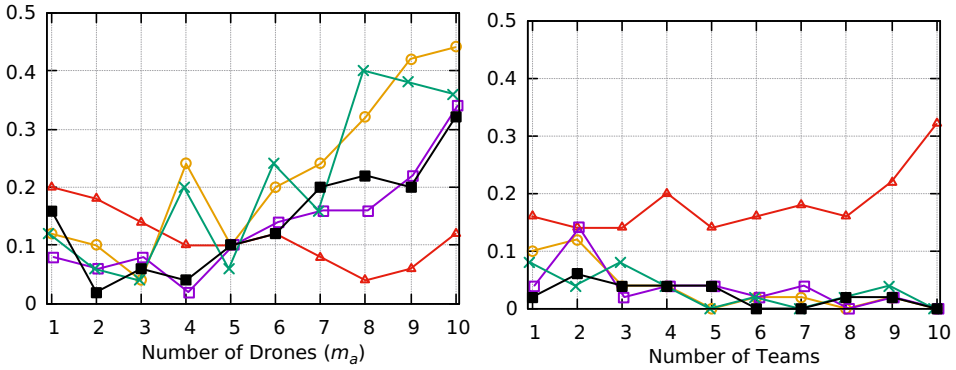
Figure 4.8 shows our solution quality results from the $IncM_a$ data set. The LORS algorithm again performs the best, with an average reduction in PAR of 0.14%, 1.00%, 2.44%, and 20.6% when compared against the LOR, LOS, LO, and Baseline algorithms, respectively. Figure 4.10(b) shows the failure rate of each algorithm on this data set. The LO, LOS, LOR, and LORS algorithms all perform better as the number of teams increases, failing on 3.6%, 3.6%, 3.2%, and 2.4% of the inputs, respectively. The Baseline struggles with more teams, failing an average of 18.2% of all inputs. The majority of these failures come

from OMPL hitting a timeout without finding a valid solution. We speculate that the difference in performance between the LORS algorithm (and its variants – LO, LOS, and LOR) and the Baseline algorithm is that the former are able to reduce the distance that the UGVs travel, which in turn reduces the reliance on the *Local-Planner()* function, where as the Baseline algorithm requires the UGVs to travel further, running the local path planner more often and increasing the probability that OMPL will timeout without finding a solution.



(a) *IncN* (b) *IncO*

Figure 4.9 Failure rate of each algorithm on the *IncN* (*a*) and *IncO* (*b*) data sets.



(a) $IncM_a$ (b) $IncM_g$

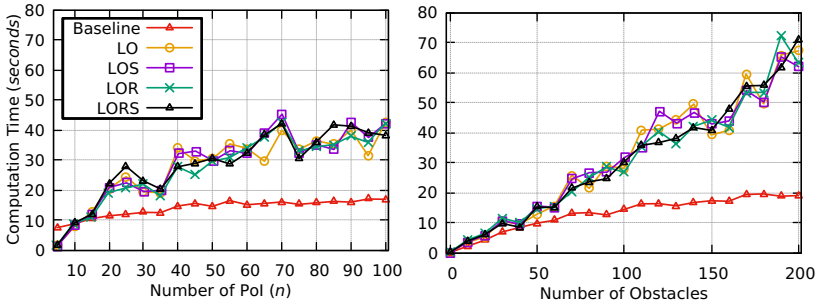
Figure 4.10 Failure rate of each algorithm on the $IncM_a$ (*a*) and $IncM_g$ (*b*) data sets.

4.5.2.5 Computation Time

Figures 4.11 and 4.12 show the average computation time for each algorithm on each of the data sets. We withheld standard deviation to avoid cluttering the graphs, but discuss these numbers as well.

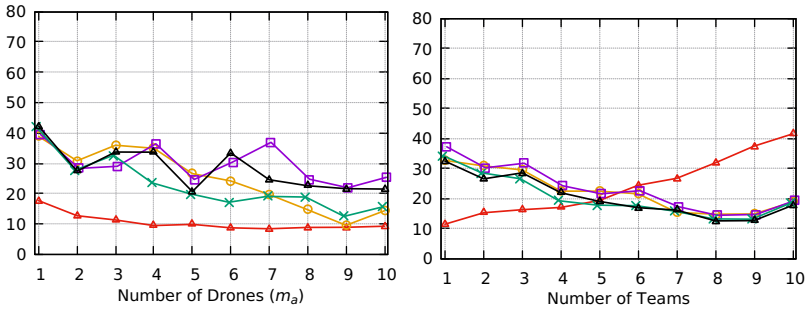
The LORS algorithm and its variants all follow the same trend, with no notable difference between them across all data sets. These algorithms increase the computation time of the Baseline algorithm by an

average factor of 1.9, 2.1, and 2.7 on the *IncN*, *IncO*, and $IncM_a$ data sets, respectively. Interestingly, the computation times of all considered algorithms are trending downwards as the number of drones increases is the $IncM_g$ data sets. We believe that the increase in drones reduces the distance that the UGV must travel, reducing the number of times we must run the *Local-Planner()* function – which is computationally expensive relative to the other algorithms. The Baseline initially performs better on the $IncM_g$ data set, where the LORS algorithm increases computation time by a factor of 2.1 when there are three or fewer teams. However, this trend reverses as the number of teams increases and the Baseline algorithm increases computation time by a factor of 2.4 compared to the other algorithms when there are seven or more teams. We speculate that the increased computation time of the Baseline on the $IncM_g$ data set is caused by the same reason for the increased failure rate on this data set: farther UGV travel distances increase reliance on the *Local-Planner()* function and increase the probability that OMPL will timeout. We believe that this same effect causes the increase in computation time among the LORS variants on the *IncO* data set.



(a) *IncN* (b) *IncO*

Figure 4.11 Computation time of each algorithm on the *IncN* and *IncO* data sets. We withheld standard deviation to avoid cluttering the graphs.



(a) $IncM_a$ (b) $IncM_g$

Figure 4.12 Computation time of each algorithm on the $IncM_a$ and $IncM_g$ data sets. We withheld standard deviation to avoid cluttering the graphs.

The variance in computation time was quite large for all variations of the LORS algorithm. In fact, the standard deviation of the LORS algorithm was 82.9% of the mean on the *IncN* data set, 85.8% on the *IncO* data set, 108.6% on the $IncM_a$ data set, and 90.3% on the $IncM_g$ data set. The large variance on these algorithms is caused by two opposing factors. The first is the while-loop on lines 8 through 11, where the *Local-Planner()* function attempts to move actions out of obstacles and the *LO-Obstacles()* function occasionally pushes actions back into new obstacles. This causes the local path planner to run more frequently, which greatly increases computation time. The other factor impacting computation time is that the SOC program tends to move drone and UGV actions closer to the UGV base, reducing the likelihood that the UGV will collide with an obstacle. This helps avoid the local path planner and reduces computation time. The result of these two factors together is that the LORS algorithm terminates quite quickly on some inputs but struggles on others. The standard deviation in computation time for the baseline is much more stable, averaging 4.3, 3.6, 3.4, and 6.1 seconds on the *IncN*, *IncO*, $IncM_a$, and $IncM_g$ data sets, respectively.

4.5.3 Results Discussion & Limitations

Our simulation results show that the LORS algorithm consistently produces better quality solutions when compared to the other considered methods. This is particularly true when comparing against the Baseline algorithm, with the LORS algorithm improving solution quality from 20.6% to up to 37.7%. The Baseline tends to fail to find a solution on more inputs when compared to the other algorithms, with the only exception being when there is a large number of drones. The only advantage to the Baseline algorithm is the computation time, where the Baseline tends to outperform the LORS algorithm and its variants, except for when there is a large number of robot teams.

Although the complete LORS algorithm only sees marginal improvements at times over its other variants (e.g., 0.14% compared to the LOR algorithm on the $IncM_g$ or 1.72% compared to the LOS algorithm on the $IncM_a$ data set), the complete algorithm consistently finds the best solution by leveraging the strengths of each sub-component of the algorithm. For example, the LOR algorithm is a strong contender on the *IncN* data set, where the LOS algorithm is weaker, but then struggles on the $IncM_a$ data set, where the LOS algorithm performs particularly well. In comparison, the LORS algorithm performs well on both. Furthermore, there is no notable difference in computation time between the various variants of the algorithm due to the steep cost of running the local path planner in the *Local-Planner()* function that all four versions of the algorithm must compute. These factors make the complete LORS algorithm a favorable choice over the other three.

One of the major limitations of the algorithm is that the SOC program may not always have a valid solution after adding the additional obstacle avoidance constraints discussed in Section 4.4.4. By adding constraints that force the UGVs out of obstacles, this process reduces the size of the search space and may box itself into an invalid solution. This is the primary cause for the LORS algorithm and its variants to fail. The other limitation is the local path planner. Although some of these algorithms, such as RRT*, are asymptotically optimal (that is, they converge onto the optimal solution if given enough time), they can be computationally expensive and may not converge within a reasonable timeframe.

4.6 Case Study

To evaluate the effectiveness of our Drone-UGV teams running the LORS Algorithm in a real-world context, we constructed an infrastructure monitoring case study based on a large wind farm in Birds Landing, CA. Growing maintenance costs have accompanied worldwide wind energy growth [146], with 65-70% of maintenance costs on wind farms coming from unscheduled corrective maintenance [147]. These costs could be curtailed using teams of autonomous robots that perform routine inspections, providing early fault detection. Additionally, many wind farms consist of hundreds of turbines spread across remote areas with little-to-no supporting infrastructure [148], making autonomous robots well suited for sustained monitoring missions.

Wind turbine maintenance spans both structural monitoring of the turbine and its load bearing elements as well as conditional monitoring of mechanical elements. Various monitoring strategies include visual inspection, acoustic emissions, ultrasonic testing, infrared thermography, radiographic testing, electromagnetic testing, and oil monitoring [149]. Many of these testing strategies require either close/direct proximity to a turbine or ability to perform analysis on large amounts of produced monitoring data [148]. Drones can easily be equipped with the appropriate sensors to handle these monitoring tasks.

To create our case study, we overlaid PoIs at 36 turbines and defined 15 obstacle areas around buildings and regions we wanted the UGV to avoid, using satellite imagery from Google Maps. We placed the depot at a central location and configured the LORS algorithm with a single UGV and two drones. We assume the UGV can traverse all terrain not explicitly designated as obstacles.

Figure 4.13 shows our case study setup and generated solution. The LORS algorithm computed the solution in 20.3 seconds with a PAR score of 33.1. This computation time matches what we found in Section 4.5.2.5. The average PAR on the $IncM_g$ data set with a single UGV and two drones was 59.2 for 50 PoI, so a PAR of 33.1 with the same robot team for 36 PoI seems to be reasonable. This case study demonstrates one possible real-world use case for the Team-PITH problem and helps validate our numerical evaluation in Section 4.5.

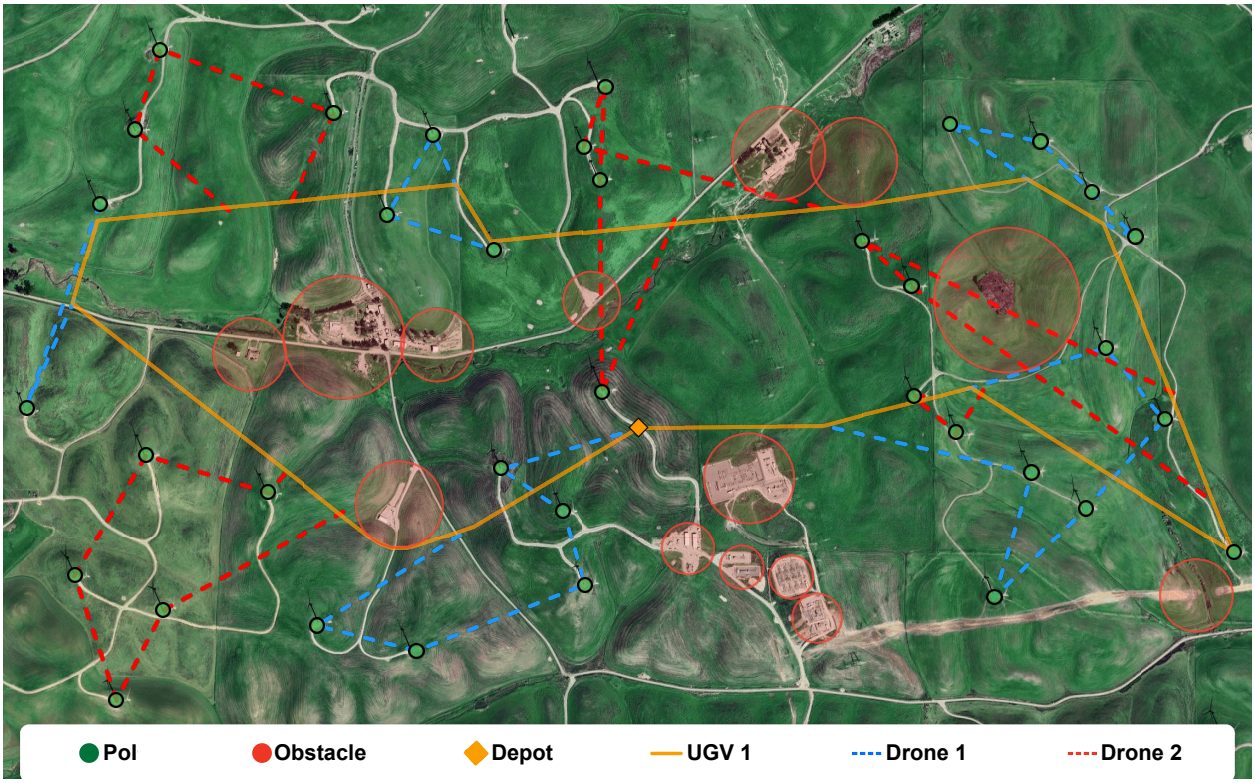


Figure 4.13 Case study example output path

4.7 Prototype on Real Drone-UGV Team

To demonstrate the Team-PITH problem and to determine limitations of our proposed algorithm, we ran a field prototype of the considered problem scenario on a physical UGV and drone testbed. We used a Clearpath™ Warthog and a custom built drone platform. The Warthog was configured to wirelessly transfer energy to the drone. We set a series of PoIs in an empty field and had the Warthog ferry the drone to different launch points.

Figure 4.14 shows the robots and plan generated by the ILO algorithm and Figure 4.15 shows the GPS trace of each vehicle. The drone traveled all assigned routes successfully but the Warthog became stuck halfway through its assigned route due to issues with its local planner and did not reach the final rendezvous location. As a result, the drone had to be manually flown back to the Warthog after the second sortie. However, the first drone sortie provides a clear example of the Team-PITH problem. Possible solutions to deployment issues include strengthening information sharing between the UGV and drone and online replanning.

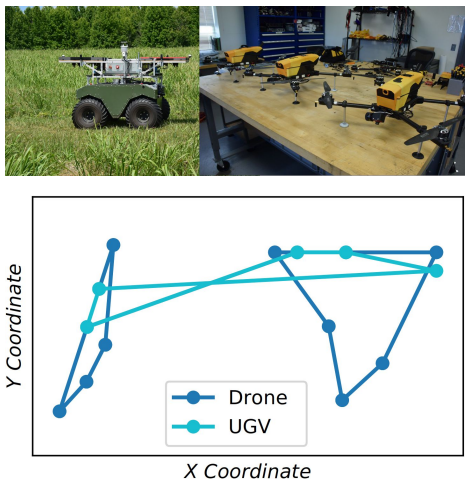


Figure 4.14 Prototype setup.

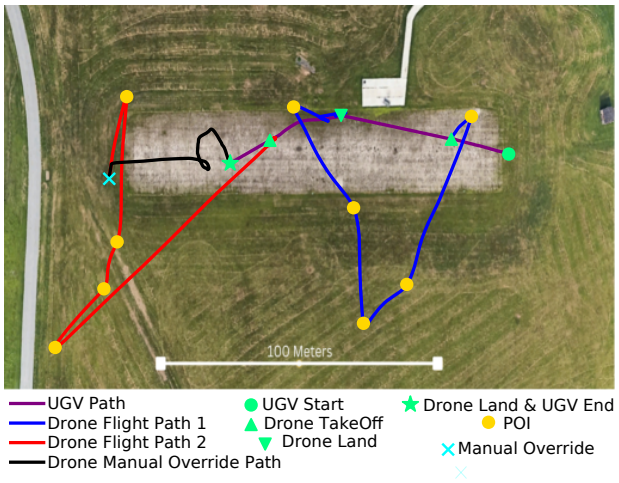


Figure 4.15 Drone and UGV paths from field experiment.

4.8 Conclusions

In this article, we introduced the Team-PITH problem, developed a new taxonomy for classifying drone-UGV planning problems, proposed a novel metric for evaluating solution quality in patrolling problems, and presented the Launch Optimizer with Replanning and Swapping algorithm. Given an initial solution, the LORS algorithm uses a Second-Order Cone program to find locally optimal solutions (which can be solved in polynomial time), resolves smaller Traveling Salesman Problems, and using an action swapping heuristic to further improve solution quality. We demonstrated through extensive numerical simulations, based on field data, that our algorithm can reduce PAR by up to 37.7% when compared to a baseline method from existing literature.

Limitations of this work include possibly excluding feasible solutions from the search space of the SOC program when avoiding ground obstacles and a reliance on sampling-based motion planners to find paths through free space to guide the UGVs. Additionally, we identified a need for online adaptation during our field prototype on a team of real robots.

For future work, we plan to address these limitations (particularly online adaptive planning) as well as handling limited communication and varying PoI priorities. Another interesting avenue to consider is allowing the drones to break from their initial team and rendezvous with other UGVs. There are many possible use cases for the core problem addressed in this article and we believe that there will be many application-specific challenges that arise when deploying drone-UGV teams for indefinite patrolling. Using this work as a building block, future work should focus on addressing these application-specific challenges.

CHAPTER 5

FAILURE-AWARE TASKING FOR TEAMS OF DRONES

Modified from [150], accepted for publication at the 45^{th} International Conference on Distributed Computing Systems[1].

Jonathan Diller[2,3], Yee Shen Teoh[2], Robert Byers[2], Qi Han[2], John G. Rogers[4], Neil T. Dantam[2]

Teams of drones have been proposed for many monitoring and data collection applications, including forest fire monitoring, search and rescue, disaster response, and infrastructure inspection. However, robot systems can be stochastic, and uncertainty arises when operating environments are dynamic or hostile. This paper investigates the problem of assigning drones to tasks where the probability that a given group of drones can cooperatively complete a task follows a Poisson-Binomial distribution. We show how to determine if a solution exists and how to calculate an upper bound on the optimal solution. We present a variation of the branch-and-bound algorithm – termed Branch-and-Match – that is tailored to our problem and always finds an optimal solution at the cost of computation time. For a more tractable approach, we present a heuristics-based algorithm – termed M+ILS – that turns the problem into a balanced matching problem to find an initial solution then runs a variation of the Iterated Local Search (ILS) algorithm. Our M+ILS algorithm is applicable to distributed scenarios but finds suboptimal solutions. We evaluate these various algorithms in a simulated forest fire monitoring scenario based on the characteristics of a fleet of real drones. Our empirical results show that the M+ILS algorithm finds solutions with an average performance gap of 2.68% compared to the optimal solution found using the Branch-and-Match algorithm.

5.1 Introduction

In July of 2022, the Washburn Fire of Mariposa Grove, CA broke out in Yosemite National Park and threatened a grove of 500 giant sequoias and 1,600 people when it burned 19.7 km^2 of the park and national forest [151]. In this paper, we study how teams of drones could have been deployed to aid firefighters in containing and monitoring the movements of the fire.

Using data captured by NASA's Fire Information for Resource Management System[5] (shown in Fig. 5.1), we have created a forest fire monitoring case study based on the Washburn Fire. From this data,

[1]Reprinted, with permission, from Jonathan Diller, Failure-Aware Tasking for Teams of Drones, International Conference on Distributed Computing Systems, 07/2025

[2]Department of Computer Science, Colorado School of Mines, USA

[3]Primary researcher and author

[4]DEVCOM, Army Research Laboratory, USA

[5]firms.modaps.eosdis.nasa.gov/map/#d:2022-07-08;@-119.60,37.50,14.00z

we have designed four drone monitoring tasks to support the initial response to the fire (depicted as green flags in the figure). For an effective response, we need all tasks completed. Given a fleet of heterogeneous drones, the problem becomes determining which drones should perform which tasks.

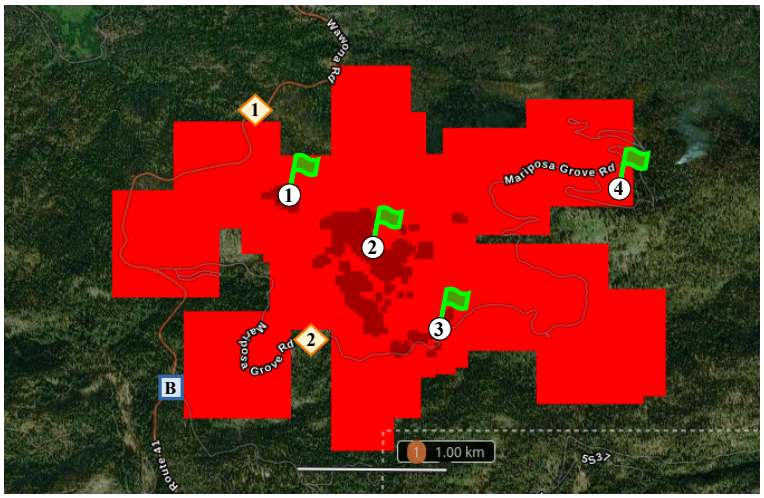


Figure 5.1 Case study on 2022 Washburn Fire in Mariposa Grove, California. Green flags are monitoring tasks, orange diamonds are drone launch locations, and the blue square is the base station. Light red markers show low resolution hot spots while dark red markers indicate active fires within 30 meters.

Initially, assigning drones to the monitoring task may appear to be a standard assignment problem, a well-studied area [152]. However, drones exhibit non-deterministic behavior, particularly in terms of energy consumption rates, making their performance and operational endurance challenging to predict accurately. Environmental factors, such as wind, hardware factors, and the age of a battery will impact performance and make a drone's operational abilities stochastic. We argue that the stochastic nature of energy consumption and the probability that a drone may run out of energy before successfully executing a task (that is, system failure) should be considered when assigning tasks to drones for data collection and monitoring applications.

When faced with uncertainty, it is advantageous to assign additional drones to a task to increase the probability that the task is completed. Suppose that Task 4 is an early fire detection task that only requires two drones to cover the area. However, if we assign more than two drones to Task 4 then the probability that at least two of the drones successfully monitor the area will increase. If we treat the probability that any individual drone successfully participates in the task as an independent trial and have a minimum number of required participating drones for the task to be completed, then the probability that the task is completed will follow a Poisson-Binomial distribution [153].

Furthermore, the tasks themselves are expected to be dynamic. Suppose that after deploying teams of drones for each task in our case study, the epicenter of the fire moves, changing the location and demands

of each task. The dynamic nature of a forest fire creates the need for distributed algorithms that can quickly replan drone tasking during deployment.

In this work, we investigate a new class of task assignment problems where the objective function is a product of Poisson-Binomial trials. We term this class of problems the Maximum Probability of Task Success (MPTS) problem, which includes compatibility and minimum-worker constraints. To the best of our knowledge, we are the first to consider this class of problem. *We make the following contributions in this article*:

1. We show how to determine if a solution exists to the MPTS problem in polynomial time and provide a theoretical upper bound on a solution.

2. We propose the Branch-and-Match algorithm, a complete solution based on the classic branch-and-bound algorithm that reduces branching by utilizing our work on solution existence.

3. We propose the M+ILS algorithm, a heuristics-based approach to solving the MPTS problem that can be implemented as a distributed algorithm and can quickly run online during mission execution.

4. We show how to assign a probability to the operational capabilities of drones and use this to motivate the MPTS problem setup.

Our extensive numerical simulations and deployment on a physical, distributed testbed demonstrate the utility of our two proposed algorithms and highlight their tradeoffs.

Although we focus on drone tasking for forest fire monitoring in this work, we argue that our problem setup and proposed algorithms can function in many other applications and problem domains. Possible other areas of application include infrastructure inspection, agriculture, manufacturing, or disaster response. To help further the frontiers of research and to promote good research integrity, our algorithm implementation and data sets have been made open-source[6].

5.2 Related Works

In this section we review related bodies of work, starting with various assignment problems and then look at distributed algorithms commonly used for solving assignment problems.

5.2.1 Task Assignment Problems for Robotics

In the linear assignment problem, we are given n workers (agents), m jobs (tasks), worker-to-job costs and are asked to find a one-to-one matching such that we minimize the summation of worker-to-job costs, where $m \geq n$. This problem can be solved using the well studied Hungarian method (Kuhn-Munkres

[6]https://github.com/pervasive-computing-systems-group/MPTS

algorithm), which runs in polynomial time [47]. A related problem is the general task assignment (GTA) problem, where we are given m items with profits and n bins with capacities and are asked to find an assignment of items to bins such that we maximize the expected profit without exceeding bin weight limits. This problem is known to be NP-hard [152] and is often applied to robot-task assignment scenarios [44].

Many works have expanded the GTA problem to incorporate probabilities. Notably, in the Weapon-Target Assignment (WTA) problem we are given a set of targets with rewards, a set of weapons with a probability of destroying a target, and are asked to find weapon-to-target assignments that maximize the expected reward. The WTA problem is also known to be NP-complete [154]. A variety of solutions have been proposed for the WTA problem, such as branch-and-bound algorithms that exploit some aspect of the problem and find optimal solutions [46]. Among heuristics-based approaches, Very Large Scale Neighborhood Search (VLSN) algorithms are quite popular [50, 155], as are biology-based solutions (e.g. bee colony algorithms [156]). Similar in nature to exact methods for the WTA problem, we design a custom branch-and-bound algorithm that exploits the problem structure of the MPTS problem to find optimal solutions.

Although the WTA problem and the MPTS problem are similar in nature, there are some key differences between the two. The MPTS problem seeks to maximize the probability that *all* tasks are completed while the WTA problem seeks to maximize a reward for completing tasks. Furthermore, the WTA problem maximizes the probability that one or more agents complete a task while our MPTS problem maximizes the probability that $d_j \geq 1$ or more agents are successful, where d_j is the minimum number of agents required to complete task j. The former is a product of Bernoulli trials while the latter requires one to optimize the cumulative distribution function (CDF) of a Poisson-Binomial Distribution (PBD), which changes the dynamics of how moving an agent from one task to another impacts the objective.

There are few examples of assignment problems that involve phenomenon that exhibit a PBD. Examples include sensing wireless channel activity [157] or randomly occurring jobs [158]. These works optimize some cost function and only use the PBD as a constraint to limit errors or future infeasibility. In this work, our objective is to maximize the CDF of a PBD rather than incorporating the CDF into problem constraints.

5.2.2 Distributed Task Assignment Algorithms

Given the dynamic nature of the considered scenario, we seek a distributed assignment algorithm for the MPTS problem that is tractable enough for online decision making. Furthermore, we prefer a solution that does not require the robots to share all of their capabilities or energy status due to privacy concerns.

A common approach to solving assignment problems is to run distributed versions of centralized algorithms involving matchings over bipartite graphs. This is seen in [159], where they search for

augmenting paths over a bipartite graph in a distributed solution to the Bottleneck Assignment problem (which seeks to maximize the minimum cost assignment). Similarly, the authors in [160] propose a distributed version of the Hungarian method for solving the linear assignment problem to optimality that builds a weighted covering in a bipartite graph of agents to tasks by finding minimum cost matchings then searches for augmenting paths over the graph. Another common distributed approach for the linear assignment problem is for the agents to share costs and then solve the bipartite matching locally, as seen in [161, 162], while others have investigated how to reduce message passing by limiting what information is shared [163]. Similar to the idea presented in [161], our M+ILS algorithm forms a balanced matching problem out of the MPTS problem and uses a distributed version of the Hungarian method to find an initial solution. To reduce information sharing, we recommend Giordani's distributed implementation of the Hungarian method [160], but this can be changed for more efficient algorithms when information privacy is not a concern.

Among distributed algorithms for finding near-optimal solutions, a common approach is to allow agents to run local searches over iterations. One example of this is gradient decent algorithms, which are popular for training machine learning models [164] and for multi-agent, multi-objective convex optimization problems [165]. In [166], the authors use a distributed local search to find graph partitions in large social networks. The authors in [167] propose an iterative Monte-Carlo tree search for searching local robot-action spaces for actions that improve cumulative team reward.

An interesting aspect of the MPTS problem is that not all drones are required for a valid solution, giving the drones more freedom to search their local solution space without creating an invalid solution. Inspired by this fact, our M+ILS algorithm utilizes the ILS algorithm [168] to improve on the initial solution. In the original ILS algorithm, we perform a local search then randomly "perturb" the search space to move out of local minimum. There is limited work on distributed versions of the ILS algorithm (see [169, 170] where they still require a leader-follower structure). In this work, we propose a distributed variation of the ILS algorithm where each drone searches its local solution space then perturbs its neighbor without sharing all of its capability information. When combined with the initial solution, our M+ILS algorithm greatly improves solution quality over the classic ILS algorithm.

5.3 Maximum Probability of Task Success Problem Formulation

In this section we demonstrate how to calculate the probability that a drone has enough energy to complete an assigned task then formally define our problem, termed the Maximum Probability of Task Success (MPTS) problem.

5.3.1 Drone Energy-based Probability of Contribution

Drone energy depletion has been shown to depend on the physical characteristics of the drone platform and on the speed that the drone travels at [5]. Building on the model presented in [7, 171], let the total energy budget of drone i be ${}^{m}s_i$, the total time duration that the drone can travel at max speed ${}^{m}v_i$ starting with a fully charged battery and operating in an ideal environment, which is measured in seconds and termed *seconds-moving*. Mathematically,

$$ {}^{m}s_i = \frac{V_{bat} B_{rate}}{\mathcal{P}({}^{m}v_i)} \tag{5.1} $$

where V_{bat} is the voltage rating of the equipped battery, B_{rate} is the depletion rate of the battery, and $\mathcal{P}({}^{m}v_i)$ is the power used by drone i when moving at ${}^{m}v_i$, which is further discussed in detail in [5]. We will assume that all drones travel at max speed in this work as this has been shown to generally outperform moving at slower speeds [102, 172]. From (5.1), we see that the total time duration that a drone can hover (i.e. when $v_i = 0$) with a fully charged battery will be

$$ {}^{h}s_i = \frac{V_{bat} B_{rate}}{\mathcal{P}({}^{h}v_i)} \tag{5.2} $$

where $\mathcal{P}({}^{h}v_i)$ is power consumption when hovering.

Based on this model for drone energy, if task j requires drone i to travel a total of $dist_{ij}$ meters and hover for t_j seconds, then the predicted energy budget required (in *seconds-moving*) to complete the task will be:

$$ s_{ij} = \frac{dist_{ij}}{{}^{m}v_i} + t_j \frac{{}^{m}s_i}{{}^{h}s_i} \tag{5.3} $$

However, wear and age will impact the capacity and depletion rate of a battery. The authors in [173] found experimentally that the capacity of a lithium ion battery will decrease linearly as a function of the number of charging cycles. Building on this work, let the percent reduction in battery capacity after c recharging cycles be:

$$ r(c) = 1 - \varphi c \tag{5.4} $$

where φ is a constant determined experimentally. In addition to battery wear, several external factors such as wind, precipitation, elevation, and others will also impact how quickly the drone consumes energy during deployment. All of these factors are challenging to accurately predict. Instead, we propose adding an error term to our energy budget that follows a continuous probability distribution. If we choose to model the error as a normal distribution, our energy budget becomes:

$$ s_i = r(c) \frac{V_{bat} B_{rate}}{\mathcal{P}({}^{m}v_i)} \sim \mathcal{N}(\mu_i, \sigma_i) \tag{5.5} $$

where mean $\mu_i = {}^m s_i$ and standard deviation $\sigma_i = \gamma\, {}^m s_i$ for some interval of confidence $\gamma \in [1, 0]$. We chose a normal distribution due to its universality but acknowledge that other continuous probability distributions could be substituted into our formulation without major change. Determining the exact characteristics of the distribution requires further field experimentation and is left for future work.

The probability that drone i completes task j without running out of energy will be

$$p_{ij} = \int_{s_{ij}}^{\infty} \frac{1}{\sigma_i \sqrt{2\pi}} e^{\frac{-(t-\mu_i)^2}{2\sigma^2}} dt. \tag{5.6}$$

In other words, p_{ij} is the probability that the actual energy budget of drone i is greater than or equal to s_{ij}.

5.3.2 Maximum Probability of Task Success Problem

We now build on (5.6) to formally define the MPTS problem. We refer the reader to Table 5.1 for a list of symbols used in our formulation.

Table 5.1 Symbols

Symbol	*Type*	*Description*
$\boldsymbol{C}$	Parameter Matrix	$c_{ik} = 1$ if drone i has capability k, 0 otherwise
$\boldsymbol{c}_i$	Parameter Array	Capability row for agent i in parameter matrix $\boldsymbol{C}$
d_j	Parameter	Number of drones required for task j
I	Set	Set of all available drones
I_j	Set	Set of drones assigned to task j
J	Set	Set of all tasks
K	Set	Set of all considered capabilities
$p_{ij}(t)$	Function	Probability that drone i contributes to task j
$\boldsymbol{R}$	Parameter Matrix	$r_{jk} = 1$ if task j requires capability k, 0 otherwise
$\boldsymbol{r}_j$	Parameter Array	Requirement row for task j in parameter matrix $\boldsymbol{R}$

We are given a set I of n drones and a set J of m tasks, where $n \geq m$, and are asked to assign drones to tasks such that the probability of all tasks being completed is maximized. Each drone has a set of capabilities (e.g. sensors) and each task has capability requirements (e.g. sensors required). Let $\boldsymbol{c}_i$ be a binary array of capabilities for drone i and $\boldsymbol{C}$ be the capability matrix for all drones. Let $\boldsymbol{r}_j$ be a binary array of capability requirements for task j with $\boldsymbol{R}$ being the matrix of all task requirements. Each $c_{ik} \in \boldsymbol{c}_i$ is 1 if drone i has capability k and 0 otherwise. Similarly, each $r_{jk} \in \boldsymbol{r}_j$ is 1 if task j requires capability k and 0 otherwise. From (5.6) we get p_{ij}, the probability that drone i successfully contributes to task j, for $i \in I$ and $j \in J$. Although we focus on drones, our problem scenario can be reapplied to any scenario where one can assign a value to p_{ij}.

Each task requires a minimum of d_j drones although we allow additional drones (so called "floating agents") to be assigned to a task. For task j to successfully complete, at least d_j drones assigned to task j must successfully contribute to the task. For example, suppose we determined that we need two drones to

canvas the area with thermal cameras for eight minutes to determine the risk to the giant sequoias for Task 4 ($j = 4$) from our case study. However, we can increase the probability that at least two drones return with thermal images by assigning more than two drones to survey j. Determining how to coordinate data collection between the drones in the event of failures such that required data is returned will depend on the specifics of the monitoring task, which we leave for future work.

Let I_j be the set of drones assigned to task j, where $I_j \subseteq I$. The probability that task j gets completed (denoted as $P_s(d_j, I_j)$) is the probability that d_j or more drones from I_j successfully contribute to the task. We treat each drone $i \in I_j$ as a sequential independent binomial trial where each trial has probability p_{ij} of success. This means that $P_s(d_j, I_j)$ is the probability that d_j or more trials are successful. Because each trial is independent (i.e. $\sum_{i \in I_j} p_{ij}$ may not be 1 and we are not guaranteed that $p_{ij} = p_{i'j}$ or $p_{ij} = p_{ij'}$), the probability that d_j or more drones successfully contribute to a task follows a PBD [153]. Given set I_j, the probability that *exactly* k of the trials are successful is found using the probability mass function (PMF) of a PBD:

$$P_r(k, I_j) = \sum_{A \in F_k} \prod_{i \in A} p_{ij} \prod_{i' \in A'} (1 - p_{i'j}) \tag{5.7}$$

where F_k is the set of all combinations of size k of the drones in I_j and $A' = I_j \setminus A$ is the compliment of A. For example, suppose $I_j = \{1, 2, 6\}$ and $k = 2$, then $F_k = \{\{1, 2\}, \{1, 6\}, \{2, 6\}\}$ and when $A = \{1, 2\}$ we get $A' = \{6\}$.

To determine $P_s(d_j, I_j)$, the probability that d_j *or more* drones contribute to the task, we must compute the complement of the cumulative distribution function (CDF), termed the "survival function". In the general, case where $|I_j| = n_j \geq d_j$,

$$P_s(d_j, I_j) = \sum_{k=d_j}^{n_j} P_r(k, I_j) \tag{5.8}$$

We can now formally define the MPTS problem:

Definition 3 (Maximum Probability of Task Success Problem): *Given set of drones I and set of tasks J with minimum numbers of drones d_j for each $j \in J$, find drone-to-task assignment sets $I_j \subseteq I$ that:*

$$maximize \prod_{j \in J} P_s(d_j, I_j) \tag{5.9}$$

subject to

$$I_j \cap I_{j'} = \emptyset, \; \forall j, j' \in J \tag{5.10a}$$
$$|I_j| \geq d_j, \; \forall j \in J \tag{5.10b}$$
$$(\boldsymbol{c}_{ik} - \boldsymbol{r}_{jk}) \geq \mathbf{0}, \; \forall k \in K, j \in J, i \in I_j \tag{5.10c}$$

Constraint (5.10a) ensures drones are assigned at most once. Constraint (5.10b) requires each task to be assigned at least the minimum number of drones, d_j, for the task to be completed. Constraint (5.10c) forces task requirements to be met.

Observe that in (5.7) there will be $\frac{n_j!}{(n_j-k)!\ k!}$ elements in F_k, making (5.9) difficult to compute as $n_j \gg d_j$, even when given a solution. Before we can consider how to solve (5.9), we would like to reformulate PMF of a PBD. In [174], they show how to rewrite the PMF as a Vandermonde matrix and simplify it into a DFT matrix, the Fourier transform of which leads to a closed-form expression of the PMF for the PBD. From their work, we find that the PMF can be computed as

$$P_f(k, I_j) = \frac{1}{n_j+1} \sum_{l=0}^{n_j} \left\{ e^{\frac{-2\pi lk\sqrt{-1}}{n_j+1}} \prod_{i \in I_j} \left\{ p_{ij} e^{\frac{2\pi lk\sqrt{-1}}{n_j+1}} + (1 - p_{ij}) \right\} \right\} \tag{5.11}$$

For brevity, we do not show the steps to derive (5.11) but refer the reader to [174] for these details. The survival function then simplifies to

$$P_s(d_j, I_j) = \sum_{k=d_j}^{n_j} P_f(k, I_j) \tag{5.12}$$

Note that the number of terms in (5.12) grows as a polynomial with the size of I_j and we can compute $P_s(d_j, I_j)$ in $\mathcal{O}(n_j^2)$.

We can formulate an integer non-linear program to solve the MPTS problem as follows: Let x_{ij} be a binary variable, where $x_{ij} = 1$ if drone i is assigned to task j and 0 otherwise. Let a_{jl} be a binary variable, for $j \in J$ and $l \in \{0, 1, \cdots n\}$, where $a_{jl} = 1$ if there are at least l drones assigned to task j and 0 otherwise. We re-write (5.11) as

$$P_{IP}(k, j) = \frac{1}{n_j+1} \sum_{l=0}^{n} \left\{ (a_{jl}) e^{\frac{-2\pi lk\sqrt{-1}}{n_j+1}} \prod_{i \in I} \left\{ p_{ij} e^{\frac{2\pi lk\sqrt{-1}}{n_j+1}} + (1 - p_{ij}) \right\}^{x_{ij}} \right\} \tag{5.13}$$

The survival function for our Integer Program is

$$P_{sIP}(j) = \sum_{k=d_j}^{n} P_{IP}(k, j) \tag{5.14}$$

The Integer Non-Linear program to determine the optimal drone-to-task assignment to maximize the probability that all tasks are completed is:

$$maximize \prod_{j \in J} P_{sIP}(j) \tag{5.15}$$

subject to

$$\sum_{j \in J} x_{ij} \leq 1, \forall i \in I \tag{5.16a}$$

$$\sum_{i \in I} x_{ij} \geq d_j, \forall j \in J \tag{5.16b}$$

$$(\boldsymbol{c}_{ik} - \boldsymbol{r}_{jk}) x_{ij} \geq \mathbf{0}, \forall k \in K, j \in J, i \in I \tag{5.16c}$$

$$\sum_{i \in I} x_{ij} \geq l a_{jl}, \forall j \in J, l \in \{0, 1, \cdots n\} \tag{5.16d}$$

We focus on drone energy to determine p_{ij}, but the MPTS problem can be reapplied to new domains where p_{ij} can be defined for tasking independent events. For example, p_{ij} could be the probability that a machine fails while producing widgets in a manufacturing scenario.

As previously stated, the objective function for the MPTS problem as presented in (5.9) will take exponential time to compute, making the problem formulation sit in EXPTIME. The integer program presented in (5.15) is non-convex, a class of problems known to be NP-Hard. At the time of writing, there is no known method for formulating the MPTS problem in a form that can be solved in polynomial time. We note that we provided both (5.9) and (5.15) to give the reader better context on the considered problem and the difficulties around solving it.

5.4 Solution Existence and Upper Bound

Although the MPTS problem is hard to solve to optimality, we can determine if a solution exists in polynomial time by mapping the problem to a minimum cost balanced matching problem – a variation of the linear assignment problem where there is an equal number (n) of workers and jobs, with cost k_{ij} for each worker i to job j assignment.

$d_1 = 2$
$d_2 = 3$

$r_1 = [1\ 0]$
$r_2 = [0\ 1]$

$c_1 = [1\ 0]$
$c_2 = [1\ 0]$
$c_3 = [0\ 1]$
$c_4 = [0\ 1]$
$c_5 = [1\ 1]$
$c_6 = [1\ 1]$

Workers \ Jobs	*1.1*	*1.2*	*1.x*	*2.1*	*2.2*	*2.3*	*2.x*
1	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1
3	1	1	1	0	0	0	0
4	1	1	1	0	0	0	0
5	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0
x	1	1	0	1	1	1	0

Figure 5.2 Balanced matching problem example to prove solution existence.

Figure 5.2 helps demonstrate how to map the MPTS problem into a balanced matching problem that can determine if a valid solution exists. We start by creating d_j jobs for each $j \in J$. These jobs are depicted in the figure as $j.k$ where $1 \leq j \leq m$ and $1 \leq k \leq d_j$. These jobs represent the minimum number of drones that must be assigned to tasks in the MPTS formulation. Let n_r be the total number of drones required to meet d_j for every task and n_f the number of floating agents. That is, $n_r = \sum_{j \in J} d_j$ and $n_f = n - n_r$. For each task j in MPTS, we create n_f jobs in the balanced matching problem in addition to the d_j jobs for j. These are shown in the figure as $j.x$ for $1 \leq j \leq m$. Let n_t be the total number of jobs required to form a balanced matching. That is, $n_t = \sum_{j \in J} d_j + n_f|J|$. To make the problem balanced, we add $n_t - n$ new "phantom workers", which are indicated as "x" in the workers column.

For each worker i and job j, we set $k_{ij} = 0$ if $(\boldsymbol{c}_{ik} - \boldsymbol{r}_{jk}) \geq \boldsymbol{0}$ and to 1 otherwise. The cost of assigning a phantom worker to any of the required d_j jobs is 1 (i.e. a phantom worker will fail to complete a required task), and the cost of assigning a phantom worker to a floating job is 0 (i.e. a phantom worker is guaranteed to complete a floating task). We call our formulated matching problem $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$. We can find a least cost assignment to our balanced matching using the Hungarian method [47], which runs in polynomial time.

Theorem 6: *There exists a solution to the MPTS problem formulated from $I, J, \boldsymbol{C}$ and $\boldsymbol{R}$ if and only if there exists a minimum cost matching of zero in $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$.*

Proof. Assume that there is a minimum cost matching of zero for $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$ and no solution exists to the MPTS problem. Because there is a minimum cost matching of zero for $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$, and job assignment costs in $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$ are, by construction, zero for drones in the MPTS problem that can actually perform each task, then matching of workers to jobs in $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$ must be a valid matching of drones to tasks in the MPTS problem. Therefore, our assumption that no solution exists to the MPTS problem must have been wrong.

The same proof can be applied in reverse to show that if a solution exists for the MPTS problem then the minimum cost matching in $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$ must be zero. □ □

If the minimum cost matching in $\mathcal{B}(I, J, \boldsymbol{C}, \boldsymbol{R})$ is greater than zero, this tells us that there are not enough drones to meet the capability requirements of every task. That is, at least one task will have less than d_j drones that meet requirements $\boldsymbol{r}_j$ and objective function (5.15) will evaluate to zero.

Additionally, we can provide an upper bound on a solution by dropping constraint (5.16a), allowing robots to be assigned to an unlimited number of tasks. In this case, we set $x_{ij} = 1$ for all drone i and task

j that do not break constraint (5.16c) and then compute:

$$\prod_{j \in J} P_{sIP}(j) \tag{5.17}$$

Equation (5.17) provides an upper bound on a given problem input, which we use in the following section to speed-up a branch-and-bound algorithm for the MPTS problem.

5.5 Algorithms to Solve the MPTS Problem

In this section we present various algorithms for solving the MPTS problem. We first present the *Match + Iterated-Local-Search* (M+ILS) algorithm, a distributed algorithm that builds on the problem structure introduced in Theorem 6 to quickly find a solution to the MPTS problem. We then look at a modified version of the classic branch-and-bound algorithm – termed branch-and-match – that utilizes our solution existence work in Section 5.4 and our upper bound on a solution from (5.17) to speed-up solve time.

Algorithm 6 M+ILS

Function: *Start*():
1: $\boldsymbol{w}_i \leftarrow calculateWeights()$
2: $\mathcal{X} \leftarrow HungarianMethod(\boldsymbol{w}_i)$
3: *Iterated-Local-Search*($\mathcal{X}$)

Function: *Iterated-Local-Search*($\mathcal{X}$):
4: **if** no changes made to $\mathcal{X}$ since last iteration **then**
5: *stop algorithm*
6: **else if** $\exists j \in J$ where $\sum_{i \in I} x_{ij} < d_j$ **then**
7: *assign robot to* j in $\mathcal{X}$
8: **else**
9: $j \leftarrow localSearch(\mathcal{X})$
10: *assign robot to* j in $\mathcal{X}$
11: **end if**
12: **if** neighbor still up **then**
13: *send* $\mathcal{X}$ *to neighbor*
14: **else**
15: *erase neighbor's entry in* $\mathcal{X}$
16: *send* $\mathcal{X}$ *to next neighbor*
17: **end if**

5.5.1 Match + Iterated-Local-Search Algorithm

Observe that (5.15) will increase as more drones are assigned to tasks that already have d_j drones. This is particularly true for tasks with smaller values for (5.14). Furthermore, if any task j has less than d_j drones assigned to it, then (5.15) evaluates to zero. This observation motivates our proposed M+ILS algorithm. For each task j, the algorithm attempts to assign the d_j best suited drones to the task in an initial solution and leaves the remaining drones free. The remaining drones can then be used to perturb the search space while running a distributed variation of the ILS algorithm. We will first look at how to find an

initial solution then discuss our adapted ILS routine. Algorithm 6 shows the outline of the M+ILS algorithm.

We introduce the algorithm as a distributed solution but a centralized version can easily be implemented by placing the algorithm into a loop and iterating over each drone. For the distributed version of the algorithm, we assume that each drone knows its own probability of contributing to each task. We argue that this assumption is reasonable because each drone could monitor the status of its battery and calculate (5.6). We also assume that each drone knows d_j, its own capabilities vector c_{ik}, and the requirements vector of each task r_{jk}, but has no knowledge of the probability that any other drone can contribute to a task or what the capabilities of the other drones are. We also assume that the drones have a numbered ordering and are wirelessly connected through one or more hops.

The idea behind our approach to finding an initial solution is to map the problem into a balanced matching problem with n workers (drones) and n jobs (tasks), where some jobs represent actual tasks and others are "null-jobs" that are not associated with any task. Figure 5.3 shows an example of how we form this balanced matching problem. We first create d_j jobs for each $j \in J$, shown as $j.k$ for $1 \leq j \leq m$ and $1 \leq k \leq d_j$ in the figure. These jobs represent the minimum number of drones that must be assigned to tasks in the original MPTS formulation. We are required to assign a total of $n_r = \sum_{j \in J} d_j$ drones to the tasks in MPTS and have $n_f = n - n_r$ floating drones that are not required to complete the tasks but can be used to reinforce the other drones in the case of failure. We create n_f additional null-jobs, indicated as an "x" in the example. We now have n workers and n jobs.

$d_1 = 2$
$d_2 = 3$

$r_1 = [1\ 0]$
$r_2 = [0\ 1]$

$c_1 = [1\ 0]$
$c_2 = [1\ 0]$
$c_3 = [0\ 1]$
$c_4 = [0\ 1]$
$c_5 = [1\ 1]$
$c_6 = [1\ 1]$

Workers \ Jobs	1.1	1.2	2.1	2.2	2.3	x
1	q_{11}	q_{11}	M	M	M	1
2	q_{21}	q_{21}	M	M	M	1
3	M	M	q_{32}	q_{32}	q_{32}	1
4	M	M	q_{42}	q_{42}	q_{42}	1
5	q_{62}	q_{62}	q_{52}	q_{52}	q_{52}	1
6	q_{61}	q_{61}	q_{62}	q_{62}	q_{62}	1

Figure 5.3 Matching problem example for the M+ILS algorithm.

For each worker-job combination, where the worker represents drone i and the job represents task j, we set the cost of the worker doing the job to $q_{ij} = 1 - p_{ij}$, the probability that drone i fails to contribute to task j. For any drone-to-task combination where the drone does not meet the capabilities of the task, we

set a cost of M, some arbitrarily large number. For all null-jobs (i.e. "x" in the figure), we assign a cost of 1 because drones that do not get assigned to a task will fail to complete a task with probability 1. This entire process is done on each individual drone on line 1 of Algorithm 6, where $\boldsymbol{w}_i$ is a vector of task weights for drone i. For example, $\boldsymbol{w}_1 = [q_{11}, q_{11}, M, M, M, 1]^T$ in the example from Fig. 5.3. We note that this method is slightly different from the mapping discussed in Section 5.4 but can be used to determine if a solution exists by the same proof used previously.

The drones then find an initial matching of drones to tasks using the Hungarian method (on line 2). There are several distributed versions of the Hungarian method in existing literature (see [160–163]). We recommend Giordani's version of the algorithm in [160] because it does not require the drones to share all of their capability information and can adapt to failures. A brief description of this algorithm is found in Section 5.2 but we refer the reader to [160] for a more detailed description.

After finding an initial matching, the drones form solution $\mathcal{X}$, a $2 \times m$ matrix where x_{i1} stores which task drone i is currently assigned to and x_{i2} stores the probability that the drone can contribute to this task. The drones then begin the ILS portion of the algorithm (line 3) and pass $\mathcal{X}$ from drone-to-drone in ascending order.

When drone i receives table $\mathcal{X}$, it searches for a task j that does not have at least d_j drones assigned, checking tasks in descending order of p_{ij} (line 6). If there exists a task j with less than d_j drones assigned, and the drone is capable of performing task j, then drone i assigns itself to task j (i.e. the drone sets $x_{i1} = j$ and $x_{i2} = p_{ij}$). This section of the algorithm is needed should a drone drop-out of the system or if the *Iterated-Local-Search*() function is run by itself (as shown in Section 5.6 as a baseline method). If no such task exists, then the drone runs a local search by iterating through all tasks that the drone is eligible to perform and computing (5.15) to find the task that maximizes the objective (line 9). Once assigned to a task, the drone passes $\mathcal{X}$ to its next neighbor. Should the next drone in the order fail (line 12 is false), then drone i removes drone $i+1$'s entry in $\mathcal{X}$ and sends $\mathcal{X}$ to drone $i+2$. The algorithm ends when n iterations have passed without any drone making an update in $\mathcal{X}$.

As presented, the computation time of the algorithm will depend on the selected distributed Hungarian method. Giordani's implementation has a global computation time of $\mathcal{O}(n^3)$ with a message complexity of $\mathcal{O}(n^3)$. In the worst-case, the *Iterated-Local-Search()* function will run in $\mathcal{O}(m \cdot n^2)$ - the time required to compute (5.12) for every task - on each drone for each iteration. However, because each drone can only be assigned to a single task, the computation time for (5.12) is expected to be $\mathcal{O}\big((\frac{n}{m})^2\big)$, making the amortized runtime for a single iteration of the function $\mathcal{O}\big(\frac{n^2}{m}\big)$. We can guarantee that the *Iterated-Local-Search()* function converges in a fixed number of iterations by only allowing task changes if $\delta_j \geq \epsilon$, $\epsilon \in [0, 1]$. That is, a drone can change tasks only if the improvement in the objective function is

greater than ϵ. In this case, the function will run for at most $\frac{1}{\epsilon}n$ iterations and will pass at most $\frac{1}{\epsilon}n$ messages in the worst-case, giving the *Iterated-Local-Search()* function a computation time of $\mathcal{O}\big(\frac{n^3}{\epsilon m}\big)$. This means that the Hungarian method will dominate and the computation complexity of the M+ILS algorithm is $\mathcal{O}(n^3)$. However, we hypothesize that the *Iterated-Local-Search()* function is likely to converge after m iterations, making it complete in n^2 time. Additionally, there are alternative distributed implementations of the Hungarian method that trade-off information privacy for speed (see [162]) that could be used without changing the structure of the M+ILS algorithm.

5.5.2 Optimal Solution: Branch-and-Match (BnM)

Our branch-and-match algorithm conceptually works the same as the classic branch-and-bound approach. We iterate through each drone, assigning them to individual tasks in turn and record when we find a better solution. We modify the traditional algorithm by considering two methods for pruning a branch. Given a partial assignment, we prune a branch if:

1. The upper bound on a solution by holding the partial assignment and solving (5.17) on unassigned drones is less than the incumbent solution, or

2. No solution exists to the sub-problem formed from the unassigned drones and unfilled tasks.

To check for the second condition, we remove drones that were assigned from the problem and reduce each d_j (the minimum number of drones required to complete task j) by the number of drones that were already assigned to task j. We then use the linear matching routine described in the proof of Theorem 6 to determine if a solution exists in the sub-problem. Additionally, we can further speed up the algorithm by using a heuristics-based algorithm to find an initial incumbent solution (a so-called "warm start") or by sorting drones so that the drones that are able to perform the most tasks are considered first. The motivation for sorting the drones is that this locks in the more capable drones early in a branch and lowers the upper bound by leaving less capable drones unassigned.

Our custom branch-and-bound algorithm will find the optimal solution. However, the algorithm will run in $\mathcal{O}\big(m^n\big)$ in the worst-case and is not feasible for larger inputs. This approach is also not ideal in a distributed environment, should the drones need to rerun the algorithm online, further motivating the need for our M+ILS algorithm.

5.6 Performance Evaluation Via Simulation

We implemented both centralized and decentralized versions of the M+ILS algorithm, as well as the branch-and-match (BnM) solver, in C++. The centralized M+ILS uses a centralized, open source

implementation of the Hungarian method[7]. For the decentralized implementation, we used socket programming and implemented Giordani's distributed version of the Hungarian method, as described in [160]. In this section, we will use the abbreviation "dM+ILS" to refer to the distributed version and M+ILS to refer to the centralized version.

5.6.1 Realistic Inputs

To the best of our knowledge, there is no public dataset on drone energy statistics or thorough experiments needed to determine γ. To cover this gap, we used drone data from the drone fleet used by the U.S. Department of Interior (DOI) [8] to make our simulations and case study more realistic. The drones considered for this problem are listed in Table 5.2, where model 1 is the Matrice 600 Pro[9], 2 is the Mavic Pro[2], 3 is the FireFLY6 Pro[10], 4 is the 3DR Solo[11], 5 is the Pulse Vapor 55TM[12], and 6 is the Parrot Anafi[13]. The sensor types are 1 for EO/IR Camera, 2 for Thermal Camera, 3 for HD Camera, 4 for 4K Camera and 5 for a Lidar. The values in the table were either found on the DOI's website, provided by the manufacturer, or were estimated as indicated.

Table 5.2 Drone Specifications

Drone Model	1	2	3	4	5	6
${}^{m}v_i$ (m/s)	18	20	30.5	25.5	11	15
${}^{m}s_i$ (s)	740*	990*	2400	270*	1800*	720*
${}^{h}s_i$ (s)	1080	1740	1200	1080*	2700	1080*
Sensors	2, 3, 5	1, 4	2, 4	3	1, 3, 5	4

* estimated value

To calculate the probability that a drone can contribute to a task, we used Equ. 5.6 with $\gamma = 0.25$ and the values provided in Table 5.2. This value for γ means that we have a 25% interval of confidence on our prediction of the total time duration that each drone can travel at max speed. In [173] they experimentally determined that the degradation rate of a battery $\varphi \approx 0.0005$, which allows us to determine $r(c)$, the percent drop in battery capacity after recharging c times.

Using the drone data in Table 5.2, we generated four sets of MPTS problem inputs. *Set 1* has randomly located tasks in a 5 km $\times$ 5 km space with task times varying from 5 to 15 minutes and randomized sensor requirements. We deployed drones from Table 5.2 at random starting points with randomly set values for c

[7] github.com/mcximing/hungarian-algorithm-cpp
[8] https://www.doi.gov/aviation/uas/fleet
[9] DJI, www.dji.com
[10] BirdsEyeView Aerobotics
[11] 3DR, www.3dr.com
[12] AeroVironment, www.avinc.com
[13] Parrot Drone SAS, www.parrot.com

between 0 and 400. We varied the number of drones from 5 to 30 (in increments of 1) and set the number of tasks to be 25% the number of drones with 25% of the drones available as floating agents. *Set 2* has the same setup as *Set 1*, but increments the number of drones from 4 up to 40, initially in increments of 1 up to 8 drones, then in increments of 4. *Set 3* again copies the general setup of *Set 1* except that the number of drones is fixed at 15 with 3 tasks and the number of floating agents varying from 0 up to 12. *Set 4* varies the number of drones from 50 up to 150, in increments of 10. There are multiple base stations (equal to 10% of the number of drones) that are scattered throughout a 3 $km \times 3$ km space. Drones are assigned to a specific base station and begin within a 50 m radius of their assigned base station. For all data sets we generated 50 inputs for each distinct configuration, giving us a total of 1,300 inputs for *Set 1*, 650 inputs for both *Set 2* and *3*, and 550 inputs for *Set 4*.

5.6.2 Baseline Methods

We consider four different baseline approaches to compare against our proposed heuristics-based algorithm. In the first baseline method we form a balanced matching problem as described in Section 5.4 with slight modifications. Rather than setting costs to 1 or 0, we set the cost for each drone-to-task combination to $q_{ij} = 1 - p_{ij}$ if drone i can perform task j, and some arbitrarily large number M otherwise. We then solve the balanced matching problem using the centralized Hungarian method. We refer to this approach as the Balanced Matching (BM) algorithm. The intuition behind BM algorithm is that minimizing the sum of probabilities that the drones will fail at their assigned task is a good proxy for solving (5.15) and promises to always find a solution, if one exists. Our second baseline approach is to run the *Iterated-Local-Search()* function from the M+ILS algorithm without running the matching portion of the algorithm first (ILS). The first two baseline approaches were chosen to determine the utility of each part of the M+ILS algorithm individually.

As a third baseline, we implemented a distance minimizing algorithm (MinDist) that puts the probability of failure into its constraints. We form a balanced matching problem by setting the cost for drone i to perform task j as the total distance that i would have to travel to perform j and restrict i from being assigned to j if $p_{ij} < 0.5$. We then find the minimum drone-to-task assignment using the Hungarian method. We chose this as a baseline because many related works use a "limit risk" type approach for assignments while minimizing some other criteria [111, 157, 158, 171].

For a fourth baseline, we implemented the VLSN algorithm, as initially described in [155]. The VLSN algorithm takes an initial solution and iteratively searches for a reassignment by attempting to swap the assignment of two or more drones. If a reassignment is found that yields a higher objective function value, then that solution is stored as the incumbent solution. The algorithm checks all possible swaps from 2 up

to 5 drones at once and uses our first baseline algorithm to find an initial solution. We chose this baseline because it was found to perform well for the WTA problem [50, 155].

5.6.3 Simulation Results

We ran our proposed algorithms on the input sets described above to compare their performance against the various baseline methods. All experiments in the subsection were run on a machine with an Intel 3.4 GHz 16-Core CPU and 64 GiB of RAM.

5.6.3.1 Performance Versus Optimal Solution

We used *Set 1* to evaluate how increasing the number of drones impacts performance by comparing each algorithm's output against the optimal solution found using our BnM algorithm. The top graph in Fig. 5.4 shows the average ratio of the found solution over the optimal solution of each input (a ratio of 1 means that the algorithm found the optimal solution). We stopped running the experiment when the number of drones reached 19 because the BnM algorithm became too slow after this point. On average, our M+ILS algorithm was within 2.68% of the optimal solution. All baseline algorithms initially performed fair but then began to diverge from the optimal as the number of drones increases. The best performing baseline approach was the ILS algorithm, with solutions averaging within 10.61% of the optimal. The bottom graph in Fig. 5.4 shows the average percent gap between the optimal solution and the upper bound found using (5.17). The average gap is 24.3% but is much tighter when there are fewer drones (eight or less).

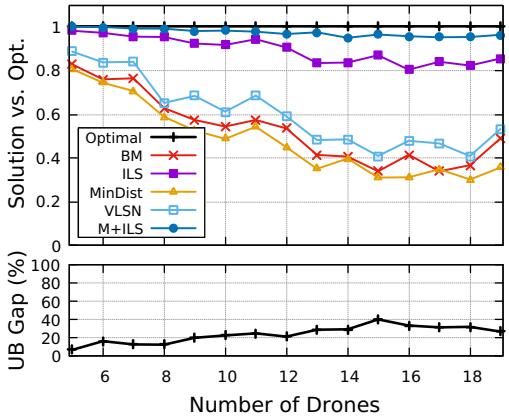


Figure 5.4 Impact of increasing drones on solution.

5.6.3.2 Performance at Scale

We next used *Set 4* to evaluate how well the heuristic-based algorithms perform on much larger inputs. The top graph in Fig. 5.5 shows the average ratio of the found solution for each algorithm over the best

found solution for each input (a ratio of 1 means that the algorithm found the best known solution). The M+ILS algorithm found the best solution for every input while the ILS algorithm, although in clear second place and better than the other baseline approaches, begins to struggle as the number of drones greatly increases. The bottom graph of Fig. 5.5 shows the average probability that any individual task is completed – calculated using Equ. (5.12). Although Equ. (5.12) is not our considered objective function, we include this data because if an algorithm is finding assignments that are consistent the objective should trend downwards as the number of tasks increases (e.g. $0.99^2 \gg 0.99^{100}$). The plot shows us that the ILS algorithm struggles to find good assignments across all tasks when there are a large number of drones while the other approaches trend slightly upward with the M+ILS algorithm consistently well ahead of all other approaches.

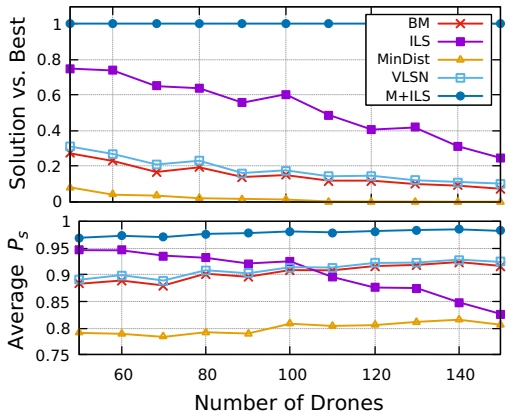


Figure 5.5 Impact of increasing drones at large scale.

5.6.3.3 Impact Of Floating Agents

We evaluated how the number of floating agents impacts performance by running our centralized solution on *Set 3*, where the number of drones and tasks are fixed at 15 and 3, respectively, but the number of floating agents varies from 0 up to 12 (80% of the drones). The top graph in Fig. 5.6 shows the average ratio of the found solution over the optimal solution found using the BnM algorithm. The plot shows that when all drones are required for a problem input (i.e. 0% floating agents) the M+ILS algorithm performs comparably to both the BM and VLSN methods. However, as the number of floating agents increases the BM and VLSN algorithms begin to struggle while the M+ILS improves and averages within 2.45% of the optimal over all inputs from data *Set 3*. The ILS algorithm initially performs very poorly compared to all other methods. Without any floating agents to switch between tasks, the ILS algorithm is essentially a greedy algorithm. However, the algorithm quickly trends towards the optimal as the percent of floating agents increases, beating all other baseline algorithms once 13% of the drones are floating agents and

averages within 7.64% of the optimal solution after this point.

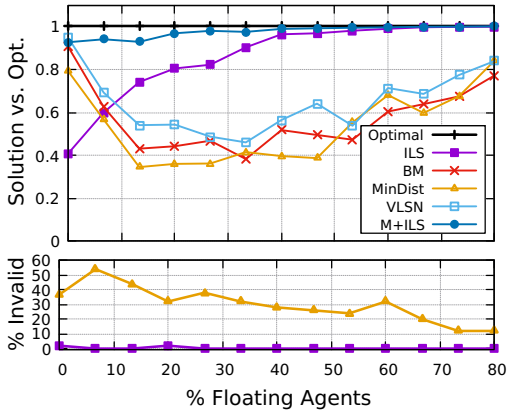


Figure 5.6 Impact of increasing "floating agents."

5.6.3.4 Finding Valid Solutions

We also used *Set 3* to evaluate how often each algorithm finds a valid solution. The bottom graph of Fig. 5.6 shows the percent of inputs where the algorithm fails to find a valid solution. The ILS algorithm slightly struggles when there are no floating agents available, failing on 2% of the inputs. This algorithm acts as a greedy solution when there are no floating agents available so we can anticipate that the algorithm will sometimes fail here. The MinDist algorithm struggles greatly at finding valid solutions, failing on 30.06% of the inputs. This is because of our constraint that no task will be considered when $p_{ij} < 0.5$, leading the algorithm to not consider valid solutions that might expose one or more of the drones to higher risk of failure. All other considered algorithms found valid solutions for all inputs and were omitted from the bottom graph of Fig. 5.6 to reduce clutter.

Table 5.3 Computation Time (Seconds) – Exact Methods

# Drones	9	11	13	15	17	19
BnB	$1.02{\cdot}10^3$	$7.68{\cdot}10^3$	0.740	1.23	582.3	-
BnM	$0.56{\cdot}10^3$	$7.90{\cdot}10^3$	0.183	0.921	26.1	541.4

5.6.3.5 Computation Time (Centralized)

Table 5.3 shows the computation time of the BnM algorithm compared to the classic branch-and-bound (BnB) approach that does not use the branch cutting techniques discussed in Section 5.5.2. The classic BnB algorithm got stuck on inputs larger than 17 drones and 5 tasks. Our proposed BnM algorithm reached 19 drones before becoming too slow to compute solutions in a reasonable time. Table 5.4 shows the

average computation time of centralized versions of the heuristics-based algorithms as the number of drones grows. Our M+ILS algorithm handled large inputs quite well, averaging only 154 *ms* on inputs with 150 drones and 38 tasks. This was only topped by the MC algorithm, which averaged 118 *ms* on the 150 drone inputs. Notably, the VLSN algorithm struggled greatly as the number of drones increased.

Table 5.4 Computation Time (Seconds) – Heuristics-Based (Centralized)

# Drones	30	50	70	90	110	130	150
BM	$1.1{\cdot}10^3$	$5.2{\cdot}10^3$	$1.8{\cdot}10^2$	$5.9{\cdot}10^2$	0.170	0.406	0.856
ILS	$3.9{\cdot}10^3$	$6.3{\cdot}10^3$	$1.2{\cdot}10^2$	$2.7{\cdot}10^2$	$4.8{\cdot}10^2$	$7.5{\cdot}10^2$	**0.118**
MinDis	$1.2{\cdot}10^3$	$4.3{\cdot}10^3$	$1.5{\cdot}10^2$	$5.2{\cdot}10^2$	0.139	0.339	0.713
VLSN	0.273	3.47	26.6	97.1	340.9	800.7	1461
M+ILS	$2.4{\cdot}10^3$	$6.3{\cdot}10^3$	$1.4{\cdot}10^2$	$3.1{\cdot}10^2$	$5.8{\cdot}10^2$	0.100	**0.154**

5.6.3.6 Packet Count and Computation Time (Distributed)

We used *Set 2* to evaluate the performance of the distributed implementation of the M+ILS algorithm and a distributed version of the ILS algorithm (dILS). We chose to only run the ILS baseline for this evaluation because it was shown to be notably better in solution performance and computation time compared to the other baseline approaches on *Set 1* (which has a similar setup to *Set 2*). Figure 5.7 shows the average number of packets sent over a socket (left y-axis in log scale) and the average computation time (right y-axis). The number of packets required for the dM+ILS algorithm grows much faster than the dILS algorithm, averaging 26,400 packets compared to only 219, respectively, when there are 40 drones. This leads to polynomial growth in computation time for the dM+ILS algorithm while the dILS algorithm grows near linearly with the number of drones.

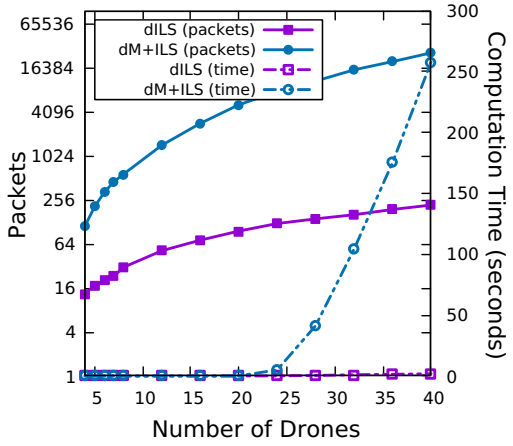


Figure 5.7 Packet count and runtime of distributed algorithms.

5.6.3.7 Summary of Major Findings

Our simulation results show that the centralized version of our M+ILS algorithm performs very well across all considered scenarios (increasing number of drones, increasing percent of floating agents, finding valid solutions). The algorithm also runs very fast when implemented as a centralized solution. However, the distributed version of our algorithm (dM+ILS) requires a lot of message passing, leading to higher computation times when compared to the most competitive baseline approach. The computation time and message complexity of the algorithm could be improved by swapping out our chosen implementation of the distributed Hungarian method with the version presented in [162], which sacrifices capability privacy for algorithm efficiency. Changing the distributed Hungarian method implementation would not require structural change to our proposed algorithm.

5.6.4 Case Study Results

Returning to our case study in Fig. 5.1, on page 85, let Tasks 1 and 3 be to monitor how the fire is moving along its boundaries while Task 2 is to provide personnel monitoring for firefighters close to the center of the fire. Task 4 is a preventative monitoring task for early fire detection near the giant sequoias. Tasks 1 and 3 require at least two drones with HD cameras for 10 minutes, Task 2 requires four drones equipped with both IR and HD cameras for 14 minutes, and Task 4 requires two drones with thermal cameras for 8 minutes. A total of 15 drones are to be launched from the two launch locations shown in Fig. 5.1. Five model 5 drones with older batteries (350 cycles) will be launched from location 1. Four model 1 (10 cycles), two model 2 (100 cycles), two model 3 (150 cycles), and two model 6 (250 cycles) drones will all be launched at location 2. The drones must fly from their respective launch points to their assigned tasks, then must fly out to the base station after completing their assigned tasks.

We ran our case study problem input on our two proposed algorithms and the ILS baseline. Our BnM algorithm found an optimal assignment with a 91.67% chance of completing all tasks while the M+T algorithm found a near optimal assignment with a 90.96% chance of completing all tasks. The BM, ILS, MinDist, and VLSN approaches all trailed behind, finding solutions of 76.28%, 84.88%, 19.32% and 76.28%, respectively. The BnM algorithm finished in 120 ms while the M+T, BM, ILS, and MinDist algorithms all completed in under 10 ms. The VLSN algorithm took 30 ms. These results follow the same pattern we found previously and support our result analysis from the numerical simulations.

5.7 Performance Evaluation in the Field

To better understand the impact of increased message passing in a realistic distributed environment, we deployed both the dM+ILS and dILS algorithms on a physical testbed. We again chose to run only the

dILS baseline on the test bed because of its superior performance compared to the other baseline methods. Our physical testbed consisted of an array of eight Raspberry Pi 3B+ (one mounted on a drone and the other seven free standing) that we deployed in a large courtyard area shown in Fig. 5.8. The Raspberry Pi were arranged throughout the courtyard, varying between 5 and 50 meters apart, and communicated via WiFi using Ad-Hoc mode. We used Raspberry Pi 3B+ because they are commonly used as an onboard computer for drones.

Figure 5.8 The Raspberry Pi 3B+s used in physical testbed.

Table 5.5 Computation Time (seconds) on Testbed

# Drones	4	5	6	7	8
dILS	0.142	0.206	0.552	0.211	0.244
dM+ILS	0.961	1.742	2.804	4.412	4.785

Table 5.6 Found solution over best found solution on Testbed

# Drones	4	5	6	7	8
dILS	0.998	0.991	0.981	0.945	0.956
dM+ILS	0.999	0.998	0.999	0.999	0.996

We ran inputs from the second test set (where the number of drones ranges from four up to eight) on the testbed. Table 5.5 shows the computation time as the number of drones increases while Table 5.6 shows the average ratio of the found solution for each algorithm over the best found solution (a ratio of 1 means that the algorithm found the best known solution). We again see the same trend, where the dM+ILS algorithm outperforms the baseline but at the cost of computation time. The time delay becomes exacerbated on the physical test bed where data packets must be transmitted over WiFi. As discussed in Section 5.6.3, we could improve the performance of the algorithm by implementing a more efficient version

of the distributed Hungarian method at the cost of data privacy.

5.8 Conclusions

In this paper we formulated the Maximum Probability of Task Success (MPTS) problem, a task assignment problem that seeks to maximize the probability that all tasks are complete. We applied the problem to a wildfire monitoring scenario and proposed both a centralized, optimal solution (Branch-and-Match) and a distributed approach (M+ILS) that finds near-optimal solutions in polynomial time. We evaluated our two proposed algorithms in extensive simulations and ran our M+ILS algorithm on a physical testbed. Our Branch-and-Match algorithm is computationally faster than the classic branch-and-bound algorithm but does not scale well with the number of drones. Our M+ILS algorithm averages solutions within 2.68% of the optimal solution but at the cost of time and message complexity when compared to other alternative baseline methods.

Future work should focus on further developing and evaluating methods for modeling the probability that a drone can complete a task based on environmental factors and onboard energy. This will require extensive field experiments on physical hardware. Furthermore, we made the assumption that the probability of each drone contributing to a task is an independent event. Removing this assumption could lead to new variation of the MPTS problem and should be considered in future work. We also see area for expanding the considered application, such as including wait times when drones move from one task to another or extending the problem to include cooperative teams of both drones and ground robots.

CHAPTER 6

COMMUNICATION MAPPING FOR ROBOT TEAMS

Modified from [175], as published in the IEEE Transactions on Field Robotics[1].

Jonathan Diller[2,3], John Rogers[4], Neil Dantam[2], Qi Han[2]

Communication is a fundamental building block in almost every robotics application, acting as the thread that holds together teams and connects humans to robots deployed in the field. However, accurately predicting how well robots will be able to communicate during deployment and incorporating such information into planning algorithms is particularly challenging. In this article, we present the Link Quality Communication Map (LQCM) - a method for mapping the potential for two robots to communicate. Our communication map builds a discretized representation of the environment and uses Expected Transmission Count (ETX), a link quality metric commonly used for data routing in wireless networks, to represent the ability of two robots to communicate in the environment. We also present a method for predicting ETX for pairs of robots. This article lays out the details of building an LQCM, highlights various properties inherent to these maps, discusses how these maps can be used in a variety of robotics applications, and reviews the results and lessons learned from our own deployments of LQCMs in the field. To validate our theoretical results, we generated communication maps for a variety of environments and used them for various robotics applications including multi-robot environmental monitoring and determining regions with guaranteed communication quality to a base station. Our results show an average decrease of 18.2% in data transmission times when compared to the current community standard for representing communication in robot planning and a decrease of over 90% in extreme cases. We also ran extensive experiments on ETX datasets from our field experiments to evaluate the accuracy of our ETX prediction method and evaluated how well the method handles malicious communication jamming.

6.1 Introduction

Communication is required for deploying robots into the world, serving as a fundamental element in almost every field robotics application. This is particularly the case in multi-robot and robot-human collaboration scenarios, and crucial to success for many applications, including search and rescue, agriculture, mining and resource extraction, environmental monitoring and surveillance. However,

[1]Reprinted, with permission, from Jonathan Diller, Communication Mapping for Robot Teams, IEEE Transactions on Field Robotics, 05/2025
[2]Department of Computer Science, Colorado School of Mines, USA
[3]Primary researcher and author
[4]DEVCOM, Army Research Laboratory, USA

accurately representing the ability for robots to communicate in planning algorithms is an open challenge in research today.

Traditional approaches for representing communication in robot planning can be divided into two main approaches: (1) the disk method where communication is assumed to be possible when within a predefined range and (2) the signal-to-noise ratio (SNR) method where algorithms attempt to maximize SNR between robots. However, decades of research in the networking community have found that the disk method is simultaneously overly conservative in prohibiting long range line of sight communications and overly optimistic in allowing short range communications through obstructions while SNR does not accurately represent the potential to transmit large volumes of data over a wireless network and cannot provide insight on multi-hop performance [72, 73, 176]. Over the last two decades, the networking community has developed various link quality metrics that more accurately capture wireless transmission throughput at the application level [6, 177], yet the robotics community continues to rely on limited and inaccurate approaches for incorporating communication into multi-robot planning algorithms.

Motivated by the shortcomings of methods commonly used in robotics, we pose the following research question: *What is a communication model for field robotics that accurately reflects application layer performance and how do we enable robots to build such a model?* The model should be able to inform us of how many robots are required and where to position robots to move data from robot to robot and from robot to humans. The disk method has been used to fill this need but again is not an accurate method for representing communication, particularly in cluttered environments. Furthermore, robots must be able to apply the model in areas that have not been fully explored. Many existing works in the community have attempted to do this by predicting point-to-point signal strength using machine learning. However, as previously stated, signal alone does not accurately represent the ability to communicate at the application level and these existing methods tend to require large volumes of training data. This paper provides a method for the robotics community to get on track with where the networking community has already gone.

To address this research question, we propose the Link Quality Communication Map (LQCM) - a model for representing the potential for a team of robots to communicate - and provide a method for predicting link quality in a given environment. Our LQCM uses the Expected Transmission Count (ETX) [178] link quality metric to represent the potential for robot communication. Expected Transmission Count provides application level insight on how likely a data packet is to be delivered to the receiver. Using ETX as the underlying metric gives LQCMs inherent properties, including determining where to position robots and the number of robots needed to maximize data-throughput in wireless networks formed by mobile robots. These properties make LQCMs ideal for multi-robot planning and help overcome the downfalls of previously proposed methods for representing communication in robot planning, such as making over

simplifying assumptions about communication or losing application-level insight. To the best of our knowledge, we are the first to propose mapping ETX for planning large robot team deployments.

Our contributions include:

1. the Link Quality Communication Map (LQCM), a discrete representation of the environment that depicts the potential for heterogeneous robots to communicate, which we show can be used to determine the optimal robot configuration and an optimal number of robots to deploy to maximize data-throughput;

2. an aggregated regression model, termed SVRF, for predicting ETX between any two points in the environment that can be trained using sparse datasets (200 to 400 data points) and adapt to changing radio frequency (RF) environments; and

3. experimental evaluation that validates the utility of the LQCM in a variety of robotics applications and that demonstrates the accuracy of our SVRF regression model.

We review related literature on the topics of robot team deployments and current approaches to mapping communication in the following section. Notably, we make the argument for a new networking metric in robotics in Section 6.2.1 where we demonstrate the shortcomings of existing models. In Section 6.3 we provide relevant background on link quality in wireless networks. In Section 6.4 we introduce the concept of an LQCM, and in Section 6.4.2 we describe our ETX prediction method. In Section 6.5 we discuss various properties of an LQCM and how they can be utilized in a variety of robotics applications. Section 6.6 summarizes our field experiments and results with concluding remarks in Section 6.7.

6.2 Related Work

In this section we summarize related work on communication-aware robot team deployment, the ETX link quality metric, and mapping communication.

6.2.1 Communication-Aware Robot Team Deployment

Most works on planning communication-aware robot team deployments utilize communication models that can be divided into two main approaches. The first approach is commonly referred to as the disk method. This is an all-or-nothing approach to communication, where whenever two robots are within a predefined radius they have maximum bandwidth but are unable to communicate whenever outside of this circle. This method has been widely adopted in robot planning research, with examples in multi-robot data collection using ground vehicles [66], data collection using drones [7, 20, 65], data relaying through ground vehicles [70], and relaying using drones [22].

The disk method relies on distance as the criteria for determining the ability to communicate. However, it is well known that data-throughput over a wireless link can vary greatly based on many factors beyond distance, including obstructing objects and the materials they are made from, multi-path signals (both as a means for communication and as a source of interference), environmental noise, interference from other devices, and even the time of day [179, 180]. It has been well documented how these complex factors invalidate the disk method approach for representing communication between two wireless devices [73] and are not useful for planning communication between multiple devices [176, 181].

The second approach is the signal-to-noise ratio (SNR) method. The concept behind the SNR method is to either maximize SNR between devices [3, 68] or to set a bound on an acceptable SNR value for communicating [182]. There are several examples where SNR and its close relative Received Signal Strength (RSS) are used as a network metric for maintaining peer-to-peer communication between collaborating robots [183, 184] or for data streaming applications via multi-robot teams [3, 185, 186]. These metrics have also been found to work well for certain robotics applications not specific to data transfer, such as localization [187, 188], because their behavior can be position-specific when one of the transmitters remains static.

However, SNR fails to give us application-layer insight on how well wireless devices will be able to transmit data [72, 73]. To demonstrate this, we will review the results from our own previous work where maximizing SNR improved the packet delivery rate, but paradoxically failed to improve video streaming quality.

Table 6.1 Average packet loss ratio (PLR) and VMAF scores from using SNR as a networking metric, compared to an agnostic baseline method, as presented in [3].

	Environment 1		**Environment 2**	
	Baseline	SNR	Baseline	SNR
PLR	**42.2%**	**0.1%**	59.9%	**26.9%**
Std. dev.	0.528	0.002	0.292	0.302
VMAF	**8.014**	**8.660**	0	**9.720**
Std. dev.	1.651	2.787	0	4.124

In [3], we deployed two robots in a network of hallways and placed a WiFi jamming device between the two robots. We fit the site-specific basic transmission loss model recommended by the International Telecommunication Union [67] with experimentally collected data and used Gaussian Process Regression for mapping noise from the WiFi jammer to predict the SNR between a relay robot and the two static robots. Using our SNR prediction method, we deployed the relay robot by maximizing the minimum SNR between any two robots and created a three-robot-long relay chain. We then measured two metrics for

moving data from one end of the chain to the other: (1) the Packet-Loss-Ratio (PLR) and (2) Netflix's VMAF score for streamed video [189], which assigns a numerical value to video based on expected user experience. A smaller PLR is better while a larger VMAF score is better. Table 6.1 summarizes our results from using SNR as a networking metric in two different indoor environments, comparing against an agnostic approach that only attempted to maximize signal strength while ignoring noise from a signal jammer. We conducted five trials for each approach in each environment.

Although utilizing SNR outperformed the agnostic approach, we still found a concerning trend in our data. Observe that in Environment 1, the VMAF scores are nearly identical yet the PLR for the agnostic method is significantly worse than the SNR method. In fact, the PLR is almost zero when maximizing the minimum SNR between any two robots in the chain, suggesting that our method for predicting SNR performed well, but this has marginal impact on data streaming. Furthermore, the SNR method found a worse PLR in Environment 2 compared to the near zero PLR found in Environment 1, but the resulting VMAF score was better in Environment 2 than it was in Environment 1. This is because SNR – and other metrics such as PLR – do not give us application-level insight on data transmission performance among multiple robots.

The networking community has since derived a variety of link quality metrics that better represent transmission throughput in wireless networks at the application level [6, 177] but the robotics community continues to use these dated and limited traditional approaches for handling communication in planning. Instead of using the agnostic disk method or an SNR approach with limited insights, we propose using the networking metric known as ETX [178] to represent the potential to communicate in multi-robot systems. The details of this metric are further discussed in Section 6.3.

6.2.2 Predicting & Mapping Communication for Robotics

To utilize ETX in robot planning, we need a method for creating a map of ETX. Although we are focused on mapping communication, we also consider predicting communication in this section because both problems involve relatively similar communication prediction methods with the former storing the predictions in some type of data structure. We will use the two terms interchangeably here.

There are several works that address the mapping of static networks [190–192]. On a related topic, there also is extensive work on estimating wireless link-quality in wireless networks [193–195]. Both bodies of work are mainly focused on improving packet routing between static nodes and do not consider how to create a map of the potential to communicate for mobile wireless nodes (robots).

Among works that focus on mapping communication for robots, almost all existing research has focused on predicting and mapping RSS or SNR, though works that state SNR tend to predict RSS and assume RF

noise is constant. We would like to note that some of the following works state that they are dealing with Received Signal Strength Indicator (RSSI) – an arbitrary and vendor specific non-negative integer to describe RSS for varying purposes on network interface controllers [196]. For clarity, RSSI is not the same as RSS and works that claim to be measuring RSSI present negative, real values in dBm (i.e. RSS measurements and not RSSI). To the best of our knowledge, no existing work predicts or maps RSSI and those that claim to do so are in fact dealing with RSS.

Works predicting and mapping RSS can be divided into two parts: (1) predicting signal strength for robot-to-fixed infrastructure communication and (2) predicting signal strength for robot-to-robot communication. Examples of predicting signal strength for robot-to-fixed infrastructure include [197, 198] where they use a distance-based method or [199, 200] where they use ray-tracing techniques in addition to distance to further refine predictions. These methods usually rely on propagation-loss models [67] and experimentally determine model parameters. Machine learning has been proposed as an alternative to using propagation-loss models, as seen in [3, 201] and [202] where they use Gaussian Process (GP) regression or in [203, 204] where they train Convolutional Neural Networks. The authors in [205, 206] propose an online approach to predicting signal strength that uses filtering, though is limited to predicting signal along a robot's trajectory and cannot predict signal in areas further away. Applications involving robot-to-fixed infrastructure include robot localization [202], path planning for data collection [70, 200], supporting disaster response operations and exploration [197, 198], or evading malicious communication jamming [3].

Predicting and mapping robot-to-robot communication has also been well studied with a sizeable body of literature. The authors in [69, 207] use a distance-based model that includes obstructions while in [208] they break the environment up into regions and propose directly measuring region-to-region RSS. However, the most common approach for predicting robot-to-robot communication is to use learning-based methods, namely GP. The most common way to use a GP for robot-to-robot communication is to use pair-wise robot position (i.e. $\mathbb{R}^4$ in two-dimensional space and $\mathbb{R}^6$ for three-dimensional space) as the input to the model. For examples of this on ground robots, see [209, 210] and [211], with an example using drones found in [212]. Although these methods have been shown to work well for predicting RSS in simplistic environments, we argue that using position alone will perform poorly in more complex environments with changing RF noise and showcases where this approach fails in our field experiments. The authors in [213] proposed using the ratio of occupied versus free cells in an occupancy grid as their model input in addition to position. In [184], the authors propose incorporating a path-loss model (such as those proposed in [67]) into the mean function prior of the GP. Breaking from GP, the authors in [214] propose using a three layer Neural Network (NN) for predicting signal strength in underground environments. Although GP and NN are powerful learning tools, we should expect them to require more data to train on as the number of

inputs increase, making them challenging to use when given limited data samples. Furthermore, none of these methods address heterogenous radio types. If each robot has a unique radio, then how well two robots will be able to communicate will not only depend on the location of each robot but also the radio type of each robot. To address these shortcomings, we present a regression model for predicting ETX that considers additional model inputs beyond distance and we show empirically that our model can be trained using smaller datasets.

As previously discussed, signal is not a good indicator of application-level performance. To the best of our knowledge, data transfer rate is the only other metric that has been mapped for robot-to-robot communication. The authors in [215] propose using the Minstrel algorithm – a rate control algorithm for mac80211 in the Linux kernel [216] – to predict the data transfer rate between a mobile robot and a static wireless device from direct measurements. However, this work does not address robot-to-robot communication, it requires taking point-to-point measurements for all region pairs, and it is not clear how data transfer rate alone can be used to plan multi-hop communication over large robot teams.

Among works that store communication information (i.e. construct a communication map) and use it for robot-centric planning problems, the most common method for representing this information is as a graph. The authors in [22, 210] and [211] form a graph where physical locations are represented as vertices and non-directional edges are added to the graph when they predict that two robots will be able to communicate when positioned at the location pair represented by the vertices. In [69], they form a fully connected graph with edge weights based on predicted RSS. However, these works assume that communication quality is bidirectional. That is, they assume the rate that robot a at position z_i can send data to robot b at position z_j is equal to the rate that robot b can send data back to robot a. This assumption does not always hold true, especially when one of the robots is in an area with excessive RF noise. In this work, we propose a communication map in the form of a directed graph with multiple layers, where robot-to-robot communication is not assumed to be bidirectional and different robot radio types are represented through new layers to the graph.

6.3 Background on Cumulative Link Quality Metrics

A cumulative link quality metric is a metric common in wireless networks where a numerical value is assigned to point-to-point links to describe the cost of transmitting data over that link. For multi-hop routes through wireless networks, the cost of the route is the sum of costs of all links in the route [6]. A cumulative link quality metric termed ETX was proposed in [178] and shown to outperform the traditional hop-count approach for data packet routing. ETX is a real number from 1 to ∞ and is defined as the

expected number of transmissions required to send a data packet over a network link. Mathematically,

$$ETX = \frac{1}{p_f p_r} \tag{6.1}$$

where p_f is the probability that the data packet is delivered to the receiving node and p_r is the probability that an acknowledgement packet is returned to the sending node. In multi-hop wireless networks, the ETX of a data packet route is the sum of ETX for each link in the route.

Expected Transmission Count is commonly used in multi-hop routing [177] and many variations of ETX have been proposed for specific wireless protocols [6]. We use ETX as our metric for this study because it is one of the fundamental building blocks for most other proposed cumulative link quality metrics and we argue that these more advanced methods can be substituted into our LQCM formulation and used in the same manner as presented here.

Furthermore, we chose ETX as our metric for mapping robot communication because it allows us to make theoretical assertions on networking performance. From [178] we get:

Assumption 1: *The highest data-throughput route through a wireless network is the route with the smallest ETX.*

Many experimental works have shown that this assumption holds true in most cases [178, 217, 218]. If we can predict ETX between arbitrary points in the environment, then *Assumption* 1 implies that we can determine robot configurations that optimize network performance. Neither RSS nor SNR can provide a guarantee on performance and it is not clear how such metrics could be used to determine an optimal multi-hop data-throughput route in wireless networks.

Although we focus on IEEE 802.11 (WiFi) in this article, ETX can be utilized to characterize peer-to-peer communication for other RF protocols, such as LoRaWAN (Long Range Wide Area Network) [219, 220] – a communication protocol that operates in the 863-928 MHz range [221]. Beyond RF, ETX has been proposed and investigated in underwater acoustic communication networks [222, 223] and in optical communication [224]. Although not investigated in this work, our proposed LQCM should also be applicable for robot-to-robot communication when using acoustic or optical communication technology.

6.4 Link Quality Communication Map

In this section we formally define a Link Quality Communication Map (LQCM) and propose a method for building an LQCM using aggregated regression models.

6.4.1 Link Quality Communication Map

Conceptually, an LQCM is a representation of the potential for robots to communicate in a given environment. In practice, an LQCM is a directed, weighted, fully-connected graph. The vertices of the graph represent physical locations in the deployment environment and static, wireless infrastructure. Edge weights of the graph are the ETX value (either measured or predicted) between two robots if they were each located at the respective physical locations represented by the vertices on either end of the edge. Figures 6.1(a) through 6.1(d) show a toy-example of building an LQCM for a simple environment.

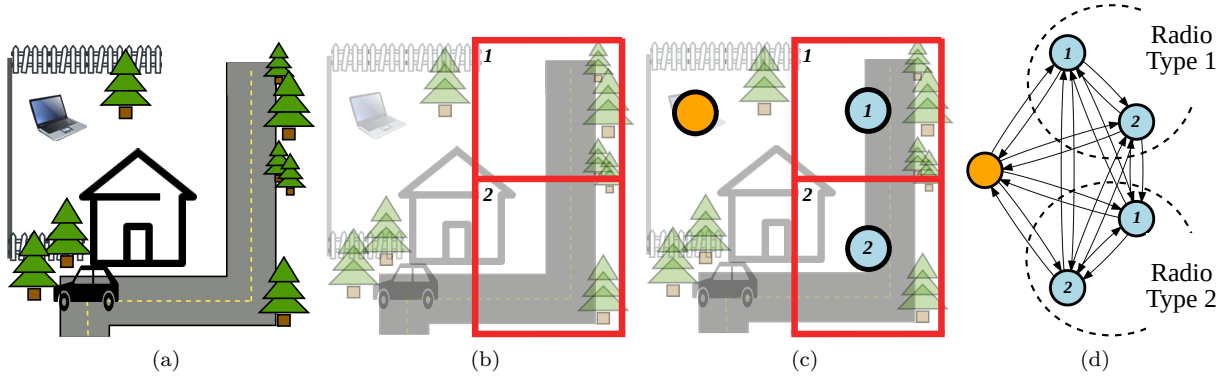


Figure 6.1 A simple example of a given environment $\mathcal{X}$ (6.1(a)), a discretization of $\mathcal{X}$ into Z (6.1(b)), generating vertices from cells in Z (6.1(c)), and a complete LQCM with a layer of vertices for each unique robot radio setup (6.1(d)).

Given continuous space $\mathcal{X}$ (Fig. 6.1(a)), we build an LQCM by discretizing $\mathcal{X}$ into a set of cells Z where each cell is a unique robot-accessible location in $\mathcal{X}$ (as depicted in Fig. 6.1(b)). The granularity of the cells is up to the user and can be based on the particular use case. Robots with identical wireless radio equipment (e.g. WiFi cards and antennas) can be assumed to be homogeneous in the LQCM. However, robots with different wireless radios will have varying ETX values between them when compared to a homogeneous radio configuration. Because of this fact, we add layers to the LQCM for each unique radio setup. Suppose we are given a team of robots with K unique radio setups. For each $z_i \in Z$, create K vertices $v_i^1, v_i^2, \cdots v_i^K$ where each vertex v_i^j represents a possible location for a robot with radio setup type j at physical location z_i (as depicted in Fig. 6.1(c) and 6.1(d)). Additionally, any static, wireless-enabled infrastructure, such as base stations, will have different ETX values with each unique robot radio setup. Let S be the set of all wireless infrastructure where each $s_i \in S$ represents the location in $\mathcal{X}$ where wireless device i is located.

Mathematically, an LQCM is the digraph $G = \{V,\ A,\ q\}$ defined as:

- $V = \bigcup_{j=1}^{K} V^j \cup S$,

- $A = V \times V$, and
- $q(v_i, v_j)$ is the ETX weight of arc $(v_i, v_j) \in A$.

where each $V^j = \{v_1^j, v_2^j, \cdots v_n^j\}$ is the set of deployment vertices for robot radio setup j. In Fig. 6.1(d), V^1 is the top set of blue vertices that represent radio type 1, V^2 is the lower set of vertices, and S contains the single orange vertex that represents the base station in the environment (depicted as a laptop).

For simplicity, we disregard occupied cells and cells that are inaccessible by a robot in $\mathcal{X}$. We assume that the ETX between two robots in a pair of cells is the same regardless of where the robots are positioned within each cell.

Observe that the size of the map depends on the size of the environment, the granularity of the cells, the number of robot radio types, and the number of static devices. Suppose that the number of cells required for a given environment is ϕ. For each K unique radio setup, we will add an additional ϕ vertices to G while each static device in S will add a single node to the graph. This means that the number of vertices in G will be $\phi K + |S|$. If the area of each cell remains fixed, then doubling the size of the environment will double the size of ϕ. Alternatively, if we fix ϕ and allow the granularity (i.e. the area of each cell) to change, then doubling the size of the environment will not impact the size of the graph. We note that the number of robots does not impact the size of G, only the number of unique robot radio setups (K).

Determining the granularity of the discretization of $\mathcal{X}$ depends on both the environment and the application for which the LQCM will be applied. More feature-dense environments will likely require a finer cell dimension. For example, an LQCM for Clearpath Jackal robots operating in a cluttered office space could consist of cells that are one-half square meters (roughly the size of a Jackal robot). Alternatively, large environments and applications where the robots communicate over long ranges can relax the size of each cell. For example, Clearpath warthog robots operating in large, open fields for an agricultural monitoring application could use cells with an area of 25 square meters. Furthermore, we use square cells in this work for convenience, but the cells could be any cellular decomposition of the environment [225].

We chose to use a discretized representation of the environment because it lends itself well for planning the actions of multi-robot teams. Examples of this are seen in [22, 210, 211], where communication is represented as a discrete graph to facilitate planning and coordinating robot actions. However, one possible way to build an LQCM in continuous space would be to remain in $\mathcal{X}$. Each robot would be represented as a discrete point in $\mathcal{X}$ and we would build a fully connected graph where each robot and static device would be vertices in the graph and edge weights would be defined as the ETX between each robot. A similar setup is seen in [69]. We further discuss advantages and drawbacks to a continuous-space representation of an LQCM in Section 6.5.2.

A limitation to using an LQCM is that it requires an accurate function $q(v_i, v_j)$. Directly measuring ETX for all possible arcs in A would require $n(n-1)$ measurements at $\frac{n(n-1)}{2}$ unique position pairs, where $n = |V|$. Alternatively, one could estimate $q(v_i, v_j)$ by predicting the ETX between points z_i and z_j, as discussed below.

6.4.2 Predicting ETX Between Robots

To build our LQCM, we need a method for predicting ETX between any two points in $\mathcal{X}$. In general, our prediction model extracts features from an occupancy grid and uses them as inputs for aggregated regression models.

In Section 6.3, ETX is defined mathematically as the inverse of the product of p_f and p_r. Observe that ETX is a rational function that grows to ∞ as p_f and p_r decrease. Rather than predicting ETX, we propose predicting $p_d = p_f p_r$, the probability that the data packet is received and acknowledged, which is defined in the range $[0, 1]$.

To predict p_d, we extract various features from occupancy grids to use as inputs to a regression model. Given two cells z_i and z_j in Z, let d_{ij} be the straight line distance between z_i and z_j. In most environments, the straight line between two random points will be obstructed by obstacles. Let o_{ij} be the number of occupied cells and u_{ij} be the number of unknown cells in an occupancy grid between z_i and z_j. By unknown cells, we mean cells that are neither known to be occupied nor known to be empty (often represented as '-1' in the grid). Both o_{ij} and u_{ij} can be found using Bresenham's line algorithm [226]. Let N_i and N_j be the RF noise at positions z_i and z_j, which can be measured directly or predicted using methods such as the one seen in [3], where Gaussian Process Regression is used to create a map of noise. Note that interference and RF chatter from other devices is included in both N_i and N_j. Let $\mathcal{G}_i$ be the radio gain of the robot type that we plan to deploy at z_i.

In many cases the obstacles between z_i and z_j will prevent RF signals from traveling directly from z_i to z_j. In such cases, if there still exists a connection between two robots located at z_i and z_j it is because the RF signal is following a multi-path route that goes around the obstacles between z_i and z_j. Let d^r_{ij} be the distance of the freespace path between z_i and z_j found using A* search from point z_i to z_j in the occupancy grid. To determine the likelihood that d^r_{ij} is more representative to the distance that a signal must travel from sender to receiver, we also consider the ratio between the straight-line and the freespace path distance, $\delta_{ij} = \frac{d^r_{ij}}{d_{ij}}$. The closer δ_{ij} gets to 1, the more likely it is that the two robots have line-of-sight with one another. Conversely, a larger value for δ_{ij} suggests that there is a large object between the two robots.

We define our prediction model's training set as $\{(\boldsymbol{x}_1, \boldsymbol{y}_1) \ \cdots \ (\boldsymbol{x}_n, \boldsymbol{y}_n)\}$, where $\boldsymbol{x}_l = [d_{ij}, o_{ij}, u_{ij}, \mathcal{G}_i, \mathcal{G}_j, N_i, N_j, d^r_{ij}, \delta_{ij}]^T$ is an input to our model and $\boldsymbol{y}_i = p_d$ is the desired model output.

We form our prediction model by combining a Support Vector Regression (SVR) model [227] and a Random Forest Regression model [228]. We use a Radial Basis kernel function [229] for the SVR. We run each model separately then average together the output of each model to get a final prediction. We chose a Random Forest because parts of our problem appear to be discrete decisions (e.g. does $o_{ij} = 0$?). However, we found that the Random Forest was prone to over fitting our data so we selected an SVR to help generalize our model.

We term this the Aggregated SVR+Random Forest (SVRF) model. Given a prior dataset, we can train our SVRF model to make p_d predictions for location pairs that we have not previously taken ETX measurements in. These predictions allow us to fill in weight values $q(v_i, v_j)$ for the LQCM digraph G.

6.5 Applying Link Quality Communication Maps

In this section we look at various properties of LQCM and discuss how to find paths in the maps. We then summarize how LQCMs can be applied in various robotics applications.

6.5.1 Properties of Link Quality Communication Maps

In wireless networks, the ETX of a data packet route is the sum of ETX for each link in the route, as discussed in Section 6.3. In the LQCM graph G, the ETX of a path through G is the sum of weights of the links in the path.

Suppose we would like to transmit data from z_i to z_j and need to determine where to place robots to transmit the data. Let the ordered sequence of vertices W_{ij} be the least ETX path in G from a vertex representing z_i to a vertex representing z_j. Building on *Assumption* 1, we get:

Proposition 7: *The robot configuration (i.e., where to place each robot) that maximizes data-throughput from a source device at z_i to a sink device at z_j is at the vertices in W_{ij}.*

Proof. By contradiction, assume that there exists a robot configuration $W_{ij}^{'} \neq W_{ij}$ that achieves higher data-throughput than the vertices in W_{ij}. From *Assumption* 1, $W_{ij}^{'}$ must be the least ETX path from z_i to z_j, but this contradicts the definition of W_{ij}. □

Furthermore, we can also determine how many robots are required to transmit data from z_i to z_j. We define $|W_{ij}|_r$ as the number of vertices in W_{ij} that represent physical locations. That is, $|W_{ij}|_r = |W_{ij} \cap (\bigcup_{j=1}^{k} V^j)|$. With $|W_{ij}|_r$, we get:

Proposition 8: *The optimal number of robots to deploy to maximize data-throughput from a source device at z_i to a sink device at z_j is $l = |W_{ij}|_r$.*

Proof. By contradiction, assume that the optimal number of robots to deploy to maximize data-throughput is $l^{'} \neq l$. From *Assumption* 1, the least ETX path from z_i to z_j must therefore have $l^{'}$ vertices. However, this contradicts the definition of W_{ij}. □

In other words, if we find W_{ij} in G, then the vertices in W_{ij} will represent the physical locations to position robots, with specific radio types, to minimize ETX from a source device at z_i to a sink device at z_j. Because putting robots at the locations in W_{ij} will minimize the ETX from the source device to the sink device, then the cardinality of W_{ij} tells us the exact number of robots needed to minimize ETX. *Assumption* 1 tells us that the highest data-throughput route through a wireless network is the route with the smallest ETX. This means that finding a robot configuration that minimizes ETX from the source device to the sink device will maximize data-throughput from the source to the sink. Determining this robot configuration simply requires us to find W_{ij} in G.

As an example of *Propositions* 7 and 8, consider the environment shown in Figure 6.1. We have two robots, a and b, where robot a has radio type 1 and robot b has radio type 2, and we want to send a robot to z_2 to stream data back to the base station. Suppose that the least cost path from the orange node to a vertex representing z_2 in the graph in Fig. 6.1(d) is $W_{ij} = \{s_1, v_1^2, v_2^1\}$. This tells us that we should place robot b (with radio type 2) at z_1 and robot a (with radio type 1) at z_2 to maximize data-throughput from z_2 back to the base station. *Propositions* 7 and 8 can be restated to include wireless infrastructure in S by finding $W_{ij'}$ from a vertex representing z_i to infrastructure vertex $s_{j'}$, or in reverse by finding $W_{j'i}$.

Note that the optimal number of robots to transmit data from z_i to z_j is not the same as the minimum number of robots to transmit data from z_i to z_j. As shown in our field experiments in Section 6.6, using more robots can sometimes improve data transmission performance compared to using fewer robots. Furthermore, communication quality can change based on a variety of factors, as discussed in Section 6.2.1, including the time of day. This means that *Propositions* 7 and 8 depend on an accurate prediction of ETX.

Propositions 7 and 8 assume that there are enough robots of the correct type for W_{ij}. However, in practice we will not always have enough robots for W_{ij}. Let R_k be the set of robots with radio setup k and $m_k = |R_k|$. We assume that $R_k \cap R_k = \emptyset$. Let $|W_{ij}|_r^k$ be the number of vertices in W_{ij} that represent robot radio setups of type k. That is, $|W_{ij}|_r^k = |W_{ij} \cap V^k|$. Let $\tilde{W}_{ij}$ be the least cost path from a vertex representing z_i to a vertex representing z_j in G where the following constraint holds:

$$|\tilde{W}_{ij}|_r^k \leq m_k,\ k \in \{1, 2, \cdots K\}. \tag{6.2}$$

Given $\tilde{W}_{ij}$, we get the following corollaries to *Propositions* 7 and 8:

Corollary 1: *Given robot team $R_1 \cup R_2 \cup \cdots R_k$, the robot configuration that maximizes data-throughput from a source device at z_i to a sink device at z_j is at the vertices in $\tilde{W}_{ij}$.*

Corollary 2: *Given robot team $R_1 \cup R_2 \cup \cdots R_k$, the optimal number of robots to deploy to maximize data-throughput from a source device at z_i to a sink device at z_j is $|\tilde{W}_{ij}|_r$.*

The proofs for *Corollaries* 1 and 2 are identical to the proofs for *Propositions* 7 and 8. Note that these corollaries are generalized to include wireless infrastructure. That is, $\tilde{W}_{ij}$ may include existing wireless infrastructure, along with multiple robots, as intermediate relay nodes to maximize data-throughput.

6.5.2 Finding Minimum Cost Paths in an LQCM

Propositions 7 and 8 require us to find minimum cost path W_{ij} through G with no restrictions on the length of the path or which nodes are included in the path. This can be done using Dijkstra's algorithm. Furthermore, we could improve the efficiency of this process by removing edges from our LQCM with excessively high cost or vertices that are unlikely to be used in W_{ij}, a topic that we cover in more detail later in the section.

We acknowledge that the A* search algorithm is more efficient than Dijkstra's algorithm for least-cost path problems when there exists an admissible heuristics function $h(v_i, v_j)$ (a function that guarantees a lower bound on the cost from vertex v_i to vertex v_j). However, it is challenging to define an admissible $h(v_i, v_j)$ for G that is meaningful due to the stochastic nature of wireless communication. Notably, $q(v_i, v_j)$ will not be an admissible heuristic because there may exist an alternative path through G from v_i to v_j that is less than $q(v_i, v_j)$. For example, suppose that there is a large metal object between the cells z_i and z_j that are represented by v_i and v_j, respectively, in G and $q(v_i, v_j) = \infty$, but a third cell $z_{i'}$ has line-of-sight with both z_i and z_j and $q(v_i, v_{i'}) + q(v_{i'}, v_j) = 3$. Another possible heuristics function would be to predict ETX between all pairs of cells in Z based on distance while assuming all cells have line-of-sight. However, this strategy again does not guarantee admissibility because ETX with line-of-sight at a given distance will change depending on the environment. This is seen in our field experiments in Section 6.6 where there is a notable difference in ETX between robots and the base station in an outdoors environment compared to an indoors environment. At the time of writing, the only guaranteed lower bound on ETX is a value of 1 (i.e. assuming there exists a perfect connection between z_i and z_j).

Corollaries 1 and 2 require us to find $\tilde{W}_{ij}$, a minimum cost path through G that adheres to the limited number of robots of each radio type. If $K = 1$ (i.e. all robots have the same radio setup) then $\tilde{W}_{ij}$ can be found using a variation of weighted breadth-first-search that limits the depth of the search to m_1 (the total number of robots available). However, if $K > 1$ (i.e. we are giving multiple radio types) then

Constraint 6.2 makes it more challenging to find $\tilde{W}_{ij}$.

Theorem 9: *Finding $\tilde{W}_{ij}$ in G when $K \geq 2$ is at least as hard as NP-Complete problems.*

Before proving *Theorem* 9, we would like to remind the reader of the 0-1 knapsack problem: Given a set I of n items, each with weight w_i and value p_i, form a subset of items I^s such that $\sum_{i \in I^s} w_i \leq W$ and $\sum_{i \in I^s} p_i \geq P$, for some given weight limit W and desired value P. The 0-1 knapsack problem is known to be NP-Complete [230]. The idea to our proof is to create a sequence of widgets that represent items in the 0-1 knapsack problem. We impose a "total cost" for traversing the widget sequence and give a cost discount for selecting items (activating widgets – as defined below) but we limit the number of widgets that can be activated using W. Figure 6.2 shows the basic concept of the widget sequence used in our proof.

Proof. We reduce the 0-1 knapsack problem into finding $\tilde{W}_{ij}$ in a special graph G_k as follows: For each item i, create a widget $\rho_i = \{\omega_i, v_i\}$, where ω_i is w_i sub-vertices in a directed tree with no branches and edge weights of 0 and v_i is a singular vertex that is not connected to ω_i. Form G_k by creating a start vertex v_s, an end vertex v_t, and then lining up all widgets in an ordered row between v_s and v_t. Add directed edges from v_s to the first ρ_1, where the edge weight going into v_1 is p_{max} and the edge weight going into the root of ω_1 is $p_{max} - p_1$, where $p_{max} = max(p_1, p_2, \ldots, p_n)$. For each ρ_i and ρ_{i+1}, add directed edges from ω_i and v_i to both ω_{i+1} and v_{i+1} with weights as described for edges going into ρ_1. The last pair ρ_n is then connected to v_t with a weight of 0. Designate all ω_i as robot type 1 and accompanying vertices v_i as robot type 2. With $m_1 = W$ and $m_2 = n$, find $\tilde{W}_{ij}$ in G_k. We consider widget i to be "activated" if $\tilde{W}_{ij}$ includes ω_i.

The activated widgets in $\tilde{W}_{ij}$ represent the items to include in I^s for the 0-1 knapsack problem. If $np_{max} - cost(\tilde{W}_{ij}) \geq P$, then there exists a subset (I^s) that meets the requirements of the given 0-1 knapsack problem. If $np_{max} - cost(\tilde{W}_{ij}) \ngeq P$, then no subset of I exists that meets the requirements of the given 0-1 knapsack problem because $\tilde{W}_{ij}$ will be a minimum cost path from v_s to v_t.

We have found a polynomial time reduction from the 0-1 knapsack problem to finding $\tilde{W}_{ij}$ in G when $K \geq 2$. Therefore, finding $\tilde{W}_{ij}$ in G when $K \geq 2$ is at least as hard as any NP-Complete problem. □

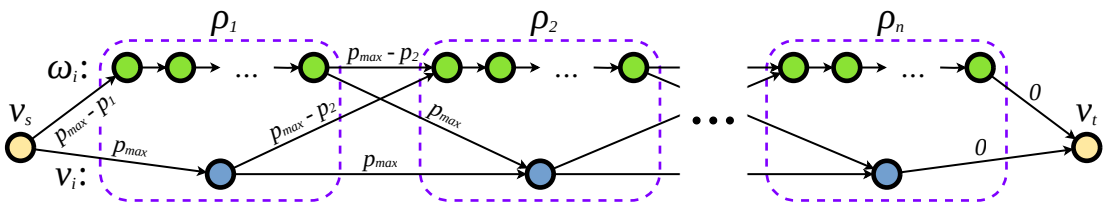


Figure 6.2 Graph G_k constructed from the 0-1 knapsack problem. Vertices grouped in purple represent a single widget, where green vertices are ω_i and blue vertices are v_i.

We note that the reduction to the knapsack problem is helpful to show the hardness of finding $\tilde{W}_{ij}$ in G while a more intuitive approach to finding $\tilde{W}_{ij}$ would be to search for constrained shortest paths (CSP) in G rather than modeling our problem as an instance of the knapsack problem. Finding CSPs in a graph is also known to be NP-hard [231], with well documented exact and approximate solutions [232]. In practice, we recommend attempting to find W_{ij} in G using Dijkstra's algorithm first. That is, attempt to find a robot configuration without restricting the number of robot radio types. If the found path breaks constraint (6.2) (i.e. the path requires more robots with a certain radio type than what are available), then an alternative method must be used to find a CSP in G. In our field experiments in Section 6.6, we found that using Dijkstra's algorithm is usually sufficient for finding $\tilde{W}_{ij}$.

Given the complexity of finding $\tilde{W}_{ij}$, it could be advantageous to remove vertices from G that are unlikely to be used. For any location $z_i \in Z$, it is unlikely that more than one vertex in G that represents z_i will be in $\tilde{W}_{ij}$. However, in the general case, removing vertices will break optimality guarantees from *Corollaries* 1 and 2. A possible case where one could remove cells is when robots with a certain radio type are not able to access an area of the environment that other robots with a different radio type are able to access. The layer that represents the restricted robot class could be trimmed down based on the accessibility of those robots without losing optimality guarantees. Another method for improving the search process for $\tilde{W}_{ij}$ is to remove unnecessary edges from G. For example, if there exists a path from v_i to v_j with a cheaper ETX cost than the edge (v_i, v_j), then we can safely remove (v_i, v_j) without compromising our optimality guarantees from *Corollaries* 1 and 2.

In Section 6.4.1 we briefly described one possible representation of an LQCM in continuous space, where each robot and static device would form vertices in a fully connected graph. The advantage to this approach is that we would move away from the accuracy limitations of discretizing our deployment space. The disadvantage of using continuous space is that determining where to deploy robots and how many robots to deploy to maximize data-throughput (that is, achieving *Corollaries* 1 and 2) becomes more complicated. To keep the guarantees of *Corollaries* 1 and 2 in continuous space, we would need to define a function, $f(z_i, z_j) : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, that takes the positions $z_i, z_j \in \mathcal{X}$ of robots i and j and determines the ETX of transmitting from robot i to robot j. To determine an optimal robot configuration when moving data from location z_i to location z_j would require us to minimize $f(z_i, z_1) + f(z_1, z_2) + \cdots f(z_n, z_j)$ by selecting the location of $z_1, z_2, \cdots z_n$, determining a value for n, and determining which robot should be placed at each position z. This is a new combinatorial problem that is further complicated when considering physical obstacles. The main advantage of our discretized space representation for the LQCM is that it simplifies this combinatorial problem and in most cases finding $\tilde{W}_{ij}$ in an LQCM can be done using Dijkstra's algorithm, as shown in Section 6.6.

6.5.3 Applications of Link Quality Communication Maps

We argue that LQCMs can be applied to any multi-robot scenario where the robots rely on wireless communication between one another and between robots and human operators. In this section we highlight how an LQCM could be applied to a variety of common multi-robot applications where high-level planning must account for wireless communication.

The maps can be used to determine how to place robots for data relaying and streaming applications such as [3, 233–236]. From *Propositions* 7 and 8, we can determine the optimal number of robots to deploy and where to place each robot for streaming applications by finding $\tilde{W}_{ij}$.

The map can also be used to determine regions of the environment that a team of robots can explore while guaranteeing a minimum wireless connection to a base station, similar to the problems addressed in [182, 215]. Suppose we have determined that an acceptable connection to the base station is an ETX of 3. Any cell in Z with a vertex in G that has a least cost path of 3 or better to the base station is a cell that can be explored while guaranteeing an acceptable wireless connection to the base station.

Link Quality Communication Maps can also be used when planning data muling problems where teams of robots are used to collect data from sensors, as seen in [3, 22, 66]. The algorithms discussed in these works could be further augmented by adding an LQCM and determining data collection and relaying configurations that reduce the time required to deliver data back to a base station.

(a) *Mock-Town* Environment (b) *Camp* Environment

Figure 6.3 Jackal robots in the *Mock-Town* (a) and *Camp* (b) environments.

6.6 Evaluation in the Field

To evaluate the utility of our LQCM and our proposed ETX prediction method, we ran field experiments in three outdoor environments (*Mock-Town*, *Courtyard*, and *Camp* shown in Fig. 6.4) and one indoor environment (*Office*). The *Mock-Town* is a robotics experimental arena set up to replicate an urban

environment [237]. The *Courtyard* is located on the Colorado School of Mines campus and is an open space with small concrete barriers situated between academic buildings. The *Camp* is a group of camp lodgings in a heavily wooded area located in West Point, New York. The *Office* is a long hallway that connects office spaces and class rooms and also located on the Colorado School of Mines campus. In the *Mock-Town* and *Camp* we used a team of three Clearpath Jackal robots and then added a Raspberry Pi 3 as a base station for the *Courtyard* and the *Office*. Figure 6.3 shows robots in the *Courtyard* and the *Camp* environment. Each robot was equipped with an USRP B205mini-i software-defined radio (SDR) that functions as a spectrum analyzer to measure RF noise. We lowered the gain of the TX power on the base station to reduce the distance that a robot needs to travel to break connection with the base station. The Raspberry Pi 3 had its TX power reduced to 2 dBm for the *Courtyard* and 1 dBm for the *Office*. The further reduction in the *Office* was because the Raspberry Pi was able to communicate noticeably further indoors. At this setting, the robots would lose communication with the base station at roughly 40 m in line-of-sight conditions. In the *Camp* environment we also deployed another USRP SDR to broadcast RF noise, similar to the Wi-Fi jammer setup described in [3]. We used OmniMapper [238] on a single Jackal robot to create occupancy grids for each environment.

(a) *Mock-Town* Environment (b) *Courtyard* Environment (c) *Camp* Environment

Figure 6.4 Locations of outdoor field experiments.

We created two networks for our experiments: a test network and an administrative network. The test network consisted of the Jackal on-board Intel WiFi cards provided by Clearpath and the WiFi peripheral built into the Raspberry Pi. The test network was used to take ETX measurements and stream data. The potential to communicate over this network is what we mapped out in an LQCM for each scenario. The administrative network consisted of a powerful UniFi access point, a commercial WiFi router, and an additional Waveshare WiFi card on each Jackal robot. We used the administrative network to control the robots and coordinate their actions. The motivation for two separated networks is to move the robots beyond the limits of their communication range (introducing communication failure) over the test network while still allowing us to safely control the robots on the stronger administrative network.

To measure ETX between two robots, or between a robot and the Raspberry Pi, we had the sending node transmit 1,000 data packets. The receiving node would then send 1,000 acknowledgement packets back to the transmitter, regardless of how many packets successfully arrived at the receiver. We then counted the total number of data and acknowledgement packets received and used these numbers to calculate p_f and p_r, respectively, by dividing the number of packets received by 1,000. The data packets were each 1,400 bytes large and the acknowledgement packets were 2 bytes. We used User Datagram Protocol (UDP) to transmit packets for ETX data collection because we wanted to count the number of packets that successfully transmitted over connectionless sockets without any handshake dialogue.

In each environment, we used two or three robots to take point-to-point ETX measurements over the test network and (when applicable) had each robot measure ETX with the Raspberry Pi base station. We manually drove the robots into different locations in each environment, seeking to find configurations where we got representative samples of both extremes for each model input discussed in Section 6.4.2. We took ∼200 measurements in the *Mock-Town*, ∼290 in the *Courtyard*, ∼400 in the *Camp* (300 with the Wi-Fi jammer and 100 without) and ∼260 in the *Office*.

6.6.1 ETX Prediction Accuracy

To evaluate our proposed ETX prediction method, we trained our proposed Aggregated SVR+Random Forest (SVRF) model on the ETX measurements collected from all three environments. As discussed in Section 6.4.2, we predict p_d, the probability that a data packet is successfully delivered and acknowledged, as oppose to ETX directly because ETX is defined on the interval $[1, \infty)$ where as $p_d \in [0, 1]$.

We compare our SVRF model against an adaptation of the ETX model found in [184], where they suggest that p_d can be derived from a signal prediction. We predict signal between two robots using a similar method to [3] by line fitting data to the signal-to-distance function $f_{dB}(d) = -10nlog_{10}(d) + c$, proposed in [67, 180], where n and c are constants that fit $f_{dB}(d)$ to our data. We then predict p_d from bidirectional SNR, assuming a linear mapping between the two. We fit $f_{dB}(d)$ and our SNR-to-p_d mapping using our experimental data and the Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm [239, 240]. We term this approach the Signal-Model (sigMod). We also constructed a Bayesian Ridge Regression (BR) model [241] and a Gaussian Process Regression model (GP) with a radial basis function kernel [242], both using the same inputs discussed in Section 6.4.2. For a fourth baseline approach, we trained a second GP model that only uses location-pairs as model inputs (GP-pos). We chose the GP methods because using a GP model was proposed in other works to map RSS between robots using position [209–212] and position with obstruction [213], and for mapping RF jamming signals [3]. To compare against an NN approach, we implemented the simple NN proposed in [214] for predicting signal strength in a mine. The NN takes the

same input as our SVRF model and has a single, fully connected 16 unit hidden layer. Additionally, we trained a Deep NN (DNN) that has four fully connected hidden layers (a 16 unit layer, two 32 unit layers, and another 16 unit layer), all using a restricted linear activation function. We also compare our model against the individual SVR and Random Forest (R-F) models without averaging together the prediction, as discussed in Section 6.4.2. We used TensorFlow [243] to build both the NN and the DNN models and used scikit-learn [244] to build the other models.

6.6.1.1 Model Accuracy

To assess the accuracy of the various ETX prediction models, we trained all approaches 50 times on randomized training-test set splits and averaged together the Mean Squared Error (MSE) and R-Squared score (R2). A smaller MSE score is better while a higher R2 score is better. The MSE metric tells us how accurately each approach can predict p_d (the probability that a data packet is successfully delivered and acknowledged), penalizing predictions that are further away from the desired output. An MSE value of $2.0{\cdot}10^{-2}$ equates to roughly 14% error when the true p_d value is 1 while an MSE value of $6.0{\cdot}10^{-2}$ equates to roughly 26% error. We would like an MSE value close to or below $2.0{\cdot}10^{-2}$ and do not want values above $6.0{\cdot}10^{-2}$. The R2 score is a "goodness of fit" metric that tells us how well the inputs predict the desired output. We prefer an R2 score above 0.5, which tells us that there is positive correlation between the model inputs and the outputs, and we do not want a negative R2 score, which indicates that the model is a poor choice for the data. Due to the limited number of measurements, we chose an 80-20 train-test split ratio.

Table 6.2 Prediction Accuracy at *Mock-Town* and *Courtyard*

	Mock-Town				*Courtyard*			
	Training Set		Test Set		Training Set		Test Set	
	MSE	R2	MSE	R2	MSE	R2	MSE	R2
sigMod	$4.47{\cdot}10^{-2}$	0.083	$4.71{\cdot}10^{-2}$	0.022	$3.55{\cdot}10^{-2}$	0.106	$3.51{\cdot}10^{-2}$	0.089
GP	$9.17{\cdot}10^{-3}$	0.812	$3.60{\cdot}10^{-2}$	0.242	$1.28{\cdot}10^{-2}$	0.678	$1.66{\cdot}10^{-2}$	0.556
GP-pos	$9.55{\cdot}10^{-3}$	0.804	$3.55{\cdot}10^{-2}$	0.254	$1.18{\cdot}10^{-2}$	0.704	$\mathbf{1.50{\cdot}10^{-2}}$	**0.600**
NN	0.200	-3.13	0.214	-3.37	0.347	-7.83	0.390	-9.39
DNN	$2.06{\cdot}10^{-2}$	0.577	$3.15{\cdot}10^{-2}$	0.345	$1.07{\cdot}10^{-2}$	0.731	$1.61{\cdot}10^{-2}$	0.565
BR	$3.12{\cdot}10^{-2}$	0.360	$3.35{\cdot}10^{-2}$	0.302	$1.99{\cdot}10^{-2}$	0.500	$1.98{\cdot}10^{-2}$	0.474
R-F	$5.22{\cdot}10^{-3}$	0.893	$\mathbf{2.98{\cdot}10^{-2}}$	**0.377**	$3.39{\cdot}10^{-3}$	0.915	$1.78{\cdot}10^{-2}$	0.527
SVR	$1.52{\cdot}10^{-2}$	0.688	$3.53{\cdot}10^{-2}$	0.259	$1.21{\cdot}10^{-2}$	0.695	$1.69{\cdot}10^{-2}$	0.552
SVRF	$8.23{\cdot}10^{-3}$	0.831	$\mathbf{3.00{\cdot}10^{-2}}$	**0.371**	$6.17{\cdot}10^{-3}$	0.845	$1.58{\cdot}10^{-2}$	0.579

Tables 6.2 and 6.3 summarize our results, listing the average MSE and R2 values from both the training and test datasets. In all four environments, both the sigMod and BR approaches struggled to capture meaning from the data, with both having negative R2 scores in the *Office*. The GP model performed well in certain environments, such as the *Office* where it tied for best, and had similar results at times between

the training set and test set, as seen on the *Courtyard* dataset, which suggests that the model is robust against over fitting. However, this model did not perform particularly well in the other environments, with performance comparable to the BR model and noticeably struggling on the *Camp* dataset. The GP-pos model performed better than the other models on the *Courtyard* dataset. This environment is fairly open and using position information was sufficient to determine peer-to-peer ETX here. However, the GP-pos model did not perform as well on the *Mock-Town* and *Office* data and had a negative R2 score in the *Camp* environment with changing RF noise. Interestingly, both the GP and GP-pos models performed the best on the training dataset in the *Camp* environment then saw a major drop in performance on the test data, suggesting major over fitting. Although GP models are powerful regression tools, we believe that they are struggling to predict ETX in most environments due to a limited number of data points. The GP-pos model specifically is failing in the *Camp* environment due to the changing RF noise that was not position dependent.

Table 6.3 Prediction Accuracy at *Camp* and *Office*

	Camp				*Office*			
	Training Set		Test Set		Training Set		Test Set	
	MSE	R2	MSE	R2	MSE	R2	MSE	R2
sigMod	0.355	-1.19	0.358	-1.29	$6.28{\cdot}10^{-2}$	-0.29	$6.49{\cdot}10^{-2}$	-0.37
GP	$4.43{\cdot}10^{-3}$	0.973	0.112	0.286	$1.28{\cdot}10^{-2}$	0.738	$\mathbf{2.03{\cdot}10^{-2}}$	**0.581**
GP-pos	$5.15{\cdot}10^{-3}$	0.968	0.223	-0.425	$1.44{\cdot}10^{-2}$	0.704	$2.42{\cdot}10^{-2}$	0.490
NN	0.468	-1.89	0.493	-2.12	0.245	-4.01	25.3	-430
DNN	$4.97{\cdot}10^{-2}$	0.694	$7.15{\cdot}10^{-2}$	0.548	$1.88{\cdot}10^{-2}$	0.615	4.07	-65.6
BR	$9.71{\cdot}10^{-2}$	0.400	0.107	0.321	$2.15{\cdot}10^{-2}$	0.560	0.432	-6.875
R-F	$1.34{\cdot}10^{-2}$	0.917	$6.88{\cdot}10^{-2}$	0.563	$4.07{\cdot}10^{-3}$	0.917	$2.25{\cdot}10^{-2}$	0.534
SVR	$4.83{\cdot}10^{-2}$	0.702	$7.35{\cdot}10^{-2}$	0.535	$1.35{\cdot}10^{-2}$	0.723	$2.07{\cdot}10^{-2}$	0.574
SVRF	$2.39{\cdot}10^{-2}$	0.852	$\mathbf{6.39{\cdot}10^{-2}}$	**0.596**	$7.17{\cdot}10^{-3}$	0.853	$\mathbf{2.02{\cdot}10^{-2}}$	**0.583**

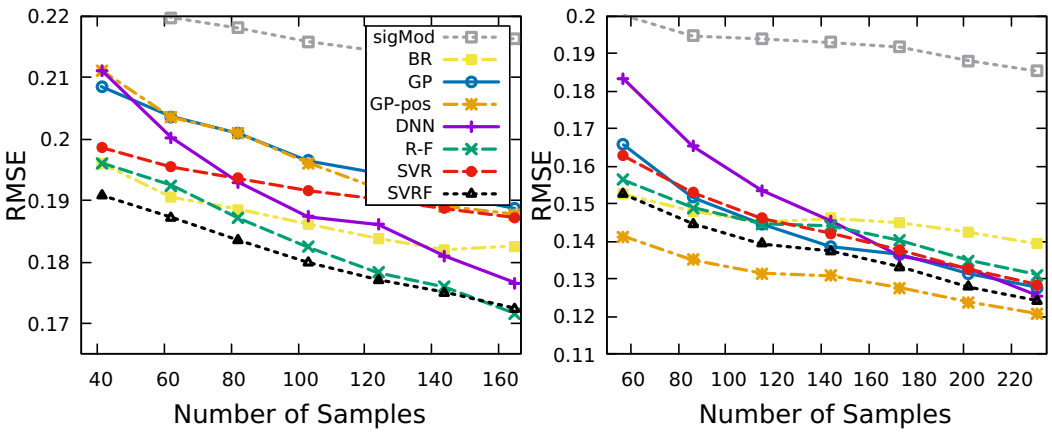
The NN model performed worse than all other models on every dataset, with consistently high MSE and a negative R2 score in every environment. This same model was found to perform quite well in [214], where they had a dataset with over one million measurements and trained with a batch size of 2,048, suggesting that this NN would have performed better for predicting p_d if we had a substantially larger dataset. The DNN performed well at times, ranking second best in the *Camp* and third in the *Mock-Town* and *Courtyard*. However, the DNN struggled greatly with the *Office* data. We used the *Courtyard* dataset when designing and initially evaluating the DNN. We believe that this causes the DNN's design to work well for outdoor environments but not for indoor environments, suggesting that the design of the model must change from environment-to-environment for this approach to work well.

The Random Forest model performed well on the *Mock-Town* and *Office* datasets but was not as consistent on the other datasets. This model outperformed all of the other models on the training data for the *Mock-Town*, *Courtyard*, and *Office* datasets but sees a large drop in performance on these test sets. This suggests that this model is over fitting to the data and does not generalize well. The SVR model does not appear to suffer from over fitting and had a reasonable drop in performance from the training data to the test data for all datasets. However, the SVR model by itself does not uniformly perform better than the other models.

The trend in the Random Forest and SVR performance motivates our proposed SVRF model. The SVRF model outperformed all other models on the test data in the *Camp* and *Office* environments, while performing second best in the *Mock-Town* and *Courtyard* environments, with a near tie on the former of the two. These results show that the SVRF model provides a nice balance between the superior results of the Random Forest and the generalizability of the SVR model.

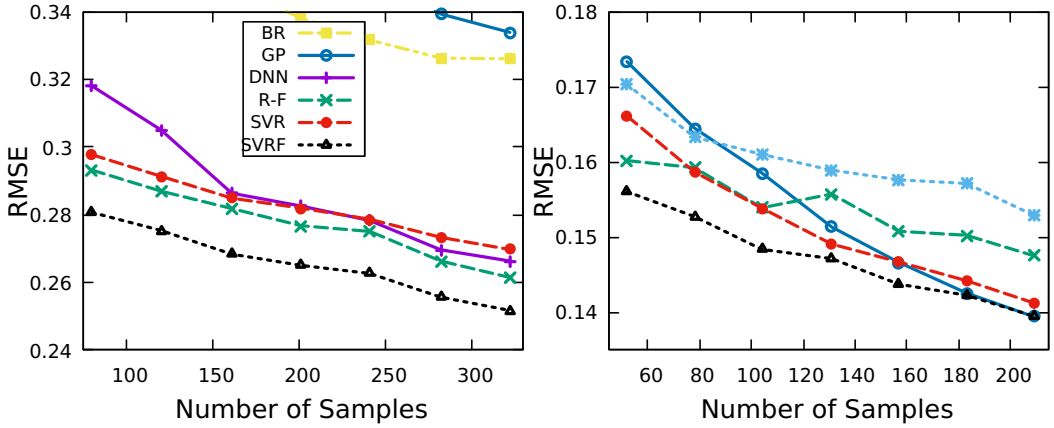
6.6.1.2 Training on Sparse Datasets

To evaluate how data sparsity impacts performance, we varied the size of the training set, training each regression model 50 times at each interval, and measured the average Root Mean Squared Error (RMSE). The RMSE tells us the average prediction error in p_d, where a lower RMSE is better. We prefer an RMSE at or below 0.15 and do not want a value above 0.25, which is an error of roughly 15% and 25%, respectively, when p_d is close to 1.



(a) *Mock-Town* Dataset (b) *Courtyard* Dataset

Figure 6.5 Graphs (a) and (b) show the average RMSE as the size of the training set varies on the *Mock-Town* and *Courtyard* datasets, respectively. Image (c) shows two of the Jackal robots (second robot at top right) in the *Mock-Town*.



(a) *Camp* Dataset (b) *Office* Dataset

Figure 6.6 Graphs (a) and (b) show the average RMSE as the size of the training set varies on the *Camp* and *Office* datasets, respectively. Image (c) shows two of the Jackal robots in the *Camp*.

Figures 6.5 and 6.6 graph our results for each of the datasets. We note that some of the models had a noticeably worse RMSE, causing them to not appear on the graph. For example, the NN performed very poorly in the *Mock-Town* environment, where the minimum achieved RMSE was 0.39, so the results for the NN do not appear in any of the figures while the sigMod and GP-pos models did not perform well enough to appear in the figures for the *Camp* and *Office* datasets.

Our SVRF model reaches the desired accuracy (0.15) with around 80 to 100 measurements on the *Courtyard* and *Office* datasets. No model reached our preferred error level on the *Mock-Town* and *Camp*datasets, with the latter only reaching an acceptable error level (0.25) after 300 measurements. However, our SVRF model is trending downward on all datasets as the number of measurements increase, suggesting that the model would eventually hit the preferred error level if given more data points.

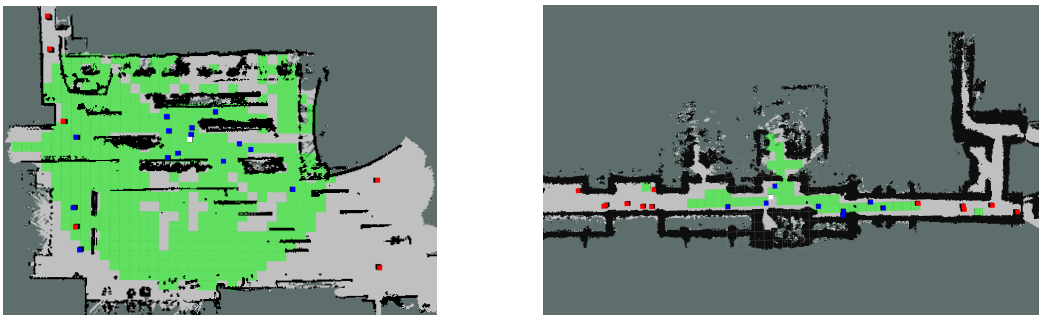
The model-versus-model performance is mostly the same as already discussed in Section 6.6.1.1, with the SVRF model consistently stronger than the other approaches on the *Mock-Town*, *Camp*, and *Office* datasets and second best on the *Courtyard* dataset. The only model that has a notable performance trend different from the rest is the DNN, which is trending downwards at a steeper rate than most of the other models on the outdoor datasets (*Mock-Town*, *Courtyard*, and *Camp*). The same trend is seen with the GP model on the *Office* dataset. These trends suggest that the DNN and GP models could eventually outperform the SVRF model if given enough data in the environments where they perform well (outdoor environments for the DNN and indoors for the GP). All of the considered datasets each took several hours to collect due to the process required to measure ETX, meaning that the SVRF model's robustness to sparse datasets makes it more ideal than the DNN and GP. Furthermore, neither of these alternative

approaches are as universally adaptable to different environments as the SVRF model.

6.6.2 Maintaining Connectivity with Another Device

Our LQCM can be used to determine what subregions of the environment a robot can manuever through while maintaining a desired communication link with another device. The other device could be another robot or static infrastructure. To demonstrate this, we used an LQCM in the *Courtyard* and *Office* environments to determine which areas in the environment a robot could operate in while still maintaining a desired ETX connection with the base station. Possible applications for this setup include a robot exploring the environment while streaming sensor data back to the base station or when a robot is remotely operated from the base station, as seen in works such as [245, 246].

In the *Courtyard*, we cycled through every vertex in the LQCM and checked the predicted probability of delivery, p_d, from the *robot* to the *base station*. If the p_d was above 0.5 (an ETX of 2) we highlighted the cell green that is represented by that vertex in the LQCM. In a similar manner, we checked the predicted p_d from the *base station* to the *robot* vertices in the *Office* environment and highlighted all corresponding cells green where $p_d \geq 0.75$ (and ETX of 1.33). For each setup, we randomly selected 20 locations throughout the environment and measured p_d (from the robot to the base station for the *Courtyard* and from the base station to the robot in the *Office*).



(a) *Courtyard* (b) *Office*

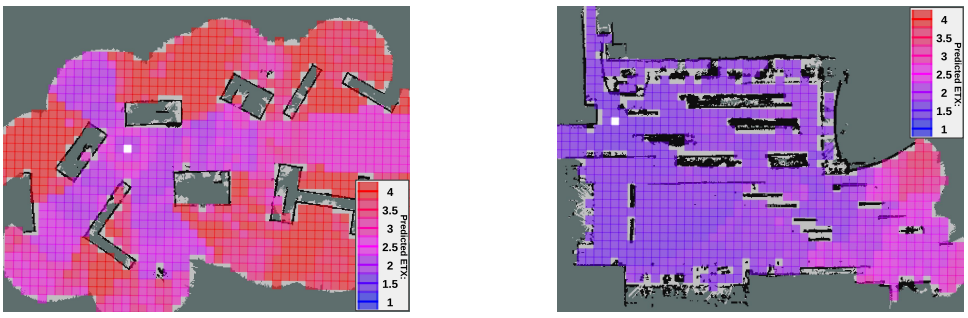
Figure 6.7 Connectivity results from the *Courtyard* and *Office*. Green shading are subregions that are predicted to meet the desired communication quality, blue markers are locations where p_d was measured above the desired threshold, red markers are locations where p_d was measured below the threshold, and the white markers are base stations.

Figure 6.7 shows the subregions of each environment (shaded in green) where the robot can maintain the desired communication link with the base station. The blue markers show where p_d was above the desired threshold and the red markers are where p_d was below the threshold. For clarity, we desire blue markers in shaded regions and red markers outside of shaded regions. In the *Courtyard* only two out of 20

measurements were not as predicted while in the *Office* only a single measurement out of 20 did not match the prediction. These experiments show how an LQCM can be used for identifying subregions of the environment where a robot can operate while still maintaining a desired communication link with another device.

6.6.3 Generating and Searching Maps

We generated LQCMs for each of the environments. As discussed in Section 6.4.1, we discarded occupied cells and cells that are inaccessible by a robot to avoid generating an excessive number of cells for locations that are not feasible to deploy a robot. The outdoor environments (the *Mock-Town*, the *Courtyard*, and the *Camp*) have several open areas and each cover a large space, so we chose to use 2×2 meter-sized cells when discretizing these environments. This created 1,197 cells for the *Mock-Town*, 819 cells for the *Courtyard*, and 1,158 for the *Camp*. We chose a cell size of 0.6×0.6 for the *Office* because the hallways are much more narrow, with several small spaces that a robot can access but are too small to be accurately represented with a 2×2 meter cell. This created 1,046 cells for the *Office*.

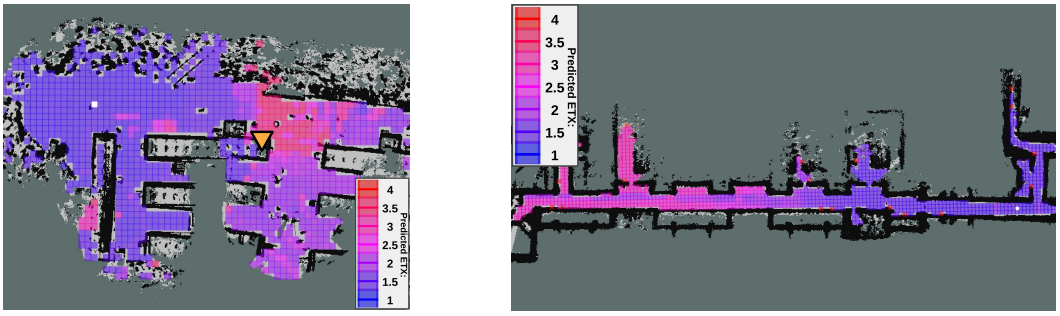


(a) *Mock-Town* (b) *Courtyard*

Figure 6.8 Link Quality Communication Maps for the *Mock-Town* and *Courtyard* environments. Each cell's color represents the predicted ETX between a robot at the white cell and a robot at that specific cell, where blue is a perfect connection (ETX = 1) and red is a weak connection (ETX = 4).

Figures 6.8 and 6.9 show the LQCMs for each environment. The color of each cell depicts the predicted ETX between a robot at the white cell and a robot at that specific cell. Blue represents a perfect connection (ETX = 1) and red is a weak connection (ETX = 4). In Fig. 6.8(a), we can see the strong shadowing of each building in the *Mock-Town*. These buildings are all made out of metal and are extremely hard to penetrate with an RF signal. The affect of obstacles is much lower in the *Courtyard* because the small concrete barriers are not as difficult to communicate around, as seen in Fig. 6.8(b). In Fig. 6.9(a), we can see the impact of the WiFi jammer at the *Camp*, located at the orange triangle in the figure. These buildings are easier to communicate through than in the *Mock-Town*, so the impact of obstacles is not as

noticeable but the impact of the WiFi jammer is very pronounced. Figure 6.9(b) shows the *Office* environment, which is mostly a narrow hallway. There are two sets of double doors that separate the two main segments of the hallway, which are not fully aligned. This misalignment is what creates the noticeable shadowing along the sides of the hallway on the left side of the figure, which is what we expect to see.



(a) *Mock-Town* (b) *Courtyard*

Figure 6.9 Link Quality Communication Maps for the *Camp* and *Office* environments. Each cell's color represents the predicted ETX between a robot at the white cell and a robot at that specific cell, where blue is a perfect connection (ETX = 1) and red is a weak connection (ETX = 4).

To evaluate the computational cost of searching for paths in an LQCM, we randomly selected robot source and sink locations in each of the environments and used Dijkstra's algorithm to get robot configurations from the LQCM, as discussed in Section 6.5.2. We chose Dijkstra's algorithm instead of other CSP algorithms because we argue that, in most cases, Dijkstra's algorithm is sufficient for finding valid robot configurations, as discussed in Section 6.5.2 and further investigated in Section 6.6.4. We used the disk method as an alternative approach for finding robot configurations. For the disk method, we assume that any two robots will be able to communicate when within a predefined distance, which was determined experimentally. When required, we selected additional robot deployment locations by minimizing the maximum distance between any two neighboring robots. We chose this baseline because it is often used to plan robot-to-robot communication [22, 66] and to plan robot-to-infrastructure data collection applications [7, 20]. For a second baseline we used the SNR method, as presented in [3]. For this approach, we predicted robot-to-robot signal strength using the method described in Section 6.6.1 and predicted noise using k-nearest neighbors and noise measurements collected when exploring the maps. To determine if additional robots are needed to connect the source and sink robots, we set a minimum SNR threshold required for two robots to be considered connected. We then incrementally increased the number of relaying robots, searching the map for positions that maximize the minimum connection between any two robots, until the predicted robot-to-robot SNR was above the predefined minimum.

Table 6.4 Average computation time / standard deviation (both in milliseconds) to search for robot configurations.

Environment	*Mock-Town*	*Courtyard*	*Camp*	*Office*
DM	< 1 / < 1	< 1 / < 1	< 1 / < 1	< 1 / < 1
SNR	6.05 / 0.22	6.2 / 0.41	4,516 / 3,114	1,946 / 3,047
LQCM	8.65 / 2.25	4.0 / 0.32	8.65 / 0.88	6.75 / 1.02

Table 6.4 shows the results of searching for 20 robot configurations in each environment. The disk method was the most computationally efficient and always computed in under one millisecond. The SNR method took around six milliseconds to find configurations in the *Mock-Town* and *Courtyard* but noticeably struggled in the *Camp* and *Office*. The latter two areas a very cluttered and the SNR method is very sensitive to obstructions, preferring line-of-sight when possible. This caused the algorithm to struggle to find satisfactory configurations at times in the *Camp* and *Office* where there is less room for additional robots to be positioned with line-of-sight between one another. Searching the LQCM with Dijkstra's algorithm was not as fast as the disk method but consistently solved in under 10 milliseconds. Our envisioned use for LQCMs is for them to be integrated into the mission planning phase of a robot deployment, where the search only runs periodically. Therefore, we argue that a computation time at or below 10 milliseconds is reasonable for the intended use.

6.6.4 Environmental Monitoring Application

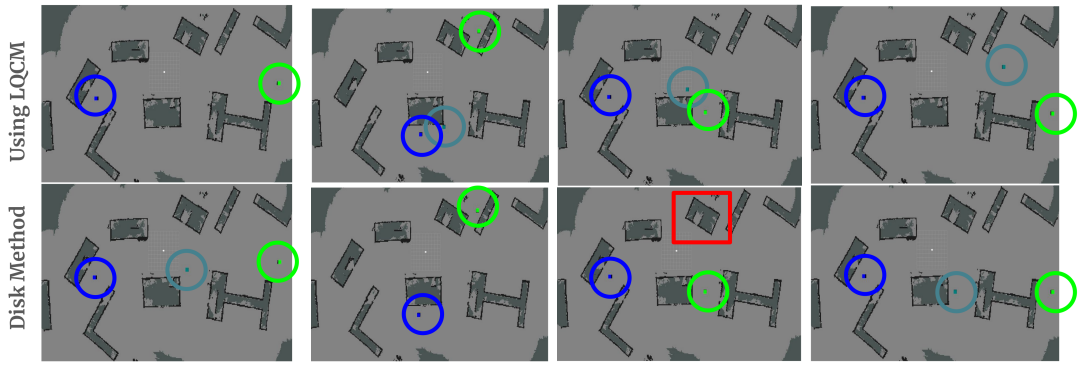
We ran data transmission experiments in the two outdoor environments to demonstrate how an LQCM can be used for relaying data in an environmental monitoring application. We first looked at robot-to-robot data relaying in the *Mock-Town* then looked at streaming data from a robot to a base station in the *Courtyard*. In both environments, we transmitted large chunks of data and recorded the time required to move the data. We used total transmission time as our metric because this encompasses other common networking metrics (e.g. packet latency and packet error rate) while focusing on application-layer performance. We kept the robots stationary during data transfer because we are interested in comparing how well the LQCM represents robot-to-robot communication when compared to other classical methods. However, depending on the application, the robots may need to continue moving while communicating. For these types of scenarios, our LQCM would be incorporated into application-specific path planning algorithms, similar to how communication graphs have been used in works such as [22, 69, 211].

We again used the disk method and maximizing SNR as baselines. We were able to find valid robot configurations in the LQCM for every scenario using Dijkstra's algorithm (avoiding the need to solve for CSPs as discussed in Section 6.5.2).

Table 6.5 Transmission Times for Environmental Monitoring Application

	Mock-Town Environment				*Courtyard* Environment			
Scenario	1	2	3	4	5	6	7	8
DM	12.4	66.6	**1.7**	291.85	48.1	37.2	168.4	**17.5**
SNR	-	-	-	-	∞	40.0	207.2	27.4
LQCM	**7.3**	**4.1**	3.9	**11.6**	**38.6**	41.7	**23.4**	27.1

For robot-to-robot data relaying, we designated one robot as a data source and another robot as a sink to replicate a scenario where robots are collecting and relaying data between each other. We moved the source robot and sink robot around the *Mock-Town* and used the LQCM to determine if additional robots were required to improve data-throughput and to decide where to position additional relay robots when needed. We only considered scenarios where the LQCM proposed a different robot configuration than the configuration found using the baseline. For each considered configuration, we had the source robot send the sink robot 100 data packets, each 110 kB in size for a total of 11 MB, and recorded the time required to complete the transmission. We repeated this experiment 10 times for each setup. This data relay experiment is meant to imitate transmitting larger data files, such as data muling applications where robots are used to collect sensor data [7, 22, 66].



(a) *Scenario 1* (b) *Scenario 2* (c) *Scenario 3* (d) *Scenario 4*

Figure 6.10 Environmental monitoring application in the *Mock-Town* environment. Blue markers show the source robot, green markers show the sink robot and turquoise are relay robots. The red box marks a building discussed in Section 6.6.4.

Figure 6.10 shows the various source-to-sink scenarios we experimented with. Table 6.5 shows the results for each scenario. Except for *Scenario 3*, using our LQCM resulted in an increase in the data transmission rate for all scenarios in the *Mock-Town* environment. In *Scenario 1*, we see an example of where the disk method added an extra robot that slowed down data streaming while *Scenarios 2* and *3* show examples of where the disk method did not add an additional robot when the LQCM did. The

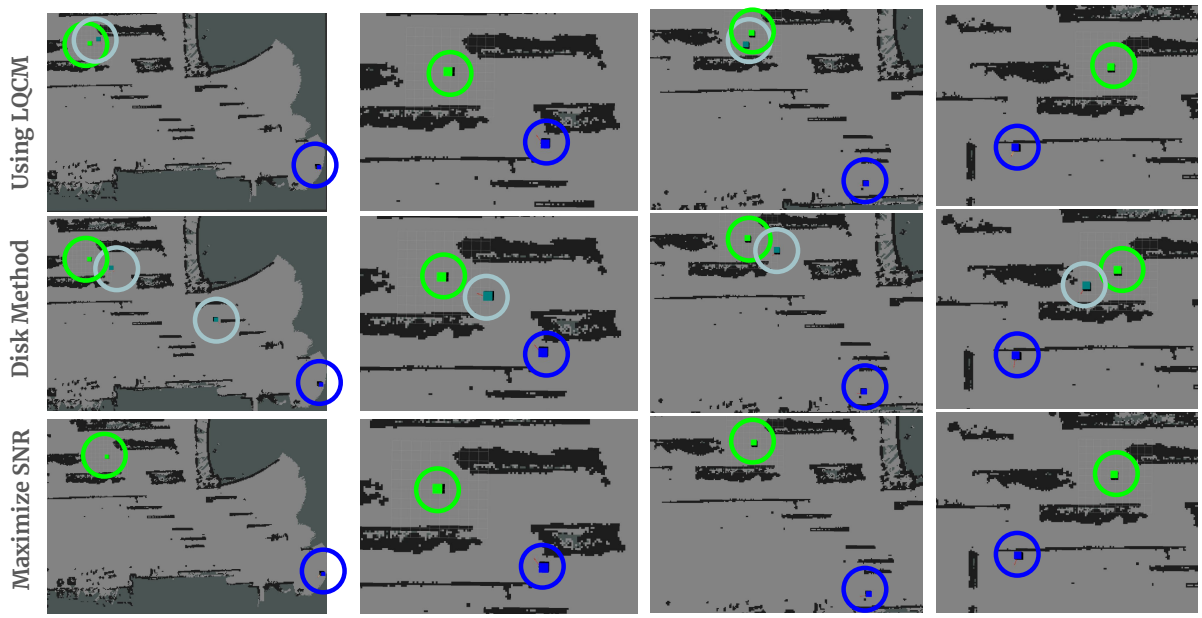
additional robot was needed in *Scenario 2*, where the LQCM clearly outperforms the disk method, but was not needed in *Scenario 3*. We hypothesize that our method failed to outperform the baseline in *Scenario 3* because the large building (marked red in 6.10) was positioned perfectly to reflect radio waves directly from the source to the sink robot and our ETX prediction method was unable to accurately predict this behavior. Interestingly, the fastest data-transfer rate found by the LQCM was in *Scenario 3*, suggesting that using the LQCM is still a competitive method in scenarios where the ETX prediction may have some error. Finally, *Scenario 4* shows an example where both the LQCM and the disk method chose the same number of robots but placed them in different locations. The disk method was agnostic to how the buildings would impact communication and placed a relay robot between two buildings. This caused the disk method to struggle greatly, averaging over 290 seconds to transmit the data, while using the LQCM only took 11.6 seconds, a transfer rate comparable to the other scenarios. On average, using the LQCM decreased transmission time by 26.6% in the *Mock-Town* environment, which extreme cases (*Scenarios 2* and *4*) seeing over 90% decrease in transmission time.

For the robot-to-base-station streaming scenarios in the *Courtyard* environment, we designated one robot as a data source and positioned the other robots to help relay the data back to the base station (as needed). We again used an LQCM to determine when any additional robots were needed to connect the source robot to the base station. For each configuration, we had the source robot send the base station 714 data packets, each 1.4 kB in size for a total of 1 MB, and recorded how long it took to transmit the data. We repeated the experiment 10 times for each robot configuration. This experiment imitates streaming sensor readings, such as a video feed as seen in [3, 247, 248].

Figure 6.11 shows the various robot-to-base-station scenarios and Table 6.5 shows the results for each scenario. The disk method outperformed using an LQCM in *Scenario 8* while the LQCM outperformed the disk method in *Scenarios 5* and *7*, and both methods performed comparable in *Scenario 6* despite finding different robot configurations. The disk method was more eager to add additional relay robots than the LQCM, doing so in all but *Scenario 7*. We believe that the LQCM failed to outperform the disk method in *Scenario 8* because the prediction model is pessimistic on how well the robot could communicate with a relay robot and determined that communicating directly with the base station would be more efficient. On average, using the LQCM still decreases the transmission time by 9.7% when compared against the disk method in the *Courtyard* environment and by a total of 18.2% across all scenarios in both environments. The SNR method found the same robot configuration as using an LQCM in *Scenarios 6* and *8* but differed from the LQCM in *Scenarios 5* and *7*, using one less robot than the LQCM. In *Scenario 5*, the SNR method picked a robot configuration where the source robot was unable to connect to the base station at all. In *Scenario 7*, using the LQCM decreased transmission time by 88.7% compared to using the SNR

method.

These environmental monitoring experiments highlight the downfalls of each baseline and demonstrate how using an LQCM helps avoid these downfalls and improves multi-robot team performance. The disk method tends to be too conservative while also being agnostic to how obstacles will impact communication, which became particularly problematic in the *Mock-Town* environment where the buildings are more difficult to communicate through. On the other hand, the SNR method is responsive to the environment but tends to be too liberal, leading to scenarios where using SNR results in complete communication failure. Using the LQCM gives us a balance between the other two methods, where we are aware of how obstacles will impact communication while not being too optimistic about peer-to-peer performance.



(a) *Scenario 5* (b) *Scenario 6* (c) *Scenario 7* (d) *Scenario 8*

Figure 6.11 Data streaming in the *Courtyard* environment.

6.7 Conclusions, Limitations, and Future Work

This work presents the Link Quality Communication Map (LQCM), a way of mapping the potential for two robots to communicate using the Expected Transmission Count (ETX) link quality metric. We presented various properties of LQCM and highlighted how they can be used in a variety of distributed, multi-robot applications. Our field test results support our claims for why communication-aware robot planning should use an LQCM instead of traditional methods for representing communication. We also presented an aggregated regression model, termed SVRF, for predicting ETX between points by extracting features from an occupancy grid, including robot-to-robot distance, obstruction, RF noise, radio power

settings, and a prediction on signal multi-path distance. Using data collected on physical robots, we showed that our ETX prediction model outperforms other approaches. To promote research integrity, we have made the ROS package used for generating an LQCM and our experimental data publicly available[5].

This work shows great promise in closing the gap between agnostic communication models used in robot planning and real-world robot-to-robot communication but has limitations. In this work, we assumed that the physical environment is static and known a priori. This limitation could be addressed by rapidly updating the structure of an LQCM online, as changes to the environment are discovered. Future work should focus on incorporating transfer learning methods (for example, see [249]) to overcome this limitation and make our prediction models more robust to changing domains.

Another limitation is that collecting ETX measurements to train our SVRF model is time consuming – each of the presented datasets took several hours to collect. Possible approaches to overcome this limitation include adapting prior knowledge to novel environments with similar characteristics, using simulations to generate training data as suggested in [203, 204], or using filtering methods such as those seen in [194, 205, 206] to speed-up the data collection process. If given a larger dataset, more powerful learning approaches, such as Gaussian Process Regression or Deep Neural Networks, may prove to be better ETX predictors. Additionally, other factors, such as the material type of obstacles and the time of day, impact ETX but were not incorporated into our ETX prediction model. One possible extension to this work is to create scene graphs, similar to the ones found in [250], that hold information on predicted object material types, which could be incorporated into our ETX prediction process.

To simplify the process of determining how many robots are required and where to position robots to move data from robot to robot and from robot to humans, we chose to discretize the environment. We showed that this discrete representation provides optimality guarantees when selecting robot configurations but at the limitation that we may be losing communication quality accuracy through the coarseness of the regions or from a poor decomposition of the environment. Future work could further investigate methods for building communication maps using ETX in continuous space. Another limitation of our proposed LQCM design is that it currently has no mechanism for handling how a single robot can be used for multiple data paths through the graph. The ETX along any given data path would depend on how strong the connection is along the other data paths, the size of the data packets, and the interval of communication (a continuous data stream versus periodical data transfer). To the best of our knowledge, this topic has not been fully investigated.

In addition to addressing the current limitations of LQCM as presented here, we see many areas for future work. We plan to investigate how to integrate building an LQCM with the multi-robot

[5]https://github.com/JonD07/CommunicationMap

Simultaneous Localization and Mapping problem. We also plan to expand our LQCM for three-dimensional maps and include multi-modal robots (e.g. accommodating both ground vehicles and drones). As discussed in Section 6.5.2, finding certain types of paths in an LQCM is hard and more work is needed in identifying and evaluating algorithms for finding such paths in an LQCM.

CHAPTER 7
HOLISTIC FRAMEWORK FOR MULTI-DRONE DATA COLLECTION IN IOT SYSTEMS

Jonathan Diller[1,2], Qi Han[1], Matthew Desaulniers[1]

Rotary-type Unmanned Aerial Vehicles (commonly referred to as drones) are often proposed as a method for collecting data from stationary wireless sensors. However, drones may need to adapt their behavior online due to the stochastic nature of wireless communication. This causes offline planners to be too optimistic, often ignoring online adaptive behavior, which can lead to data collection plans that are not plausible due to the limited onboard energy of drones. Conversely, relying solely on online decision making will lead to poor performance. To address this problem, we propose a holistic drone planning framework for planning and validating how to deploy teams of drones to collect data from wireless sensors. Our framework forms an offline plan while adhering to communication and energy constraints, then runs Monte-Carlo simulations with an online adaptive planner to evaluate the plausibility of the offline plan. Using data collected from real-time simulation, the framework updates planning parameters to improve the offline plan. For offline planning, we propose both a complete solution using Mixed Integer Programming and a polynomial-time algorithm that turns the problem into a variation of the Vehicle Routing Problem and solves a convex optimization program, which finds locally optimal solutions. For our online algorithm, we solve a simplified variation of the convex problem from our offline planner. We evaluate our framework and algorithms through extensive numerical simulations as well as a proof-of-concept implementation on a physical drone and wireless sensor testbed.

7.1 Introduction

Wi-Fi enabled Internet of Things (IoT) sensors are used in a variety of application areas including environmental, flora and fauna, industrial, urban, military and health monitoring. Traditionally, these sensors have been connected to form larger networks so that data can be forwarded to a central data center through multi-hop routing [251]. A drawback to this approach is that the nodes closest to data sinks become weak points in the network. To mitigate this problem, a lot of research has looked into collecting data via mobile agents, so called "data-mules" [18]. Drones are often proposed for data-muling because they can easily traverse rugged terrain, hover in place while communicating with ground nodes, and reach areas that are challenging for humans and ground robots to access. Figure 7.1 depicts the multi-drone data-muling scenario.

[1]Department of Computer Science, Colorado School of Mines, USA
[2]Primary researcher and author

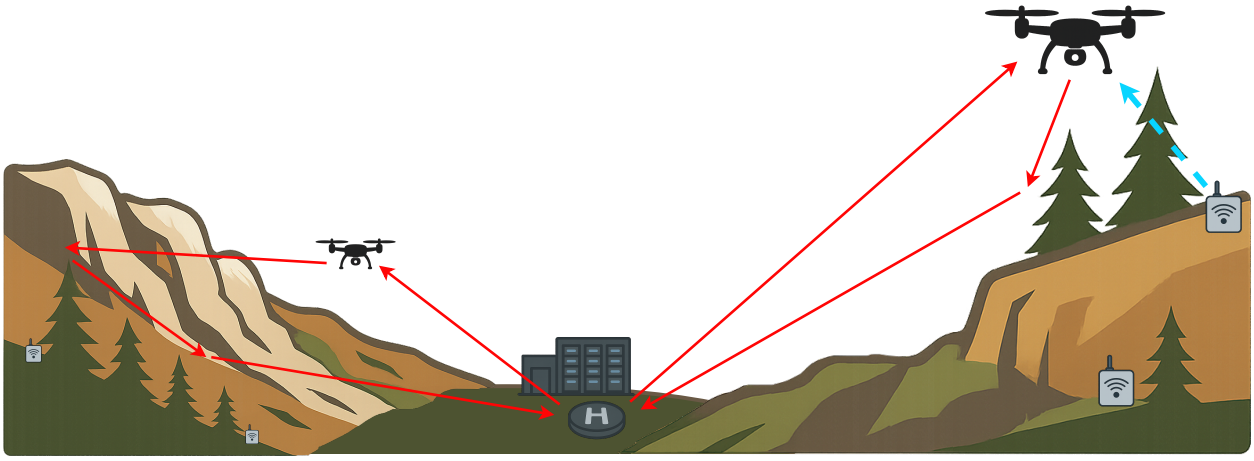


Figure 7.1 Multi-drone IoT data-muling scenario.

Although drones have many advantages for data-muling, there are several challenges that must be considered when planning routes for IoT data collection tasks. One major challenge in using drones is their finite on-board energy storage, requiring them to periodically recharge or change out on-board batteries. Another challenge is that drones take much longer to collect data from IoT sensors compared with the traditional connected network approach. Furthermore, wireless communication can be challenging to accurately predict and failed or weak communication scenarios require the drones to adapt their behavior during deployment. This motivates the need to form efficient collection paths for multi-drone teams while accounting for limited energy and the stochastic nature of wireless communication.

There are many proposed solutions in recent literature on planning efficient data collection routes for drones. These works generally fall into one of two categories: offline mission planning and online adaptive behavior. However, none of the existing literature takes a holistic approach to this problem. The stochastic nature of wireless communication, particularly from an IoT sensor that is deployed in harsh, outdoor environments, means that any solution must include some means for adapting behavior online when drone-to-sensor communication fails. We not only need a strategy for adapting behavior, but our offline planner must incorporate this behavior into planning. An overly optimistic offline planner will not save energy for adaptive behavior during deployment, which means the drone may fail to collect all of the data or risk running out of energy.

Another shortcoming in existing literature is a lack of data-driven approaches. Offline planners are great at finding optimal or near optimal solutions, but they tend to be disconnected from the performance we see during deployment on physical hardware. In our previous work [7], one of the major limitations that we found in our field prototype was that neither our offline planner nor our drone simulator could mimic

the performance we saw on our physical hardware.

In this study, we formulate a holistic framework for planning and deploying drone swarms for the IoT data-muling problem. Our framework first finds an idealistic offline plan using powerful solvers in an Offline Global Planner. The framework then refines this plan by iteratively running Monte Carlo simulations to determine realistic energy and dynamics statistics on the found plan and feeds this back into the offline planner. Additionally, the framework includes an Online Adaptive Planner that can adapt the plan found offline should the drone fail to communicate with an IoT device.

This work extends our previous work in [7], and makes the following major additional contributions:

1. We extend our planning framework by including energy allocation and drone behavior feedback via data-driven Monte Carlo simulations (in Section 7.4).

2. We look at how the distance between two devices impacts data transfer rate (in Section 7.3.3) and then investigate the tradeoff between increasing the transfer rate versus the time spent moving closer to a wireless sensor (in Sections 7.8.1 and 7.8.3).

3. We provide a complete solution that finds the optimal answer to the offline path planning problem (in Section 7.5.1).

4. We present a more refined polynomial-time algorithm for offline planning that finds locally optimal solutions and outperforms our previously proposed solution (in Section 7.5.2).

5. We lay out a new algorithm for adapting drone behavior online should drone-to-sensor communication fail (in Section 7.6).

To the best of our knowledge, this work is the first to consider planning for drone data-muling in a coupled, offline-online manner and the first to propose using a combined drone and network simulator to provide performance feedback to an offline planner. To promote good research integrity and to help further the frontiers on science, we have made the source code to our framework and planning algorithms open-source[3].

7.2 Related Work

Existing literature has addressed a variety of open problems in multi-drone data-muling. One common topic is how to cluster together sensors and select a "cluster head" [24, 252–254]. A cluster head is a single wireless node that pools together the data of neighboring nodes to transmit to the drone. A similar problem focuses on selecting hovering locations to collect data from multiple nodes at once [20, 255, 256].

[3]https://github.com/JonD07/HolisticFramework

A natural follow-on problem is to then determine an efficient ordering to the hovering locations [5, 21, 23, 28, 65, 257, 258]. An alternative to grouping together sensors is to have drones visit sensors individually and focus on finding efficient drone paths [35, 259–262]. In this work, we do not directly consider cluster heads and focus on collecting data from nodes individually. However, existing solutions could be applied to select cluster heads that are then treated as individual nodes in our problem setup.

Solutions for solving these various problems can be grouped into two categories: offline planning and online planning. The majority of online approaches involve some form of Reinforcement Learning (RL). Works such as [263, 264] use deep-RL approaches to control a single drone. Multi-drone planning is often done using multi-agent RL algorithms, as seen in [265–268]. The main motivation for the online path planning problem setup is that some aspect of the problem is not known *a priori*. For example, in [263, 266] they consider a problem where data generation is unknown prior to deployment while the authors in [265, 268] consider dynamic environments. Although not as common, there do exist works that assume prior knowledge of the system but choose to plan and coordinate actions online [253, 267]. This work is primarily focused on planning with total system knowledge (e.g. known sensor locations, data quantities, etc.) but also incorporates how to adapt behavior when data collection fails during deployment.

There is a large variety of solution approaches for offline planning, many of which involve transforming the data collection problem into some other, well studied problem. One common approach for multi-drone scenarios is to divide up the sensors into groups then treat each group of sensors as a Traveling Salesman Problem (TSP) [65, 258]. Treating the problem as a variation of the TSP is also common for single drone scenarios [20]. Popular approaches for solving TSP in these works include the Genetic Algorithm (GA) [253, 260, 269], the Ant Colony algorithm [255, 270, 271], or mathematical optimization techniques [5]. Another common strategy is to treat the data collection problem as a variation of the Vehicle Routing Problem (VRP) [252, 254, 257, 272], which is solved using similar approaches as the TSP. Other related graph theory problems (and their common solutions) have also been used, such as the Orienteering Problem [23].

There are also many works that do not convert the problem into an input instance of well-studied existing solutions. Among these, formulating mathematical optimization problems is quite common [34, 261, 262, 273]. Other heuristic-based solutions include iteratively building solutions [28] or nearest-neighbor methods [35]. There are also examples of learning methods for offline single drone problems, such as [24, 259, 274] where they propose a deep-RL approach or [21] where they use a transformer. In this work, we propose both an optimization-based approach that finds optimal solutions and a VRP approach. However, our drone planning framework can be used with any offline planner.

Among these problem setups and algorithms, there are a variety of metrics for evaluating a solution. Some of the more common metrics include drone energy [24, 259], the total volume of data collected [23, 266], the volume of data collected from any single sensor [261, 262], the average data latency (sometimes referred to as Age-of-Information - AoI) [255], the maximum data latency [19, 21, 65, 269], or some combination of these metrics [256, 274]. This work uses average data latency, but our framework can be utilized with any of these metrics.

7.3 Problem Formulation

We consider a scenario where there are n heterogeneous, wireless-enabled IoT sensors deployed throughout a three-dimensional space $\mathcal{X}$ based on application needs. Each sensor is gathering application-specific data that must be collected by a drone and returned to a data center located at $x_b \in \mathcal{X}$. Let $I = \{s_1, s_2, \cdots, s_n\}$ be the set of sensors. Let the position of s_i in three-dimensional space be $x_i \in \mathcal{X}$ and q_i be the number of megabytes of data to be collected from s_i, both of which are known a priori. Let J be a set of m heterogeneous drones that will be launched from the data center at x_b and must collect all of the captured data from the sensors and return it to x_b.

We formally define a drone sub-tour $\mathcal{T}$ as a sequence of tuples $\langle(\tilde{x}_i, s_i) \mid i \in \mathcal{T}\rangle$, where $\tilde{x}_i \in \mathcal{X}$ and s_i is a sensor that the drone will collect data from while at $\tilde{x}_i$. It is assumed that the drone will move from x_b to the first $\tilde{x}_i$ in $\mathcal{T}$ and from the final $\tilde{x}_i$ in $\mathcal{T}$ back to the data center. For operational safety and mathematical simplicity, we assume that x_b is located at a predefined distance above the ground at the data center. We argue that this assumption is justified because ascending and descending between the ground and x_b at the beginning and end of $\mathcal{T}$ will take a constant amount of time and energy and is independent of $\mathcal{T}$.

7.3.1 Communication Model

We define *servicing* a sensor as a drone connecting wirelessly with a sensor and collecting all data held on that sensor. From our field experiments, we found that the data transfer rate (in megabytes per seconds) from the sensor to the drone can be modeled as:

$$tx(d_{ji}) = min(\frac{a_i}{d_{ji}^2 + c_i} + b_i,\ r_{max\text{-}i}) \tag{7.1}$$

where d_{ji} is the distance from drone j to sensor i, values a_i, b_i and c_i are constants and $r_{min\text{-}i}$ is the maximum expected data transfer rate. These latter four values must be determined experimentally. We assume there there is a known minimum data transfer rate, $r_{min\text{-}i}$, that must be met to successfully transmit data between two devices.

Suppose that a drone is hovering at $\tilde{x}_i$ when it services s_i (which is located at x_i). The time required to service i will be:

$$t_h(\tilde{x}_i, x_i) = \frac{q_i}{tx(||\tilde{x}_i - x_i||_2)} \tag{7.2}$$

where $||x||_2$ is the Euclidean norm of x and q_i is the volume of data to collect from i. Given a drone sub-tour $\mathcal{T}_j$, the total time spent servicing the sensors on $\mathcal{T}_j$ will be:

$$t_s(\mathcal{T}_j) = \sum_{(\tilde{x}_i, s_i) \in \mathcal{T}_j} t_h(\tilde{x}_i, x_i) \tag{7.3}$$

We assume that only a single sensor will communicate with the drone at a time but acknowledge that in practice packet collision can be handled by the MAC layer protocol.

7.3.2 Energy Model

Drones have limited on-board energy, usually in the form of batteries, and can only operate for so long before needing to either swap-out batteries or recharge. Previous works have shown that drone power consumption is a function of drone speed [4, 5], which was verified by [63]. Let $\mathcal{P}(v)$ be the function that maps speed v to power consumed, in watts. If a drone travels for t seconds at speed v (in meters per second), the drone will consume $\mathcal{P}(v)t$ joules. Let β_j be the total usable joules on drone j.

Assuming that drone j moves at an average speed of v_j on sub-tour $\mathcal{T}_j$, the total time required to travel $\mathcal{T}_j$ (excluding time spent hovering while servicing sensors) will be:

$$t_m(\mathcal{T}_j) = \frac{1}{v_j}\big(||x_b - \tilde{x}_1||_2 + \sum_{i \in \mathcal{T}_j} ||\tilde{x}_i - \tilde{x}_{i+1}||_2 + ||\tilde{x}_\varphi - x_b||_2\big) \tag{7.4}$$

where $|\mathcal{T}_j| = \varphi$. The total time to complete sub-tour $\mathcal{T}_j$ will be the sum of the time to travel through the sub-tour and the time to service all sensors on the sub-tour. Mathematically,

$$t_{tot}(\mathcal{T}_j) = t_s(\mathcal{T}_j) + t_m(\mathcal{T}_j) \tag{7.5}$$

Similarly, let the total energy to complete $\mathcal{T}_j$ be:

$$\mathcal{J}(\mathcal{T}_j) = \mathcal{P}(0)t_s(\mathcal{T}_j) + \mathcal{P}(v_j)t_m(\mathcal{T}_j) \tag{7.6}$$

Each drone a_j will have a sequence of sub-tours, $\mathcal{K}_j$, that contains one or more sub-tours, $\mathcal{T}_j$. Between consecutive sub-tours, the drone must have its battery swapped out at the data center. Let t_b be the time required to swap out batteries. Then the total time for drone a_j to complete $\mathcal{K}_j$ will be:

$$t_{\mathcal{K}}(\mathcal{K}_j) = \sum_{\mathcal{T}_j \in \mathcal{K}_j} t_{tot}(\mathcal{T}_j) + t_b * (||\mathcal{K}_j|| - 1) \tag{7.7}$$

Note that we assume that no drone will need a battery swap before the first tour, but this assumption can easily be reversed.

7.3.3 Travel-To-TX Tradeoff

Observe that the time required to service a node (Equ. 7.2) depends on the drone-to-sensor distance, d_{ji}, and the quantity of data on the sensor, q_i. Reducing the distance will increase the data transmission rate (Equ. 7.1), which reduces the service time. However, moving close to a sensor takes time - which depends on the drone's speed.

Without loss of generality, assume that there is a single sensor s_i that is x_i meters from the data center. If we assume that the drones do not communicate while traveling, a drone must travel $x_i - d_{ji}$ meters, at v_j meters-per-second, before collecting q_i MB of data to service s_i and then travel $x_i - d_{ji}$ meters back to the data center. We want to select d_{ji} such that we minimize the time to move $x_i - d_{ji}$ meters, collect data, and return $x_i - d_{ji}$ meters to the data center. The total time to move and collect data will be

$$t_{tot}(d_{ji}) = \frac{2(x_i - d_{ji})}{v_j} + \frac{q_i(d_{ji}^2 + c_i)}{a_i + b_i(d_{ji}^2 + c_i)}. \tag{7.8}$$

Note that the second term is the result of simplifying the fraction $\frac{q_i}{tx(d_{ji})}$. If we take the partial derivative of Equ. (7.8), we get

$$\frac{\partial t_{tot}}{\partial d_{ji}} = \frac{-2}{v_j} + \frac{2q_i d_{ji}}{a_i + b_i(d_{ji}^2 + c_i)} - \frac{2b_i q_i d_{ji}(d_{ji}^2 + c_i)}{(a_i + b_i(d_{ji}^2 + c_i))^2}. \tag{7.9}$$

Solving for the roots of Equ. (7.9) will give us the locally minimum values for $t_{tot}(d_{ji})$, one of which will be the optimal value for d_{ji} to minimize the total time required to service s_i. Apart from the sensor-specific parameters ($a_i,\ b_i,\ c_i$), observe that the optimal value of d_{ji} depends on v_j and q_i. In general, we expect d_{ji} to decrease when q_i increases and d_{ji} to increase when v_j decreases. That is, it is better to get closer to collect larger volumes of data and it is better to stop early when the drone moves slowly. We refer to this as the *travel-to-TX* tradeoff.

7.3.4 Problem Definition

We can now formally define the Drone Data-Muling Problem (DDMP). Given a set of n sensors, data center location (x_b), and a team of m drones, find drone sub-tours such that

$$\min \sum_{j=1}^{m} t_{\mathcal{K}}(\mathcal{K}_j) \tag{7.10}$$

where every sensor s_i is in at least one $\mathcal{K}_j$, and

$$\mathcal{J}(\mathcal{T}_j) \leq \beta_j,\ \forall \mathcal{T}_j \in \mathcal{K}_j, j \in J \tag{7.11}$$

7.4 Holistic Drone Planning Framework

There is often a disparity between centralized, high-level offline planning and real-world execution caused by the simplifying assumptions made to render the problem easier to solve offline. This is particularly true when dealing with wireless devices due to the stochastic nature of radio frequencies. A variety of factors can impact a drone's ability to collect data from a sensor including mis-predicting the communication range of the sensor or a weakened sensor battery. This motivates the need to consider both offline, high-level planning and online adaptive behavior jointly.

To address the disparity in planning, we present a holistic drone planning framework for solving the DDMP that combines offline mission planning with online adaptive behavior. Figure 7.2 shows an overview of our proposed framework.

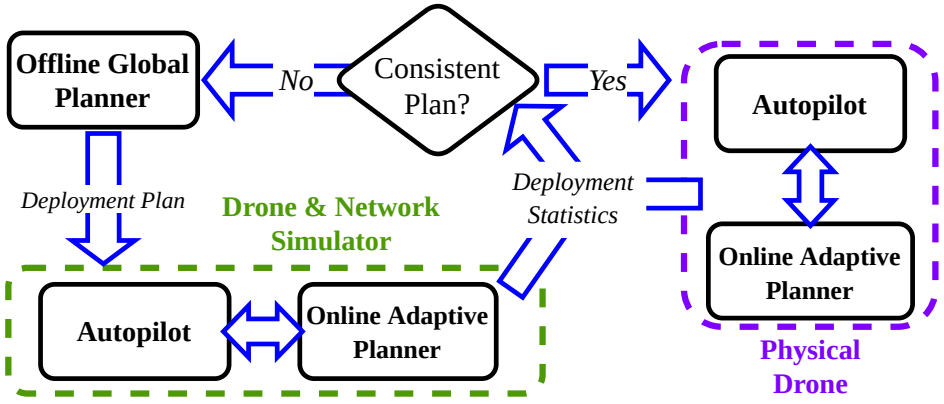


Figure 7.2 Overview of our drone planning framework.

The framework first finds an efficient solution to the DDMP problem via a Global Planner. The Global Planner assumes that drones must stop and hover to service a sensor and that each drone will move at a fixed speed. Additionally, the Global Planner modifies Constraint (7.11) to be

$$\mathcal{J}(\mathcal{T}_j) \leq \alpha\beta_j, \ \forall \mathcal{T}_j \in \mathcal{K}_j, j \in A \tag{7.12}$$

where $\alpha \in (0, 1]$ is the battery share alloted for offline planning. The motivation for adjusting the energy budget of each drone is to leave excess energy for adaptive behavior during deployment. Setting α too high will cause the drone to not have enough energy to adapt online while setting α too low will restrict the Global Planner and lead to less efficient plans. Ideally, α should be as large as possible without causing a drone to run out of energy during deployment.

The Global Planner sends its deployment plan to a drone and network simulator. The drone portion of the simulator handles realistic drone dynamics while the network portion of the simulator handles

drone-to-sensor communication and simulates stochastic communication performance. In the simulator, an autopilot carries out the deployment plan and invokes the Online Adaptive Planner should the drone fail to communicate with a sensor. In the case of failure, the online planner moves the drone as close to the sensor as allowed and plans a new route back to the data center. Unlike the Global Planner, the autopilot does not force the drone to stop and hover to service a node and will attempt to communicate with node $i+1$ as soon as it has finished servicing node i.

The framework simulates the plan multiple times in a Monte-Carlo fashion, collecting average drone speeds and total energy used. After several iterations of the simulator, the framework considers the performance of the deployment plan. The plan is considered *consistent* if the total energy used on each sub-tour j is below but within a desirable range of β_j and the predicted average drone speeds v_j is within a desirable range of the measured v_j. If the deployment plan is not consistent, the framework adjusts the values of α and the average drone speeds, then sends this information back to the Global Planner to restart the cycle. If a deployment plan is consistent, then it is ready to be deployed on the actual drone to collect data.

The physical drone has the same autopilot and Online Adaptive Planner as the simulator with the exception that the online planner in the simulator does not observe any drone energy constraint while the physical drone does. This is because we want the simulation to determine the actual energy required to service all of the sensors but we do not want the physical drone to crash during deployment.

To determine α, we initially set it to 1 then run the Drone and Network Simulator several times to determine if the drones will run out of energy. If the drones do run out of energy, then we adjust α. Let the maximum observed jules consumed per drone sub-tour on the i^{th} iteration of the framework be $\tilde{\jmath}_i$. If $\tilde{\jmath}_i > \beta$ (that is, the drones are running out of energy during deployment), then it may seem natural to assume that the ratio between α and $\tilde{\jmath}$ are equivalent. That is,

$$\frac{\alpha_i}{\tilde{\jmath}_i} \equiv \frac{\alpha_{i+1}}{\tilde{\jmath}_{i+1}}, \tag{7.13}$$

which means we could update α using:

$$\alpha_{i+1} = \alpha_i \frac{\beta}{\tilde{\jmath}_i}. \tag{7.14}$$

However, there is no guaranteed correlation between α_j and $\tilde{\jmath}_i$ and Equ. (7.14) may update α too aggressively. Therefore, we propose using a convergence rate $\mu \in (0, 1]$ and updating α using:

$$\alpha_{i+1} = \alpha_i\big(1 - \mu(1 - \frac{\beta}{\tilde{\jmath}_i})\big). \tag{7.15}$$

Setting $\mu = 1$ makes Equ. (7.15) equivalent to Equ. (7.14) while setting $\mu = 0.5$ means that α updates at half the rate as Equ. (7.14). We experimentally determined that setting μ between 0.5 and 0.75 causes

the framework to converge nicely without changing α too aggressively.

In the following two sections we further discuss the Global Planner and the Online Adaptive Planner. We describe the Drone and Network Simulator used for this study in Section 7.8 and give details on our drone and physical testbed in Section 7.9.

7.5 Offline Global Planner

In this section, we present two methods for solving DDMP. We first formulate a Mixed Integer Non-Linear Program that will find an optimal solution but is not tractable. We then present a polynomial time algorithm that divides the problem into multiple Traveling Salesman Problems (TSP), that are solved using well studied heuristics-based algorithms, and then improves the performance of the solution using convex optimization. All sets used in this section are provided in Table 7.1 and all decision variables are listed in Table 7.2.

Table 7.1 Sets for Mixed Integer Non-Linear Program

Set	Magnitude	Description
I	n	Set of sensors, indexed with i
J	m	Set of drones, indexed with j
K	$\leq n$	Set of drone sub-tours, indexed with k

Table 7.2 Decision Variables for Mixed Integer Non-Linear Program

Variable	Range	Description
$\mathbf{x}_i, \mathbf{y}_i, \mathbf{z}_i$	$\mathbb{R}$	Coordinates of servicing waypoint for node i
$\mathbf{d}_{i\hat{i}}$	$\mathbb{R}$	Distance from waypoint i to waypoint $\hat{i}$
$\mathbf{d}_{bi}$	$\mathbb{R}$	Distance from the data center to waypoint i
${}^{n}\mathbf{d}_i$	$\mathbb{R}$	Distance from servicing waypoint i to node i
${}^{o}\mathbf{d}_i$	$\mathbb{R}$	Distance to servicing waypoint i from servicing waypoint $i-1$ in $\mathcal{T}_j$
${}^{f}\mathbf{d}_b$	$\mathbb{R}$	Distance from final servicing waypoint in $\mathcal{T}_j$ to the data center
$\mathbf{t}_{jk}$	$\mathbb{R}_{\geq 0}$	Time duration of drone j's k^{th} sub-tour
${}^{s}\mathbf{t}_i$	$\mathbb{R}_{\geq 0}$	Time required to service node i
$\mathbf{r}_i$	$\mathbb{R}_{\geq 0}$	Data transfer rate when servicing node i
$\mathbf{e}_{ji\hat{i}k}$	$\mathbb{B}$	1 if j travels from i to $\hat{i}$ on sub-tour k, 0 otherwise
$\mathbf{e}_{jbik}$	$\mathbb{B}$	1 if j travels from the base to i on sub-tour k, 0 otherwise
$\mathbf{e}_{jibk}$	$\mathbb{B}$	1 if j travels from i to the base on sub-tour k, 0 otherwise
$\mathbf{w}_{jk}$	$\mathbb{B}$	1 if j deploys on sub-tour k, 0 otherwise
$\mathbf{u}_i$	$\mathbb{N}$	Sequence number for node i
$\mathbf{n}_{jk}$	$\mathbb{N}$	Number of nodes drone j visits on sub-tour k

7.5.1 Mixed Integer Non-Linear Solution

To solve the DDMP to optimality, we propose a Mixed Integer Non-Linear Program (MINLP). The MINLP formulation is motivated by similar formulations for solving the Vehicle Routing Problem (VRP) and uses the well-known Miller-Tucker-Zemlin cycle elimination constraint to prevent sub-cycles in drone sub-tours. One important way that our formulation deviates from the classic VRP is that we are selecting the location of each waypoint where a drone will service each sensor.

We define continuous variables $\mathbf{x}_i, \mathbf{y}_i$, and $\mathbf{z}_i$ as the waypoint coordinates from which a drone will service node i. Let K be the set of sub-tours that a drone has the option to fly on. Let binary variable $\mathbf{e}_{ji\hat{\imath}k}$ be a visit flag where if $\mathbf{e}_{ji\hat{\imath}k} = 1$ then drone j will travel from waypoint i to waypoint $\hat{\imath}$ on sub-tour k, and 0 otherwise. Likewise, binary variable $\mathbf{e}_{jbik}$ determines if drone j travels from the data center to waypoint i on sub-tour k and $\mathbf{e}_{jibk}$ determines if j goes from waypoint i to the data center. We use binary variable $\mathbf{w}_{jk}$ to determine if drone j will run sub-tour k. Continuous variables $\mathbf{t}_{jk}$ is the time duration of drone j's k^{th} sub-tour, $\mathbf{d}_{i\hat{\imath}}$ is distance from waypoint i to waypoint $\hat{\imath}$, $\mathbf{d}_{bi}$ is the distance from the data center to waypoint i, and ${}^{n}\mathbf{d}_i$ is the distance from waypoint i to sensor i. Integer variable $\mathbf{u}_i$ assigns a visit number to each waypoint. Continuous variables ${}^{s}\mathbf{t}_i$ track the time required to service node i and $\mathbf{r}_i$ determines the data transfer rate while servicing node i. Integer variable $\mathbf{n}_{jk}$ for the number of nodes that drone j visits on sub-tour k.

Our objective is to minimize the sum of individual sub-tour times:

$$\min \ \frac{1}{n}\Big(\sum_{j\in J}\sum_{k\in K}\big(\mathbf{n}_{jk}\sum_{\hat{k}\in K,\hat{k}\le k}\mathbf{t}_{j\hat{k}}\big)\Big) \tag{7.16}$$

subject to

$$\frac{1}{v_j}\big(\sum_{i\in I}\sum_{\hat{\imath}\in I}\mathbf{d}_{i\hat{\imath}}\mathbf{e}_{ji\hat{\imath}k}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jbik}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jibk}\big)+\sum_{i\in I}\sum_{\hat{\imath}\in I}{}^{s}\mathbf{t}_i\mathbf{e}_{ji\hat{\imath}k}+t_b\mathbf{w}_{jk}\le\mathbf{t}_{jk},$$
$$\forall j\in J,\ k\in K \tag{7.17a}$$
$$\mathbf{n}_{jk}=\sum_{\hat{\imath}\in I}\sum_{k\in K}\mathbf{e}_{j\hat{\imath}ik}+\sum_{i\in I}\mathbf{e}_{jbik},\quad j\in J,\ k\in K \tag{7.17b}$$
$$\frac{\mathcal{P}(v_j)}{v_j}\big(\sum_{i\in I}\sum_{\hat{\imath}\in I}\mathbf{d}_{i\hat{\imath}}\mathbf{e}_{ji\hat{\imath}k}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jbik}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jibk}\big)+\mathcal{P}(0)\sum_{i\in I}\sum_{\hat{\imath}\in I}{}^{s}\mathbf{t}_i\mathbf{e}_{ji\hat{\imath}k}\le\alpha\beta_j,$$
$$\forall j\in J,\ k\in K \tag{7.17c}$$
$$\mathcal{M}\mathbf{w}_{jk}\ge\sum_{i\in I}\sum_{\hat{\imath}\in I}\mathbf{e}_{ji\hat{\imath}k},\quad\forall j\in J,\ k\in K \tag{7.17d}$$
$$\mathbf{w}_{jk-1}\ge\mathbf{w}_{jk},\quad\forall j\in J,\ k\in K,k>1 \tag{7.17e}$$
$$\sum_{i\in I}\mathbf{e}_{jbik}=\mathbf{w}_{jk},\quad\forall j\in J,\ k\in K \tag{7.17f}$$

$$\sum_{i \in I} \mathbf{e}_{jibk} = \mathbf{w}_{jk}, \quad \forall j \in J,\ k \in K \tag{7.17g}$$

$$\sum_{j \in J} \sum_{\hat{\imath} \in I} \sum_{k \in K} \mathbf{e}_{j\hat{\imath}ik} + \sum_{j \in J} \sum_{k \in K} \mathbf{e}_{jbik} = 1, \quad \forall i \in I \tag{7.17h}$$

$$\sum_{j \in J} \sum_{\hat{\imath} \in I} \sum_{k \in K} \mathbf{e}_{ji\hat{\imath}k} + \sum_{j \in J} \sum_{k \in K} \mathbf{e}_{jibk} = 1, \quad \forall i \in I \tag{7.17i}$$

$$\sum_{\hat{\imath} \in I} \mathbf{e}_{ji\hat{\imath}k} + \mathbf{e}_{jbik} - \big(\sum_{\hat{\imath} \in I} \mathbf{e}_{j\hat{\imath}ik} + \mathbf{e}_{jibk} \big) = 0, \quad \forall j \in J,\ i \in I,\ k \in K \tag{7.17j}$$

$$\mathcal{M}(1 - \mathbf{e}_{ji\hat{\imath}k}) + \mathbf{u}_i + 1 \geq \mathbf{u}_{\hat{\imath}} \geq \mathbf{u}_i + 1 - \mathcal{M}(1 - \mathbf{e}_{ji\hat{\imath}k}), \quad \forall j \in J,\ i, \hat{\imath} \in I,\ k \in K \tag{7.17k}$$

$$\mathbf{d}^2_{i\hat{\imath}} \geq (\mathbf{x}_i - \mathbf{x}_{\hat{\imath}})^2 + (\mathbf{y}_i - \mathbf{y}_{\hat{\imath}})^2 + (\mathbf{z}_i - \mathbf{z}_{\hat{\imath}})^2, \quad \forall i, \hat{\imath} \in I \tag{7.17l}$$

$$\mathbf{d}^2_{bi} \geq (\mathbf{x}_i - \pi_1(x_b))^2 + (\mathbf{y}_i - \pi_2(x_b))^2 + (\mathbf{y}_i - \pi_3(x_b))^2, \quad \forall i \in I \tag{7.17m}$$

$${}^{n}\mathbf{d}^2_i \geq (\mathbf{x}_i - \pi_1(x_i))^2 + (\mathbf{y}_i - \pi_2(x_i))^2 + (\mathbf{z}_i - \pi_3(x_i))^2, \quad \forall i \in I \tag{7.17n}$$

$$\mathbf{r}_i \leq \frac{a_i}{{}^{n}\mathbf{d}^2_i + c_i} + b_i, \forall i \in I \tag{7.17o}$$

$$r_{max\text{-}i} \geq \mathbf{r}_i \geq r_{min\text{-}i}, \forall i \in I \tag{7.17p}$$

$${}^{s}\mathbf{t}_i \geq \frac{q_i}{\mathbf{r}_i}, \forall i \in I \tag{7.17q}$$

Constraint (7.17a) bounds the total time of sub-tour k for drone j. Constraint (7.17c) enforces the drones energy budget on each sub-tour, where $\mathcal{P}(v)$ maps speed v to power consumed, in watts, β_j is the capacity of the drone's battery, and α is the battery share for offline planning. Constraint (7.17d) determines if a particular sub-tour k is actually used for drone j while constraint (7.17e) forces lower numbered sub-tours to be used first. Constraints (7.17f) and (7.17g) force each sub-tour that is used to start and end at the data center. Constraints (7.17h) and (7.17i) ensure that each waypoint is visited exactly once while constraint (7.17j) ensures that the "in" edge and "out" edge of each waypoint is on the same sub-tour for the same drone. Constraint (7.17k) enforces a numbering system on each waypoint to eliminate smaller cycles in drone sub-tours. Constraint (7.17l) tracks the distance between each waypoint, constraint (7.17m) tracks the distance from each waypoint back to the data center, and constraint (7.17n) tracks the distance from waypoint i to sensor i. Note that $\pi_i(x)$ is the projection function that returns the i^{th} values of vector x. Together, constraints (7.17o) and (7.17p) restrict the data transmission rate for sensor i based on (7.1), and constraint (7.17q) limits sensor service time based on the data transmission rate.

Although solving our MINLP will yield an optimal solution, MINLP are particularly hard to solve to optimality. In the following subsection we present a more tractable approach to solving the DDMP.

7.5.2 Polynomial Time Solution

Our MINLP has two major factors that make it hard to solve. The first is the combinatorial problem, where we must select which drones visit which sensors and in what order. The second major challenge is

Constraints (7.17o) and (7.17p), which limit the transmission rate using Equ. (7.1) - a non-convex exponential decay function.

For a more tractable solution to the DDMP, we propose a divide and conquer approach that overcomes the major challenges of the MINLP solution. Our algorithm treats the problem as a VRP instance and uses a heuristics-based solver to find an initial solution, which solves the combinatorial problem. After finding an initial solution, our algorithm formulates a convex optimization problem for selecting drone hovering locations to service nodes by using a straight-line approximation of Equ. (7.1), making the optimization problem solvable in polynomial time. We term this approach *VRP+CO*.

Algorithm 7 Multi-Drone Data-Muling Algorithm

Input: I : set of sensor, x_b : data center position
Output: $\mathcal{K}$: set of drone tours

1: $k \leftarrow 1$
2: **while** *stopping condition not met* **do**
3: $\quad \mathcal{C} \leftarrow k\text{-}means(I,\ x_b,\ k)$
4: $\quad$ **for** $\mathcal{C}_j \in \mathcal{C}$ **do**
5: $\quad\quad \mathcal{T}_j \leftarrow solve\text{-}TSP(\mathcal{C}_j, x_b)$
6: $\quad\quad \mathcal{T}_j \leftarrow improve\text{-}tour(\mathcal{T}_j)$
7: $\quad\quad$ *append* $\mathcal{T}_j$ *on* $\mathcal{T}$
8: $\quad$ **end for**
9: $\quad \mathcal{K} \leftarrow assign\text{-}drones(\mathcal{T})$
10: $\quad$ **if** *each* $\mathcal{T}_i \in \mathcal{T}$ *is under energy budget* **then**
11: $\quad\quad$ *Stopping condition met*
12: $\quad$ **else**
13: $\quad\quad m \leftarrow m + 1$
14: $\quad$ **end if**
15: **end while**
16: **return** *best found* $\mathcal{K}_m$

Algorithm 7 lists the details of our algorithm. We start by fixing k, the total number of sub-tours per drone, on line 1. We then use Lloyd's k-means clustering algorithm on line 3 to form k clusters. On line 5 we solve a TSP on nodes in each cluster along with the data center. We use the Lin-Kernighan-Helsgaun (LKH) heuristic [109] to solve each TSP instance, which runs in polynomial time [31]. We then run the *improve-tour*() function on the newly found sub-tour, which is further described below. On line 9 we assign drone to sub-tours by sorting the drone based on total energy budget available and sorting each sub-tour by total distance, then assigning drones to sub-tours based on this ordering. On line 10 we verify that each drone will have enough energy to perform each sub-tour assigned to it. If true, then we have met the stopping condition and the algorithm ends. Otherwise, we increase k and restart the algorithm.

To improve each sub-tour, the *improve-tour*() function attempts to find better hovering locations $\tilde{x}_i$ for each sensor i in sub-tour $\mathcal{T}_j$. We do this by solving a convex optimization problem that simplifies the MINLP presented in Section 7.5.1. In addition to the variables present previously, let ${}^o\mathbf{d}_i$ be the distance

from the hovering location before i to hovering location i in $\mathcal{T}_j$ and let ${}^f\mathbf{d}_b$ be the distance from the last hovering location in $\mathcal{T}_j$ back to the data center. We can formulate a convex optimization problem that minimizes time to collect data from the sensors on $\mathcal{T}_j$ as follows:

$$\min \frac{1}{v_j}\Big(\sum_{i\in\mathcal{T}_j} {}^o\mathbf{d}_i + {}^f\mathbf{d}_b\Big) + \sum_{i\in\mathcal{T}_j}^{s} T_i \tag{7.18}$$

subject to: Constraints (7.17n), (7.17q), (7.17a), (7.17p), and

$${}^o\mathbf{d}_i^2 \geq (\mathbf{x}_i - \mathbf{x}_{i-1})^2 + (\mathbf{y}_i - \mathbf{y}_{i-1})^2 + (\mathbf{z}_i - \mathbf{z}_{i-1})^2, \quad \forall i \in \mathcal{T}_j \tag{7.19}$$

$${}^f\mathbf{d}_b^2 \geq (\mathbf{x}_i - \pi_1(x_b))^2 + (\mathbf{y}_i - \pi_2(x_b))^2 + (\mathbf{y}_i - \pi_3(x_b))^2, \quad i = last_element(\mathcal{T}_j) \tag{7.20}$$

$$\mathbf{r}_i \leq \frac{r_2 - r_{max\text{-}i}}{d_2 - d_1}({}^o\mathbf{d}_i - d_1) + r_{max\text{-}i}, \forall i \in \mathcal{T}_j \tag{7.21}$$

Constraint (7.19) tracks the waypoint-to-waypoint distance while constraint (7.20) tracks the distance from the last waypoint in $\mathcal{T}_j$ to the data center. Constraint (7.21) limits the data transfer rate for each sensor in $\mathcal{T}_j$ by linearly approximating the curve in Equation (7.1), where

$$d_1 = \sqrt{\frac{a_i}{r_{max\text{-}i} - b_i} - c_i} \tag{7.22}$$

$$r_2 = \frac{a_i}{d_2^2 + c_i} + b_i \tag{7.23}$$

and $d_2 > d_1$.

We would like to exploit the *travel-to-TX* tradeoff by solving for roots in Equ. (7.9) and using the optimal drone-to-sensor distance for d_2. However, observe that Equ. (7.9), when simplified, is a fifth-order polynomial, which is not solvable in radicals per the Abel-Ruffini theorem. To estimate an optimal value for d_2 we could use numerical methods, such as Newton's method, or by creating a look-up table with precomputed values. We chose to use a look-up table for our implementation of the *VRP+CO* algorithm to reduce computation time in our Online Adaptive Planner (as discussed in Section 7.6), but either method can be used.

Observe that objective function (7.18) is linear and constraints (7.17n), (7.17q), (7.17a), (7.17p), (7.19), (7.20) and (7.21) are all convex, creating a second-order cone program (SOCP), which can be solved efficiently using interior-point methods [275]. Further more, solving the VRP will be dominated by the underlying TSP solver. The LKH solver in [31] has an approximate time complexity of $\mathcal{O}(n^{2.2})$ while the time to solve the SOCP will depend on the solver employed (such as Gurobi), which we assume is more efficient than the LKH solver. The outer while-loop in our algorithm will run at most n times, making the total runtime of our Multi-Drone Data-Muling Algorithm $\mathcal{O}(n^{3.2})$. However, in the average case we do not

anticipate the outer while-loop to run n times, giving us a runtime between $\mathcal{O}(n^{2.2})$ and $\mathcal{O}(n^{3.2})$.

7.6 Online Adaptive Planner

Our planning framework requires a method for quickly adapting the plan found using the Offline Global Planner. For this, we propose an Online Adaptive Planner that utilizes the SOCP presented in Section 7.5.2. When communication failure occurs, the autopilot from our framework in Section 7.4 gives the online planner the drone's current position, the sensor that failed to respond to the drone, and a list of sensors from that sub-tour that have not yet been visited.

Suppose that sensor $\hat{\imath}$ has failed to communicate with the drone (which is located at $\tilde{x}_{\hat{\imath}}$) and the remaining sensors to visit are in $\mathcal{T}_{\hat{\jmath}}$. We formulate a new SOCP by adding the distance from $\tilde{x}_{\hat{\imath}}$ to the point at the safety altitude directly above sensor $\hat{\imath}$ into objective function (7.18) and to energy constraint (7.17c). We then solve the rest of the SOCP from Section 7.4 using the points in $\mathcal{T}_{\hat{\jmath}}$, giving us a new data collection plan that attempts to connect from the failing sensor while adapting the original deployment plan from the offline planner but avoids the combinatorial aspect of the DDMP.

7.7 Drone & Network Simulator

Our framework requires us to run Monte Carlo simulations to validate drone deployment plans. To create a realistic environment for validating plans, we joined together a real-time drone simulator, a drone autopilot that supports adaptive behavior, and a simplified networking simulation. We discuss each of these pieces in-depth below. Although we chose specific tools for each of these elements, any of these individual pieces can be swapped out with alternative options and our framework would still be applied the same.

We used the ArduPilot Software-In-The-Loop (SITL) simulator[4] as our real-time drone simulator. The Ardupilot SITL models drone dynamics and can be set to run in any real-world location. The simulator utilizes MAVProxy's mavproxy_map module for simulating location-specific terrain features, meaning that the drone in the simulation can crash into the ground if the terrain is not accounted for (we will address this concern in the following subsection). The ArduPilot SITL supports a variety of control commands through the MAVLink protocol[5], and mirrors common commands used in commercially available flight controllers, such as the CubePilot Cube Orange Flight Controller[6], meaning that the same commands used to control a real drone can be used to control a simulated drone in the SITL simulator. This feature allows our autopilot to work on both the ArduPilot SITL simulator and on our physical drone.

[4]https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html
[5]https://mavlink.io/en/
[6]https://docs.px4.io/main/en/flight_controller/cubepilot_cube_orange.html

7.7.1 Drone Autopilot

For an autopilot, we have adapted the DroNS3 autopilot described in [276]. This autopilot can read in a list of commands as a plan text file and will execute each command in the given order. At a high level, these commands fall into a handful of basic operations: takeoff, move to a waypoint, collect data from a sensor, return to the home position, and land.

We have adapted this autopilot framework in a few ways. The first adaptation is that we have combined the move-to-waypoint and the collect-data commands so that the drone will attempt to collect data while moving. We have also added an adaptive behavior feature, where the autopilot will call the Online Adaptive Planner when drone-to-sensor communication fails. Another notable adaptation is ground-avoidance. When the autopilot executes a move-to-waypoint command, we check the trajectory of the drone to verify that the drone will not collide with the ground. If a ground collision is predicted, then we replace that move-to-waypoint command with a series of shorter move commands that direct the drone to gain altitude and avoid the ground.

7.7.2 Network Simulator

To simulate drone-to-sensor communication, we have implemented our own simplified network simulator (termed SimpleNetSim or SNS) that utilizes a data-driven approach to simulate peer-to-peer communication. At a high level, SNS simulates a single peer-to-peer transaction by taking in the quantity of data to transmit and the distance between the two devices, then predicts the data transfer rate and simulates the time delay to transmit the given volume of data. Although SNS can be used with any volume of data, we recommend breaking up large volumes of data into smaller chunks to accommodate the drone's movement and changing drone-to-sensor distance.

SimpleNetSim uses a Weibull distribution to predict peer-to-peer data transfer rates. We chose a Weibull distribution because it was previously shown to nicely model network traffic [277] and our own field test data reaffirmed these findings. Given peer-to-peer distance d, the Weibull distribution's probability density function is defined as

$$f(d;\lambda,c) = \frac{c}{\lambda}\left(\frac{d}{\lambda}\right)^{c-1} e^{-(c/\lambda)^c} \tag{7.24}$$

where λ defines the shape of the distribution and c defines the scale, both of which depend on distance d. That is, a given distance d should have unique values for λ and c. Let $f_W(d)$ be a function that maps distance to values for λ and c. SimpleNetSim tunes function $f_W(d)$ using transmission rate measurements at varying distances from field experiments on physical hardware. Once function $f_W(d)$ has been tuned, SNS randomly draws a transmission rate for distance d by drawing random variable

$R_{tx} \backsim Weibull(f_W(d))$. Note that λ and c are both device dependent, meaning that function $f_W(d)$ must be tuned for each unique sensor type.

We acknowledge that we previously used NS3, a powerful discrete event networking simulator[7], in our prior work [7]. However, we found several limitations in using NS3 that SNS attempts to overcome. The first limitation is that NS3 does not readily provide stochastic behavior. SimpleNetSim overcomes this by sampling the Weibull distribution each time the simulator is called, which emulates the inconsistent behavior commonly seen during deployment. The second limitation that we found with NS3 is that it is not easy to accurately replicate the performance seen on our own hardware. SimpleNetSim is data-driven from field test data, making it easy to model our own hardware in the simulation.

7.8 Numerical Evaluations

In this section we evaluate our planning framework through various numerical simulations with field data. We first discuss how we ran field tests to collect data transmission measurements and used the results to find various parameters used in our framework. We then evaluate the performance of our proposed framework and various algorithms for both the Offline Global Planner and the Online Adaptive Planner.

Figure 7.3 Drone and Raspberry Pies used in field experiments.

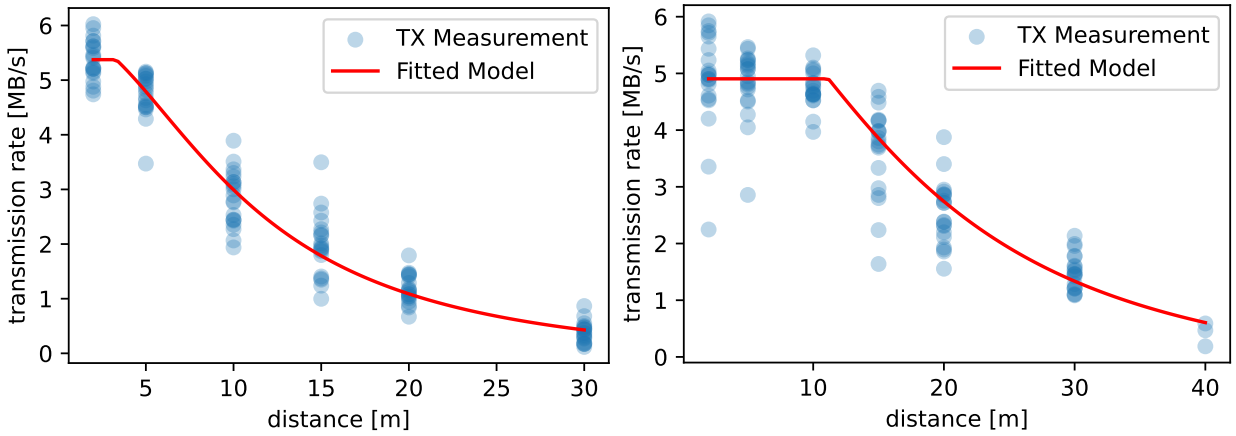
7.8.1 Data Transmission Experiments

To determine values for Equ. (7.1), λ, and c, we deployed two Raspberry Pies (one RPi3 and one RPi4) in a field and flew a drone above the Pies at varying distances. The drone was also equipped with an RPi4 as its on-board computer. Figure 7.3 shows the drone and Raspberry Pies. At each distance, we had the

[7]https://www.nsnam.org/

Pies transmit data to the drone over Wi-Fi and measured the data transfer rate. Both of the Pies and the drone were in ad-hoc mode, transmitting at the 2.4 GHz range.

Figure 7.4 shows the results of our experiments with Equ. (7.1) fitted to the data. We used the random sample consensus algorithm to fit the model to our data because this approach was shown to work well with very noisy data [278]. For the RPi3 (Fig. 7.4(a)), we found $a = 681,\ b = -0.246,\ c = 110,\ r_{max} = 5.37$ and $r_{min} = 0.2$, and for the RPi4 (Fig. 7.4(b)), we found $a = 2403,\ b = -0.654,\ c = 308,\ r_{max} = 4.9$ and $r_{min} = 0.4$. We used SciPy[8] to determine λ and c at each of the various distances to build function $f_W(d)$. The values for λ and c listed in our open source repository.



(a) (b)

Figure 7.4 Results from field experiments to determine data transfer rates between a drone using a Raspberry Pi 4 and a Raspberry Pi 3 (a) and Pi 4 (b) sensors on the ground.

Using the parameters for Equ. (7.1), we computed optimal values for d_* in Equ. (7.9), for both Raspberry Pies (RPi3 in Fig. 7.5(a) and RPi4 in Fig. 7.5(b)), to build a look-up table for Constraint (7.21). We solved the equation with varying values for v, from 2 meters-per-second up to 20 meters-per-second in increments of 1. At each unique speed setting, we used values of $q = 0.001 * 2^{n/8}$ MB, for $n \in \{0, 1, \cdots 159\}$, giving us a total of 160 values for d_* at each speed setting. The results of these computations is shown in Fig. 7.5. These graphs clearly show the *travel-to-TX* tradeoff and demonstrate that characteristics of the tradeoff are dependent on the speed of the drone and the volume of data on the sensor. The flat section at the bottom of each graph occurs when $\frac{a}{d^2+c} + b = r_{max}$, at which point there is no longer a gain in data transmission rate for moving closer to the sensor and corresponds with the flat segment of $tx(d)$ on the left side of the graphs in Fig. 7.4. The red dashed line shows the distance at which $\frac{a}{d^2+c} + b = r_{mix}$ and is the max drone-to-sensor distance that we expect to be able to communicate at.

[8]https://docs.scipy.org/doc/scipy/

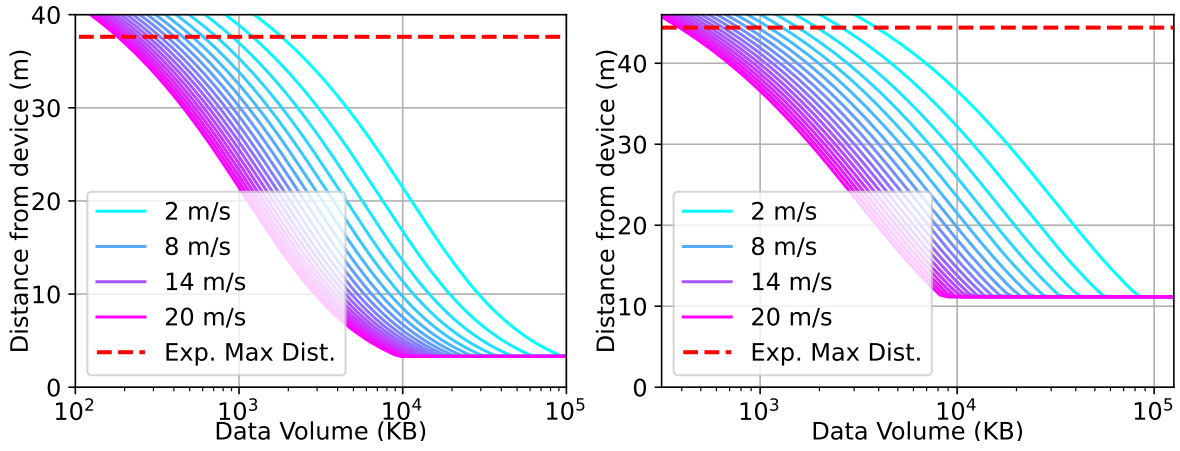


Figure 7.5 Optimal distance to collect data from Raspberry Pi 3 (a) and Pi 4 (b) sensors based on drone speed.

7.8.2 Framework Evaluation

To evaluate our framework, we ran it on various inputs and analyzed how well the framework could (1) find valid solutions and (2) find efficient solutions. For this experiment, we generated a data set where the number of sensors varies from five up to 30, in increments of 5. We created 10 unique sensor deployments at each increment by selecting random locations within a 577×577 meter space, with the data center located at the center, which gives us a sensor density of 15 $sensors/km^2$ up to 90 $sensors/km^2$. We used MAVProxy's mavproxy_map module to determine the altitude (z value) of each of the randomly placed sensors.

We compared our proposed framework against non-adaptive approaches that fix the value for α. These baseline solutions are equivalent to running our framework (same Offline Global Planner, Online Adaptive Planner, and simulation setup) but without restarting the offline planner with an updated value for α whenever the drone goes over energy in the simulation. We compared against α fixed at 1.0, 0.75, and 0.5.

Figure 7.6 shows the results of our simulations. For clarity, our adaptive approach (the purple line) is beneath $\alpha = 1$ (black line) in the top plot and beneath $\alpha = 0.5$ (red line) in the bottom graph. The top graph shows the average found average latency compared to the best-found solution. We see a fair amount of variation when the number of sensors is small (≤ 10), which is expected due to the stochastic nature of the networking simulator. However, as the number of sensors grows, our adaptive framework and α fixed at 1.0 are consistently performing the best, while fixing $\alpha = 0.5$ has high variance and averages a 15.8% increase in average latency when there are 15 or more sensors. This is because fixing α to a lower value causes the offline planner to be too conservative and generate more sub-tours than needed, leading to

higher data latency.

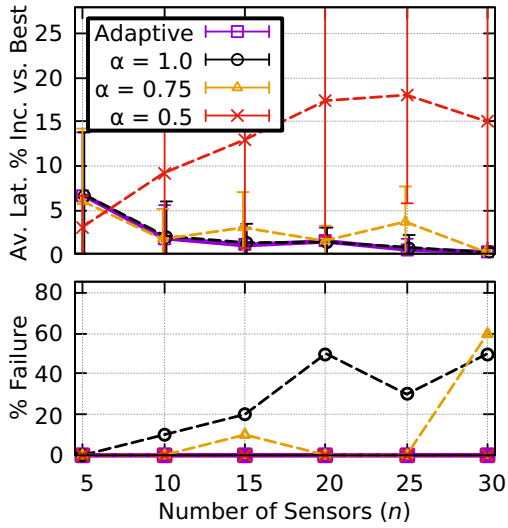


Figure 7.6 The percent increase in average latency compared to the best-found solution (top) and the percentage of inputs that each approach failed to find a valid plan for (bottom).

The bottom graph in Figure 7.6 shows the percentage of inputs where the approach failed to find a valid solution (where the drone would have ran out of energy). When there are only 5 sensors, all approaches find valid solutions. However, as the number of sensors grows, fixing $\alpha = 1.0$ and $\alpha = 0.75$ leads to failures, with the former failing 26.7% of all inputs. Our adaptive approach and α fixed at 0.5 achieve valid solutions for every input. This is because fixing α too high allows the offline planner to be too optimistic about how well the drones will be able to follow a plan found offline.

These results show the need for our holistic approach to planning. Allowing an Offline Global Planner to be too optimistic (setting a higher value for α) gives us better quality solutions but can generate infeasible plans, where the drones will run out of energy when adapting online. Conversely, allowing the Offline Global Planner to be too conservative (setting a lower value for α) avoids failures but leads to poor quality solutions. These trends intensify as the number of sensors increases. Interestingly, the fixed-α approaches were each able to find a valid solution that matched the best-found solution at least once with 30 sensors. This tells us that there is no "perfect" value for α and it needs to be experimentally determined.

7.8.3 Offline Global Planner Evaluation

To evaluate the performance of our proposed Offline Global Planner algorithm ($VRP+CO$), we compared our solution against two baseline approaches: (1) the VRP algorithm in listing 7 without the convex optimization step (VRP), and (2) the approach introduced in [7], where they solve a VRP and use a "tightening heuristic" to improve the solution ($VRP+TH$). The VRP algorithm is simply flying the drones

as close as possible to the sensor and ignoring the *travel-to-TX* tradeoff, as done in [35, 270, 272, 274]. The notable difference between our *VRP+CO* algorithm and the *VRP+TH* approach is that the former optimizes the *travel-to-TX* tradeoff while the latter algorithm assumes a constant transmission rate when within some larger distance to a sensor and simply tries to optimize travel distance while still passing through the transmission range of each sensor. This is similar to [65, 252, 260, 269] and a variation of other works that focus on choosing a hovering location to service multiple nodes at once, such as [20, 28, 253]. For the *VRP+TH* approach, we set a transmission range of 30 meters for the RPi3 and 35 meters for the RPi4. As an additional benchmark, we implemented our MINLP introduced in Section 7.5.1. We used Gurobi, version 12.0, for the MINLP and our *improve-tour*() function in the *VRP+CO* algorithm. We used the LKH solver, version 3.0, to solve the VRP in all three heuristics-based approaches.

We generated three sets of randomized sensor deployments for this evaluation. The first set has a varying number of sensors from four to 30 in increments of 1 (50 layouts at each increment) where the sensors are either Raspberry Pi 3s (RPi3) or Raspberry Pi 4s (RPi4) with an average sensor density of 75 $sensors/km^2$. The sensors have varying volumes of data to collect, defined as $q = 0.001 * 2^n$ MBs for some randomly picked $n \in \{0, 1, \cdots 15\}$. We term this the *n4-30* input set. We again used MAVProxy to determine the altitude of each sensor. The second sensor deployment set has the same parameters as the *n4-30* set, but with five up to 200 sensors, in increments of 5. The second set is termed *n5-200*. Our third sensor deployment set has 50 sensors with a uniform volume of data to collect from each sensor. We set the volume of data to $q = 0.001 * 2^n$, with n incrementing from zero up to 16, and generated 50 sensor deployments at each increment. We term this set *q0-16*.

We consider three types of drones in our numerical evaluation. The first drone is the model discussed in [4]. We estimate that this aircraft normally operates at 15 meters per second and carries 112 kilojoule of usable energy. The second considered drone is the one discussed in [63], which likely operates at 12 meters per second. We estimate that this drone could carry a 10.0 ampere-hour, 15.2 Volt battery, which would give the aircraft approximately 438 kilojoule of usable energy. The final considered drone is one described in [97], which operates at 10 meters per second and an estimated 360 kilojoule of usable energy. The parameters needed for $\mathcal{P}(v)$ for each drone are provided in each of the referenced publications.

7.8.3.1 Small Varying Number of Sensors

We ran the MINLP and the heuristics-based approaches on the *n4-30* set with two drones (the drones from [63] and [97]). To prevent excessive computation times, we set the MINLP to timeout after 4 hours. We stopped the MINLP approach at nine sensors because the solver began to hit the timeout too frequently. The top graph of Fig. 7.7 shows the percent increase in average latency of the heuristics-based

algorithms when compared against the optimal solution found using the MINLP and the bottom graph shows how frequently the MINLP hit the timeout. The *VRP+CO* algorithm averages 5.4% of the optimal solution while the *VRP* and *VRP+TH* averaged within 24.6% and 27.0% of the optimal, respectively. The MINLP is reasonably efficient until nine sensors, when it timed out on 36% of the inputs.

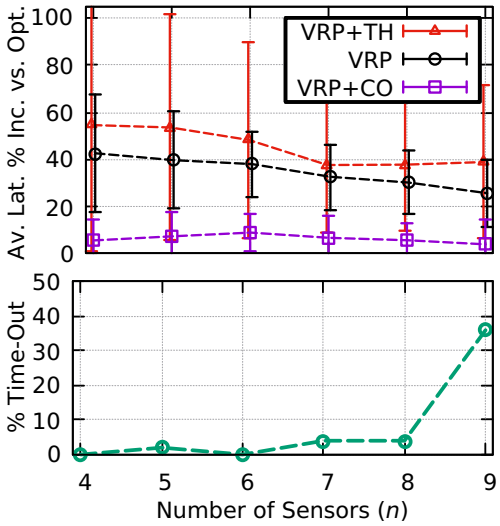


Figure 7.7 The average latency compared to the optimal solution (top) and the percentage of inputs that the MILP timed-out on (bottom).

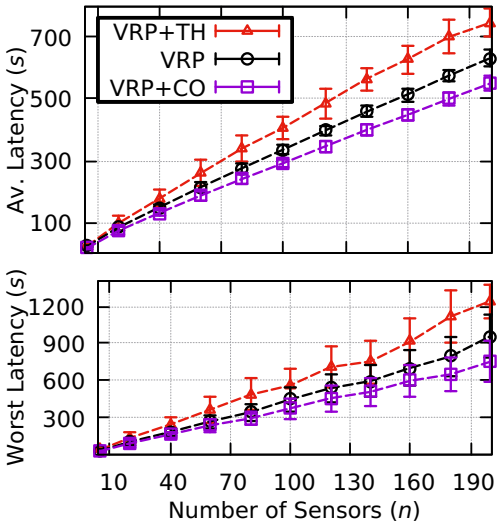


Figure 7.8 The average latency (top) and worst latency (bottom) as the number of sensors (n) increases.

7.8.3.2 Large Varying Number of Sensors

To determine how well each of the heuristics-based algorithms can handle larger input sizes, we ran all three algorithms on the *n5-200* set with all three drones available. Figure 7.8 shows the average found average latency (top graph) and the worst latency (bottom graph) using each algorithm. On average, the *VRP+CO* algorithm improves the *VRP* approach by 13.3% and the *VRP+TH* algorithm by 27.7%.

7.8.3.3 Varying Data Volume

To investigate how optimizing the *travel-to-TX* tradeoff impacts performance, we ran each of the heuristics-based algorithms on the *q0-16* input set. Figure 7.9 shows the average found average latency (top graph) and the worst latency (bottom graph) using each algorithm. The *VRP+TH* algorithm initially performs well when the volume of data is at or below 1 MB. However, this method begins to suffer greatly when the data volume surpasses 2 MBs. This is because this method uses a simplified model for communication and is agnostic to the *travel-to-TX* relationship. The *VRP* is also agnostic to data transfer rate and data volume, taking instead the more conservative approach and always minimizing the drone-to-sensor distance. This behavior causes the algorithm to not perform as well as the other two when the data volume is below 2 MBs. However, at around 8 MBs this approach merges with our *VRP+CO* algorithm, at which point it pays off to minimize the drone-to-sensor distance to maximize the data transfer rate. Our proposed *VRP+CO* algorithm consistently outperforms the other two approaches at both small and large data volumes.

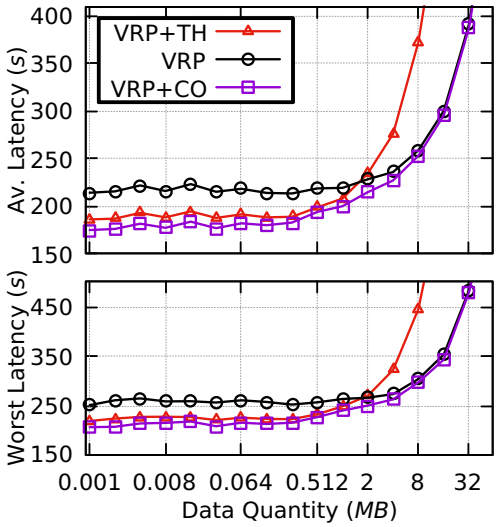


Figure 7.9 The average latency (top) and worst latency (bottom) while increasing the quantity of data to collect from each sensor.

7.8.3.4 Computation Time

Table 7.3 shows the computation time for the various heuristics-based algorithms. On smaller inputs (10 sensors) we see almost twice the computation to run the *VRP* algorithm when compared to the other two algorithms (*VRP+TH* and *VRP*). However, the computation time remains under 100 milliseconds until around 40 sensors. At larger inputs (200 sensors) we no longer see a notable difference between the *VRP+CO* and the *VRP+TH* algorithms, with the simpler *VRP* algorithm only slightly outperforming the others.

Table 7.3 Average computation time (in seconds) as the number of sensors (n) increases.

$n =$	10	20	30	40	50	100	150	200
VRP+TH	0.04	0.05	0.06	0.07	0.08	0.34	0.82	1.83
VRP	0.05	0.06	0.07	0.07	0.09	0.34	0.79	1.70
VRP+CO	0.09	0.09	0.10	0.11	0.13	0.41	0.91	1.81
OAP	0.11	0.22	0.33	0.45	0.58	-	-	-

7.8.3.5 Offline Global Planner Evaluation Summary

These results show that our *VRP+CO* algorithm performs very near the optimal solution and is able to exploit the *travel-to-TX* tradeoff. We believe that the latter trait is the main reason that the algorithm finds better solution quality when compared against the *VRP* and the *VRP+TH* algorithms. Although the *VRP* and the *VRP+TH* algorithms are faster than the *VRP+CO* algorithm at smaller input sizes, the *VRP+CO* algorithm can still solve inputs within a reasonable time (under 100 milliseconds for less than 40 sensors) and matches the computation time of the other two on larger inputs.

7.8.4 Online Adaptive Planner Evaluation

To evaluate our proposed Online Adaptive Planner (*OAP*), we ran the algorithm multiple times on the RPi4 mounted on our drone and recorded how quickly it could solve various input sizes. We created randomized problem setups by selecting sensors from the *n5-200* input set and then running our online planner to find an adapted plan. We varied the number of sensors from 10 up to 50.

Table 7.3 shows the computation time required to run the online planner. The planner can handle inputs of 10 sensors or less in about 100 milliseconds and appears to scale nearly linearly with the number of sensors, taking 580 milliseconds for 50 sensors. Although this time is reasonably fast, in practice this computation is done while the drone hovers. If online computation time becomes a concern, then we recommend cutting down the considered problem. For example, only planning for the next 10 sensors rather than 50 sensors and using the original plan from the Offline Global Planner to service the latter 40 sensors.

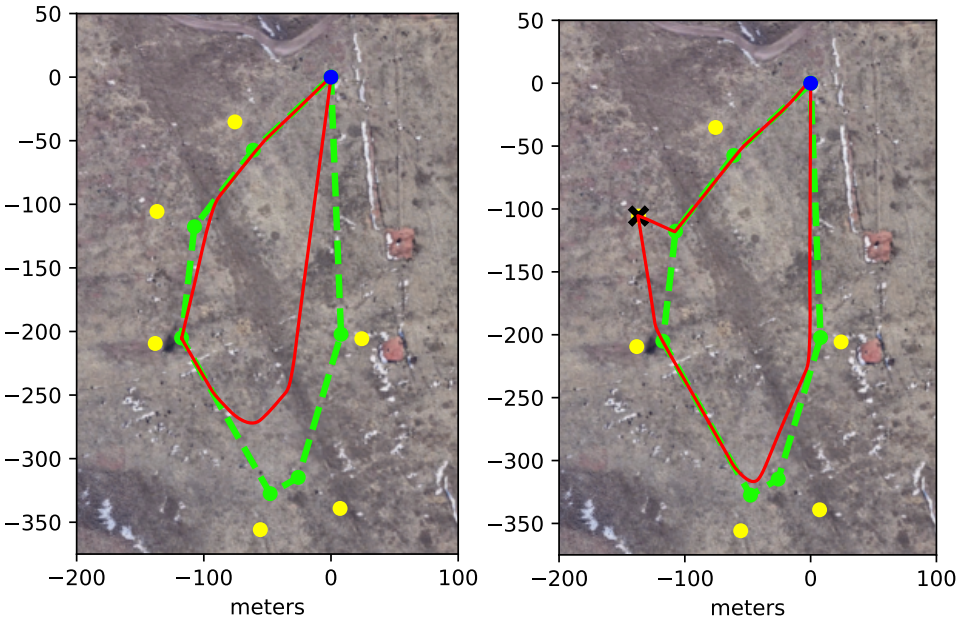


Figure 7.10 Field experiment results. Blue marker is the data center, yellow markers are IoT devices, black 'X' shows a dead sensor, green lines are the Offline Global Planner's plan, and red lines are the drone's GPS trace. Panel (a) shows standard execution and panel (b) shows execution with dead sensor.

7.9 Field Test

To validate our data-driven methodology, we prototyped the problem setup on a physical testbed. We deployed six Raspberry Pies (three RPi3s and three RPi4s) in a large field and used our planning framework to find a consistent deployment plan. We then flew our drone through the field, guided by our autopilot and Online Adaptive Planner, and collected data from the sensors over WiFi. The results of our prototype are shown in Fig. 7.10.

Figure 7.10(a) shows the final plan produced by the Offline Planner (in green) and the GPS trace of the drone while executing the plan (in red). We executed the plan five times and recorded the following flight times (in seconds): 84.0, 114.0, 94.0, 88.8, 123.0. This is an average of 100.8 seconds. Interestingly, the drone was able to collect data from the sensors before reaching most of the planned waypoints and did not actually stop at most of them. We ran the same setup in our Drone and Networking Simulator and recorded the following times (in seconds): 120.1, 107.5, 108.4, 108.9, 113.4. This gives us an average of 111.7 seconds in the simulator. The Offline Global Planner predicted that it would take the drone 108.4 seconds. This gives us an error of 10.8% between the simulation and the physical hardware, an error of 3.0% between the offline planner and the simulator, and an error of 7.5% between the offline planner and the hardware. We speculate that the difference between the simulator and our field test comes from the

acceleration. Our drone can accelerate very aggressively but the drone simulator does not readily support setting a custom acceleration. Other factors that may be contributing to these differences include wind and air pressure, which are not modeled in our simulation setup nor in our offline planner. Although not exact, these numbers are reasonably close, and much more accurate than the results in [7], where they had over 45% error between their planner and the physical testbed.

We repeated our experiment with one of the sensors turned off to prototype how the Online Adaptive Planner reacts to a dead sensor. Figure 7.10(b) shows the results of this experiment. We see that the online planner successfully redirected the drone to fly over the dead sensor before continuing on to the next sensor.

7.10 Conclusions

In this work we presented a holistic drone planning framework for solving the Multi-Drone Data-Muling problem. Our framework utilizes an Offline Global Planner to determine how to divide sensors up between drones and what route each drone should take to collect data from its assigned sensors. The framework then validates these offline plans through Monte Carlo simulations in a data-driven drone and network simulator that is designed to replicate real-world networking performance with an Online Adaptive Planner for adapting behavior online in response to communication failures. Using feedback from the simulations, the framework makes input parameter adjustments to the offline planner to prevent the drones from running out of energy during deployment. Our simulation experiments show that our proposed framework finds a balance between planning to utilize all of a drone's battery in the offline planner and saving energy for adaptive behavior during deployment.

Furthermore, we proposed two Offline Global Planners and an Online Adaptive Planner that exploit a tradeoff between the time required to collect data from a sensor and the time to move closer to a sensor to increase data transmission rates. Our results show that this tradeoff is important and can greatly improve solution performance.

Future work could further investigate other limitations, such as limited drone data storage or deadlines for data delivery. Other metrics could also be explored, such as prioritizing certain sensors over others and limiting the number of allowed battery swaps.

CHAPTER 8

SEMPER GUMBY: ADAPTIVE MULTI-ROBOT TASKING IN DYNAMIC ENVIRONMENTS

Robots are conglomerates of multiple sub-systems that all work together to perform the actions that make robots useful in a multitude of applications. However, these subsystems – such as energy or communication – tend to behave stochastically when deployed in the real world, making it challenging to accurately represent multiple robots in team tasking algorithms. In this work, we investigate how to incorporate the stochastic nature of robot systems into team-level tasking and decision-making. We demonstrate how to integrate stochastic systems into multi-robot task planning algorithms and show how to calculate an upper bound on model error and external interference. Inspired by our theoretical results, we propose a generalized planning framework for adjusting team-level decisions during deployment in response to unknown model error and external interference. We show how to apply our framework in two applications: (1) multi-drone site inspection and (2) drone-unmanned ground vehicle teaming for disaster response. We evaluated our proposed framework in simulations grounded in field data. Our results show that our framework improves performance by up to 38% when compared against baseline methods.

8.1 Introduction

Teams of robots have the potential to automate many monotonous and dangerous tasks, extending the reach of their human operators and removing humans from hazardous environments. For example, drones are able to quickly maneuver around infrastructure and industrial facilities to perform visual inspections. In natural disaster scenarios, unmanned ground vehicles (UGVs) can deliver emergency supplies and aid in search and rescue efforts, while drones travel ahead of the UGVs to identify open paths and locate survivors.

Many robot systems, notably energy and communication, behave non-deterministically in the real world. These systems create constraints on how we assign tasks to robots. For example, drones cannot operate indefinitely without recharging their batteries and wireless robot-to-robot communication will not be available everywhere when deployed in large environments. To handle these constraints in tasking algorithms, we must form a mathematical representation for how these systems operate.

However, robots depend on stochastic systems, and accounting for this noise when planning how to effectively deploy a team of robots is particularly challenging. For example, the energy consumption rate of a drone can be estimated using data-driven approaches (see [4, 279]), but these techniques still leave room for error. This error is further exacerbated by external factors such as wind or atmospheric pressure. The parameters of these stochastic system models will follow some probabilistic distribution. To compensate for

this, many offline planning approaches for making team-level tasking decisions use the expected value (or mean) of these parameters. This method becomes problematic during deployment when the actual performance of these systems do not follow the mean, creating a need to dynamically update plans online.

This problem is further complicated when the characteristics of the stochastic system change during deployment. For example, an increase in wind during deployment changes how a drone consumes energy. Although there is existing work that looks at how to incorporate probability models and "risk" into decision-making (see [111, 280]), these works tend to assume that the characteristics of a stochastic system are known *a priori* or only focus on how changing environments impact an individual robot's, or robot pair's, decision-making process.

A major aspect of team tasking with stochastic systems that is missing from existing literature is discussion on bounding allowable model error and external disturbance, particularly in the context of multi-robot tasking. After identifying an initial tasking assignment for each of the robots in the team, we would like to verify that this plan is still feasible after deploying the robots. Robots should be able to identify how much external interference they can tolerate before their current plan is no longer feasible. Additionally, we see a lack of discussion on the connection between multiple stochastic systems. For example, task planning for a data relaying application with drones must consider the nondeterministic nature of both energy and communication.

In this article, we seek to answer the question: "How do system model error and environmental disturbances impact team-level tasking and decision-making?" Particularly when model error and environmental disturbances are not fully known *a priori* or and when multiple stochastic systems impact team-level decisions.

We demonstrate how to determine a bound on allowable model error and external interference in multi-robot tasking. We also present a novel framework for team-level task planning that integrates stochastic robot system models in to team-level decision making and adapts these plans in response to changing external disturbance. Our framework utilizes our theoretical error bounds to determine when team-level task assignments must be replanned due to changing environmental interference and deviation from an offline plan. This framework generalizes a multitude of multi-robot tasking scenarios, and we demonstrate how to apply it to two specific applications: (1) multi-drone site inspection and (2) drone-UGV collaborative data acquisition and offloading. In the latter scenario, we highlight the interconnected relationship between energy and communication that arises from their stochastic nature.

Our unique contributions include:

1. We show how to calculate an upper bound on allowable model error given some existing task sequence.

2. We propose a novel framework for updating team-level tasking decisions that combines powerful offline planners with online controllers.

3. We demonstrate how to apply our tasking framework in two different multi-robot applications.

4. We provide rigorous numerical and simulation experiments, using field data from real robots, to validate our proposed framework and to compare it against current methods for multi-robot tasking.

To promote good research integrity and the reuse of our work, we have made our source code and data publicly available[1].

8.2 Related Work

There are several works that already consider stochastic systems and their impact on robot tasking. We will first look at relevant works that incorporate non-deterministic system models into tasking, with a focus on energy and robot-to-robot communication. Then, we will look at modeling robot component failure and how this impacts tasking. Finally, we will briefly discuss model error outside of the context of multi-robot tasking.

In this work, we are primarily focused on energy and communication, but we acknowledge that other robot models, such as dynamics, are stochastic and subject to external interference. The concepts introduced in this work can be readily extended to other systems.

8.2.1 Stochastic System Models in Robot Tasking

We first review works that incorporate stochastic representations for energy into tasking algorithms and then look at communication.

8.2.1.1 Energy Models in Tasking

There is a growing body of literature that looks at how to incorporate the stochastic nature of robot energy consumption into planning. For example, the authors in [110, 111] proposed a stochastic model for how drones consume energy that incorporates wind and vehicle weight. They argue that the energy consumption model for drones should include a random variable term that follows a Weibull distribution. They demonstrate how to form chance constraints based on user provided limits to acceptable risk for drones running out of energy and apply these constraints to a generalized drone-UGV site inspection

[1] https://github.com/JonD07/ProativeTasker

application. This energy model was further leveraged in [281], where they used it to determine what actions risk energy failure when training a deep reinforcement learning model for deciding between accepting a new task or recharging. The authors in [150] proposed a competing model, focused on stored energy, that incorporates battery recharge cycles and a normally distributed random variable term. They use this model to calculate the probability that an individual drone will fail to complete a task in a wildfire monitoring scenario. Breaking from drone-focused models, the authors in [132] consider a more generalized robot energy model that is uniformly distributed. They use this model at the agent-level to determine when robots should deviate from their given plan to recharge.

These works focus on how the stochastic nature of nondeterministic systems impact agent-level decision making. That is, they consider the risk of an individual robot running out of energy and adapt that robot's task plan without considering how this change impacts the performance of the entire team. In this work, we consider how changing system model parameters and external disturbances during deployment impact the performance of individual robots and then integrate this into online team-level decision making.

8.2.1.2 Communication Models in Tasking

There are many methods for modeling robot-to-robot communication, with different communication metrics considered. Some of the more common representations include estimating signal strength [184], signal-to-noise ratio [185], received signal strength indicator [201], and the data transmission rate [282]. The authors in [184] present a stochastic robot-to-robot communication model that includes path-loss, shadowing, and a Gaussian random variable for fading. The authors use this model to predict robot-to-robot signal strength and avoid losing peer-to-peer communication in multi-robot exploration tasks. There are several other works that investigate different algorithms applied to similar exploration tasks with near identical communication models [283, 284]. Building on this concept, the authors in [285] look at how to add a probability guarantee to robot-to-robot links, then use this for a multi-robot patrolling application. Rather than considering the existence of robot-to-robot links, we propose a stochastic model for the data transmission rate between robots and show how to incorporate this into multi-robot task assignment problems. An alternative approach is to build a Gaussian Process regression model to represent robot-to-robot communication based on robot positions [182] and then use the predicted mean from the regression model to guide tasking decisions. This is seen in [201] for investigating points of interest, in [3] for multi-robot data streaming, and in [212] for drone information relays.

The majority of these works assume that the model parameters of these stochastic robot systems are known *a priori*. Although we can run preliminary experiments prior to deployment to get an initial understanding of how to model these systems, their actual performance depends on the environment during

the time of deployment, which may even change during deployment. Multi-robot tasking algorithms must be able to not only adjust for the current environment but must also adjust team-level tasking decisions during deployment in response to changes in the environment. Although the works that propose using Gaussian Process regression models could be updated online, they rely on the predicted mean produced by the regression model to drive decisions. We demonstrate why using the expected value, or mean, of a stochastic model exposes the robots to undue risk of breaking application and system constraints.

8.2.2 Other Related Topics

There are several other related topics in the literature that we would like to highlight and distinguish from our own work. There are several works that consider the probability that robot capabilities change during deployment (e.g., sensor failure or increasing carrying capability – see [280, 286, 287]). In this work, we assume that the probability of complete component failure is negligible compared to the probability that an environmental change requires us to update how we model robot systems. There are also several works that address other causes for uncertainty in decision making, including observation [288], object properties such as size and weight [289], and terrain durability [290]. These works consider uncertainty in the context of single-robot decision making and do not focus on changing robot system models.

There are several other works that consider model error outside of the context of planning. For example, model error in control is a well studied topic in robotics [291]. This leads into topics covering uncertainty in mobility models [292, 293]. This work is focused on planning for robot teams and not control.

8.3 Problem Formulation

In this section, we first review how to define stochastic system models for energy and robot-to-robot communication, then we formally define the multi-robot tasking problem.

8.3.1 Energy Model

The rate that a robot consumes energy at is most commonly modeled on the robot's speed. This has been thoroughly investigated for drones [4, 5, 63]. The same behavior has also been observed on UGV [64]. In general, the expected energy consumption of a robot (in watts) can be modeled as the polynomial

$$\mathcal{P}(v) = c_1 v^3 + c_2 v^2 + c_3 v + c_4, \tag{8.1}$$

where v is the robot's speed and c_1, c_2, c_3 and c_4 are constants that can be determined experimentally. In practice, (8.1) is very noisy (see [4, 279]) and the values for c_1, c_2, c_3 and c_4 represent the average rate observed. As presented, $\mathcal{P}(v)$ does not represent the stochastic nature of the physical system. We instead

represent the rate that a robot consumes energy at as the normally distributed random variable

$$\mathrm{R}_e(v) \sim \mathcal{N}(\mu, \sigma^2), \tag{8.2}$$

with mean $\mu = \mathcal{P}(v)$ and standard deviation σ.

We assume that c_1, c_2, c_3, and c_4 are known but the standard deviation term σ is either unknown *a priori* or may change during deployment due to changing environmental factors. We can estimate an initial value for model noise (i.e. σ) through field experimentation but its true value will need to be adjusted appropriately to account for current wind and air moister conditions during deployment. Noise parameter σ will also change during deployment as the environment changes (e.g., a change in wind speed or severity in gusting). We provide example values for c_1, c_2, c_3, c_4 and σ from field experiments on physical hardware in Section 8.7.

We acknowledge that other works, such as [110, 111] advocate for a noise term in (8.1) that follows a Weibull distribution. We chose a normal probability distribution to define our noise term because our own field data (presented in Section 8.7) exhibits a normal distribution. However, the noise term can be readily exchanged to any probability distribution that has a defined inverse cumulative distribution function (quantile function).

8.3.2 Communication Model

The authors in [282] determined through field experiments that the rate that two robots transmit data between one another (in megabytes per second) can be modeled as

$$tx(d) = min(\frac{c_{\mathrm{I}}}{d^2 + c_{\mathrm{II}}} + c_{\mathrm{III}},\ r_{max}) \tag{8.3}$$

where d is the distance between the two robots, r_{max} is the maximum possible data throughput, and $c_{\mathrm{I}}, c_{\mathrm{II}}$, and c_{III} are constants that can be determined experimentally. Similar to energy, (8.3) represents the expected value of a non-deterministic system. The authors in [282] noted that the noise in (8.3) exhibits a Weibull distribution. Building on these findings, we propose modeling robot-to-robot communication as the Weibull-distributed random variable

$$\mathrm{R}_c(d) \sim \mathcal{W}(k, \lambda), \tag{8.4}$$

where k and λ are shape and scale parameters, respectively, and

$$\mathbb{E}[\mathrm{R}_c] = tx(d). \tag{8.5}$$

The parameters of R_c will depend on several factors, such as obstacles between the robots and channel interference from other devices. We provide example values for k and λ from our own field experiments in Section 8.7. Many existing works have argued in favor of adding a Gaussian distributed term to

communication models [184, 284]. We note that these works are modeling signal strength and not data transmission rate.

8.3.3 Formal Problem Definition

We consider a tasking scenario where we are given n tasks to complete and m heterogenous robots to complete these tasks. In general, this problem scenario can be formulated mathematically as

$$\min f(\mathcal{X}) \tag{8.6}$$

Subject to:

$$h(\mathcal{X}, \Xi) \tag{8.7}$$

where $\mathcal{X}$ is a set of decision variables, Ξ is a set of input parameters that define the problem setup, $f(\mathcal{X})$ is a function that maps $\mathcal{X}$ to the objective of the task assignment problem, and $h(\mathcal{X}, \Xi)$ is a function that constrains the assignment variables in $\mathcal{X}$ to the problem input parameters.

Some of the input parameters $\xi \in \Xi$ are stochastic. For example, the parameters that define how much energy a robot will consume while performing a task or the rate at which robots transmit data between one another.

We can solve for a drone-to-task assignment in $\mathcal{X}$ using advanced mathematical solvers, heuristics-based algorithms, or even learning methods. We assume that there exists a means for solving (8.6). The best solution approach will depend on the specific scenario and available computational resources, which falls out of the scope of this research.

In many cases, finding an optimal assignment for $\mathcal{X}$ is NP-Hard. For example, see the Multi-Robot Task Allocation (MRTA) problem [294], the Vehicle Routing Problem (VRP) [138], or collaborative Drone-UVG routing problems [14, 97, 295]. Even heuristic-based algorithms can be too computationally expensive to compute online, particularly on robot platforms such as drones with limited computational power. More powerful resources, such as a centralized computer, can be leveraged to find an initial, offline solution. However, individual robots should be able to identify when this plan is no longer feasible due to changing environmental factors that impact robot systems, such as energy and communication.

8.4 Bounding System-Model Error

In this section, we look at how to bound allowable variance in a system model. As discussed in the previous section, robot systems, such as energy and communication, behave non-deterministically when our robots are deployed in the real world. A naive approach to integrating these systems into tasking algorithms is to use the expected value, or mean, of the random variables in our models when constraining

allowable actions. However, using the expected value of a stochastic system equates to exposing the robots to a 50% chance of breaking operational constraints in our problem.

As a precursor to bounding model error, we will first demonstrate how to guarantee a user-defined probability of meeting problem constraints (8.7) as they pertain to stochastic systems. Then, we will define model error and show how to find a bound on allowable error when given a task plan. We cover each of these aspects for both energy and communication system models.

8.4.1 Energy Model Error

Let $x_{ji} \in \mathcal{X}$ be a decision variable that determines if robot j completes task i. The standard approach to defining an energy constraint in $h(\mathcal{X}, \Xi)$ for robot j is

$$\sum_{x_{ji} \in \mathcal{X}} \mathcal{P}(v_j) t_{ji} x_{ji} \leq e_j, \tag{8.8}$$

where v_j is the speed that robot j moves at while completing task j (which may be zero), t_{ji} is the time required for j to complete task i, and e_j is the energy budget of robot j (in joules). As discussed above, using $\mathcal{P}(v_j)$ (the average energy consumption rate) is equivalent to accepting a 50% risk of running out of energy during deployment. That is, if we observe a realization of $\mathrm{R}_e > \mu = \mathcal{P}(v_j)$ during deployment (which is expected to occur with probability 0.5), then we are at risk of breaking constraint (8.8).

To control the risk of running out of energy during deployment, we can convert (8.8) into the chance constraint

$$P\Big[\sum_{x_{ji} \in \mathcal{X}} \mathrm{R}_e t_{ji} x_{ji} \leq e_j \Big] \geq \alpha. \tag{8.9}$$

Our updated constraint (8.9) ensures that the probability that we do not break our original constraint (8.8) is greater than some user provided probability of guarantee α. Let Φ_N^{-1} denote the quantile function (inverse CDF) of the standard normal distribution. We can rewrite our chance constraint as the linear constraint

$$\sum_{x_{ji} \in \mathcal{X}} (\mu - z_{1\text{-}\alpha}\sigma) t_{ji} x_{ji} \leq e_j, \tag{8.10}$$

where $z_{1\text{-}\alpha} = \Phi_N^{-1}(1-\alpha)$.

We define error in our energy model, γ_e, as

$$\gamma_e = \frac{\sigma}{\mu}. \tag{8.11}$$

For ease of notation, let $r_e(v_j) = (\mu_j - z_{1\text{-}\alpha}\sigma_j)$, where $\mu_j = \mathcal{P}(v_j)$ and $\sigma_j = \gamma_e \mu_j$. Let $\mathcal{X}^*$ be a robot-to-task assignment found by solving (8.6) with updated energy constraints in $h(\mathcal{X}, \Xi)$ updated to (8.10). We can determine the maximum allowable error in the energy model that the robot can tolerate without violating constraint (8.10) by solving

$$\max \quad \gamma_e \tag{8.12}$$

subject to:

$$\mu(1-\gamma_e z_{1\text{-}\alpha}) \sum_{\substack{x_{ij}\in\mathcal{X}^* \\ x_{ij}=1}} t_{ji} \leq e_j \tag{8.13}$$

Observe that (8.12) with constraint (8.13) is linear and we can directly solve for γ_e by rearranging the terms in the constraint to be

$$\gamma_e = \frac{1}{\mu z_{1\text{-}\alpha}}\big(\mu - e_j \sum_{\substack{x_{ij}\in\mathcal{X}^* \\ x_{ij}=1}} t_{ji}\big). \tag{8.14}$$

That is, solving (8.14) tells us the maximum error from mis-classifying our energy model and from external interference that the robot can tolerate without needing to resolve for $\mathcal{X}^*$.

8.4.2 Communication Model Error

Similar to energy, the standard method for incorporating communication constraints into tasking problems, in the general case, is

$$\frac{q}{tx(d_{j\hat{j}})} x_{j\hat{j}} \leq t_*, \tag{8.15}$$

where $x_{j\hat{j}} \in \mathcal{X}$ determines if robot j communicates with robot $\hat{j}$, $d_{j\hat{j}}$ is the distance between robots j and $\hat{j}$, q is the volume of data (in megabytes) to be transmitted, and $t_* \in \mathcal{X}$ is some other continuous variable that connects constraint (8.15) with objective (8.6) or other constraints in $h(\mathcal{X}, \Xi)$.

Using expected data transmission rate $tx(d_{j\hat{j}})$ in constraint (8.15) exposes the robot to a 50% chance of breaking the constraint during deployment. We control this risk by transforming (8.15) into the chance constraint

$$P\left[\frac{q}{\mathrm{R}_c(d_{j\hat{j}})} x_{j\hat{j}} \leq t_*\right] \geq \alpha, \tag{8.16}$$

for user defined probability of guarantee α. Using the quantile function of the Weibull distribution, we can rewrite (8.16) as

$$\frac{q}{\lambda\big(-ln(\alpha)\big)^{1/k}} x_{j\hat{j}} \leq t_*. \tag{8.17}$$

We define communication error γ_c, in terms of k and λ, as

$$k = c_5\gamma_c + c_6 \tag{8.18}$$

$$\lambda = c_7\gamma_c + c_7 \tag{8.19}$$

where c_5, c_6, c_7, and c_8 are constants that depend on specific hardware and can be determined experimentally. Given a robot-to-task assignment $\mathcal{X}^*$, with updated communication constraints in $h(\mathcal{X}, \Xi)$ based on (8.17), the maximum allowable error in robot-to-robot communication without violating constraint (8.17) will be

$$\max \quad \gamma_c \tag{8.20}$$

subject to:

$$\begin{aligned} &\frac{qx_{j\hat{j}}}{t_*} \leq (c_7\gamma_c + c_7)\big(-ln(\alpha)\big)^{1/(c_5\gamma_c+c_6)}, \\ &\forall x_{ij} \in \mathcal{X}^*, x_{ij} = 1. \end{aligned} \tag{8.21}$$

Solving (8.20) will tell us the maximum allowable communication disturbance that the robots can tolerate before the robot-to-task assignment in $\mathcal{X}^*$ is no longer valid.

8.5 Error-Aware Tasking Framework

When a robot determines that it has surpassed the allowable error in a system model and the current robot-to-task assignment in $\mathcal{X}^*$ is no longer feasible, we must re-solve (8.6) using updated model parameters. In this section we propose a framework for replanning team-level task assignment during deployment.

Figure 8.1 depicts the conceptual design of our tasking framework. The framework is divided up into two major components: (1) a mission planner, and (2) an autopilot. The mission planner runs on a centralized coordinator, which could be a designated robot with strong computational power or a stationary device, and is responsible for finding robot-to-task assignment $\mathcal{X}^*$ using whatever planning methodology is best suited for the application. The mission planner sends *task plans* to each of the robots by extracting the robot-to-task assignments in $\mathcal{X}^*$ for each robot.

The autopilot runs on each robot and performs multiple functions. The primary function of the autopilot is to execute the current *task plan* as set by the mission planner. The autopilot also coordinates the behavior of multiple sub-modules on the robot. These sub-modules include the status module, the validation module, and system modules (e.g., energy and communication). As the robot operates in its environment, the system modules measure how the robot interacts with the environment and update system model parameters. In addition to monitoring communication performance, the communication module also handles robot-to-robot communication. The validation module uses the updated system model

parameters to validate the robot's *task plan* by calculating upper bounds on system model error, as discussed in Section 8.4. If the validation module determines that the robot cannot safely complete the assigned task plan, it tells the status module to send a *replan request* to the centralized coordinator. The status module is responsible for handling robot-to-coordinator communication. In addition to sending out *replan requests*, the status module also sends *status reports* back to the centralized coordinator, which include the robots current position, battery level, and updated system model parameters from the system modules.

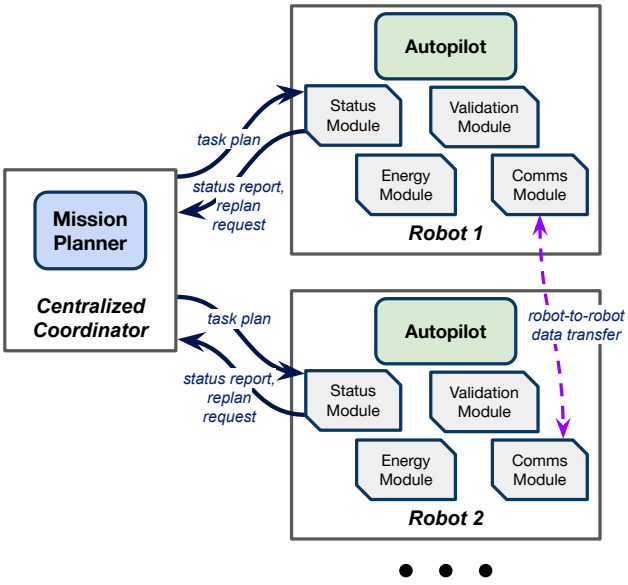


Figure 8.1 Proposed error-aware tasking framework.

When the mission planner receives a *replan request*, it uses the most recent *status report* from each robot and re-solves (8.6). It is expected that this planning process will be time consuming. When solving for new robot-to-task assignments during mission execution, the mission planner has a time-out limit and projects the robots progress forward in time based on this time-out window and the previously assigned $\mathcal{X}^*$. The mission planner uses the projected, future position as the initial position of each robot as its input when solving for a new $\mathcal{X}^*$. Once the mission planner either finds an optimal robot-to-task plan, or hits the timeout, it sends updated *task plans* to each of the robots.

Our proposed framework uses a pull approach to replanning, as opposed to a push approach. In the pull methodology, the robots must request a replan from the centralized coordinator. In a push design, the centralized coordinator would collect system model changes and progress updates from the robots and determine for itself when a replan is needed, then push these changes out to the robots when an update is made. A push architecture would proactively update team-level plans and improve team performance

without waiting until the current plan is no longer feasible, but would heavily rely on a fully connected communication network. A pull architecture only makes updates when needed and can accommodate intermittent communication between the robots and the centralized coordinator. Although this approach may underperform when compared to the push methodology, we chose the pull approach due to its flexibility with peer-to-peer communication. Existing research on how to plan for intermittent communication in robot teams (see [22, 66, 70]) can be applied to our proposed framework, which would not be the case in a push architecture.

8.6 Multi-Robot Tasking Scenarios

To demonstrate the impact of external interference on robot systems and how our proposed tasking framework handles these disturbances in team-level decision-making, we consider two multi-robot tasking scenarios. The first scenario is a multi-drone site inspection setup that we show how to model as a VRP. The second scenario is a mixed drone-UGV data acquisition and offloading scheme that we model as a multistage VRP. After introducing these two scenarios in this section, we will apply each of them to various case studies in Section 8.7.

In each of the following subsections, we will use italic, uppercase letters to indicate sets (e.g., 'I'). Bold, lowercase text letters are decision variables (e.g., '$\mathbf{x}$') and italic, lowercase letters are constants (e.g., 'c').

Table 8.1 Sets for Multi-Drone Site Inspection

Set	Magnitude	Description
I	n	Set of PoI, indexed with i and $\hat{\imath}$
J	m	Set of drones, indexed with j
K	$\leq n$	Set of drone sorties, indexed with k

Table 8.2 Decision Variables for Multi-Drone Site Inspection

Variable	Range	Description
$\mathbf{x}_{ji\hat{\imath}k}$	$\{0,1\}$	Does drone j travel from PoI i to $\hat{\imath}$ on sortie k?
$\mathbf{w}_{jk}$	$\{0,1\}$	Does drone j deploy on sortie k?
$\mathbf{t}_{jk}$	$\mathbb{R}_{\geq 0}$	Completion time of sortie k for drone j
$\mathbf{u}_i$	$\mathbb{N}$	Sequence number for PoI i

8.6.1 Multi-Drone Site Inspection

We consider a multi-drone site inspection scenario, where a team of energy limited drones must visit several points of interest (PoI) in minimum time. The drones have limited onboard energy and must return

to a centralized base station to recharge, meaning that some drone may need to run multiple sorties (short flights) for every PoI to be visited. Possible real-world applications for this setup include search and rescue [296], infrastructure inspections [297], security and surveillance [298], and natural disaster response [299].

This scenario can be modeled as a Mixed Integer, Linear Program (MILP). Tables 8.1 and 8.2 provide a full list of all sets and decision variables used in our formulation. We use an adaptation of the vehicle flow model of the VRP [138], with Miller, Tucker, and Zemlin (MTZ) cycle-elimination constraints [116] to model this problem. Our objective is to minimize accumulated penalty, which we define as the time that each drone completes its last sortie, squared. For example, if drone 1 finishes its final sortie after 60 seconds and drone 2 finished after 65 seconds, the total penalty would be $60^2 + 65^2 = 7{,}825$ seconds-squared, or 2.17 minutes-squared. To make the penalty easier to read, we will use minutes-squared when reviewing the results of our experiments in Section (8.7). This metric has been presented as an alternative to minimizing the maximum sortie (see [17, 97]) because it considers the latency of all vehicles while also penalizing excessively long deployments. Let $\hat{k}$ be the final sortie in K.

$$\min \quad \sum_{j=1}^{m} \mathbf{t}_{j\hat{k}}^2 \tag{8.22}$$

subject to:

$$\sum_{i \in I} \left(\sum_{\hat{i} \in I} \frac{d_{i\hat{i}}}{v_j} \mathbf{x}_{ji\hat{i}k} + \frac{d_{ib}}{v_j} \mathbf{x}_{jibk} \right) \leq \mathbf{t}_{jk}, \quad j \in J, k \in K \tag{8.23a}$$

$$r_e(v_j) \sum_{i \in I} \left(\sum_{\hat{i} \in I} \frac{d_{i\hat{i}}}{v_j} \mathbf{x}_{ji\hat{i}k} + \frac{d_{ib}}{v_j} \mathbf{x}_{jibk} \right) \leq e_j, \quad j \in J, k \in K \tag{8.23b}$$

$$\sum_{\hat{i} \in I} \mathbf{x}_{j\hat{i}ik} \leq \mathbf{w}_{jk}, \quad j \in J, i \in I, k \in K \tag{8.23c}$$

$$\sum_{\hat{i} \in I} \mathbf{x}_{ji\hat{i}k} \leq \mathbf{w}_{jk}, \quad j \in J, i \in I, k \in K \tag{8.23d}$$

$$\mathbf{w}_{jk} \leq \mathbf{w}_{jk\text{-}1}, \quad j \in J, k \in K, k > 1 \tag{8.23e}$$

$$\sum_{i \in I} \mathbf{x}_{j0ik} + \mathbf{x}_{j0b1} = \mathbf{w}_{jk}, \quad j \in J, k = 1 \tag{8.23f}$$

$$\sum_{i \in I} \mathbf{x}_{jbik} = \mathbf{w}_{jk}, \quad j \in J, k \in K, k > 1 \tag{8.23g}$$

$$\sum_{i \in I} \mathbf{x}_{jibk} = \mathbf{w}_{jk}, \quad j \in J, k \in K \tag{8.23h}$$

$$\sum_{j \in J} \sum_{k \in K} \left(\mathbf{x}_{jbik} + \sum_{\hat{i} \in I} \mathbf{x}_{j\hat{i}ik} \right) = 1, \quad i \in I \tag{8.23i}$$

$$\sum_{j \in J} \sum_{k \in K} \left(\mathbf{x}_{jibk} + \sum_{\hat{i} \in I} \mathbf{x}_{j\hat{i}ik} \right) = 1, \quad i \in I \tag{8.23j}$$

$$\sum_{\hat{i}\in I} \mathbf{x}_{j\hat{i}ik} + \mathbf{x}_{jbik} = \sum_{\hat{i}\in I} \mathbf{x}_{ji\hat{i}k} + \mathbf{x}_{jibk}, \quad j \in J, i \in I, k \in K \tag{8.23k}$$

$$\mathcal{M}(1 - \mathbf{x}_{ji\hat{i}k}) + \mathbf{u}_i + 1 \geq \mathbf{u}_{\hat{i}} \geq \mathbf{u}_i + 1 - \mathcal{M}(1 - \mathbf{x}_{ji\hat{i}k}), \quad j \in J, i, \hat{i} \in I, k \in K \tag{8.23l}$$

Constraint 8.23a tracks the time required for each drone to complete each sortie. We note that subscript b indicates the base. For the first sortie ($k = 1$), we add

$$\frac{d_{0b}}{v_j}\mathbf{x}_{j0b1} + \sum_{i\in I} \frac{d_{0i}}{v_j}\mathbf{x}_{j0i1} \tag{8.24}$$

to constraint 8.23a to account for the time for drone j to travel from its initial position (indicated as subscript 0) to either PoI i or to the base. On subsequent sorties ($k > 1$), we include

$$t_j^b \mathbf{w}_{jk} + \sum_{i\in I} \frac{d_{bi}}{v_j}\mathbf{x}_{jbik} + \mathbf{t}_{jk-1} \tag{8.25}$$

to incorporate the time required to change batteries, t_j^b, the time required to travel from the base to the next PoI, and the end time of the previous sortie. Constraint (8.23b) ensures that drone j does not exceed its allowable energy budget, e_j, on sortie k and is an example of the energy chance constraints discussed in Section 8.4. Similar to Constraint (8.23a), we add

$$r_e(v_j)\left(\frac{d_{0b}}{v_j}\mathbf{x}_{j0b1} + \sum_{i\in I} \frac{d_{0i}}{v_j}\mathbf{x}_{j0i1}\right) \tag{8.26}$$

to (8.23b) when $k = 1$ and

$$r_e(v_j)\sum_{i\in I} \frac{d_{bi}}{v_j}\mathbf{x}_{jbik} \tag{8.27}$$

when $k > 1$.

Constraints (8.23c) and (8.23d) ensure that drone j does not visit PoI i if that drone does not execute sortie k. Constraint (8.23e) forces lower numbered sorties to be used before later numbered sorties. Constraints (8.23f), (8.23g) and (8.23h) forces the drone to initial and end a sortie if sortie-tracking variable $\mathbf{w}_{jk} = 1$. We add $\mathbf{x}_{j0ik}$ to (8.23h) when $k = 1$ to allow the drones to return directly to the base on the first sortie, which may be needed when replanning tasking online. Constraints (8.23i) and (8.23j) ensures that every PoI is visited exactly once. We replace $\mathbf{x}_{jbik}$ with $\mathbf{x}_{j0ik}$ in (8.23i) when $k = 1$. Constraint (8.23k) forces the edges entering and exiting PoI i to be on the same sortie. We again replace $\mathbf{x}_{jbik}$ with $\mathbf{x}_{j0ik}$ here when $k = 1$. Finally, constraint (8.23l) is a MTZ cycle-elimination constraint.

8.6.1.1 Solver Speed-Up

Our framework must solve (8.22), subject to the constraints in (8.23), during deployment when one of the drones requests a replan. Therefore, we would like to quickly find a feasible solution to $\mathcal{X}$ when addressing replan requests. We used Gurobi (v12.0.2) as our mathematical solver and found various

techniques to help speed up the solve time.

The first major solve time improvement is to give Gurobi a "warm start" solution. That is, finding an incumbent solution to give the solver prior to optimizing the model. This allows the solver to use the user found incumbent solution to better prune search tree branches. We can further improve solve time by limiting the upper bound of k – the number of drone sorties. In the worst case, a single drone will only visit one PoI on each sortie, giving us an upper bound on k of n. In practice, this scenario is unlikely and we can bound k by some more reasonable value, which will greatly reduce the search space and speed up solve time. We found that the latter technique particularly speeds up initial root relaxation time. We also tighten how we set $\mathcal{M}$, which is bounding our node ordering variables, $\mathbf{u}_i$. Observe that we can tighten the bounds on $\mathbf{u}_i$ to be $0 \leq \mathbf{u}_i \leq n$. Setting $\mathcal{M} = n$ facilitates this bound.

We additionally found that turning off cutting strategies, except for network and flow cover cuts, helps the solver quickly find quality solutions. Gurobi can be configured to focus on finding feasible solutions quickly, as oppose to reducing bounds and closing the provable solution gap. This last technique is particularly helpful when the drones are deployed and waiting for a new solution. Our framework limits solve time during deployment, so we do not anticipate reaching the optimal solution in the allotted time and prefer to focus on finding a quality solution over proving bounds on the solution.

Table 8.3 Sets for Drone-UGV Collaboration

Set	**Magnitude**	**Description**
I	n	Set of PoI
I_l	n_l	Set of PoI in sector l, indexed with i and $\hat{\imath}$
J	m	Set of drones, indexed with j
K	$\leq \max(n_l)$	Set of drone sorties per sector, indexed with k

Table 8.4 Decision Variables for Drone-UGV Collaboration

Variable	**Range**	**Description**
$\mathbf{x}_{lji\hat{\imath}k}$	$\{0,1\}$	In l, does j travel from i to $\hat{\imath}$ on sortie k?
$\mathbf{w}_{ljk}$	$\{0,1\}$	Does drone j deploy on sortie k in sector l?
$\mathbf{t}_{ljk}$	$\mathbb{R}_{\geq 0}$	Completion time of sortie k in sector l for drone j
$\mathbf{u}_i$	$\mathbb{N}$	Sequence number for PoI i
$\mathbf{o}_{ljk}$	$\{0,1\}$	Is k the final sortie for drone j in sector l?
$\mathbf{d}_{lj}$	$\mathbb{N}$	Number of PoI drone j visits in sector l
$\mathbf{e}_{ljk}$	$\mathbb{R}_{\geq 0}$	Energy used by j on sortie k in sector l
$\tilde{\mathbf{e}}_{lj}$	$\mathbb{R}_{\geq 0}$	Remaining energy held by j at the end of sector l

8.6.2 Drone-UGV Teaming for Disaster Response

We consider a drone-UGV collaborative data acquisition and offloading setup as a second multi-robot tasking scenario. In this scenario, we are given a set of drones, J, a set of drone sorties, K, and a set of PoI, I, that are divided across a set of sectors, L. Let $I_l \subseteq I$ be the set of PoI for sector $l \in L$. For each sector, the drones must capture high-definition images of the environment (at the PoI in that sector) and relay this data back to the UGV. The UGV will then analyze the images and move to the next sector. The PoI are broken up into multiple sectors. There is a centralized base that the drones may return to for battery swaps. This scenario can be applied to a variety of drone-UGV applications, such as infrastructure inspection [97], precision agriculture [9], and natural disaster response [17].

The drones transmit the HD images to the UGV over IEEE 802.11 (Wi-Fi). We assume that there exists the communication infrastructure required for the robots to send status reports and replan requests to the centralized coordinator, and robot task plans from the coordinator back to the robots, but this link is not sufficient for relaying HD images from the drones to the UGV. For example, cellular networks or long range wide area networks [221] used for low bandwidth, text-based message passing.

Tables 8.3 and 8.4 list the decision variables used in this subsection. We note that the decision variables used for the multi-drone site inspection formulation in the previous subsection are not reused here. Our objective is to minimize accumulative penalty. This scenario can be modeled as the following Mixed Integer, Non-Linear Program (MINLP).

$$\min \quad \sum_{l \in L} \sum_{j \in J} \mathbf{t}_{lj\hat{i}}^2 \tag{8.28}$$

subject to:

$$\sum_{i \in I_l} \sum_{\hat{i} \in I_l} \frac{d_{i\hat{i}}}{v_j} \mathbf{x}_{lji\hat{i}k} + \sum_{i \in I_l} \frac{d_{ib}}{v_j} \mathbf{x}_{ljibk} + \sum_{i \in I_l} d_{ig} \mathbf{x}_{ljigk} \leq \mathbf{t}_{ljk}, \quad l \in L, j \in J, k \in K \tag{8.29a}$$

$$r_e(v_j)\Big(\sum_{i \in I_l} \sum_{\hat{i} \in I_l} \frac{d_{i\hat{i}}}{v_j} \mathbf{x}_{lji\hat{i}k} + \sum_{i \in I_l} \frac{d_{ib}}{v_j} \mathbf{x}_{ljibk} + \sum_{i \in I_l} d_{ig} \mathbf{x}_{ljigk} \Big) + r_e(0)\Big(\frac{q}{\lambda\big(-ln(\alpha)\big)^{1/k}} \mathbf{d}_{lj} \Big) \mathbf{o}_{ljk} \leq \mathbf{e}_{ljk}, \tag{8.29b}$$
$$l \in L, j \in J, k \in K$$

$$\mathbf{e}_{ljk} \leq e_j, \quad l \in L, j \in J, k \in K \tag{8.29c}$$

$$\tilde{\mathbf{e}}_{lj} - \mathcal{M}\Big(1 - (\sum_{\hat{i} \in I_l} \mathbf{x}_{ljigk} + \mathbf{x}_{ljbgk}) \Big) \leq \mathbf{e}_{ljk}, \quad l \in L, j \in J, k \in K \tag{8.29d}$$

$$\sum_{i \in I_l} \Big(\mathbf{x}_{ljgi} + \sum_{k \in K} \big(\mathbf{x}_{ljbik} + \sum_{\hat{i} \in I_l} \mathbf{x}_{lji\hat{i}k} \big) \Big) = \mathbf{d}_{lj}, \quad l \in L, j \in J \tag{8.29e}$$

$$\mathbf{x}_{ljbgk} + \sum_{i \in I_l} \mathbf{x}_{ljigk} = \mathbf{o}_{ljk}, \quad l \in L, j \in J, k \in K \tag{8.29f}$$

$$\mathbf{w}_{ljk} \leq \mathbf{w}_{ljk\text{-}1}, \quad l \in L, j \in J, k \in K, k > 1 \tag{8.29g}$$

$$\sum_{k\in K} \mathbf{o}_{ljk} = 1, \quad l \in L, j \in J \tag{8.29h}$$

$$\mathbf{w}_{ljk} \leq 1 - \mathbf{o}_{ljk\text{-}1}, \quad l \in L, j \in J, k \in K, k > 1 \tag{8.29i}$$

$$\mathbf{x}_{ljbgk} + \sum_{i\in I_l} \mathbf{x}_{ljbik} = \mathbf{w}_{ljk}, \quad l \in L, j \in J, k \in K \tag{8.29j}$$

$$\mathbf{x}_{ljbgk} + \sum_{i\in I_l} (\mathbf{x}_{ljibk} + \mathbf{x}_{ljigk}) = \mathbf{w}_{ljk}, \quad l \in L, j \in J, k \in K \tag{8.29k}$$

$$\sum_{k\in K} \left(\mathbf{x}_{ljbgk} + \sum_{i\in I_l} \mathbf{x}_{ljigk}\right) = 1, \quad l \in L, j \in J \tag{8.29l}$$

$$\sum_{j\in J} \left(\mathbf{x}_{ljgi} + \sum_{k\in K} \left(\mathbf{x}_{ljbik} + \sum_{\hat{i}\in I_l} \mathbf{x}_{lj\hat{i}ik}\right)\right) = 1, l \in L, i \in I_l \tag{8.29m}$$

$$\sum_{j\in J} \left(\sum_{k\in K} \left(\mathbf{x}_{ljibk} + \mathbf{x}_{ljigk} + \sum_{\hat{i}\in I_l} \mathbf{x}_{lji\hat{i}k}\right)\right) = 1, l \in L, i \in I_l \tag{8.29n}$$

$$\sum_{\hat{i}\in I_l} \mathbf{x}_{lj\hat{i}ik} + (\mathbf{x}_{ljgi}) + (\mathbf{x}_{ljbik}) = \sum_{\hat{i}\in I_l} \mathbf{x}_{lji\hat{i}k} + \mathbf{x}_{ljigk} + \mathbf{x}_{ljibk}, \quad l \in L, j \in J, i \in I_l, k \in K \tag{8.29o}$$

$$\mathcal{M}(1 - \mathbf{x}_{lji\hat{i}k}) + \mathbf{u}_{li} + 1 \geq \mathbf{u}_{l\hat{i}} \geq \mathbf{u}_{li} + 1 - \mathcal{M}(1 - \mathbf{x}_{lji\hat{i}k}), l \in L, j \in J, i \in I_l, \hat{i} \in I_l, k \in K \tag{8.29p}$$

Constraint (8.29a) tracks the duration time of each drone sortie. When $k = 1$, we add

$$\sum_{i\in I_l} \frac{d_{gi}}{v_j} \mathbf{x}_{ljgi} + \frac{d_{gb}}{v_j} \mathbf{x}_{ljgb} \tag{8.30}$$

to the left hand side of the inequality to account for the time to move away from the drone's starting position in that sector. For $k > 1$, we add

$$t_j^b \mathbf{w}_{ljk} + \sum_{i\in I_l} \frac{d_{bi}}{v_j} \mathbf{x}_{ljbik} + d_{bg} \mathbf{x}_{ljbgk} + \mathbf{t}_{ljk-1} \tag{8.31}$$

to the left hand side of constraint (8.29a) to account for the time to swap batteries on the drone and leave the base station, in addition to adding in the previous sortie time. We include the time it takes to transmit data back to the UGV in the final sortie of each sector by adding

$$\frac{q}{\lambda\big(-ln(\alpha)\big)^{1/k}} \mathbf{d}_{lj} \tag{8.32}$$

to the inequality, making (8.29a) and example of a communication chance constraint introduced in Section 8.4. Note that the time to offload data remains in the time to complete the final sortie, even if $\mathbf{w}_{ljk} = 0$. If drone j is already carrying data from previously visited PoI that needs to be relayed to the UGV, then we additionally include

$$\frac{q_0}{\lambda\big(-ln(\alpha)\big)^{1/k}} \tag{8.33}$$

to constraint (8.29a) for $l = 1$, where q_0 is the quantity of data that the drone is holding. This term is particularly relevant when planning online and the drones have already completed tasks prior to re-running

the solver. To make sector times cumulative, we add

$$\mathbf{t}_{l-1j\acute{k}} \tag{8.34}$$

to the left hand side of (8.29a) when $l > 1$, where $\acute{k}$ is the index of the final sortie from the previous sector.

Constraint (8.29b) bounds how much energy each drone spends on each sortie. Similar to (8.29a), we add

$$r_e(v_j)\Big(\sum_{i\in I_l}\frac{d_{gi}}{v_j}\mathbf{x}_{ljgi} + \frac{d_{gb}}{v_j}\mathbf{x}_{ljgb}\Big) \tag{8.35}$$

to the left hand side of (8.29b) when $k = 1$, and

$$r_e(v_j)\Big(\sum_{i\in I_l}\frac{d_{bi}}{v_j}\mathbf{x}_{ljbik} + \frac{d_{bg}}{v_j}\mathbf{x}_{ljbgk}\Big) \tag{8.36}$$

when $k > 1$. If drone j is already carrying data that needs to be delivered to the UGV, we add

$$r_e(0)\frac{q_0}{\lambda\big(-ln(\alpha)\big)^{1/k}}\mathbf{o}_{ljk} \tag{8.37}$$

when $l = 1$. Each drone must return to the base after completing the mission. To account for this behavior, we include

$$\Big(r_e(v_j)\frac{d_{gb}}{v_j} + {}^{l}e_j\Big)\mathbf{o}_{ljk} \tag{8.38}$$

when $l = |L|$, where d_{gb} is the distance from the UGV's final position to the drone base and ${}^{l}e_j$ is the energy required for j to land.

Constraint (8.29c) restricts the total energy spent on each sortie. We replace e_j with ${}^{i}e_j$ (the initial battery level of drone j) when $l = 1$ and $k = 1$ and with $\tilde{\mathbf{e}}_{l-1j}$ (the energy remaining at the end of the previous sector) when $l > 1$ and $k = 1$. Constraint (8.29d) determines the final energy of each drone at the end of each sector.

Constraint (8.29e) allows us to count the number of PoI that each drone visits in each sector. Constraint (8.29f) determines which sortie is the drone's last sortie of each sector (needed for tracking energy in (8.29b)) and constraint (8.29g) forces earlier sorties to be selected before later sorties. Constraint (8.29h) ensures that we only mark a single sorties as the final sortie of sector l while constraint (8.29i) prevents drone j from deploying on sortie k if sortie $k-1$ was marked as the final sortie for sector l. Constraint (8.29j) forces each activated sortie to start at the base station. We change the left hand side of the inequality to

$$\mathbf{x}_{ljgbk} + \sum_{i\in I_l}\mathbf{x}_{ljgik} \tag{8.39}$$

when $k = 1$. Similarly, constraint (8.29k) forces each activated drone sortie to terminate at either the base station or the UGV. We add $\mathbf{x}_{ljgb}$ to the left hand side of the inequality when $k = 1$ to account for the

drone leaving its starting position in sector l and traveling directly to the base station. Constraint (8.29l) forces each drone to eventually end each sector at the UGV.

The remaining constraints follow the classic VRP formulation, applied to each sector. Constraints (8.29m) and (8.29n) ensure that every PoI is visited exactly once by a drone. Constraint (8.29o) forces the edges that enter and exit each PoI be associated with the same drone and the same sortie. Finally, (8.29p) is a MTZ cycle-elimination constraint.

8.6.2.1 Solver Speed-Up

We employ many of the same techniques discussed in Section 8.6.1.1 to improve the solve time for the Drone-UGV scenario. We again found that a warm start and focusing on finding feasible solutions during deployment quickly improve solution quality. Additionally, we can bound $\mathcal{M}$ in (8.29p) as discussed in 8.6.1.1.

We found a few additional techniques beyond what was discussed in Section 8.6.1.1 to further improve solve time. We need to use a different approach to bound $\mathcal{M}$ in constraint (8.29d). This constraint allows us to track how much energy drone j has at the end of each sector. Therefore, we set $\mathcal{M}$ to the maximum battery capacity of j for constraint (8.29d). We also found that clique cuts improved solve time.

8.7 Evaluation

We applied our two multi-robot tasking scenarios to various case studies to demonstrate how our proposed tasking framework, with our bounds on model error and external interference, can effectively adapt team-level tasking decisions. We will first look at our experimental setup and baseline, then discuss the case studies for each tasking scenario, in turn.

8.7.1 Experimental Setup

We implemented our tasking framework in C++ using the Robot Operating System 2 library. We used Gurobi v12.0.2 to solve the MILP and MINLP for our two tasking scenarios. All simulation experiments were run on a laptop with 16 GiB of RAM and an 11th generation, eight-core Intel i7 processor that runs at 2.8 GHz. We used the Gazebo robot simulator with the PX4 Software In the Loop (SITL) to simulate the robots.

To make our simulation setup reflective of the real work, we ran various field experiments on our drone [279] to collect an example of both a drone energy model and a robot-to-robot communication model. For our energy model, we found $c_1 = 0.0262, c_2 = 0.597, c_3 = -12.3$, and $c_4 = 349.01$. Figure 8.2 shows a histogram of the data we collected while the drone was traveling at 10 meters per second. This data was collected on a day with moderate to light wind. Based on our definition of model error, we found

that $\gamma_e = 0.089$ for this particular day. For our communication model, we found $c_5 = 11.0, c_6 = 0.48, c_7 = 4.2$, and $c_8 = 0.027$ for two Raspberry Pi4s (one mounted on the drone and the other positioned on the ground) transmitting data from 15 meters apart. Figure 8.3 displays a histogram of this data, where $\gamma_c = 0.1$. We used these constant values in our simulator but modulated the values for γ_e and γ_c to emulate changing environments.

We compare our team-level tasking framework (TL) against an agent-level decision maker (AL). The AL approach finds an initial, team-level plan offline by solving the same mathematical formulations for each scenario, respectively, using the same solver and computational resources as our framework, but ignores energy constraints. The agent-level decision maker then monitors the robot's energy during deployment, as well as updates system models in response to changes in the environment, and schedules recharge visits to the base as needed without coordinating its actions with the rest of the team. This approach is conceptually similar to the methods proposed in [110, 111, 132]. For a second baseline, we created a worst-case planner (WC) by running our proposed framework with the initial predicted value for $\gamma_e = 0.2$ and $\gamma_c = 0.2$. The concept behind this baseline is to plan for the worst case scenario and avoid the need to adapt during deployment.

For all scenarios, we set $e_j = 255{,}744$ J, which is approximately 80% of the expected capacity of a 4,000 mAh, 22.2 V LiPo drone battery. Unless stated otherwise, we initially set $\gamma_e = 0.05$ to find an initial, offline plan then update this during deployment, as described below. We converted penalty, as introduced in Section 8.6, from seconds-squared to minutes-squared to keep the output range of our metric reasonable. We set $\alpha = 0.99$ for our risk tolerance for energy and 0.8 for communication.

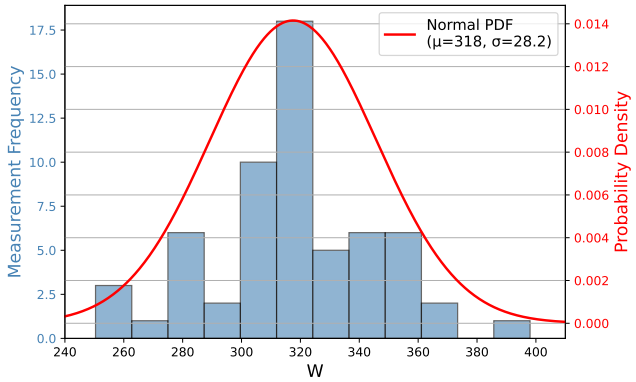


Figure 8.2 Field experiment results showing the energy consumption rate of the ROARQuad drone [279] while moving at 10 meters per second.

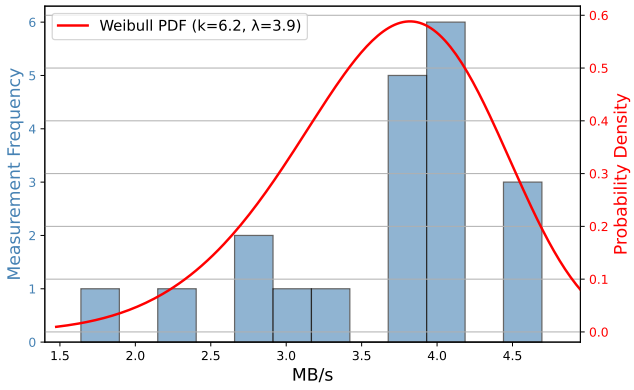
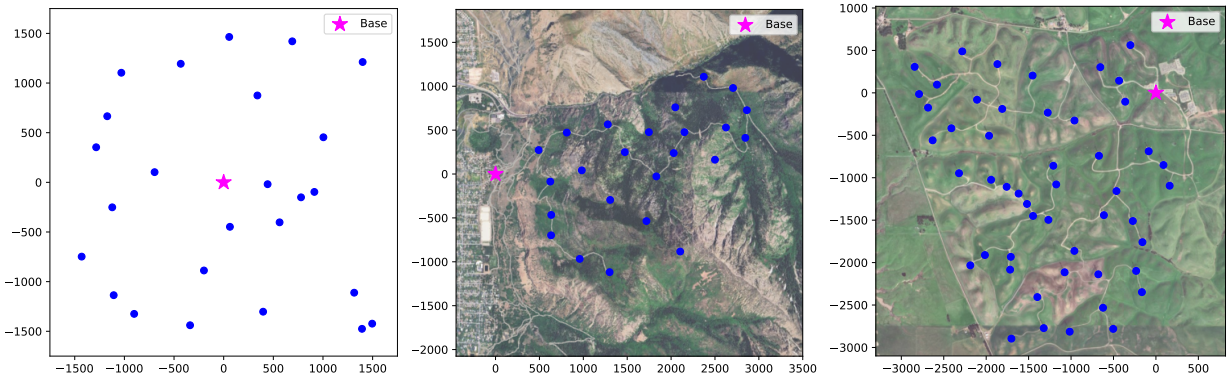


Figure 8.3 Field experiment results showing the data transmission rate between two robots 15 meters apart.

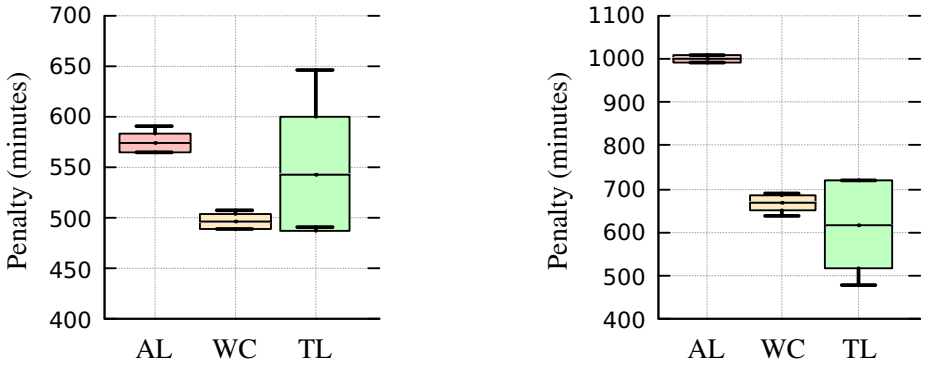


(a) Randomized (b) Utah SAR Mission (c) Wind Farm Inspection

Figure 8.4 Multi-drone site inspection case study setups.

8.7.2 Multi-Drone Site Inspection

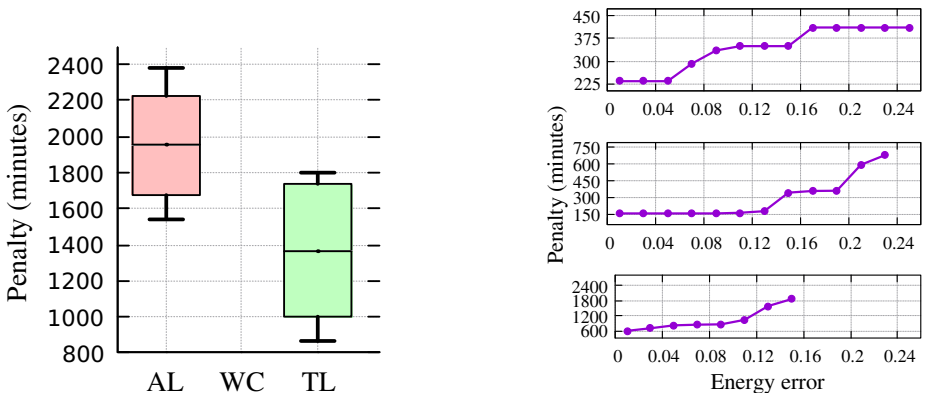
For the multi-drone site inspection scenario we considered three setups: (1) randomized waypoints, (2) a search and rescue (SAR) mission, and (3) infrastructure inspection. The randomized setup contains 25 PoI with (x, y) coordinates selected randomly over a uniform distribution between -1,500 and 1,500 meters (see Fig 8.4(a)). We deployed two drones for this setup and changed γ_e to 0.15 after 40 seconds. The online solver was given a 20 second timeout window. The SAR case study has 24 PoI and is modeled after the real-world search for a pair of lost hikers in the Wasatch Mountains in Utah, USA[300] (see Fig 8.4(b)). We again deployed two drones and changed γ_e to 0.19 after 20 seconds to emulate deploying in heavy wind with a 40 second timeout window for the online solver. The infrastructure inspection case study has 50 PoI and is built around the EDF Renewable Energy wind farm in Birds Landing, California, USA (see Fig 8.4(c)). We deployed three drones for this case study, with γ_e changing to 0.1 after 180 seconds to simulate a delayed, smaller change in wind. The online solver had a 60 second timeout window.



(a) Randomized

(b) Utah SAR Mission

Figure 8.5 Multi-drone site inspection case study results.

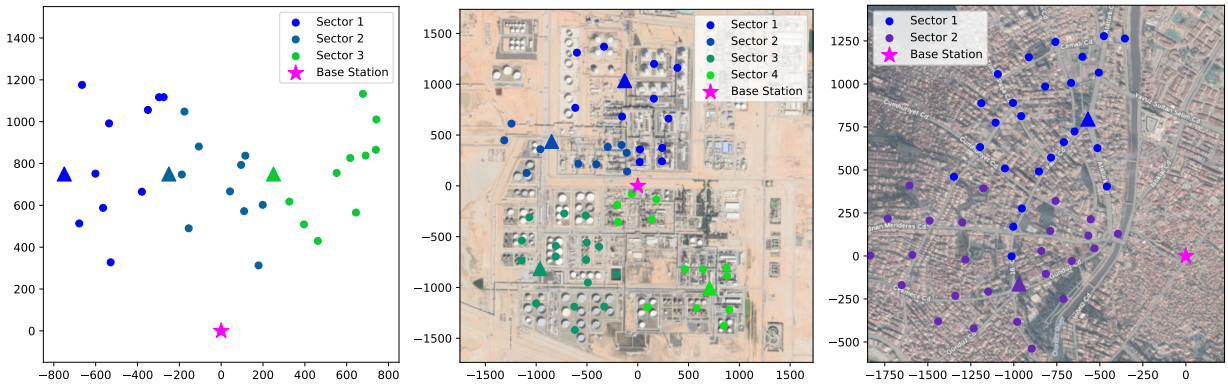


(a) Wind Farm Inspection

(b) Increasing penalty with energy error for the Randomized (*top*), Utah SAR (*middle*), and Wind Farm (*bottom*) case studies.

Figure 8.6 Multi-drone site inspection case study results.

We ran each of the case studies five times for each approach. Figures 8.5 and 8.6 shows the average, minimum, maximum, and standard deviation of the penalty for each approach on each case study. On average, our team-level tasking framework reduces penalty by 5.4% on the Randomized setup, 30.0% on the Utah SAR case study, and 38.1% on the wind farm case study when compared to the agent-level approach. Compared to the worst-case approach, our team-level approach saw a 9.4% increase in penalty on the Randomized input but reduced penalty by 7.5% on the Utah SAR case study. The offline planning for the worst-case approach was unable to find a valid solution for wind farm case study. We also ran the mission planner repeatedly while varying γ_e from 0.01 up to 0.25, in steps of 0.02 and recorded the penalty of the found solution. Figure 8.6(b) shows how penalty increases with energy model error. The mission planning failed to find a solution with $\gamma_e > 0.23$ on the Utah SAR case study and with $\gamma_e > 0.15$ for with Wind Farm case study.



(a) Randomized (b) Oil Refinery (c) Earthquake Response

Figure 8.7 Drone-UGV collaboration case studies. Triangles depict the waiting area for the UGV for each sector.

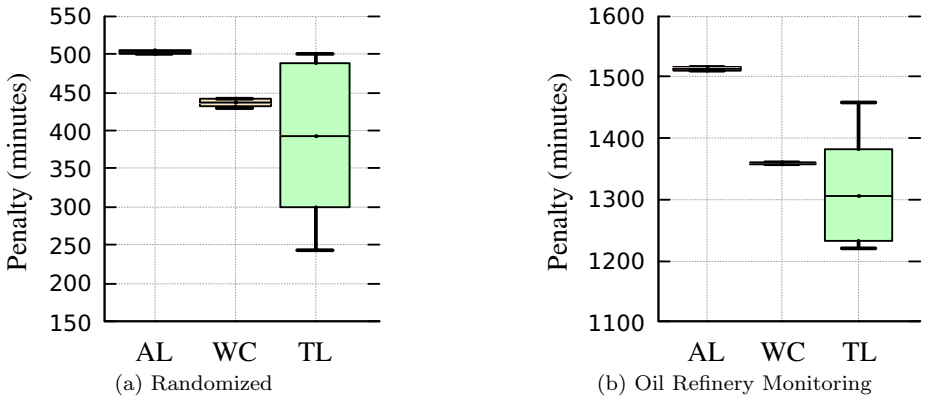
8.7.3 Collaborative Drone-UGV Teams

For the drone-UGV collaborative data acquisition and offloading scenario we again considered three setups: (1) randomized waypoints, (2) infrastructure monitoring, and (3) disaster response. Unless stated otherwise, we set $\gamma_c = 0.02$ for our predicted communication disturbance in the offline planner. During simulation, the drones updated this parameter to the stated actual value after the first time they communicated with the UGV.

The randomized setup contains 30 randomly selected PoI broken up into 3 sectors, as shown in Fig. 8.7(a). We deployed two drones for this setup. We set $\gamma_e = 0.1$ for both the initial plan and throughout the deployment and used $\gamma_c = 0.02$ for the initial plan with a real value of 0.2. We set the online solver timeout to 30 seconds for this setup. This setup shows the impact of communication disturbance. The infrastructure monitoring case study has 51 PoI and is designed around the Ruwais Petroleum Refinery in Ruwais, UAE (see Fig. 8.7(b)). We increased γ_e to 0.15 after 20 seconds and used $\gamma_c = 0.02$ for the offline planner with an actual value of 0.13. We deployed two drone for this case study and set the online solver timeout to 40 seconds. This setup demonstrates how changing energy and communication disturbance impacts performance. The disaster response case study is centered around the city of Antakya, Türkiya, where a 7.8-magnitude earthquake destroyed the city in 2023. This case study has 50 PoI selected from various locations throughout the city, spread across two sectors (see Fig. 8.7(c)). We deployed three drones for the case study, with γ_e changing to 0.18 after 30 seconds and $\gamma_c = 0.1$ for both the offline planner and throughout deployment. The online planner had a timeout window of 60 seconds.

We ran each of the case studies five times for each approach. Figures 8.8 and 8.9 shows the average, minimum, maximum, and standard deviation of the penalty for each approach on each case study. On

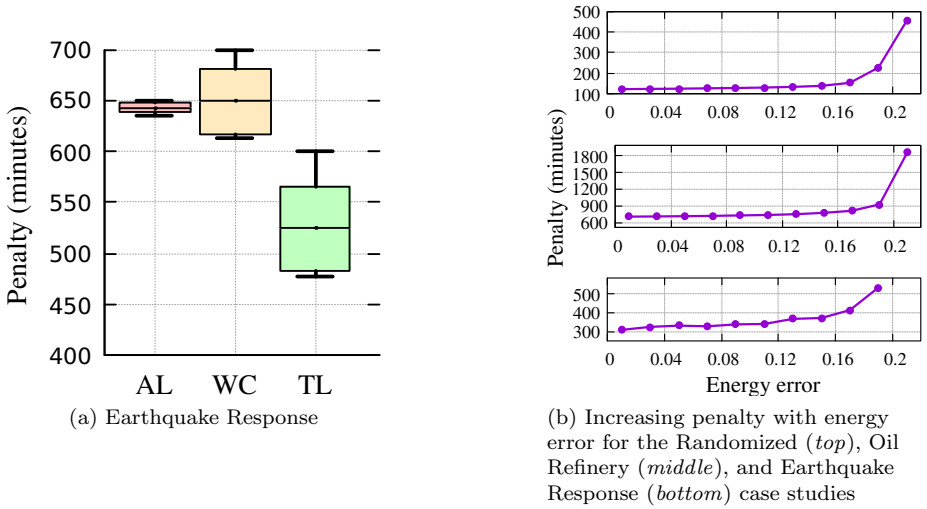
average, our team-level tasking framework reduces penalty by 21.6% on the Randomized setup, 13.8% on the Oil Refinery case study, and 18.4% on the Earthquake case study when compared to the agent-level approach. Compared to the worst-case approach, our approach reduced penalty by 9.6% on the Randomized setup, 3.8% on the Oil Refinery case study, and 19.2% on the Earthquake case study. We again ran the mission planner repeatedly while varying γ_c from 0.01 up to 0.21, in steps of 0.02, while holding $\gamma_e = 0.089$ (the value we found in our field experiments). Figure 8.6(b) shows how penalty increases with communication model error for each of the case studies. The mission planner failed when $\gamma_c > 0.19$ for the Earthquake Response case study.



(a) Randomized

(b) Oil Refinery Monitoring

Figure 8.8 Drone-UGV collaboration case study results.



(a) Earthquake Response

(b) Increasing penalty with energy error for the Randomized (*top*), Oil Refinery (*middle*), and Earthquake Response (*bottom*) case studies

Figure 8.9 Drone-UGV collaboration case study results.

8.7.4 Discussion & Limitations

Our results show consistent improvement over the baseline approach for both considered multi-robot tasking scenarios. Our proposed framework is able to respond to environmental changes and adapt how the entire team executes tasks. Although not completely ineffective, the agent-level decision maker does not adapt how the larger team responds to changes and therefore underperforms. The worst-case approach performs well at times but is not consistent. If the error value that we set for this approach is close to the error observed during deployment, then this approach performs well. However, when there is a large difference between the predicted error and actual error, and the problem input is particularly challenging to solve because it pushes the drones close to their limits, then the worst-case approach can fail to even find a solution.

Our team-level framework's reliance on a powerful, centralized solver is one of the limitations of this work. We see notably larger variance in the results when using our tasking framework. This variance comes from running the Gurobi solver online, with a limited timeout window. The timeout windows were set based on the difficulty of the problem input. From a practical standpoint, we would like to keep the timeout window as small as possible so that the robots respond as quickly as possible to changes. However, limiting the solve time may lead to less-than ideal solution quality, creating a need to find a balance between response time and response quality. We believe that a limited response time is what caused the noticeably larger standard deviation in Figures 8.5(a) and 8.5(b).

Additionally, our design requires some level of peer-to-peer communication. Although this framework can be readily adapted to accommodate intermittent communication, this is not a topic that we covered in this work and will need to be considered on a case-by-case basis with respect to the specific application and problem setup being considered.

8.8 Conclusions & Future Work

In this article we looked at the problem of adapting multi-robot tasking decisions online in response to changing environments and external disturbance. Given an initial team-level task plan, we showed how to bound allowable robot system model error caused by these changes, with a focus on energy and communication. We integrated our theoretical error bounds into a generalized, team-level tasking framework that is able to adapt tasking decisions during deployment.

To demonstrate how to apply our tasking framework, we presented two multi-robot tasking scenarios and developed several case studies based on each of these scenarios. We evaluated our proposed framework on each setup in simulation, using system models based on data from field experiments. The results from our various case studies show that our proposed framework is able to outperform current methodologies for

responding to stochastic robot systems and changing environments.

In this article we focused on adapting to changing system model parameters, but we see future work in identifying when these parameters have changed online. We see this as an extension to existing work on robot state estimation and will require more field experiments on physical hardware. Future work should also focus on addressing the limitations identified in Section 8.7.4, specifically tuning the tradeoff between response time and response quality based on application needs and adapting the decision-making process in response to intermittent communication.

CHAPTER 9
CONCLUSIONS & FUTURE WORK

In this chapter we summarize the main conclusions of this thesis and review various open questions and directions for future work.

9.1 Conclusions

This thesis pursued the broad research question:

> *How do we accurately model robot behavior and how do we incorporate these models into the decision-making process for robot teams?*

Rather than presenting a single solution to this question, we instead advocate for the design philosophy of *Reflective Planning*. In this design philosophy, we first investigate how to better model the way that robots behave in the real world through data-driven, preliminary field experiments. We then investigate how the characteristics of these refine models will impact application-level performance in multi-robot planning scenarios.

We demonstrated how the *Reflective Planning* philosophy can both improve performance and extend the capabilities of robots through various multi-robot planning problems and applications, with a focus on energy and communication. We showed that there exists a tradeoff between travel distance and speed for drones and demonstrated how to exploit this tradeoff in a drone-UGV search-and-rescue mission. We also investigated how to incorporate energy-sharing between drones and UGV into patrolling algorithms for infrastructure monitoring applications over indefinite time horizons. We then presented a method of assigning a probability to the operational capabilities of drones based on energy storage and used this to optimize the probability that a team of drones complete monitoring tasks for forest fires.

From a communication perspective, we proposed a novel method for mapping the potential for robot-to-robot communication and proved that this mapping can be used to find optimal robot configurations for various data gathering and exploration applications. We then investigated the intersection of communication and energy in a multi-drone data muling scenario and showed how to balance alloted energy between planned and unplanned actions. We then showed how to find an upper bound on the impact of external disturbances to stochastic models for energy and communication and demonstrated how to utilize these bounds to adapt team-level decisions during deployment.

In summary, this body of works demonstrates the value of developing robust, high fidelity models for how robots interact with their environment and each other. We showed that taking a closer look at how

these models impact application-level performance can not only better guide how we construct planning algorithms but can also extend the capabilities of robot platforms.

9.2 Future Work

We see various possible directions for future work. One possible course is to reapply our *Reflective Planning* philosophy to other robot capabilities that were not considered in this thesis. There are also a multitude of multi-robot planning problems that we did not consider and the knowledge that we have gained from this work can readily be reapplied and extended by considering more applications and new problem setups. Beyond extending the *Reflective Planning* philosophy to new robot systems and scenarios, we see an opportunity to reapply this philosophy as a method for guiding and validating the growing area of embodied intelligence that has arrived with the advent of large language models.

9.2.1 Modeling More Robot Capabilities

This thesis focused on applying the *Reflective Planning* design philosophy to how we model and integrate energy and communication into decision-making algorithms for multi-robot applications. There are various other robot systems that we did not consider that we argue should also be approached from a *Reflective Planning* point of view. Particularly mobility and perception.

Throughout this thesis, we utilized a simplified model for robot mobility. We assumed that robots move at a constant speed, neglecting acceleration. These assumptions helped simplify our planning process but is clearly not the case in practice. We would like to investigate the feasibility and reliability of solutions found while making simplifying assumptions about robot mobility. We would also like to investigate how to better model robot mobility and integrate this into team-level tasking algorithms.

We also assumed that our robots could travel from point a to point b without getting stuck or lost along the way. We found that this assumption does not always hold in our own field experiments throughout this thesis. Future work should investigate how to incorporate the stochastic nature of robot mobility into the decision making process for teams.

Robots require a means for perceiving their surroundings. There is already extensive research on robot perception, especially as it relates to control, motion planning, state estimation, and mapping. Many of these fields of research, particularly control, state estimation, and mapping, have focused on how to overcome perception inaccuracy and accounting for erroneous measurements. However, we see a lack of discussion on how this inaccuracy and noise will impact decisions at a team-level for task applications that fall out of mapping and data acquisition. For example, how should uncertainty in state estimation be accounted for in a multi-robot rendezvous application?

9.2.2 Expanding To New Problem Domains

An interesting problem domain that we identified in our field experiments in Chapter 4 but have not fully explored is enabling robots to project teammate actions and reason about how other robots are making decisions. Robots should be sharing these high-fidelity models of how they operate in their environment with their team. The other robots should then be able to predict the actions and decisions of their teammates. We believe that projecting the behavior of other entities will enable robots to further refine their own decisions and anticipate the level of cooperation within their team. For example, in a rendezvous scenario each of the robots scheduled to meetup at a designated time and place should be able to use high-fidelity system models of their teammates to predict the likelihood that everyone arrives at the agreed upon location at the agreed upon time.

Another interesting class of problems, with their own, unique applications, is very-large scale multi-robot systems. For example, deploying hundreds of drones over a city for security. At some point, the scale becomes too large for central coordinators to effectively make decisions for every robot and even decentralized planning will hit a communication bottleneck. This was seen in Chapter 5, where the distributed Hungarian Method struggled to perform well on our physical test bed due to delays in WiFi communication. This puts emphasis on decentralized coordination and opens up new questions about which robots need to communicate with one another and how much information they need to share for the larger group to effectively make decisions.

We would also like to extend this line of work to new types of robot systems, notably space robots, which present their own, unique applications and challenges. These vehicles are not only energy-limited, but they commonly rely solar power on to recharge, creating unique operational challenges. Unmanned aerial vehicles for lunar environments are also constrained by limited energy, solar power for recharging, and often have limited endurance due to weight conservation caused by high transportation costs. We believe that our *Reflective Planning* philosophy should be leveraged to extend the capabilities and endurance of space robots.

9.2.3 Guiding and Validating Embodied Intelligence

There is growing interest in using large language models (LLMs) in robot decision making. Large language models show great potential in being able to interpret written instructions and act on this information, making them great for bridging the technology gap between a technology novice and advanced, engineered systems.

However, LLMs are not perfect. These agents are susceptible to "hallucinations" – where an LLM generates erroneous information – which is a major limitation to incorporating LLMs in the

decision-making process for robots. Although we would like to leverage their capabilities, LLMs should not be given unchecked control over robots, especially in environments where humans have limited oversight and control of how the robots operate.

We argue that both the *Reflective Planning* design philosophy and the lessons learned for applying it to various problems can be used to validate and guide LLMs decisions when controlling teams of robots. For example, LLMs are currently bound to the cloud and our communication maps, introduced in Chapter 6, could help determine what regions of the environment an LLM-enabled robot is able to operate in, preventing the robot from disconnecting itself from the cloud. Our work on bounding allowable model error in Chapter 8 could be leveraged to validate action sequences generated by an LLM and our work on estimating the probability of a robot's operational capabilities in Chapter 5 could be used as additional input to the LLM to better guide its decisions.

REFERENCES

[1] Yongkun Zhou, Bin Rao, and Wei Wang. Uav swarm intelligence: Recent advances and future trends. *IEEE Access*, 8, 2020. doi: 10.1109/ACCESS.2020.3028865.

[2] Jonathan Diller, Peter Hall, Corey Schanker, Kristen Ung, Philip Belous, Peter Russell, and Qi Han. Iccswarm: A framework for integrated communication and control in uav swarms. In *Proceedings of the Eighth Workshop on Micro Aerial Vehicle Networks, Systems, and Applications*, DroNet '22, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394055. doi: 10.1145/3539493.3539579. URL https://doi.org/10.1145/3539493.3539579.

[3] Jonathan Diller, Neil Dantam, John Rogers, and Qi Han. Communication jamming-aware robot path adaptation. In *International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. IEEE, 2023.

[4] Zhilong Liu, Raja Sengupta, and Alex Kurzhanskiy. A power consumption model for multi-rotor small unmanned aircraft systems. In *International Conference on Unmanned Aircraft Systems (ICUAS)*, 2017. doi: 10.1109/ICUAS.2017.7991310.

[5] Yong Zeng, Jie Xu, and Rui Zhang. Energy minimization for wireless communication with rotary-wing uav. *Transactions on Wireless Communications*, 18(4), 2019.

[6] Ying Qu, Bryan Ng, and Winston Seah. A survey of routing and channel assignment in multi-channel multi-radio wmns. *Journal of Network and Computer Applications*, 65, 2016.

[7] Jonathan Diller, Peter Hall, and Qi Han. Holistic path planning for multi-drone data collection. In *International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. IEEE, 2023.

[8] Kevin Yu, Ashish Kumar Budhiraja, Spencer Buebel, and Pratap Tokekar. Algorithms and experiments on routing of unmanned aerial vehicles with mobile recharging stations. *Journal of Field Robotics*, 36(3), 2019.

[9] Pratap Tokekar, Joshua Vander Hook, David Mulla, and Volkan Isler. Sensor planning for a symbiotic uav and ugv system for precision agriculture. *Transactions on Robotics*, 32(6), 2016.

[10] Yao Liu, Zhihao Luo, Zhong Liu, Jianmai Shi, and Guangquan Cheng. Cooperative routing problem for ground vehicle and unmanned aerial vehicle: The application on intelligence, surveillance, and reconnaissance missions. *IEEE Access*, 7, 2019.

[11] Parikshit Maini, Kevin Yu, PB Sujit, and Pratap Tokekar. Persistent monitoring with refueling on a terrain using a team of aerial and ground robots. In *International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018.

[12] Shafkat Islam and Abolfazl Razi. A path planning algorithm for collective monitoring using autonomous drones. In *Annual Conference on Information Sciences and Systems (CISS)*. IEEE, 2019.

[13] Roberto G Ribeiro, Luciano P Cota, Thiago AM Euzébio, Jaime A Ramírez, and Frederico G Guimarães. Unmanned-aerial-vehicle routing problem with mobile charging stations for assisting search and rescue missions in postdisaster scenarios. *Transactions on Systems, Man, and Cybernetics: Systems*, 52(11), 2021.

[14] Kevin Yu, Jason M O'Kane, and Pratap Tokekar. Coverage of an environment using energy-constrained unmanned aerial vehicles. In *international conference on robotics and automation (ICRA)*. IEEE, 2019.

[15] Xiaoshan Lin, Yasin Yazıcıoğlu, and Derya Aksaray. Robust planning for persistent surveillance with energy-constrained uavs and mobile charging stations. *IEEE Robotics and Automation Letters*, 7(2), 2022.

[16] Subramanian Ramasamy, Jean-Paul F Reddinger, James M Dotterweich, Marshal A Childers, and Pranav A Bhounsule. Cooperative route planning of multiple fuel-constrained unmanned aerial vehicles with recharging on an unmanned ground vehicle. In *International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2021.

[17] Md Safwan Mondal, Subramanian Ramasamy, James D Humann, James M Dotterweich, Jean-Paul F Reddinger, Marshal A Childers, and Pranav Bhounsule. A robust uav-ugv collaborative framework for persistent surveillance in disaster management applications. In *International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2024.

[18] Donghyun Kim, Baraki H Abay, RN Uma, Weili Wu, Wei Wang, and Alade O Tokuta. Minimizing data collection latency in wireless sensor network with multiple mobile elements. In *International Conference on Computer Communications (INFOCOM)*. IEEE, 2012.

[19] Jie Gong, Tsung-Hui Chang, Chao Shen, and Xiang Chen. Flight time minimization of uav for data collection over wireless sensor networks. *IEEE Journal on Selected Areas in Communications*, 36(9), 2018. doi: 10.1109/JSAC.2018.2864420.

[20] Mengyu Chen, Weifa Liang, and Sajal K Das. Data collection utility maximization in wireless sensor networks via efficient determination of uav hovering locations. In *International Conference on Pervasive Computing and Communications (PerCom)*. IEEE, 2021.

[21] Botao Zhu, Ebrahim Bedeer, Ha H Nguyen, Robert Barton, and Zhen Gao. Uav trajectory planning for aoi-minimal data collection in uav-aided iot networks by transformer. *IEEE Transactions on Wireless Communications*, 2022.

[22] Jürgen Scherer and Bernhard Rinner. Multi-uav surveillance with minimum information idleness and latency constraints. *Robotics and Automation Letters*, 5(3), 2020.

[23] Yuchen Li, Weifa Liang, Wenzheng Xu, Zichuan Xu, Xiaohua Jia, Yinlong Xu, and Haibin Kan. Data collection maximization in iot-sensor networks via an energy-constrained uav. *Transactions on Mobile Computing*, 22(1), 2021.

[24] Botao Zhu, Ebrahim Bedeer, Ha H Nguyen, Robert Barton, and Jerome Henry. Uav trajectory planning in wireless sensor networks for energy consumption minimization by deep reinforcement learning. *IEEE Transactions on Vehicular Technology*, 70(9), 2021.

[25] Jan Karel Lenstra and AHG Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11(2), 1981.

[26] Pieter Vansteenwegen, Wouter Souffriau, and Dirk Van Oudheusden. The orienteering problem: A survey. *European Journal of Operational Research*, 209(1), 2011.

[27] Zheng Wang and Jiuh-Biing Sheu. Vehicle routing problem with drones. *Transportation research part B: methodological*, 122, 2019.

[28] Francesco Betti Sorbelli, Alfredo Navarra, Lorenzo Palazzetti, Cristina M Pinotti, and Giuseppe Prencipe. Wireless iot sensors data collection reward maximization by leveraging multiple energy- and storage-constrained uavs. *Journal of Computer and System Sciences*, 139, 2024. ISSN 0022-0000. doi: https://doi.org/10.1016/j.jcss.2023.103475. URL https://www.sciencedirect.com/science/article/pii/S0022000023000806.

[29] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2024. URL https://www.gurobi.com.

[30] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, 1972.

[31] Keld Helsgaun. An effective implementation of the lin–kernighan traveling salesman heuristic. *European journal of operational research*, 126(1), 2000.

[32] Yuan Tang, Rui Zhou, Guibin Sun, Bin Di, and Rongling Xiong. A novel cooperative path planning for multirobot persistent coverage in complex environments. *Sensors Journal*, 20(8), 2020.

[33] Sara Pérez-Carabaza, Jürgen Scherer, Bernhard Rinner, José A López-Orozco, and Eva Besada-Portas. Uav trajectory optimization for minimum time search with communication constraints and collision avoidance. *Engineering Applications of Artificial Intelligence*, 85, 2019.

[34] Francesco Betti Sorbelli, Alfredo Navarra, Lorenzo Palazzetti, Cristina M. Pinotti, and Giuseppe Prencipe. Optimal and heuristic algorithms for data collection by using an energy- and storage-constrained drone. In Thomas Erlebach and Michael Segal, editors, *Algorithmics of Wireless Networks*, Cham, 2022. Springer International Publishing. ISBN 978-3-031-22050-0.

[35] Ilham Benmad, Elmahdi Driouch, and Mustapha Kardouchi. Trajectory planning for data collection in multi-uav assisted wsns. In *Vehicular Technology Conference: (VTC2022-Spring)*, 2022. doi: 10.1109/VTC2022-Spring54318.2022.9861010.

[36] Jürgen Scherer and Bernhard Rinner. Short and full horizon motion planning for persistent multi-uav surveillance with energy and communication constraints. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017. doi: 10.1109/IROS.2017.8202162.

[37] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. In *Operations Research Forum*, volume 3. Springer, 2022.

[38] Cristina Bazgan, Refael Hassin, and Jérôme Monnot. Approximation algorithms for some vehicle routing problems. *Discrete Applied Mathematics*, 146(1), 2005. ISSN 0166-218X. doi: https://doi.org/10.1016/j.dam.2004.07.003. URL https://www.sciencedirect.com/science/article/pii/S0166218X04002860.

[39] Christian Zauner, Hubert Gattringer, Andreas Müller, and Matthias Jörgl. Heuristic sequencing methods for time optimal tracking of nested, open and closed paths. *Multibody System Dynamics*, 59 (1), 2023.

[40] Yongyu Chen. Approximation schemes for capacity vehicle routing problems: A survey. In *International Conference on Computational Modelling, Simulation and Optimization (ICCMSO)*, 2023. doi: 10.1109/ICCMSO59960.2023.00059.

[41] Hamza Chakraa, François Guérin, Edouard Leclercq, and Dimitri Lefebvre. Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art. *Robotics and Autonomous Systems*, 168, 2023. ISSN 0921-8890. doi: https://doi.org/10.1016/j.robot.2023.104492. URL https://www.sciencedirect.com/science/article/pii/S0921889023001318.

[42] Wanqing Zhao, Qinggang Meng, and Paul WH Chung. A heuristic distributed task allocation method for multivehicle multitask problems and its application to search and rescue scenario. *Transactions on Cybernetics*, 46(4), 2015.

[43] Xiaohua Gao, Lei Wang, Xinyong Yu, Xichao Su, Yu Ding, Chen Lu, Haijun Peng, and Xinwei Wang. Conditional probability based multi-objective cooperative task assignment for heterogeneous uavs. *Engineering Applications of Artificial Intelligence*, 123, 2023.

[44] Gennaro Notomista, Siddharth Mayya, Seth Hutchinson, and Magnus Egerstedt. An optimal task allocation strategy for heterogeneous multi-robot systems. In *European Control Conference (ECC)*. IEEE, 2019.

[45] Xiaowei Fu, Peng Feng, and Xiaoguang Gao. Swarm uavs task and resource dynamic assignment algorithm based on task sequence mechanism. *IEEE Access*, 7, 2019.

[46] Alexandre Colaers Andersen, Konstantin Pavlikov, and Túlio AM Toffolo. Weapon-target assignment problem: Exact and approximate solution algorithms. *Annals of Operations Research*, 312(2), 2022.

[47] Harold W Kuhn. The hungarian method for the assignment problem. *Naval research logistics quarterly*, 2(1-2), 1955.

[48] Lyle Ramshaw and Robert E Tarjan. On minimum-cost assignments in unbalanced bipartite graphs. *HP Labs, Palo Alto, CA, USA, Tech. Rep. HPL-2012-40R1*, 20, 2012.

[49] Haris Aziz, Hau Chan, Ágnes Cseh, Bo Li, Fahimeh Ramezani, and Chenhao Wang. Multi-robot task allocation-complexity and approximation. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '21, Richland, SC, 2021. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450383073.

[50] Xuening Chang, Jianmai Shi, Zhihao Luo, and Yao Liu. Adaptive large neighborhood search algorithm for multi-stage weapon target assignment problem. *Computers & Industrial Engineering*, 181, 2023.

[51] Michael M Zavlanos, Leonid Spesivtsev, and George J Pappas. A distributed auction algorithm for the assignment problem. In *Conference on Decision and Control*. IEEE, 2008.

[52] Han-Lim Choi, Luc Brunet, and Jonathan P How. Consensus-based decentralized auctions for robust task allocation. *transactions on robotics*, 25(4), 2009.

[53] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, TK Kumar, et al. Multi-agent pathfinding: Definitions, variants, and benchmarks. In *Proceedings of the International Symposium on Combinatorial Search*, volume 10, pages 151–158, 2019.

[54] Oren Salzman and Roni Stern. Research challenges and opportunities in multi-agent path finding and multi-agent pickup and delivery problems. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, pages 1711–1715, 2020.

[55] Zhongxia Yan, Han Zheng, and Cathy Wu. Multi-agent path finding for cooperative autonomous driving. In *International Conference on Robotics and Automation (ICRA)*, pages 12361–12367. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10611649.

[56] D. Rus, B. Donald, and J. Jennings. Moving furniture with teams of autonomous robots. In *International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, volume 1, pages 235–242 vol.1. IEEE, 1995. doi: 10.1109/IROS.1995.525802.

[57] David Šišlák, Přemysl Volf, and Michal Pěchouček. Agent-based cooperative decentralized airplane-collision avoidance. *IEEE Transactions on Intelligent Transportation Systems*, 12(1):36–46, 2011. doi: 10.1109/TITS.2010.2057246.

[58] Yang Quan Chen and Zhongmin Wang. Formation control: a review and a new consideration. In *International Conference on Intelligent Robots and Systems*, pages 3181–3186. IEEE, 2005. doi: 10.1109/IROS.2005.1545539.

[59] Manuele Brambilla, Eliseo Ferrante, Mauro Birattari, and Marco Dorigo. Swarm robotics: a review from the swarm engineering perspective. *Swarm Intelligence*, 7(1):1–41, 2013. ISSN 1935-3820. doi: 10.1007/s11721-012-0075-2. URL https://doi.org/10.1007/s11721-012-0075-2.

[60] Isaac E. Weintraub, Meir Pachter, and Eloy Garcia. An introduction to pursuit-evasion differential games. In *American Control Conference (ACC)*, pages 1049–1066. IEEE, 2020. doi: 10.23919/ACC45564.2020.9147205.

[61] Songhua Li, Minming Li, Lingjie Duan, and Victor Lee. Online ride-hitching in uav travelling. In *International Computing and Combinatorics Conference*. Springer, 2021.

[62] Chi Harold Liu, Chengzhe Piao, and Jian Tang. Energy-efficient uav crowdsensing with multiple charging stations by deep learning. In *IEEE Conference on Computer Communications*, 2020. doi: 10.1109/INFOCOM41043.2020.9155535.

[63] Feng Shan, Junzhou Luo, Runqun Xiong, Wenjia Wu, and Jiashuo Li. Looking before crossing: An optimal algorithm to minimize uav energy by speed scheduling with a practical flight energy model. In *IEEE Conference on Computer Communications*, 2020. doi: 10.1109/INFOCOM41043.2020.9155376.

[64] Arnon M Hurwitz, James M Dotterweich, and Trevor A Rocks. Mobile robot battery life estimation: battery energy use of an unmanned ground vehicle. In *Energy Harvesting and Storage: Materials, Devices, and Applications XI*, volume 11722. SPIE, 2021.

[65] Chixiong Mao, Juan Liu, and Lingfu Xie. Multi-uav aided data collection for age minimization in wireless sensor networks. In *International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE, 2020.

[66] Matthew A Schack, John G Rogers, Qi Han, and Neil T Dantam. Robot team data collection with anywhere communication. In *International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023.

[67] International Telecommunication Union. Propagation data and prediction methods for the planning of indoor radiocommunication systems and radio local area networks in the frequency range 300 mhz to 450 ghz, 2019.

[68] Marshall Sweatt, Adewole Ayoade, Qi Han, John Steele, Khaled Al-Wahedi, and Hamad Karki. Wifi based communication and localization of an autonomous mobile robot for refinery inspection. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2015.

[69] Matthew A Schack, John G Rogers, Qi Han, and Neil T Dantam. Optimizing non-markovian information gain under physics-based communication constraints. *Robotics and Automation Letters*, 6(3), 2021.

[70] Matthew A Schack, John G Rogers, and Neil T Dantam. The sound of silence: Exploiting information from the lack of communication. *Robotics and Automation Letters*, 2024.

[71] Vivek Shankar Varadharajan, David St-Onge, Bram Adams, and Giovanni Beltrame. Swarm relays: Distributed self-healing ground-and-air connectivity chains. *IEEE Robotics and Automation Letters*, 5(4), 2020. doi: 10.1109/LRA.2020.3006793.

[72] Jerry Zhao and Ramesh Govindan. Understanding packet delivery performance in dense wireless sensor networks. In *International Conference on Embedded networked sensor systems*, 2003.

[73] Daniel Aguayo, John Bicket, Sanjit Biswas, Glenn Judd, and Robert Morris. Link-level measurements from an 802.11 b mesh network. In *Conference on Applications, technologies, architectures, and protocols for computer communications*, 2004.

[74] Jonathan Diller and Qi Han. Energy-aware drone path finding with a fixed-trajectory ground vehicle. *ACM J. Auton. Transport. Syst.*, February 2025. doi: 10.1145/3716894. URL https://doi.org/10.1145/3716894. Just Accepted.

[75] Jonathan Diller and Qi Han. Energy-aware uav path planning with adaptive speed. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2023.

[76] Nils Boysen, Stefan Fedtke, and Stefan Schwerdfeger. Last-mile delivery concepts: a survey from an operational research perspective. *Or Spectrum*, 43, 2021.

[77] Vincenzo Lomonaco, Angelo Trotta, Marta Ziosi, Juan de Dios Yáñez Ávila, and Natalia Díaz-Rodríguez. Intelligent drone swarm for search and rescue operations at sea. In *Workshop on AI for Good, NeurIPS 2018 (Neural Information Processing Systems)*, 2018.

[78] Lihua Ruan, Lingjie Duan, and Jianwei Huang. Optimal uav hitching on ground vehicles. In *Global Communications Conference (GLOBECOM)*. IEEE, 2021.

[79] Mohamed Abdelkader, Samet Güler, Hassan Jaleel, and Jeff S Shamma. Aerial swarms: Recent applications and challenges. *Current robotics reports*, 2, 2021.

[80] Bander Alzahrani, Omar Sami Oubbati, Ahmed Barnawi, Mohammed Atiquzzaman, and Daniyal Alghazzawi. Uav assistance paradigm: State-of-the-art in applications and challenges. *Journal of Network and Computer Applications*, 166, 2020.

[81] Georgy Skorobogatov, Cristina Barrado, and Esther Salamí. Multiple uav systems: A survey. *Unmanned Systems*, 8(02), 2020.

[82] Tarek Bouzid, Noureddine Chaib, Mohamed Lahcen Bensaad, and Omar Sami Oubbati. 5g network slicing with unmanned aerial vehicles: Taxonomy, survey, and future directions. *Transactions on Emerging Telecommunications Technologies*, 34(3), 2023.

[83] Patchara Kitjacharoenchai, Mario Ventresca, Mohammad Moshref-Javadi, Seokcheon Lee, Jose MA Tanchoco, and Patrick A Brunese. Multiple traveling salesman problem with drones: Mathematical model and heuristic approach. *Computers & Industrial Engineering*, 129, 2019.

[84] Zhihao Luo, Mark Poon, Zhenzhen Zhang, Zhong Liu, and Andrew Lim. The multi-visit traveling salesman problem with multi-drones. *Transportation Research Part C: Emerging Technologies*, 128, 2021. ISSN 0968-090X. doi: https://doi.org/10.1016/j.trc.2021.103172. URL https://www.sciencedirect.com/science/article/pii/S0968090X2100190X.

[85] Tolga Bektas. The multiple traveling salesman problem: an overview of formulations and solution procedures. *omega*, 34(3), 2006.

[86] Kris Braekers, Katrien Ramaekers, and Inneke Van Nieuwenhuyse. The vehicle routing problem: State of the art classification and review. *Computers & industrial engineering*, 99, 2016.

[87] Baiq Zuyyina Hilyatur Rozaliya, I-Lin Wang, and Ahmad Muklason. Multi-uav routing for maximum surveillance data collection with idleness and latency constraints. *Procedia Computer Science*, 197, 2022.

[88] Raïssa G Mbiadou Saleu, Laurent Deroussi, Dominique Feillet, Nathalie Grangeon, and Alain Quilliot. The parallel drone scheduling problem with multiple drones and vehicles. *European Journal of Operational Research*, 300(2), 2022.

[89] Jinyu Fu, Guanghui Sun, Jianxing Liu, Weiran Yao, and Ligang Wu. On hierarchical multi-uav dubins traveling salesman problem paths in a complex obstacle environment. *Transactions on Cybernetics*, 54(1), 2023.

[90] Parikshit Maini and P. B. Sujit. On cooperation between a fuel constrained uav and a refueling ugv for large scale mapping applications. In *International Conference on Unmanned Aircraft Systems (ICUAS)*, 2015. doi: 10.1109/ICUAS.2015.7152432.

[91] Parikshit Maini, Kaarthik Sundar, Mandeep Singh, Sivakumar Rathinam, and PB Sujit. Cooperative aerial–ground vehicle route planning with fuel constraints for coverage applications. *Transactions on Aerospace and Electronic Systems*, 55(6), 2019.

[92] Divansh Arora, Parikshit Maini, Pedro Pinacho-Davidson, and Christian Blum. Route planning for cooperative air-ground robots with fuel constraints: An approach based on cmsa. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450361118. doi: 10.1145/3321707.3321820. URL https://doi.org/10.1145/3321707.3321820.

[93] Kevin Yu, Ashish Kumar Budhiraja, and Pratap Tokekar. Algorithms for routing of unmanned aerial vehicles with mobile recharging stations. In *international conference on robotics and automation (ICRA)*. IEEE, 2018.

[94] Charles E Noon and James C Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR: Information Systems and Operational Research*, 31(1), 1993.

[95] Shushman Choudhury, Kiril Solovey, Mykel Kochenderfer, and Marco Pavone. Coordinated multi-agent pathfinding for drones and trucks over road networks. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2022.

[96] Subramanian Ramasamy, Jean-Paul F Reddinger, James M Dotterweich, Marshal A Childers, and Pranav A Bhounsule. Coordinated route planning of multiple fuel-constrained unmanned aerial systems with recharging on an unmanned ground vehicle for mission coverage. *Journal of Intelligent & Robotic Systems*, 106(1), 2022.

[97] Jonathan Diller, Qi Han, Robert Byers, James Dotterweich, and James Humann. Hitchhiker's guide to patrolling: Path-finding for energy-sharing drone-ugv teams. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2025.

[98] Neelanga Thelasingha, A Agung Julius, James Humann, Jean-Paul Reddinger, James Dotterweich, and Marshal Childers. Iterative planning for multi-agent systems: An application in energy-aware uav-ugv cooperative task site assignments. *Transactions on Automation Science and Engineering*, 2024.

[99] Kenny Chour, Jean-Paul Reddinger, James Dotterweich, Marshal Childers, James Humann, Sivakumar Rathinam, and Swaroop Darbha. An agent-based modeling framework for the multi-uav rendezvous recharging problem. *Robotics and Autonomous Systems*, 166, 2023.

[100] Neil Mathew, Stephen L Smith, and Steven L Waslander. Planning paths for package delivery in heterogeneous multirobot teams. *Transactions on Automation Science and Engineering*, 12(4), 2015.

[101] Chase C Murray and Ritwik Raj. The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones. *Transportation Research Part C: Emerging Technologies*, 110, 2020.

[102] Ritwik Raj and Chase Murray. The multiple flying sidekicks traveling salesman problem with variable drone speeds. *Transportation Research Part C: Emerging Technologies*, 120, 2020.

[103] Mohamed R Salama and Sharan Srinivas. Collaborative truck multi-drone routing and scheduling problem: Package delivery with flexible launch and recovery sites. *Transportation Research Part E: Logistics and Transportation Review*, 164, 2022.

[104] Yong Wang, Zheng Wang, Xiangpei Hu, Guiqin Xue, and Xiangyang Guan. Truck–drone hybrid routing problem with time-dependent road travel time. *Transportation Research Part C: Emerging Technologies*, 144, 2022.

[105] Stefan Poikonen and Bruce Golden. Multi-visit drone routing problem. *Computers & Operations Research*, 113, 2020.

[106] Patchara Kitjacharoenchai, Byung-Cheol Min, and Seokcheon Lee. Two echelon vehicle routing problem with drones in last mile delivery. *International Journal of Production Economics*, 225, 2020. ISSN 0925-5273. doi: https://doi.org/10.1016/j.ijpe.2019.107598. URL https://www.sciencedirect.com/science/article/pii/S0925527319304335.

[107] M Amine Masmoudi, Simona Mancini, Roberto Baldacci, and Yong-Hong Kuo. Vehicle routing problems with drones equipped with multi-package payload compartments. *Transportation Research Part E: Logistics and Transportation Review*, 164, 2022.

[108] Neil Mathew, Stephen L Smith, and Steven L Waslander. Multirobot rendezvous planning for recharging in persistent tasks. *Transactions on Robotics*, 31(1), 2015.

[109] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde*, 2017.

[110] Guangyao Shi, Nare Karapetyan, Ahmad Bilal Asghar, Jean-Paul Reddinger, James Dotterweich, James Humann, and Pratap Tokekar. Risk-aware uav-ugv rendezvous with chance-constrained markov decision process. In *Conference on Decision and Control (CDC)*. IEEE, 2022. doi: 10.1109/CDC51059.2022.9993358.

[111] Ahmad Bilal Asghar, Guangyao Shi, Nare Karapetyan, James Humann, Jean-Paul Reddinger, James Dotterweich, and Pratap Tokekar. Risk-aware recharging rendezvous for a collaborative team of uavs and ugvs. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2023.

[112] Tomas Baca, Petr Stepan, Vojtech Spurny, Daniel Hert, Robert Penicka, Martin Saska, Justin Thomas, Giuseppe Loianno, and Vijay Kumar. Autonomous landing on a moving vehicle with an unmanned aerial vehicle. *Journal of Field Robotics*, 36(5), 2019.

[113] Yufei Zhang, Zhong Wu, and Tong Wei. Precise landing on moving platform for quadrotor uav via extended disturbance observer. *Transactions on Intelligent Vehicles*, 2024.

[114] Juan Zhang, James F Campbell, Donald C Sweeney II, and Andrea C Hupman. Energy consumption models for delivery drones: A comparison and assessment. *Transportation Research Part D: Transport and Environment*, 90, 2021.

[115] Jungyun Bae and Sivakumar Rathinam. Approximation algorithms for multiple terminal, hamiltonian path problems. *Optimization Letters*, 6(1), 2012.

[116] Clair E Miller, Albert W Tucker, and Richard A Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, 7(4), 1960.

[117] Imdat Kara, Gilbert Laporte, and Tolga Bektas. A note on the lifted miller–tucker–zemlin subtour elimination constraints for the capacitated vehicle routing problem. *European Journal of Operational Research*, 158(3), 2004.

[118] Stuart Lloyd. Least squares quantization in pcm. *IEEE transactions on information theory*, 28(2), 1982.

[119] Ronald L Rardin. *Optimization in operations research*. Prentice Hall, Upper Saddle River, NJ, 1998.

[120] Yinyu Ye and Edison Tse. An extension of karmarkar's projective algorithm for convex quadratic programming. *Mathematical programming*, 44, 1989.

[121] Ruben Sánchez-Corcuera, Adrián Nuñez-Marcos, Jesus Sesma-Solance, Aritz Bilbao-Jayo, Rubén Mulero, Unai Zulaika, Gorka Azkune, and Aitor Almeida. Smart cities survey: Technologies, application domains and challenges for the cities of the future. *International Journal of Distributed Sensor Networks*, 15(6), 2019.

[122] Weisen Shi, Haibo Zhou, Junling Li, Wenchao Xu, Ning Zhang, and Xuemin Shen. Drone assisted vehicular networks: Architecture, challenges and opportunities. *IEEE Network*, 32(3), 2018.

[123] Ahmed Bahabry, Xiangpeng Wan, Hakim Ghazzai, Gregg Vesonder, and Yehia Massoud. Collision-free navigation and efficient scheduling for fleet of multi-rotor drones in smart city. In *International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, 2019.

[124] Muhammad Asghar Khan, Bilal Ahmed Alvi, Alamgir Safi, and Inam Ullah Khan. Drones for good in smart cities: A review. In *Proc. Int. Conf. Elect., Electron., Comput., Commun., Mech. Comput.(EECCMC)*, 2018.

[125] United States Environmental Protection Agency. *Leak Detection and Repair: A Best Practices Guide*. Office of Compliance, Office of Enforcement and Compliance Assurance, Washington, DC, 2007. URL https://www.epa.gov/compliance/leak-detection-and-repair-best-practices-guide. Accessed: 2024-11-27.

[126] Wei Wei, Shuiyuan Cheng, Guohao Li, Gang Wang, and Haiyan Wang. Characteristics of volatile organic compounds (vocs) emitted from a petroleum refinery in beijing, china. *Atmospheric Environment*, 89, 2014.

[127] Chuanbo Yan and Tao Zhang. Multi-robot patrol: A distributed algorithm based on expected idleness. *International Journal of Advanced Robotic Systems*, 13(6), 2016.

[128] Mohammad Safwan Mondal, Subramanian Ramasamy, and Pranav Bhounsule. How to coordinate UAVs and UGVs for efficient mission planning? optimizing energy-constrained cooperative routing with a DRL framework. In *Proceedings of the Robotics: Science and Systems Conference (RSS)*, 2025. Accepted for publication.

[129] Md Safwan Mondal, Subramanian Ramasamy, James D Humann, Jean-Paul F Reddinger, James M Dotterweich, Marshal A Childers, and Pranav A Bhounsule. Cooperative multi-agent planning framework for fuel constrained uav-ugv routing problem. *arXiv preprint arXiv:2309.03397*, 2023.

[130] Subramanian Ramasamy, Md Safwan Mondal, James D Humann, James M Dotterweich, Jean-Paul F Reddinger, Marshal A Childers, and Pranav A Bhounsule. Optimizing routes of heterogenous unmanned systems using supervised learning in a multi-agent framework: A computational study. In *International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2024.

[131] Nare Karapetyan, Ahmad Bilal Asghar, Amisha Bhaskar, Guangyao Shi, Dinesh Manocha, and Pratap Tokekar. Ag-cvg: Coverage planning with a mobile recharging ugv and an energy-constrained uav. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2024.

[132] Ali Tevfik Buyukkocak, Derya Aksaray, and Yasin Yazıcıoğlu. Energy-aware planning of heterogeneous multi-agent systems for serving cooperative tasks with temporal logic specifications. In *International Conference on Intelligent Robots and Systems (IROS)*, pages 8659–8665. IEEE, 2023. doi: 10.1109/IROS55552.2023.10342064.

[133] Parikshit Maini, Pratap Tokekar, and PB Sujit. Visibility-based persistent monitoring of piecewise linear features on a terrain using multiple aerial and ground robots. *Transactions on Automation Science and Engineering*, 18(4), 2020.

[134] Sepehr Seyedi, Yasin Yazicioğlu, and Derya Aksaray. Persistent surveillance with energy-constrained uavs and mobile charging stations. *IFAC-PapersOnLine*, 52(20), 2019.

[135] Nicola Basilico. Recent trends in robotic patrolling. *Current Robotics Reports*, 3(2), 2022.

[136] Guangyao Shi and Pratap Tokekar. Decision-oriented learning with differentiable submodular maximization for vehicle routing problem. In *International Conference on Intelligent Robots and Systems (IROS)*, pages 11135–11140. IEEE, 2023. doi: 10.1109/IROS55552.2023.10342311.

[137] Steven S. Skiena. *The Algorithm Design Manual*. Springer Publishing Company, Incorporated, 2nd edition, 2008. ISBN 1848000693.

[138] Paolo Toth and Daniele Vigo. *The vehicle routing problem*. SIAM, 2002.

[139] Ilya Builuk. A new solver for rich Vehicle Routing Problem, 2023. URL https://doi.org/10.5281/zenodo.4624037.

[140] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 1st edition, 2004. ISBN 978-0-521-83378-3.

[141] Lixing Liu, Xu Wang, Xin Yang, Hongjie Liu, Jianping Li, and Pengfei Wang. Path planning techniques for mobile robots: Review and prospect. *Expert Systems with Applications*, 227:120254, 2023. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2023.120254. URL https://www.sciencedirect.com/science/article/pii/S095741742300756X.

[142] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7), 2011. doi: 10.1177/0278364911406761. URL https://doi.org/10.1177/0278364911406761.

[143] Yurii Nesterov and Arkadii Nemirovskii. *Interior-Point Polynomial Algorithms in Convex Programming*. Society for Industrial and Applied Mathematics, 1994.

[144] Ioan A. Şucan, Mark Moll, and Lydia E. Kavraki. The Open Motion Planning Library. *Robotics & Automation Magazine*, 19(4):72–82, December 2012. doi: 10.1109/MRA.2012.2205651. https://ompl.kavrakilab.org.

[145] Jonathan D. Gammell, Siddhartha S. Srinivasa, and Timothy D. Barfoot. Informed rrt*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic. In *International Conference on Intelligent Robots and Systems (IROS)*, pages 2997–3004. IEEE, 2014. doi: 10.1109/IROS.2014.6942976.

[146] Zongchang Lui. *A Systematic Framework for Unsupervised Feature Mining and Fault Detection for Wind Turbine Drivetrain Systems*. PhD thesis, University of Cincinnati, 2016. URL https://etd.ohiolink.edu/acprod/odb_etd/etd/r/1501/10?clear=10&p10_accession_num=ucin1471348052.

[147] Wei Qiao and Liyan Qu. Prognostic condition monitoring for wind turbine drivetrains via generator current analysis. *Chinese Journal of Electrical Engineering*, 4(3):80–89, 2018. doi: 10.23919/CJEE.2018.8471293.

[148] Wenxian Yang, P. J. Tavner, and Michael Wilkinson. Wind turbine condition monitoring and fault diagnosis using both mechanical and electrical signatures. In *International Conference on Advanced Intelligent Mechatronics*, pages 1296–1301. IEEE, 2008. doi: 10.1109/AIM.2008.4601849.

[149] Pierre Tchakoua, René Wamkeue, Mohand Ouhrouche, Fouad Slaoui-Hasnaoui, Tommy Andy Tameghe, and Gabriel Ekemb. Wind turbine condition monitoring: State-of-the-art review, new trends, and future challenges. *Energies*, 7(4):2595–2630, 2014. ISSN 1996-1073. doi: 10.3390/en7042595. URL https://www.mdpi.com/1996-1073/7/4/2595.

[150] Jonathan Diller, Yee Shen Teoh, Robert Byers, Qi Han, John G. Rogers, and Neil T. Dantam. Failure-aware tasking for teams of drones. In *International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2025.

[151] 2022. URL https://www.nps.gov/articles/000/yosemite-washburn-fire-suppression.htm.

[152] Marshall L Fisher, Ramchandran Jaikumar, and Luk N Van Wassenhove. A multiplier adjustment method for the generalized assignment problem. *Management science*, 32(9), 1986.

[153] William Feller. *An introduction to probability theory and its applications*. John Wiley & Sons, 1991.

[154] Stuart P Lloyd and Hans S Witsenhausen. Weapons allocation is np-complete. In *summer computer simulation conference*, 1986.

[155] K Ahuja Ravindra, K Arvind, CJ Krishna, and BO James. Exact and heuristic algorithms for the weapon-target assignment problem. *Operations Research*, 55(6), 2007.

[156] Tong Wang, Liyue Fu, Zhengxian Wei, Yuhu Zhou, and Shan Gao. Unmanned ground weapon target assignment based on deep q-learning network with an improved multi-objective artificial bee colony algorithm. *Engineering Applications of Artificial Intelligence*, 117, 2023.

[157] Lamiaa Khalid and Alagan Anpalagan. Adaptive assignment of heterogeneous users for group-based cooperative spectrum sensing. *Transactions on Wireless Communications*, 15(1), 2015.

[158] Jens Buergin, Philippe Blaettchen, Chuanqi Qu, and Gisela Lanza. Assignment of customer-specific orders to plants with mixed-model assembly lines in global production networks. *Procedia CIRP*, 50, 2016.

[159] Mitchell Khoo, Tony A Wood, Chris Manzie, and Iman Shames. A distributed augmenting path approach for the bottleneck assignment problem. *Transactions on Automatic Control*, 69(2), 2023.

[160] Stefano Giordani, Marin Lujak, and Francesco Martinelli. A distributed algorithm for the multi-robot task allocation problem. In *International Conference on Industrial Engineering and Other Applications of Applied Intelligent Systems*. Springer, 2010.

[161] Smriti Chopra, Giuseppe Notarstefano, Matthew Rice, and Magnus Egerstedt. A distributed version of the hungarian method for multirobot assignment. *Transactions on Robotics*, 33(4), 2017.

[162] Sarah Ismail and Liang Sun. Decentralized hungarian-based approach for fast and scalable task allocation. In *International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2017.

[163] Samuel Friedman and Qi Han. Request and share then assign (rasta): Task assignment for networked multi-robot teams. In *International Conference on Mobile Ad Hoc and Sensor Systems (MASS)*. IEEE, 2020.

[164] Alham Aji and Kenneth Heafield. Sparse communication for distributed gradient descent. In *Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics (ACL), 2017.

[165] John C Duchi, Alekh Agarwal, and Martin J Wainwright. Dual averaging for distributed optimization: Convergence analysis and network scaling. *Transactions on Automatic control*, 57(3), 2011.

[166] Bin Zheng, Ouyang Liu, Jing Li, Yong Lin, Chong Chang, Bo Li, Tefeng Chen, and Hao Peng. Towards a distributed local-search approach for partitioning large-scale social networks. *Information Sciences*, 508, 2020.

[167] Andrew J Smith, Graeme Best, Javier Yu, and Geoffrey A Hollinger. Real-time distributed non-myopic task selection for heterogeneous robotic teams. *Autonomous Robots*, 43, 2019.

[168] Helena Ramalhinho Lourenço, Olivier C Martin, and Thomas Stützle. Iterated local search: Framework and applications. *Handbook of metaheuristics*, 2019.

[169] Ornella Pisacane, Andrea Attanasio, Francesca Guerriero, and Roberto Musmanno. A general distributed framework based on iterated local search. In *International Workshop on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications*. IEEE, 2009.

[170] Mario Levorato, Lúcia Drummond, Rosa Figueiredo, and Yuri Frota. A distributed gpu-based correlation clustering algorithm for large-scale signed social networks. In *Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD)*. SBC, 2017.

[171] Mario Valenti, Brett Bethke, Jonathan P How, Daniela Pucci de Farias, and John Vian. Embedding health management into mission tasking for uav teams. In *American Control Conference*. IEEE, 2007.

[172] Jonathan Diller and Qi Han. Energy-aware uav path planning with adaptive speed. In *International Conference on Autonomous Agents and Multiagent Systems*, 2023.

[173] C Love, A Gaskins, and NAVAL RESEARCH LAB WASHINGTON DC. Performance loss of lithium ion polymer batteries subjected to overcharge and overdischarge abuse. *Memorandum Report, NRL/MR/6110-12-9455*, 2012.

[174] Manuel Fernández and Stuart Williams. Closed-form expression for the poisson-binomial probability density function. *IEEE Transactions on Aerospace and Electronic Systems*, 46(2), 2010.

[175] Jonathan Diller, John G. Rogers, Neil T. Dantam, and Qi Han. Communication mapping for robot teams. *IEEE Transactions on Field Robotics*, 2, 2025. doi: 10.1109/TFR.2025.3568794.

[176] Douglas SJ De Couto, Daniel Aguayo, Benjamin A Chambers, and Robert Morris. Performance of multihop wireless networks: Shortest path is not enough. *SIGCOMM Computer Communication Review*, 33(1), 2003.

[177] Nessrine Chakchouk. A survey on opportunistic routing in wireless communication networks. *Communications Surveys & Tutorials*, 17(4), 2015.

[178] Douglas SJ De Couto, Daniel Aguayo, John Bicket, and Robert Morris. A high-throughput path metric for multi-hop wireless routing. In *International Conference on Mobile computing and networking*, 2003.

[179] International Telecommunication Union. Effects of building materials and structures on radiowave propagation above about 100 mhz, 2015.

[180] Mehmet Safak. *Digital communications*. John Wiley & Sons, 2017.

[181] Evşen Yanmaz, Samira Hayat, Jürgen Scherer, and Christian Bettstetter. Experimental performance analysis of two-hop aerial 802.11 networks. In *Wireless Communications and Networking Conference (WCNC)*. IEEE, 2014.

[182] Jeffrey N Twigg, Jonathan R Fink, Paul L Yu, and Brian M Sadler. Efficient base station connectivity area discovery. *The International Journal of Robotics Research*, 32(12), 2013.

[183] Michael M Zavlanos, Alejandro Ribeiro, and George J Pappas. Network integrity in mobile robotic networks. *IEEE Transactions on Automatic Control*, 58(1), 2012.

[184] Jonathan Fink, Alejandro Ribeiro, and Vijay Kumar. Robust control of mobility and communications in autonomous robot teams. *IEEE Access*, 1, 2013.

[185] Cory Dixon and Eric W Frew. Maintaining optimal communication chains in robotic sensor networks using mobility control. *Mobile Networks and Applications*, 14, 2009.

[186] Boda Ning, Qing-Long Han, Zongyu Zuo, Jiong Jin, and Jinchuan Zheng. Collective behaviors of mobile robots beyond the nearest neighbor rules with switching topology. *IEEE transactions on cybernetics*, 48(5), 2017.

[187] MSJNM Ocana, LM Bergasa, MA Sotelo, J Nuevo, and R Flores. Indoor robot localization system using wifi signal measure and minimizing calibration effort. In *International Symposium on Industrial Electronics*, volume 4. IEEE, 2005.

[188] Joydeep Biswas and Manuela Veloso. Wifi localization and navigation for autonomous indoor mobile robots. In *international conference on robotics and automation*. IEEE, 2010.

[189] Zhi Li, Christos Bampis, Julie Novak, Anne Aaron, Kyle Swanson, Anush Moorthy, and JD Cock. Vmaf: The journey continues. *Netflix Technology Blog*, 25(1), 2018.

[190] Anthony D Wood, John A Stankovic, and Sang Hyuk Son. Jam: A jammed-area mapping service for sensor networks. In *Real-Time Systems Symposium*. IEEE, 2003.

[191] Yongxiang Peng, Yaling Yang, Xianliang Lu, and Xuyang Ding. Coding-aware routing for unicast sessions in multi-hop wireless networks. In *Global Telecommunications Conference GLOBECOM*. IEEE, 2010.

[192] N Anyameluhor and E Peytchev. Routing and communication path mapping in vanets. In F Squazzoni, F Baronio, C Archetti, and M Castellani, editors, *European Conference on Modelling and Simulation (ECMS 2014), Brescia, Italy, 27-30 May 2014*. European Council for Modelling and Simulation, 2014. ISBN 9780956494481. doi: 10.7148/2014-0359. URL http://dx.doi.org/10.7148/2014-0359.

[193] Kyu-Han Kim and Kang G Shin. On accurate measurement of link quality in multi-hop wireless mesh networks. In *International conference on Mobile computing and networking*, 2006.

[194] Murat Senel, Krishna Chintalapudi, Dhananjay Lal, Abtin Keshavarzian, and Edward J Coyle. A kalman filter based link quality estimation scheme for wireless sensor networks. In *Global Telecommunications Conference*. IEEE, 2007.

[195] Jinglong Zhou, Cheng Guo, Przemyslaw Pawelczak, and Ignas Niemegeers. Adaptable link quality estimation for multi data rate communication networks. In *Vehicular Technology Conference*. IEEE, 2009.

[196] IEEE. Draft standard for information technology - telecommunications and information exchange between systems - local and metropolitan area networks - specific requirements - part 11: Wireless lan medium access control (mac) and physical layer (phy) specifications. *IEEE P802.11-REVma/D89.0*, 2010.

[197] Hyun-ja Im, Chang-Eun Lee, and Young-Jo Cho. Radio mapping scheme using collective intelligent robots for teleoperation in unstructured environments. In *international symposium on robot and human interactive communication*. IEEE, 2014.

[198] Martin Zoula and Jan Faigl. Wireless communication infrastructure building for mobile robot search and inspection missions. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2024.

[199] Chitra R Karanam and Yasamin Mostofi. A foundation for wireless channel prediction and full ray makeup estimation using an unmanned vehicle. *Sensors Journal*, 2023.

[200] Zhiyou Ji, Guoliang Li, Ruihua Han, Shuai Wang, Bing Bai, Wei Xu, Kejiang Ye, and Chengzhong Xu. Mpcom: Robotic data gathering with radio mapping and model predictive communication. *arXiv preprint arXiv:2404.10541*, 2024.

[201] Carlos Nieto-Granda, John G Rogers III, Nicholas Fung, Stephanie Kemna, Henrik I Christensen, and Gaurav Sukhatme. On-line coordination tasks for multi-robot systems using adaptive informative sampling. In *International Symposium on Experimental Robotics*. Springer, 2020.

[202] Renato Miyagusuku, Atsushi Yamashita, and Hajime Asama. Precise and accurate wireless signal strength mappings using gaussian processes and path loss models. *Robotics and Autonomous Systems*, 103, 2018.

[203] Ron Levie, Çağkan Yapar, Gitta Kutyniok, and Giuseppe Caire. Radiounet: Fast radio map estimation with convolutional neural networks. *Transactions on Wireless Communications*, 20(6), 2021.

[204] Mert Torun, Hong Cai, and Yasamin Mostofi. Spatial prediction of channel signal strength map using deep fully convolutional neural network. In *Asilomar Conference on Signals, Systems, and Computers*. IEEE, 2022.

[205] Ramviyas Parasuraman, Byung-Cheol Min, and Petter Ögren. Rapid prediction of network quality in mobile robots. *Ad Hoc Networks*, 138, 2023.

[206] Ramviyas Parasuraman, Petter Oegren, and Byung-Cheol Min. Kalman filter based spatial prediction of wireless connectivity for autonomous robots and connected vehicles. In *vehicular technology conference (VTC-fall)*. IEEE, 2018.

[207] Qiuyi Gu, Jincheng Yu, Zihan Lin, Jinggao Bai, Bangyan Zhang, Yuan Shen, Jian Wang, and Yu Wang. Md-radiomap: Multi-drone radio map building via single-anchor ultra-wideband localization network. In *International Conference on Automation Science and Engineering (CASE)*. IEEE, 2023.

[208] Jonathan Fink, Jeffrey Twigg, Paul Yu, and Brian M Sadler. A parsimonious model for wireless connectivity in robotic networks. In *Global Conference on Signal and Information Processing*. IEEE, 2013.

[209] Jacopo Banfi, Alberto Quattrini Li, Nicola Basilico, Ioannis Rekleitis, and Francesco Amigoni. Multirobot online construction of communication maps. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2017.

[210] Alberto Quattrini Li, Phani Krishna Penumarthi, Jacopo Banfi, Nicola Basilico, Jason M O'Kane, Ioannis Rekleitis, Srihari Nelakuditi, and Francesco Amigoni. Multi-robot online sensing strategies for the construction of communication maps. *Autonomous Robots*, 44, 2020.

[211] Francesco Amigoni, Jacopo Banfi, Nicola Basilico, Ioannis Rekleitis, and Alberto Quattrini Li. Online update of communication maps for exploring multirobot systems under connectivity constraints. In *International Symposium on Distributed Autonomous Robotic Systems*. Springer, 2019.

[212] Pawel Ladosz, Hyondong Oh, Gan Zheng, and Wen-Hua Chen. Gaussian process based channel prediction for communication-relay uav in urban environments. *Transactions on Aerospace and Electronic Systems*, 56(1), 2019.

[213] Martin Zoula, Miloš Prágr, and Jan Faigl. On building communication maps in subterranean environments. In *International Conference on Modelling and Simulation for Autonomous Systems*. Springer, 2020.

[214] Lillian Clark, Jeffrey A Edlund, Marc Sanchez Net, Tiago Stegun Vaquero, and Ali-akbar Agha-Mohammadi. Propem-l: Radio propagation environment modeling and learning for communication-aware multi-robot exploration. In *Robotics: Science and Systems*, 2022.

[215] Jason M Gregory, Jeffrey N Twigg, and Jonathan R Fink. Enabling autonomous information-gathering and self-recovery behaviors in partially-known, communication-constrained environments. In *International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 2016.

[216] Linux Wireless Documentation Project. Rate control, 2023. URL https://wireless.docs.kernel.org/en/latest/en/developers/documentation/mac80211/ratecontrol/minstrel.html. Accessed: 2024-10-15.

[217] Szymon Chachulski, Michael Jennings, Sachin Katti, and Dina Katabi. Trading structure for randomness in wireless opportunistic routing. *SIGCOMM Computer Communication Review*, 37(4), 2007.

[218] Xian Ni, Kun-chan Lan, and Robert Malaney. On the performance of expected transmission count (etx) for wireless mesh networks. In *International conference on performance evaluation methodologies and tools*, 2008.

[219] Po-Yu Chen, Laksh Bhatia, Roman Kolcun, David Boyle, and Julie A McCann. Contact-aware opportunistic data forwarding in disconnected lorawan mobile networks. In *International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2020.

[220] Khaled Q Abdelfadeel, Dimitrios Zorbas, Victor Cionca, and Dirk Pesch. $free$—fine-grained scheduling for reliable and energy-efficient data collection in lorawan. *Internet of Things Journal*, 7 (1), 2019.

[221] ITU-T. Low power protocol for wide area wireless networks. Recommendation Y.4480, International Telecommunication Union, November 2021. URL https://www.itu.int/rec/T-REC-Y.4480.

[222] Edward A Carlson, Pierre-Philippe Beaujean, and Edgar An. Location-aware routing protocol for underwater acoustic networks. In *OCEANS*. IEEE, 2006.

[223] Daeyoup Hwang and Dongkyun Kim. Dfr: Directional flooding-based routing protocol for underwater sensor networks. In *OCEANS*. IEEE, 2008.

[224] Bowen Yan, Lei Wang, Yu Su, Yu Tian, and Bingxian Lu. A novel routing metric for underwater optical wireless communication using ns-3. In *International Conference on Big Data Computing and Communications (BigCom)*. IEEE, 2021.

[225] Jean-Claude Latombe and Jean-Claude Latombe. Exact cell decomposition. *Robot motion planning*, 1991. doi: 10.1007/978-1-4615-4022-9_5.

[226] J. E. Bresenham. Algorithm for computer control of a digital plotter. In *Seminal Graphics: Pioneering Efforts That Shaped the Field, Volume 1*. Association for Computing Machinery, New York, NY, USA, 1998. ISBN 158113052X. URL https://doi.org/10.1145/280811.280913.

[227] Harris Drucker, Christopher J Burges, Linda Kaufman, Alex Smola, and Vladimir Vapnik. Support vector regression machines. *Advances in neural information processing systems*, 9, 1996.

[228] Leo Breiman. Random forests. *Machine learning*, 45, 2001.

[229] Yin-Wen Chang, Cho-Jui Hsieh, Kai-Wei Chang, Michael Ringgaard, and Chih-Jen Lin. Training and testing low-degree polynomial data mappings via linear svm. *Journal of Machine Learning Research*, 11(4), 2010.

[230] Michael R Garey and David S Johnson. *Computers and intractability*, volume 174. freeman San Francisco, 1979.

[231] Luigi Di Puglia Pugliese and Francesca Guerriero. A survey of resource constrained shortest path problems: Exact solution approaches. *Networks*, 62(3), 2013.

[232] Leonardo Lozano and Andrés L Medaglia. On an exact method for the constrained shortest path problem. *Computers & operations research*, 40(1), 2013.

[233] Michael R Souryal, Johannes Geissbuehler, Leonard E Miller, and Nader Moayeri. Real-time deployment of multihop relays for range extension. In *International Conference on Mobile systems, applications and services*, 2007.

[234] Daniel Mox, Miguel Calvo-Fullana, Mikhail Gerasimenko, Jonathan Fink, Vijay Kumar, and Alejandro Ribeiro. Mobile wireless network infrastructure on demand. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2020.

[235] Daniel Mox, Vijay Kumar, and Alejandro Ribeiro. Learning connectivity-maximizing network configurations. *Robotics and Automation Letters*, 7(2), 2022.

[236] Jingbang Wu, Huimei Lu, Yong Xiang, Rui Wu, and Feng Wang. Mbr: A map-based relaying algorithm for reliable data transmission through intersection in vanets. *Transactions on Intelligent Transportation Systems*, 20(10), 2018.

[237] Marshal A Childers, Arnon Hurwitz, Jeffrey Westrich, and John Millemaci. *Robotics research collaboration campus: Operations for industry challenges*. DEVCOM, Army Research Laboratory, 2021.

[238] A. J. B. Trevor, J. G. Rogers, and H. I. Christensen. Omnimapper: A modular multimodal mapping framework. In *IEEE International Conference on Robotics and Automation (ICRA)*, May 2014. doi: 10.1109/ICRA.2014.6907122.

[239] Roger Fletcher. *Practical methods of optimization*. John Wiley & Sons, 2000.

[240] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 2020. doi: 10.1038/s41592-019-0686-2.

[241] David JC MacKay. Bayesian interpolation. *Neural computation*, 4(3), 1992.

[242] Eric Schulz, Maarten Speekenbrink, and Andreas Krause. A tutorial on gaussian process regression: Modelling, exploring, and exploiting functions. *Journal of mathematical psychology*, 85, 2018.

[243] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL https://www.tensorflow.org/. Software available from tensorflow.org.

[244] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2011.

[245] Joel Meyer, Ahalya Prabhakar, Allison Pinosky, Ian Abraham, Annalisa Taylor, Millicent Schlafly, Katarina Popovic, Giovani Diniz, Brendan Teich, Borislava Simidchieva, et al. Scale-invariant specifications for human-swarm systems. *Transactions on Field Robotics*, 2024.

[246] Gautier Hattenberger, Titouan Verdu, Nicolas Maury, Pierre Narvor, Raphael Bailon-Ruiz, Grégoire Cayez, Fleur Couvreux, Gregory C Roberts, and Simon Lacroix. Experimental uav flights to collect data within cumulus clouds. *Transactions on Field Robotics*, 2024.

[247] Yogesh Girdhar, Levi Cai, Stewart Jamieson, Nathan McGuire, Genevieve Flaspohler, Stefano Suman, and Brian Claus. Streaming scene maps for co-robotic exploration in bandwidth limited environments. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2019.

[248] Admir Kaknjo, Muzaffar Rao, Edin Omerdic, Luke Robinson, Daniel Toal, and Thomas Newe. Real-time video latency measurement between a robot and its remote control station: Causes and mitigation. *Wireless Communications and Mobile Computing*, 2018.

[249] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1), 2020.

[250] Antoni Rosinol, Arjun Gupta, Marcus Abate, Jingnan Shi, and Luca Carlone. 3d dynamic scene graphs: Actionable spatial perception with places, objects, and humans. *Robotics: Science and Systems XVI*, 2020.

[251] Alan Mainwaring, David Culler, Joseph Polastre, Robert Szewczyk, and John Anderson. Wireless sensor networks for habitat monitoring. In *international workshop on Wireless sensor networks and applications*. ACM, 2002.

[252] Safwan Alfattani, Wael Jaafar, Halim Yanikomeroglu, and Abbas Yongacoglu. Multi-uav data collection framework for wireless sensor networks. In *Global Communications Conference (GLOBECOM)*. IEEE, 2019.

[253] Rezoan Ahmed Nazib and Sangman Moh. Energy-efficient and fast data collection in uav-aided wireless sensor networks for hilly terrains. *IEEE Access*, 9, 2021.

[254] Oussama Ghdiri, Wael Jaafar, Safwan Alfattani, Jihene Ben Abderrazak, and Halim Yanikomeroglu. Energy-efficient multi-uav data collection for iot networks with time deadlines. In *Global Communications Conference (GLOBECOM)*. IEEE, 2020.

[255] Xingxia Gao, Xiumin Zhu, and Linbo Zhai. Aoi-sensitive data collection in multi-uav-assisted wireless sensor networks. *IEEE Transactions on Wireless Communications*, 22(8), 2023. doi: 10.1109/TWC.2022.3232366.

[256] Manish Kumar Singh, Amit Choudhary, Sandeep Gulia, and Anurag Verma. Multi-objective nsga-ii optimization framework for uav path planning in an uav-assisted wsn. *J. Supercomput.*, 79(1), July 2022. ISSN 0920-8542. doi: 10.1007/s11227-022-04701-2. URL https://doi.org/10.1007/s11227-022-04701-2.

[257] Zhen Qin, Aijing Li, Chao Dong, Haipeng Dai, and Zhengqin Xu. Completion time minimization for multi-uav information collection via trajectory planning. *Sensors*, 19(18), 2019.

[258] Pejman A Karegar and Adnan Al-Anbuky. Travel path planning for uav as a data collector for a sparse wsn. In *International conference on distributed computing in sensor systems (DCOSS)*. IEEE, 2021.

[259] Peng Zhang, Baoyu Liu, Sihua Gao, Bingyang Zhao, and Weifeng Xu. A multi-uav trajectory planning algorithm for time-sensitive data collection. In *International Conference on Next Generation Data-driven Networks (NGDN)*, 2024. doi: 10.1109/NGDN61651.2024.10744152.

[260] Zheng Zhou, Juan Liu, and Chixiong Mao. Age of information oriented data collection via energy-constrained uavs in wireless sensor networks. *IEEE Access*, 12, 2024. doi: 10.1109/ACCESS.2024.3356175.

[261] Kehao Wang, Zheng Tang, Pei Liu, Yirui Cong, Xiangke Wang, Dejin Kong, and Yitong Li. Uav-based and energy-constrained data collection system with trajectory, time, and collection scheduling optimization. In *International Conference on Communications in China (ICCC)*. IEEE/CIC, 2021. doi: 10.1109/ICCC52777.2021.9580291.

[262] Qing Cai, Zheng Tang, and Chuan Liu. Joint trajectory, transmission time and power optimization for multi-uav data collecting system. *Heliyon*, 10(5), 2024. ISSN 2405-8440. doi: https://doi.org/10.1016/j.heliyon.2024.e26627. URL https://www.sciencedirect.com/science/article/pii/S2405844024026586.

[263] Conghao Zhou, Hongli He, Peng Yang, Feng Lyu, Wen Wu, Nan Cheng, and Xuemin Shen. Deep rl-based trajectory planning for aoi minimization in uav-assisted iot. In *International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE, 2019.

[264] Yitong Liu, Junjie Yan, and Xiaohui Zhao. Deep-reinforcement-learning-based optimal transmission policies for opportunistic uav-aided wireless sensor network. *IEEE Internet of Things Journal*, 9(15), 2022.

[265] Lin Li, Lei Wang, Jiawang Hou, Junjie Ma, and Yang Liu. Multi-uav path planning based on drl for data collection in uav-assisted iot. In *International Conference on Information Science and Technology (ICIST)*, 2024. doi: 10.1109/ICIST63249.2024.10805321.

[266] Harald Bayerlein, Mirco Theile, Marco Caccamo, and David Gesbert. Multi-uav path planning for wireless data harvesting with deep reinforcement learning. *IEEE Open Journal of the Communications Society*, 2, 2021.

[267] Eslam Eldeeb, Mohammad Shehab, and Hirley Alves. Age minimization in massive iot via uav swarm: A multi-agent reinforcement learning approach. In *Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*. IEEE, 2023. doi: 10.1109/PIMRC56721.2023.10293964.

[268] Nawaf Qasem Hamood Othman, Haithm M. Al-Gunid, and Imran Younas. Efficient multi-uav path planning for wireless data harvesting: A deep reinforcement learning approach. In *International Conference on Electronic Information and Communication Technology (ICEICT)*, 2024. doi: 10.1109/ICEICT61637.2024.10670944.

[269] Min Li, Xiao Liu, and Hong Wang. Completion time minimization considering gns' energy for uav-assisted data collection. *IEEE Wireless Communications Letters*, 12(12), 2023. doi: 10.1109/LWC.2023.3309625.

[270] Vihar Shah, Manish Kumar, and Abhishek Jindal. An energy efficient uav path planning for data collection in multi-node iot system. In *International Conference on Advanced Networks and Telecommunications Systems (ANTS)*. IEEE, 2024. doi: 10.1109/ANTS63515.2024.10898404.

[271] Hui Xiao, Zhigang Hu, Kun Yang, Yao Du, and Dongwei Chen. An energy-aware joint routing and task allocation algorithm in mec systems assisted by multiple uavs. In *International Wireless Communications and Mobile Computing (IWCMC)*, 2020. doi: 10.1109/IWCMC48107.2020.9148519.

[272] Yuwen Pan, Yuanwang Yang, and Wenzao Li. A deep learning trained by genetic algorithm to improve the efficiency of path planning for data collection with multi-uav. *IEEE Access*, 9, 2021.

[273] Tianyu Liu, Guangchi Zhang, Miao Cui, Changsheng You, Qingqing Wu, Shaodan Ma, and Wei Chen. Task completion time minimization for uav-enabled data collection in rician fading channels. *IEEE Internet of Things Journal*, 10(2), 2023. doi: 10.1109/JIOT.2022.3204658.

[274] Rongtao Zhang, Jie Hao, Ran Wang, Hui Wang, Hai Deng, and Shifang Lu. Global path planning for multi-objective uav-assisted sensor data collection: A drl approach. In Shiyu Yang and Saiful Islam, editors, *Web and Big Data. APWeb-WAIM 2022 International Workshops*, Singapore, 2023. Springer Nature Singapore. ISBN 978-981-99-1354-1.

[275] Florian A Potra and Stephen J Wright. Interior-point methods. *Journal of computational and applied mathematics*, 124(1-2), 2000.

[276] Peter Hall, Jonathan Diller, Ava Moon, and Qi Han. Drons-3: Framework for realistic drone and networking simulators. In *Workshop on Micro Aerial Vehicle Networks, Systems, and Applications*, DroNet '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702105. doi: 10.1145/3597060.3597239. URL https://doi.org/10.1145/3597060.3597239.

[277] Asad Arfeen, Krzysztof Pawlikowski, Don McNickle, and Andreas Willig. The role of the weibull distribution in modelling traffic in internet access and backbone core networks. *Journal of Network and Computer Applications*, 141, 2019. ISSN 1084-8045. doi: https://doi.org/10.1016/j.jnca.2019.05.002. URL https://www.sciencedirect.com/science/article/pii/S1084804519301547.

[278] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 1981.

[279] Matthew Hatch, Cody Fellinge, Jonathan Diller, and Qi Han. Roarquad: Robust, open academic research quadcopter. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*, MOBISYS '25, 2025.

[280] Bo Fu, William Smith, Denise M. Rizzo, Matthew Castanier, Maani Ghaffari, and Kira Barton. Robust task scheduling for heterogeneous robot teams under capability uncertainty. *Transactions on Robotics*, 39(2):1087–1105, 2023. doi: 10.1109/TRO.2022.3216068.

[281] Mohammad Safwan Mondal, Subramanian Ramasamy, Ragib Rownak, Luca Russo, James Humann, James Dotterweich, and Pranav Bhounsule. Risk-aware energy-constrained UAV-UGV cooperative routing using attention-guided reinforcement learning. In *Proceedings of the 2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025. Accepted for publication.

[282] Jonathan Diller, Qi Han, and Matthew Desaulniers. Holistic framework for multi-drone data collection in IoT systems. Manuscript under review, 2025.

[283] Shirin Rahmanpour and Reza Mahboobi Esfanjani. Decentralized energy-aware co-planning of motion and communication strategies for networked mobile robots. *Transactions on Cognitive and Developmental Systems*, 12(3):519–528, 2020. doi: 10.1109/TCDS.2019.2932751.

[284] Yaroslav Marchukov and Luis Montano. Communication-aware planning for robot teams deployment. *IFAC-PapersOnLine*, 50(1):6875–6881, 2017. ISSN 2405-8963. doi: 10.1016/j.ifacol.2017.08.1210. 20th IFAC World Congress.

[285] James Stephan, Jonathan Fink, and Alejandro Ribeiro. System architectures for communication-aware multi-robot navigation. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6395–6399, 2016. doi: 10.1109/ICASSP.2016.7472908.

[286] Jinwoo Park, Andrew Messing, Harish Ravichandar, and Seth Hutchinson. Risk-tolerant task allocation and scheduling in heterogeneous multi-robot teams. In *International Conference on Intelligent Robots and Systems (IROS)*, pages 5372–5379. IEEE, 2023. doi: 10.1109/IROS55552.2023.10341837.

[287] Amy Fang, Tenny Yin, and Hadas Kress-Gazit. Online resynthesis of high-level collaborative tasks for robots with changing capabilities. *IEEE Robotics and Automation Letters*, 10(2):2032–2039, 2025. doi: 10.1109/LRA.2025.3527337.

[288] Max Merlin, Shane Parr, Neev Parikh, Sergio Orozco, Vedant Gupta, Eric Rosen, and George Konidaris. Robot task planning under local observability. In *International Conference on Robotics and Automation (ICRA)*, pages 1362–1368. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10610876.

[289] Yan Ding, Xiaohan Zhang, Xingyue Zhan, and Shiqi Zhang. Learning to ground objects for robot task and motion planning. *Robotics and Automation Letters*, 7(2):5536–5543, 2022. doi: 10.1109/LRA.2022.3157566.

[290] Adam Polevoy, Craig Knuth, Katie M. Popek, and Kapil D. Katyal. Complex terrain navigation via model error prediction. In *International Conference on Robotics and Automation (ICRA)*, pages 9411–9417. IEEE, 2022. doi: 10.1109/ICRA46639.2022.9811644.

[291] Mark W. Spong, Seth Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, 2nd edition, 2020. ISBN 978-1-119-52399-4.

[292] Christopher D. McKinnon and Angela P. Schoellig. Context-aware cost shaping to reduce the impact of model error in receding horizon control. In *International Conference on Robotics and Automation (ICRA)*, pages 2386–2392. IEEE, 2020. doi: 10.1109/ICRA40945.2020.9197521.

[293] Derya Aksaray, Yasin Yazıcıoğlu, and Ahmet Semi Asarkaya. Probabilistically guaranteed satisfaction of temporal logic constraints during reinforcement learning. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6531–6537, 2021. doi: 10.1109/IROS51168.2021.9636598.

[294] Brian P. Gerkey and Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004. doi: 10.1177/0278364904045564.

[295] Chase C. Murray and Amanda G. Chu. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C: Emerging Technologies*, 54:86–109, 2015. ISSN 0968-090X. doi: 10.1016/j.trc.2015.03.005.

[296] Maria-Theresa Oanh Hoang, Kasper Andreas Rømer Grøntved, Niels van Berkel, Mikael B. Skov, Anders Lyhne Christensen, and Timothy Merritt. *Drone Swarms to Support Search and Rescue Operations: Opportunities and Challenges*, pages 163–176. Springer International Publishing, Cham, 2023. ISBN 978-3-031-28138-9. doi: 10.1007/978-3-031-28138-9_11.

[297] Sophie Jordan, Julian Moore, Sierra Hovet, John Box, Jason Perry, Kevin Kirsche, Dexter Lewis, and Zion Tsz Ho Tse. State-of-the-art technologies for uav inspections. *IET Radar, Sonar & Navigation*, 12(2):151–164, 2018. doi: 10.1049/iet-rsn.2017.0251.

[298] Rey Koslowski and Marcus Schulzke. Drones along borders: Border security uavs in the united states and the european union. *International Studies Perspectives*, 19(4):305–324, 2018.

[299] Maja Kucharczyk and Chris H. Hugenholtz. Remote sensing of natural hazard-related disasters with small drones: Global trends, biases, and research opportunities. *Remote Sensing of Environment*, 264:112577, 2021. ISSN 0034-4257. doi: 10.1016/j.rse.2021.112577.

[300] Dan Schwartz. Why drones are the future of outdoor search and rescue. *Outside Online*, October 2021. URL https://www.outsideonline.com/outdoor-adventure/exploration-survival/drones-search-rescue/. Accessed: 2025-08-08.

APPENDIX A

LIST OF PUBLICATIONS

Journal Papers Under Review or Ready for Submission:

[R3] Jonathan Diller, Soamar Homsi, and Qi Han. "*Semper Gumby: Adaptive Multi-Robot Tasking in Dynamic Environments.*" Ready for submission.

[R2] Jonathan Diller, Qi Han, and Matthew Desaulniers. "*Holistic Framework for Multi-Drone Data Collection in IoT Systems.*" In revision.

[R1] Jonathan Diller, Jack Sigler, and Qi Han. "*Collaborative Path Planning of Energy-Sharing Drone-UGV Teams for Persistent Patrolling.*" Under review at the ACM Journal on Autonomous Transportation Systems.

Journal Papers:

[J2] Jonathan Diller, John Rogers, Qi Han, and Neil T. Dantam. "*Communication Mapping for Robot Teams.*" Published in the IEEE Transactions on Field Robotics, 2025.

[J1] Jonathan Diller, Qi Han. "*Energy-aware drone path finding with a fixed-trajectory ground vehicle.*" Published in the ACM Journal on Autonomous Transportation Systems, 2025.

Conference Papers:

[C7] Jonathan Diller, Yee Shen Teoh, Robert Byers, Qi Han, John G. Rogers, and Neil T. Dantam. "*Failure-aware tasking for teams of drones.*" In proceedings of the International Conference on Distributed Computing Systems (ICDCS), 2025. *Acceptance rate: 19.7%.*

[C6] Jonathan Diller, Qi Han, Robert Byers, James Dotterweich, and James Humann. "*Hitchhiker's guide to patrolling: Path-finding for energy-sharing drone-ugv teams.*" In proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS), 2025. *Acceptance rate: 24.5%.*

[C5] Chenyang Wang, Jonathan Diller, and Qi Han. "*LLM for Generating Simulation Inputs to Evaluate Path Planning Algorithms.*" In proceedings of the International Conference on Machine Learning and Applications (ICMLA), 2024. *Acceptance rate: 24.3%.*

[C4] Jonathan Diller, Peter Hall, and Qi Han. "*Holistic path planning for multi-drone data collection.*" In proceedings of the International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT), 2023. *Acceptance rate: 38.4% (short paper).*

[C3] Jonathan Diller and Qi Han. "*Energy-aware uav path planning with adaptive speed.*" In proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS), 2023. *Acceptance rate: 23.3%.*

[C2] Jonathan Diller, Peter Idowu, and Javad Khazaei. "*Load-Leveling Trainer for Demand Side Management on a 45kW Cyber-Physical Microgrid.*" In proceedings of the Texas Power and Energy Conference (TPEC), 2020.

[C1] Jonathan Diller, Brennan Trussell, Javad Khazaei, and Peter Idowu. "*Hardware Development of a Sinusoidal PWM on a Three-Phase 3.5 kW SiC Converter.*" In proceedings of the Texas Power and Energy Conference (TPEC), 2020.

Workshop Papers:

[W3] Jonathan Diller, Neil Dantam, John Rogers, and Qi Han. "*Communication jamming-aware robot path adaptation.*" In proceedings of the International Workshop on Distributed Collective Intelligence (DISCOLI), 2023.

[W2] Peter Hall, Jonathan Diller, Ava Moon, and Qi Han. "*DroNS-3: Framework for realistic drone and networking simulators.*" In proceedings of the Workshop on Micro Aerial Vehicle Networks, Systems, and Applications (DroNet), 2023.

[W1] Jonathan Diller, Peter Hall, Corey Schanker, Kristen Ung, Philip Belous, Peter Russell, and Qi Han. "*ICCSwarm: A framework for integrated communication and control in uav swarms.*" In proceedings of the Workshop on Micro Aerial Vehicle Networks, Systems, and Applications (DroNet), 2022.

Extended Abstracts & Demonstrations:

[A4] Matthew Hatch, Cody Fellinge, Jonathan Diller, and Qi Han. "*ROARQuad: Robust, Open Academic Research Quadcopter.*" Accepted for demonstration at the International Conference on Mobile Systems, Applications, and Services (Mobisys), 2025.

[A3] Jonathan Diller. "*Self-Aware High-Level Planning for Robot Teams.*" Presented at the Cyber-Physical Systems Rising Stars Workshop, 2024. *Acceptance rate: 16.4%.*

[A2] Jonathan Diller. "*Planning and Coordination for Unmanned Aerial Vehicles.*" Presented at the International Conference on Autonomous Agents and Multiagent Systems' Doctoral Consortium, 2023.

[A1] Jonathan Diller, Qi Han, Chris Dreyer, Federico Rossi, Saptarshi Bandopadhyay, Jean-Pierre de la Croix, Amir Rahmani, Pamela Clark. "*Integrated Communication and Controls for Swarms of Small Satellites.*" Presented at the Inter-Planetary Small Satellite Conference, 2021.

APPENDIX B

COPYRIGHT

B.1 Permission to Reuse Material Already Published

This thesis contains material published in Association for Computing Machinery (ACM), the International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS), and the Institute of Electrical and Electronics Engineers (IEEE).

B.1.1 Association for Computing Machinery

The ACM states that "Authors can include partial or complete papers of their own (and no fee is expected) in a dissertation as long as citations and DOI pointers to the Versions of Record in the ACM Digital Library are included. Authors can use any portion of their own work in presentations and in the classroom (and no fee is expected)."[1]

B.1.2 International Foundation for Autonomous Agents and Multiagent Systems

The IFAAMAS states that "Permission to make digital or hard copies of portions of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyright for components of this work owned by others than IFAAMAS must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee."[2]

B.1.3 Institute of Electrical and Electronics Engineers

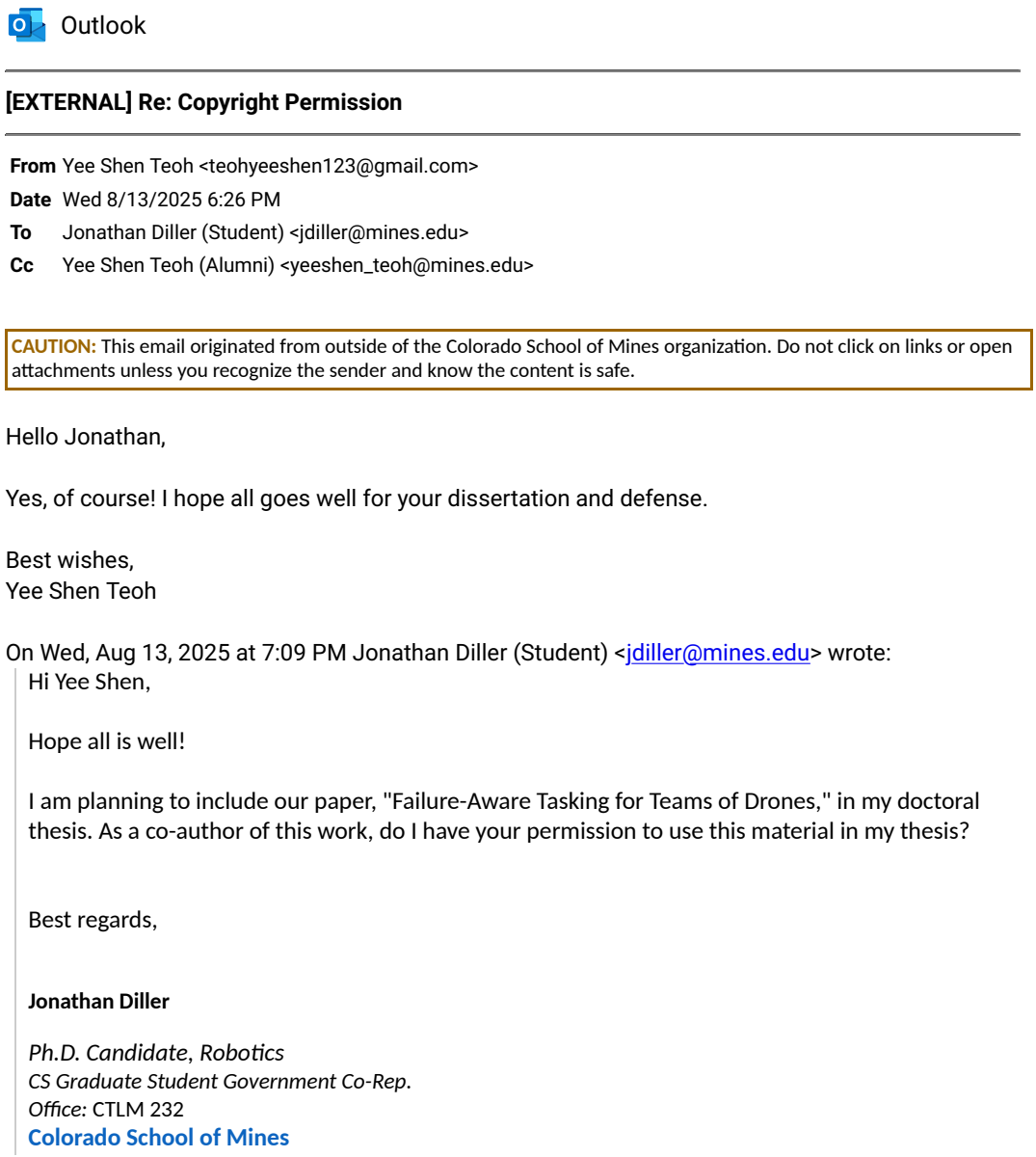
The IEEE states that "For journals and conference publications, an appropriate copyright notice shall appear on the first page of each technical contribution to simplify and facilitate reuse of individual articles."[3]

B.2 Permission from Co-Authors

Several chapters of this thesis have multiple co-authors. Permission to reuse chapters that have been published are listed below. Please note that co-authors Dr. Qi Han, Dr. Neil Dantam, and Dr. John G. Rogers were all members of the thesis committee and therefore explicit, written permission to reuse material is not required from these authors.

[1] https://authors.acm.org/author-resources/author-rights
[2] https://www.ifaamas.org/Proceedings/aamas2025/
[3] https://www.ieee.org/publications/rights/copyright-policy

- Permission from Yee Shen Teoh was granted to reproduce Chapter 5 (see Figure B.1).
- Permission from Robert Byers was granted to reproduce Chapter 5 (see Figure B.2).



Outlook

[EXTERNAL] Re: Copyright Permission

From Yee Shen Teoh <teohyeeshen123@gmail.com>
Date Wed 8/13/2025 6:26 PM
To Jonathan Diller (Student) <jdiller@mines.edu>
Cc Yee Shen Teoh (Alumni) <yeeshen_teoh@mines.edu>

CAUTION: This email originated from outside of the Colorado School of Mines organization. Do not click on links or open attachments unless you recognize the sender and know the content is safe.

Hello Jonathan,

Yes, of course! I hope all goes well for your dissertation and defense.

Best wishes,
Yee Shen Teoh

On Wed, Aug 13, 2025 at 7:09 PM Jonathan Diller (Student) <jdiller@mines.edu> wrote:

> Hi Yee Shen,
>
> Hope all is well!
>
> I am planning to include our paper, "Failure-Aware Tasking for Teams of Drones," in my doctoral thesis. As a co-author of this work, do I have your permission to use this material in my thesis?
>
> Best regards,
>
> **Jonathan Diller**
>
> *Ph.D. Candidate, Robotics*
> *CS Graduate Student Government Co-Rep.*
> *Office:* CTLM 232
> **Colorado School of Mines**

Figure B.1 Copyright Permission From Yee Shen Teoh.

Outlook

Re: Copyright Permission

From Robert Byers (Student) <robert_byers@mines.edu>
Date Wed 8/13/2025 8:02 PM
To Jonathan Diller (Student) <jdiller@mines.edu>

Hey! You have my permission! Thanks!

Get Outlook for iOS

From: Jonathan Diller (Student) <jdiller@mines.edu>
Sent: Wednesday, August 13, 2025 6:09:07 PM
To: Robert Byers (Student) <robert_byers@mines.edu>
Subject: Copyright Permission

Hi Will,

Hope all is well!

I am planning to include our paper, "Failure-Aware Tasking for Teams of Drones," in my doctoral thesis. As a co-author of this work, do I have your permission to use this material in my thesis?

Best regards,

Jonathan Diller
Ph.D. Candidate, Robotics
CS Graduate Student Government Co-Rep.
Office: CTLM 232
Colorado School of Mines

Figure B.2 Copyright Permission From Robert Byers.